

LineScope

A Literate Program for Studying
Resampling and Filtering of Images

`pfiguero@uniandes.edu.co`

September 25, 2026

Contents

1	Introduction	1
1.1	How to read this document	1
1.2	The shape of the program	2
1.3	Building	2
I	LineScope for macOS	4
2	Pixels	5
2.1	The representation	5
2.2	Construction and access	6
2.3	Premultiplication	7
2.4	Bridging to Core Graphics	8
3	Resampling	10
3.1	Theory in brief	10
3.2	The methods	11
3.3	Entry points	12
3.3.1	The two passes	13
3.4	Weights	14
3.5	Kernels	16
4	Filters and Operations	18
4.1	The filters	18
4.1.1	Dispatch	19
4.1.2	The Core Image bridge	21
4.1.3	CPU filters	22
4.2	Operations	23
5	Lines, Colors and Spectra	25
5.1	The selection line	25
5.2	Sampling along the line	26
5.3	Color models	28
5.3.1	HSL	29
5.3.2	CMYK	30
5.4	The spectrum	31
6	Test Patterns	35
6.1	The catalog	35
6.2	Specifications	37
6.3	The generator	37
6.3.1	Chirps and gratings	38

6.3.2	Edges and impulses	39
6.3.3	The Siemens star	40
6.3.4	Noise and color	40
6.4	SplitMix64	40
7	Tests	42
7.1	Spectrum	43
7.2	Resampling	43
7.3	Color conversions	45
7.4	Line sampling	45
7.5	Core Graphics and filters	46
7.6	Test patterns	47
8	Documents and the Session Model	48
8.1	The app and its scenes	48
8.2	The image document	49
8.3	Variants, zones and panes	52
8.4	The session	54
8.4.1	State	55
8.4.2	Panes	57
8.4.3	Variants and operations	57
8.4.4	Analysis	58
8.4.5	Export	59
8.4.6	Finding the session from outside the window	60
9	Views	61
9.1	The document window	61
9.2	Zones	64
9.3	Panes and tabs	66
9.4	The image canvas	69
9.5	Charts	73
9.6	The parameter sheet	75
9.7	The inspector	78
9.8	Test-pattern windows	80
10	Menus	85
10.1	Which document does a menu act on?	85
10.2	The File menu	86
10.3	Filters and resampling	86
10.4	The View menu	87
II	LineScope for Windows	89
11	The Windows Port	90
11.1	Why a native port	90
11.2	The solution	90
11.3	Same algorithms, different plumbing	91
11.4	How to read this part	91
11.5	Building	92
11.5.1	The solution file	92
11.5.2	The app project	93
11.5.3	The application manifest	94

11.5.4	Theme information	94
11.6	The application object	95
12	The Numeric Core in C#	99
12.1	The project file	99
12.2	Pixels	100
12.2.1	A class treated as a value	100
12.2.2	The 8-bit bridge	103
12.3	Resampling	104
12.4	Filters	110
12.4.1	Filter values	110
12.4.2	The filter class	111
12.4.3	Separable blurs	112
12.4.4	Sharpening, median and noise reduction	114
12.4.5	Why the results differ from macOS	117
12.4.6	CPU filters	117
12.5	Operations and number formatting	119
12.5.1	Formatting numbers	119
12.6	Lines and colors	120
12.6.1	Color models	123
12.7	The spectrum	125
12.7.1	The FFT	128
12.8	Test patterns	129
12.8.1	The generator	132
12.8.2	SplitMix64	135
13	Tests in C#	136
13.1	The test project	136
13.2	The test file	137
13.3	Spectrum	138
13.4	Resampling	139
13.5	Color conversions and line sampling	140
13.6	The bitmap bridge	142
13.7	Filters	142
13.8	Test patterns	144
14	Documents and the Session in C#	146
14.1	The image document	146
14.1.1	Decoding with WIC	147
14.1.2	Color management	149
14.2	Variants, zones and panes	151
14.3	The session	153
14.3.1	Change notification	154
14.3.2	State	155
14.3.3	Panes and variants	157
14.3.4	Analysis and export	159
14.3.5	The active session	160
15	Views and Menus in WPF	162
15.1	Views without XAML	162
15.2	The document window	164
15.3	Menus and shortcuts	168
15.4	Zones	171

15.5 Panes and tabs	176
15.6 The image canvas	179
15.7 Charts	185
15.8 The parameter dialog	191
15.9 The test-pattern window	195
15.10The inspector	201
Bibliography	207
Index of Chunks	208

Chapter 1

Introduction

LineScope is a small program, for macOS and for Windows, for *looking* at what resampling and filtering do to an image. You open an image, derive copies from it (blurred, sharpened, shrunk with nearest-neighbor, enlarged with Lanczos, and so on), and draw a line across it. For every copy the program shows three things side by side:

1. the image itself, at its true pixel scale, with the line drawn on it;
2. the *profile*: one color channel sampled along the line;
3. the *spectrum*: the Fourier transform of that profile.

Because the line is shared, moving it in one copy moves it in all of them, and the differences between the methods become visible at once. Aliasing shows as false energy below the Nyquist frequency, over-smoothing as a spectrum that dies too early, and ringing as ripples beside an edge.

1.1 How to read this document

This is a *literate program* in the sense of Donald Knuth [1]: a document written for people, from which the program for the machine is extracted. The code appears in *chunks*. Each chunk has a name and a number, and is written

$$\langle name\ n \rangle \equiv$$

where it is defined. A chunk may use other chunks by name; that is how a long function is explained a piece at a time, in the order that makes the ideas clear rather than the order the compiler wants. When the same name is defined again, the new text is appended to the old, and the header reads $+\equiv$. Below each chunk, a note tells where it is continued and where it is used.

The chunk index at the end of the book lists every chunk.

Two tools turn this document into something useful:

Tangling

follows the references from each *root* chunk and writes the source file the chunk is named after. The roots of this web are exactly the source files of the project, so tangling rebuilds LineScope from this document.

Weaving

produces the L^AT_EX you are reading, with the cross-references.

The directory LiterateP holds the web (chapters/*.nw), the tool `lit.py` that does both jobs using Norman Ramsey's noweb notation [2], and a Makefile. Running `make check` tangles the web and compares every file, byte for byte, with the real sources in the repository. So this document can't drift away from the program without the check noticing.

1.2 The shape of the program

The code is split in two.

The first part is `LineScopeCore`, a Swift package with no user interface. It holds everything numeric: the pixel buffer (Chapter 2), resampling (Chapter 3), filters (Chapter 4), line sampling, color models and the spectrum (Chapter 5), and generated test patterns (Chapter 6). It is tested on its own (Chapter 7).

The second part is the app, `LineScope`. It holds the document and the per-window session model (Chapter 8), the SwiftUI views (Chapter 9) and the menus (Chapter 10).

This split keeps the mathematics testable without a window, and keeps the views thin: a view asks the session for a profile or a spectrum and draws it.

These two parts form Part I of the book, *LineScope for macOS*. Part II, *LineScope for Windows*, presents a native port written in C# with .NET 9 and WPF (Chapter efch:win-overview). The port has the same split, file for file: a UI-free library `LineScope.Core` (Chapter efch:win-core) with its tests (Chapter efch:win-tests), and a WPF application with its session model (Chapter efch:win-model) and views (Chapter efch:win-views). The numeric algorithms are kept identical, so Part II doesn't repeat the theory. It concentrates on what had to change: an FFT of its own in place of Apple's vDSP, CPU versions of the Core Image filters, and the WPF plumbing for windows, menus and drawing. The root chunks of Part II are the files under `Windows/`, and `make check` verifies them in the same way.

1.3 Building

The Xcode project isn't kept in the repository. XcodeGen [6] generates it from a short specification. The app target depends on the local package, is a document *viewer* for any image type, and is signed ad hoc so it runs without a developer account.

```
1  <project.yml 1> ≡
    name: LineScope
    options:
      bundleIdPrefix: co.edu.uniandes
      deploymentTarget:
        macOS: "15.0"
    packages:
      LineScopeCore:
        path: Packages/LineScopeCore
    targets:
      LineScope:
        type: application
        platform: macOS
        sources: [LineScope]
        dependencies:
          - package: LineScopeCore
        info:
          path: LineScope/Info.plist
        properties:
          CFBundleDisplayName: LineScope
          LSApplicationCategoryType: public.app-category.graphics-design
          NSHumanReadableCopyright: ""
          CFBundleDocumentTypes:
            - CFBundleTypeName: Image
              CFBundleTypeRole: Viewer
              LSHandlerRank: Alternate
              LSItemContentTypes: [public.image]
        settings:
          base:
            PRODUCT_BUNDLE_IDENTIFIER: co.edu.uniandes.LineScope
            SWIFT_VERSION: "5.0"
```

```
MARKETING_VERSION: "0.1"
CURRENT_PROJECT_VERSION: "1"
CODE_SIGN_IDENTITY: "-"
CODE_SIGN_STYLE: Manual
ENABLE_HARDENED_RUNTIME: NO
GENERATE_INFOPLIST_FILE: NO
```

Root chunk, tangled to project.yml.

The rank `Alternate` means LineScope offers to open images but never claims to be their default application.

The package manifest declares one library and its tests. One line deserves comment. Swift in a Debug build is compiled without optimization, and the inner loops of resampling and filtering then run tens of times slower, since every array access is bounds-checked and nothing is inlined. The manifest therefore asks for `-O` even in Debug. This uses `unsafeFlags`, which the Swift Package Manager allows only for local packages like this one.

2 $\langle$ Packages/LineScopeCore/Package.swift 2 $\rangle \equiv$

```
// swift-tools-version: 5.10
```

```
import PackageDescription
```

```
let package = Package(
    name: "LineScopeCore",
    platforms: [.macOS(.v14)],
    products: [
        .library(name: "LineScopeCore", targets: ["LineScopeCore"]),
    ],
    targets: [
        .target(
            name: "LineScopeCore",
            // Pixel loops are unusably slow at -Onone; optimize even in Debug.
            swiftSettings: [.unsafeFlags(["-O"], .when(configuration: .debug))]
        ),
        .testTarget(
            name: "LineScopeCoreTests",
            dependencies: ["LineScopeCore"]
        ),
    ]
)
```

Root chunk, tangled to Packages/LineScopeCore/Package.swift.

With these two files in place, the commands are:

```
xcodegen generate
xcodebuild -project LineScope.xcodeproj -scheme LineScope build
cd Packages/LineScopeCore && swift test
```

Part I

LineScope for macOS

Chapter 2

Pixels

Every numeric part of LineScope works on one image representation, the `PixelBuffer`. Choosing it well settles a surprising number of later questions, so this chapter is short on code but long on decisions.

2.1 The representation

A buffer stores its pixels *interleaved*: the four components of a pixel sit next to each other, and pixels follow one another row by row, starting with the *top* row. The component of channel $c \in \{0, 1, 2, 3\}$ (red, green, blue, alpha) at column x and row y of a buffer of width W therefore lives at index

$$4(yW + x) + c.$$

Row 0 at the top matches the way images are stored in files and drawn on the screen, so no flipping is ever needed between the buffer, Core Graphics bitmaps and the views.

Components are 32-bit floats in $[0, 1]$. Floats let a filter produce intermediate values (negative lobes of a Lanczos kernel, overshoot of a sharpening filter) without wrapping around, and they make the arithmetic in resampling exact enough that the tests can compare against closed-form answers.

Two further conventions matter.

Encoded, not linear. The components are the sRGB-*encoded* values, the numbers stored in the file and sent to the display, not the linear-light intensities obtained by undoing the sRGB transfer curve. A physically careful resampler would linearize first; LineScope deliberately does not, because its purpose is to show the profile of the values a user sees and that most software filters operate on. Everything downstream (profiles, spectra, filters) sees the same encoded numbers.

Straight alpha. Colors are stored with *straight* (non-premultiplied) alpha, so a half-transparent red pixel is $(1, 0, 0, \frac{1}{2})$ rather than $(\frac{1}{2}, 0, 0, \frac{1}{2})$. Straight alpha is what a user expects to read in a profile. Filtering, however, must be done on premultiplied values $(\alpha r, \alpha g, \alpha b, \alpha)$; otherwise a fully transparent pixel, whose color is arbitrary, would bleed its color into its neighbors. The buffer therefore offers conversions in both directions (Section 2.3).

The file has two parts: the types themselves, and the bridge to Core Graphics.

3 *<Packages/LineScopeCore/Sources/LineScopeCore/PixelBuffer.swift 3> ≡*

```
import CoreGraphics
import Foundation
```

```
<PixelBuffer: the color sample RGBA 4>
```

```
<PixelBuffer: the buffer type 5>
```

```
// MARK: - Core Graphics bridging
```

```
extension PixelBuffer {
    <PixelBuffer: the sRGB color space 9>

    <PixelBuffer: decoding a CGImage 10>

    <PixelBuffer: encoding a CGImage 11>
}
```

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/PixelBuffer.swift.

A single color sample is a small value type. It is what a line sampler returns and what the color conversions of Chapter 5 consume. Alpha defaults to opaque, because most samples are.

4 *<PixelBuffer: the color sample RGBA 4> ≡*

```
/// A color sample with straight (non-premultiplied) alpha, sRGB-encoded components in 0...1.
```

```
public struct RGBA: Sendable, Equatable {
    public var r: Float
    public var g: Float
    public var b: Float
    public var a: Float

    public init(r: Float, g: Float, b: Float, a: Float = 1) {
        self.r = r
        self.g = g
        self.b = b
        self.a = a
    }
}
```

Used in chunk 3.

The buffer is a struct holding its dimensions and one flat array. Swift arrays are copy-on-write, so passing a buffer around (to a background task, into an `ImageVariant`) costs nothing until someone mutates it. The type is `Sendable`, which lets it cross from the main actor to a detached task and back, and `Equatable`, which the tests use to compare whole images.

5 *<PixelBuffer: the buffer type 5> ≡*

```
/// Interleaved RGBA Float32 image. Row 0 is the top row.
```

```
/// Components are sRGB-encoded (not linearized) with straight alpha, so values match what is displayed.
```

```
public struct PixelBuffer: Sendable, Equatable {
    <PixelBuffer: stored properties and initializers 6>

    <PixelBuffer: pixel subscript 7>

    <PixelBuffer: premultiplication 8>
}
```

Used in chunk 3.

2.2 Construction and access

The designated initializer takes the raw array and checks the one invariant everything else relies on: exactly four floats per pixel, and at least one pixel. A buffer with zero width would make the

resampler divide by zero, so it is rejected at the door. The second initializer fills the image with one color; the tests and the test-pattern generator start from it.

6 *⟨PixelBuffer: stored properties and initializers 6⟩ ≡*

```
public let width: Int
public let height: Int
public var data: [Float]

public init(width: Int, height: Int, data: [Float]) {
    precondition(width > 0 && height > 0 && data.count == width * height * 4)
    self.width = width
    self.height = height
    self.data = data
}

public init(width: Int, height: Int, fill: RGBA = RGBA(r: 0, g: 0, b: 0, a: 1)) {
    var data = [Float](repeating: 0, count: width * height * 4)
    for i in stride(from: 0, to: data.count, by: 4) {
        data[i] = fill.r
        data[i + 1] = fill.g
        data[i + 2] = fill.b
        data[i + 3] = fill.a
    }
    self.init(width: width, height: height, data: data)
}
```

Used in chunk 5.

A subscript reads and writes whole pixels using the index formula above. It is convenient rather than fast: the heavy loops in resampling and filtering index `data` directly through unsafe buffer pointers instead.

7 *⟨PixelBuffer: pixel subscript 7⟩ ≡*

```
public subscript(x: Int, y: Int) -> RGBA {
    get {
        let i = (y * width + x) * 4
        return RGBA(r: data[i], g: data[i + 1], b: data[i + 2], a: data[i + 3])
    }
    set {
        let i = (y * width + x) * 4
        data[i] = newValue.r
        data[i + 1] = newValue.g
        data[i + 2] = newValue.b
        data[i + 3] = newValue.a
    }
}
```

Used in chunk 5.

2.3 Premultiplication

To premultiply, scale the three color components of every pixel by its alpha. The result is a bare array, not a `PixelBuffer`, on purpose: premultiplied data is an internal intermediate of the resampler and of the Core Image bridge, and giving it its own type would invite someone to plot it.

Going back divides by alpha. Two safeguards make this robust against what filters produce. Alpha is first clamped to $[0, 1]$, since a Lanczos kernel can overshoot it. Where alpha is (nearly) zero the color is undefined, and dividing would amplify rounding noise into bright garbage; such pixels become transparent black. The color components are clamped too, which is where the overshoot and undershoot of sharpening kernels is finally cut off.

8 *⟨PixelBuffer: premultiplication 8⟩* ≡*// MARK: Premultiplication (used by resampling and Core Image, which expect premultiplied data)*

```

func premultiplied() -> [Float] {
    var out = data
    for i in stride(from: 0, to: out.count, by: 4) {
        let a = out[i + 3]
        out[i] *= a
        out[i + 1] *= a
        out[i + 2] *= a
    }
    return out
}

static func fromPremultiplied(width: Int, height: Int, _ p: [Float]) -> PixelBuffer {
    var out = p
    for i in stride(from: 0, to: out.count, by: 4) {
        let a = min(max(out[i + 3], 0), 1)
        out[i + 3] = a
        if a > 1e-6 {
            out[i] = min(max(out[i] / a, 0), 1)
            out[i + 1] = min(max(out[i + 1] / a, 0), 1)
            out[i + 2] = min(max(out[i + 2] / a, 0), 1)
        } else {
            out[i] = 0
            out[i + 1] = 0
            out[i + 2] = 0
        }
    }
    return PixelBuffer(width: width, height: height, data: out)
}

```

Used in chunk 5.

2.4 Bridging to Core Graphics

Images enter the program as `CGImage` values decoded by `ImageIO`, and they leave it as `CGImage` values drawn by `SwiftUI` or written to PNG. Both directions go through the `sRGB` color space, created once.

9 *⟨PixelBuffer: the sRGB color space 9⟩* ≡

```

static let sRGB = CGColorSpace(name: CGColorSpace.sRGB)!

```

Used in chunk 3.

A decoded image can have any layout: 16 bits per component, CMYK, a grayscale palette, alpha first or last. Rather than handling each case, the decoder lets Core Graphics do the conversion: it creates an 8-bit premultiplied RGBA bitmap in `sRGB` and draws the image into it. Interpolation is switched off; the image is drawn at its own size, so no resampling should happen at all, and this makes sure of it. The bitmap's memory is borrowed from a Swift array for the duration of the closure, and the bytes are then scaled to `[0, 1]` and un-premultiplied.

This does lose precision for 16-bit sources. The inspector reports the source's real bit depth, so the user knows when that happened.

10 *⟨PixelBuffer: decoding a CGImage 10⟩* ≡*/// Decodes any CGImage into 8-bit sRGB and converts it to a float buffer.*

```

public init?(cgImage: CGImage) {
    let w = cgImage.width
    let h = cgImage.height
    guard w > 0, h > 0 else { return nil }
    var bytes = [UInt8](repeating: 0, count: w * h * 4)

```

```

let drawn = bytes.withUnsafeMutableBytes { raw -> Bool in
    guard let ctx = CGContext(
        data: raw.baseAddress, width: w, height: h, bitsPerComponent: 8, bytesPerRow: w * 4,
        space: PixelBuffer.sRGB, bitmapInfo: CGImageAlphaInfo.premultipliedLast.rawValue
    ) else { return false }
    ctx.interpolationQuality = .none
    ctx.draw(cgImage, in: CGRect(x: 0, y: 0, width: w, height: h))
    return true
}
guard drawn else { return nil }
var p = [Float](repeating: 0, count: w * h * 4)
for i in 0..

```

Used in chunk 3.

The opposite direction clamps, premultiplies and rounds each component to a byte, and wraps the bytes in a data provider. The image is marked `shouldInterpolate: false` so that when SwiftUI draws it, nothing smooths the pixels behind the program's back. Seeing the blocky result of nearest-neighbor upsampling is part of the point.

11 $\langle$ PixelBuffer: encoding a CGImage 11 $\rangle \equiv$

```

/// Renders the buffer as an 8-bit premultiplied sRGB CGImage for display or export.
public func makeCGImage() -> CGImage? {
    var bytes = [UInt8](repeating: 0, count: data.count)
    for i in stride(from: 0, to: data.count, by: 4) {
        let a = min(max(data[i + 3], 0), 1)
        bytes[i] = UInt8((min(max(data[i], 0), 1) * a * 255).rounded())
        bytes[i + 1] = UInt8((min(max(data[i + 1], 0), 1) * a * 255).rounded())
        bytes[i + 2] = UInt8((min(max(data[i + 2], 0), 1) * a * 255).rounded())
        bytes[i + 3] = UInt8((a * 255).rounded())
    }
    guard let provider = CGDataProvider(data: Data(bytes) as CFData) else { return nil }
    return CGImage(
        width: width, height: height, bitsPerComponent: 8, bitsPerPixel: 32, bytesPerRow: width * 4,
        space: PixelBuffer.sRGB, bitmapInfo: CGBitmapInfo(rawValue:
            ↪ CGImageAlphaInfo.premultipliedLast.rawValue),
        provider: provider, decode: nil, shouldInterpolate: false, intent: .defaultIntent
    )
}

```

Used in chunk 3.

Chapter 3

Resampling

Resampling changes the size of an image. It is the operation LineScope was written to study, so its implementation puts clarity and exactness first. Each menu entry is *exactly* the textbook algorithm it names. Library routines (vImage, Core Image) choose their own kernels, prefilters and edge rules, and that would blur the comparison the user is trying to make.

3.1 Theory in brief

A digital image is a set of samples of a continuous picture. To resample it to a new grid we (conceptually) reconstruct the continuous picture by convolving the samples with a *kernel* k , then sample the result at the new positions. In one dimension, output pixel i at position u_i in input coordinates is

$$\text{out}_i = \frac{\sum_j k(j + \frac{1}{2} - u_i) \text{in}_j}{\sum_j k(j + \frac{1}{2} - u_i)}.$$

The denominator renormalizes the weights, so a constant image stays constant even where the kernel is cut off at the image border.

Pixel centers. Input pixel j covers the interval $[j, j+1)$ and has its center at $j + \frac{1}{2}$. With scale $s = n_{\text{out}}/n_{\text{in}}$, output pixel i covers $[i, i+1)$ in output coordinates, and its center maps back to

$$u_i = \frac{i + \frac{1}{2}}{s}$$

in input coordinates. This convention keeps the image centered: halving an image of width 4 puts the two output centers at input positions 1 and 3, exactly between the input centers 0.5, 1.5 and 2.5, 3.5. Using $u_i = i/s$ instead would shift the whole image by half a pixel, a classic bug that shows up as asymmetric profiles.

Downscaling needs a prefilter. When $s < 1$ the new grid cannot represent frequencies above its own Nyquist limit, $s/2$ cycles per input pixel. A reconstruction kernel of fixed width lets those frequencies through, and they fold back as aliases. The standard cure is to stretch the kernel by $1/s$, which turns it into a low-pass filter matched to the new grid:

$$k_s(x) = k(sx), \quad \text{with support widened by } 1/s.$$

The normalization above absorbs the factor s that a properly scaled filter would carry. Nearest neighbor is kept *without* this widening, so that the menu offers a method that aliases. Seeing aliasing is instructive.

Separability. All kernels here are applied separably, first along rows, then along columns: $k(x, y) = k(x)k(y)$. That turns an $O(r^2)$ cost per pixel into $O(r)$ and lets both passes share one routine for computing weights.

3.2 The methods

The file holds the list of methods, and the resampler that implements them.

12 *⟨Packages/LineScopeCore/Sources/LineScopeCore/Resampling.swift 12⟩* ≡
`import Foundation`

⟨Resampling: the ResampleMethod enumeration 13⟩

⟨Resampling: the Resampler namespace 14⟩

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/Resampling.swift.

The enumeration is what the Resample menu iterates over. Each method has a long name for the menu and a short one for tab titles, where a chain like “Tent ×2 → Mitchell ×0.5” must stay readable. CaseIterable lets the menu stay in sync with this list automatically.

13 *⟨Resampling: the ResampleMethod enumeration 13⟩* ≡

```
public enum ResampleMethod: String, CaseIterable, Identifiable, Sendable {
    case nearest
    case tent
    case catmullRom
    case mitchell
    case lanczos3
    case sinc
    case area

    public var id: String { rawValue }

    public var displayName: String {
        switch self {
        case .nearest: "Nearest Neighbor"
        case .tent: "Tent (Bilinear)"
        case .catmullRom: "Catmull-Rom (Bicubic)"
        case .mitchell: "Mitchell-Netravali"
        case .lanczos3: "Lanczos-3"
        case .sinc: "Sinc (truncated, r=\(Int(Resampler.sincRadius)))"
        case .area: "Area Average"
        }
    }

    public var shortName: String {
        switch self {
        case .nearest: "Nearest"
        case .tent: "Tent"
        case .catmullRom: "Catmull-Rom"
        case .mitchell: "Mitchell"
        case .lanczos3: "Lanczos"
        case .sinc: "Sinc"
        case .area: "Area"
        }
    }
}
```

Used in chunk 12.

The resampler is a caseless enum, used only as a namespace. Its documentation comment summarizes the conventions just derived. The body goes from the public entry points down to the kernels.

14 *⟨Resampling: the Resampler namespace 14⟩* ≡

```

/// Separable resampling with explicit, textbook kernels.
///
/// Coordinates follow the pixel-center convention: input pixel `j` covers `[j, j+1)` and its center is
///   ↪ `j + 0.5`.
/// Output pixel `i` maps to input position `(i + 0.5) / scale`.
/// - `nearest` picks one source pixel and does no prefiltering, so it aliases when downscaling.
/// - The kernel methods (tent, Catmull-Rom, Mitchell, Lanczos, sinc) widen their kernel by `1/scale`
///   when downscaling (antialiased).
/// - `area` weights source pixels by how much of the output pixel's footprint they cover.
public enum Resampler {
  ⟨Resampler: sinc radius and output size 15⟩

  ⟨Resampler: resampling by a scale factor 16⟩

  ⟨Resampler: resampling to a given size 17⟩

  // MARK: - Kernels

  ⟨Resampler: contributions of input pixels 20⟩

  ⟨Resampler: trimming and normalizing weights 24⟩

  ⟨Resampler: kernel supports 25⟩

  ⟨Resampler: the sinc function 26⟩

  ⟨Resampler: the BC-spline family 27⟩

  ⟨Resampler: kernel evaluation 28⟩
}

Used in chunk 12.

```

3.3 Entry points

A scale factor becomes an output size by rounding, never below one pixel. The parameter sheet uses the same function to preview the resulting size, so what the user is promised is what they get. The radius of the truncated sinc kernel is a public constant for the same reason: the menu title shows it.

15 *⟨Resampler: sinc radius and output size 15⟩* ≡

```

/// Half-width of the truncated sinc kernel, in source pixels (before widening).
public static let sincRadius = 8.0

public static func outputSize(for size: Int, scale: Double) -> Int {
  max(1, Int((Double(size) * scale).rounded()))
}

Used in chunk 14.

```

16 *⟨Resampler: resampling by a scale factor 16⟩* ≡

```

public static func resample(_ src: PixelBuffer, scale: Double, method: ResampleMethod) -> PixelBuffer {
  resample(
    src,
    width: outputSize(for: src.width, scale: scale),
    height: outputSize(for: src.height, scale: scale),
    method: method
  )
}

```

Used in chunk 14.

Resampling to a given size first premultiplies the source (Section 2.3), so that transparent pixels don't leak their color. It then computes one table of weights per axis, runs the two passes, and converts the result back to straight alpha. Note that the scale in each direction is derived from the rounded sizes, not from the requested factor, so the image maps exactly onto the new grid.

17 *⟨Resampler: resampling to a given size 17⟩* ≡

```
public static func resample(_ src: PixelBuffer, width: Int, height: Int, method: ResampleMethod) ->
    ↳ PixelBuffer {
    let premul = src.premultiplied()
    let xContrib = contributions(inSize: src.width, outSize: width, method: method)
    let yContrib = contributions(inSize: src.height, outSize: height, method: method)

    ⟨Resampler: horizontal pass 18⟩

    ⟨Resampler: vertical pass 19⟩

    return PixelBuffer.fromPremultiplied(width: width, height: height, out)
}
```

Used in chunk 14.

3.3.1 The two passes

The horizontal pass changes only the width. For every row and every output column it takes the weighted sum of a short run of consecutive input pixels, the run described by that column's Contribution. The loops use unsafe buffer pointers: this is the one place where bounds checks on every access would noticeably slow the program down, and the contribution tables guarantee the indices are in range.

18 *⟨Resampler: horizontal pass 18⟩* ≡

```
// Horizontal pass: (src.width x src.height) -> (width x src.height)
var tmp = [Float](repeating: 0, count: width * src.height * 4)
premul.withUnsafeBufferPointer { s in
    tmp.withUnsafeMutableBufferPointer { t in
        for y in 0..

```

Used in chunk 17.

The vertical pass works on the intermediate image. Instead of gathering a column pixel by pixel, which would jump through memory, it accumulates whole input rows into each output row: every weight multiplies a contiguous run of $4W$ floats. The inner loop then runs over consecutive memory, which the compiler vectorizes.

```
19  ⟨Resampler: vertical pass 19⟩ ≡
// Vertical pass: (width × src.height) → (width × height)
var out = [Float](repeating: 0, count: width * height * 4)
tmp.withUnsafeBufferPointer { t in
    out.withUnsafeMutableBufferPointer { o in
        let rowStride = width * 4
        for y in 0..

```

Used in chunk 17.

3.4 Weights

A Contribution says which input pixels feed one output pixel: a first index and a list of weights for consecutive pixels. The weights are computed once per output column (or row) and reused for every row (or column), which is what makes separable resampling cheap.

```
20  ⟨Resampler: contributions of input pixels 20⟩ ≡
struct Contribution {
    var start: Int
    var weights: [Float]
}

static func contributions(inSize: Int, outSize: Int, method: ResampleMethod) -> [Contribution] {
    let scale = Double(outSize) / Double(inSize)
    var result: [Contribution] = []
    result.reserveCapacity(outSize)

    for i in 0..

```

Used in chunk 14.

Nearest neighbor takes the single input pixel whose interval contains u_i , that is $j = \lfloor u_i \rfloor$, clamped to the image. It uses no prefilter and no blending.

21 *⟨Resampler: nearest-neighbor contribution 21⟩* $\equiv$

```

case .nearest:
  let j = min(max(Int(center.rounded(.down)), 0), inSize - 1)
  result.append(Contribution(start: j, weights: [1]))

```

Used in chunk 20.

Area averaging is the box filter done exactly. The output pixel's footprint, mapped back to input coordinates, is the interval $[u_i - \frac{1}{2s}, u_i + \frac{1}{2s}]$. Each input pixel is weighted by the length of its overlap with the footprint,

$$w_j = \max(0, \min(hi, j + 1) - \max(lo, j)),$$

so for a factor of $\frac{1}{2}$ the result is the plain mean of pixel pairs, and for non-integer factors the edge pixels count in proportion to how much of them is covered. When upscaling, the footprint is smaller than a pixel and the method degrades gracefully into nearest neighbor with blending at the seams.

22 *⟨Resampler: area-average contribution 22⟩* $\equiv$

```

case .area:
  // Output pixel footprint in input coordinates, exact coverage weights.
  let half = 0.5 / scale
  let lo = center - half
  let hi = center + half
  let first = max(0, Int(lo.rounded(.down)))
  let last = min(inSize - 1, Int(hi.rounded(.up)) - 1)
  var weights: [Float] = []
  if last >= first {
    for j in first...last {
      let overlap = min(hi, Double(j + 1)) - max(lo, Double(j))
      weights.append(Float(max(0, overlap)))
    }
  }
  result.append(normalized(start: first, weights: weights, fallback: center, inSize: inSize))

```

Used in chunk 20.

All the true kernels share one path. The filter scale is $\max(1, 1/s)$: it is 1 when upscaling, where the kernel only interpolates, and $1/s$ when downscaling, where it also prefilters. The support widens by the same factor, and each input pixel within it gets the weight $k((j + \frac{1}{2} - u_i) / \max(1, 1/s))$.

23 *⟨Resampler: kernel contribution 23⟩* $\equiv$

```

case .tent, .catmullRom, .mitchell, .lanczos3, .sinc:
  let filterScale = max(1, 1 / scale)
  let support = kernelSupport(method) * filterScale
  let first = max(0, Int((center - support).rounded(.down)))
  let last = min(inSize - 1, Int((center + support).rounded(.up)))
  var weights: [Float] = []
  if last >= first {
    for j in first...last {
      let x = (Double(j) + 0.5 - center) / filterScale
      weights.append(Float(kernel(method, x)))
    }
  }
  result.append(normalized(start: first, weights: weights, fallback: center, inSize: inSize))

```

Used in chunk 20.

Before a weight list is stored, zero weights at both ends are removed; the support bounds are conservative, and the trimmed list keeps the inner loops short. The weights are then divided by their sum. This is the renormalization in the formula at the start of the chapter: it keeps flat regions flat and handles the image border, where part of the kernel falls outside the image.

If nothing is left, or the sum vanishes (possible in principle for kernels with negative lobes), the function falls back to nearest neighbor rather than dividing by zero.

24 *⟨Resampler: trimming and normalizing weights 24⟩* ≡

```
private static func normalized(start: Int, weights: [Float], fallback center: Double, inSize: Int) ->
  ↪ Contribution {
  // Trim zero weights at both ends to keep inner loops short.
  var first = 0
  var last = weights.count - 1
  while first <= last && weights[first] == 0 { first += 1 }
  while last >= first && weights[last] == 0 { last -= 1 }
  guard first <= last else {
    let j = min(max(Int(center.rounded(.down)), 0), inSize - 1)
    return Contribution(start: j, weights: [1])
  }
  let trimmed = Array(weights[first...last])
  let sum = trimmed.reduce(0, +)
  guard abs(sum) > 1e-12 else {
    let j = min(max(Int(center.rounded(.down)), 0), inSize - 1)
    return Contribution(start: j, weights: [1])
  }
  return Contribution(start: start + first, weights: trimmed.map { $0 / sum })
}
```

Used in chunk 14.

3.5 Kernels

Each kernel is an even function $k(x)$ that is zero outside a radius, its *support*.

25 *⟨Resampler: kernel supports 25⟩* ≡

```
static func kernelSupport(_ method: ResampleMethod) -> Double {
  switch method {
  case .nearest, .area: 0.5
  case .tent: 1
  case .catmullRom, .mitchell: 2
  case .lanczos3: 3
  case .sinc: sincRadius
  }
}
```

Used in chunk 14.

Sinc. The ideal reconstruction filter for band-limited signals is

$$\text{sinc}(x) = \frac{\sin \pi x}{\pi x}, \quad \text{sinc}(0) = 1,$$

whose Fourier transform is a perfect box: it passes every frequency below Nyquist unchanged and removes everything above. Its tails decay only like $1/x$, so it has to be truncated. LineScope truncates it at $|x| < 8$ with a rectangular window, which is the textbook “sinc filter” with its textbook flaw: the sharp cutoff produces visible ringing next to edges (the Gibbs phenomenon). The step-edge test pattern shows this clearly.

26 *⟨Resampler: the sinc function 26⟩* ≡

```
private static func sinc(_ x: Double) -> Double {
  if abs(x) < 1e-9 { return 1 }
  let px = Double.pi * x
  return sin(px) / px
}
```

Used in chunk 14.

The BC-splines. Mitchell and Netravali [3] described a two-parameter family of cubic kernels with support 2 that contains most cubic filters in use:

$$k_{B,C}(x) = \frac{1}{6} \begin{cases} (12 - 9B - 6C)|x|^3 + (-18 + 12B + 6C)|x|^2 + (6 - 2B), & |x| < 1, \\ (-B - 6C)|x|^3 + (6B + 30C)|x|^2 + (-12B - 48C)|x| + (8B + 24C), & 1 \leq |x| < 2, \\ 0, & \text{otherwise.} \end{cases}$$

The choice $B = 0$, $C = \frac{1}{2}$ is the Catmull-Rom spline, which is Keys' cubic convolution kernel with $a = -\frac{1}{2}$ [4]. It interpolates ($k(0) = 1$, $k(\pm 1) = k(\pm 2) = 0$) and reproduces quadratics exactly, but it rings a little. Mitchell and Netravali chose $B = C = \frac{1}{3}$ after a user study, trading a little blur for much less ringing and less anisotropy. That kernel does *not* interpolate: $k(0) = \frac{8}{9}$ and $k(\pm 1) = \frac{1}{18}$, so even resampling at scale 1 smooths the image slightly. The tests check these values.

27 *⟨Resampler: the BC-spline family 27⟩* ≡

/// Mitchell-Netravali / Keys family: (B, C) = (1/3, 1/3) is Mitchell, (0, 1/2) is Catmull-Rom.

```
private static func bcSpline(_ ax: Double, b: Double, c: Double) -> Double {
    if ax < 1 {
        return ((12 - 9 * b - 6 * c) * ax * ax * ax + (-18 + 12 * b + 6 * c) * ax * ax + (6 - 2 * b)) / 6
    }
    if ax < 2 {
        return ((-b - 6 * c) * ax * ax * ax + (6 * b + 30 * c) * ax * ax
            + (-12 * b - 48 * c) * ax + (8 * b + 24 * c)) / 6
    }
    return 0
}
```

Used in chunk 14.

All kernels together. The box, $k(x) = 1$ for $|x| < \frac{1}{2}$, stands in for nearest neighbor and area averaging, although both have their own code paths above and never actually evaluate it. The *tent*, $k(x) = \max(0, 1 - |x|)$, is linear interpolation; applied separably it is bilinear interpolation. *Lanczos-3* tapers the sinc with a wider sinc as the window,

$$L_3(x) = \text{sinc}(x) \text{sinc}(x/3), \quad |x| < 3,$$

which keeps most of the sinc's sharpness and much less of its ringing.

28 *⟨Resampler: kernel evaluation 28⟩* ≡

```
static func kernel(_ method: ResampleMethod, _ x: Double) -> Double {
    let ax = abs(x)
    switch method {
    case .nearest, .area:
        return ax < 0.5 ? 1 : 0
    case .tent:
        return max(0, 1 - ax)
    case .catmullRom:
        return bcSpline(ax, b: 0, c: 0.5)
    case .mitchell:
        return bcSpline(ax, b: 1.0 / 3, c: 1.0 / 3)
    case .lanczos3:
        return ax < 3 ? sinc(ax) * sinc(ax / 3) : 0
    case .sinc:
        // Ideal low-pass, truncated (rectangular window): expect ringing near edges.
        return ax < sincRadius ? sinc(ax) : 0
    }
}
```

Used in chunk 14.

Chapter 4

Filters and Operations

Filters keep the size of an image and change its content. LineScope offers the handful that people most often want to compare: blurs, a median, sharpening, noise reduction, edge detectors, and two per-pixel color maps. Where Core Image already has a good, well-known implementation it is used; where a filter is simple enough to state in a few lines (the 3×3 kernels), it is written out, so the reader can see precisely what was computed.

The second half of the chapter introduces `ImageOperation`, the value that describes *one step* from one image to the next, whether that step is a filter or a resampling.

4.1 The filters

29 `<Packages/LineScopeCore/Sources/LineScopeCore/Filters.swift 29> ≡`

```
import CoreImage
import CoreImage.CIFilterBuiltins
import Foundation
```

`<Filters: the FilterKind enumeration 30>`

`<Filters: number formatting for names 31>`

```
public enum Filters {
    <Filters: dispatching a filter 32>
```

`// MARK: - Core Image`

`<Filters: the Core Image context 35>`

`<Filters: running a Core Image filter on a buffer 36>`

`// MARK: - CPU filters`

`<Filters: per-pixel color maps 37>`

`<Filters: 3x3 convolution 38>`

`<Filters: Sobel gradient magnitude 39>`

```
}
```

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/Filters.swift.

A filter is described by an enumeration whose cases carry their parameters. Because the value is `Equatable` and `Sendable`, it can be stored in an image’s history and shipped to a background task. Its `displayName` becomes part of a tab title, so it names the parameters compactly: “Gaussian r=2” tells the user what was applied.

30 *⟨Filters: the FilterKind enumeration 30⟩* ≡

```
public enum FilterKind: Sendable, Equatable {
    case gaussianBlur(radius: Double)
    case boxBlur(radius: Double)
    case median
    case unsharpMask(radius: Double, intensity: Double)
    case noiseReduction(level: Double, sharpness: Double)
    case sobel
    case laplacian
    case emboss
    case grayscale
    case invert

    public var displayName: String {
        switch self {
            case .gaussianBlur(let r): "Gaussian r=\(fmt(r))"
            case .boxBlur(let r): "Box r=\(fmt(r))"
            case .median: "Median 3×3"
            case .unsharpMask(let r, let i): "Sharpen r=\(fmt(r)) i=\(fmt(i))"
            case .noiseReduction(let l, let s): "Denoise \(fmt(l))/\(fmt(s))"
            case .sobel: "Sobel"
            case .laplacian: "Laplacian"
            case .emboss: "Emboss"
            case .grayscale: "Grayscale"
            case .invert: "Invert"
        }
    }
}
```

Used in chunk 29.

Parameters are shown with at most three decimals and no trailing zeros, so 2 reads “2” and 0.125 reads “0.125”. The helper is internal to the package; the resampling and test-pattern names use it too.

31 *⟨Filters: number formatting for names 31⟩* ≡

```
func fmt(_ v: Double) -> String {
    v.formatted(.number.precision(.fractionLength(0...3)))
}
```

Used in chunk 29.

4.1.1 Dispatch

`Filters.apply` maps each case to its implementation. The first group goes to Core Image, each case building one built-in filter with its parameters.

32 *⟨Filters: dispatching a filter 32⟩* ≡

```
public static func apply(_ filter: FilterKind, to src: PixelBuffer) -> PixelBuffer {
    switch filter {
        ⟨Filters: Core Image cases 33⟩
        ⟨Filters: CPU cases 34⟩
    }
}
```

Used in chunk 29.

The Gaussian blur’s radius is Core Image’s, roughly the standard deviation σ in pixels. The box blur averages over a square of that radius. The median is Core Image’s fixed 3×3 median, good at removing isolated outliers without softening edges. The unsharp mask adds back *intensity* $\times$ (image – blurred image), which boosts high frequencies and is visible in the spectrum as a lift just below Nyquist. Noise reduction is Core Image’s adaptive smoothing, which leaves edges above a sharpness threshold alone.

33 *⟨Filters: Core Image cases 33⟩* ≡

```

case .gaussianBlur(let radius):
    return applyCI(src) { img in
        let f = CIFilter.gaussianBlur()
        f.inputImage = img
        f.radius = Float(radius)
        return f.outputImage
    }
case .boxBlur(let radius):
    return applyCI(src) { img in
        let f = CIFilter.boxBlur()
        f.inputImage = img
        f.radius = Float(radius)
        return f.outputImage
    }
case .median:
    return applyCI(src) { img in
        let f = CIFilter.median()
        f.inputImage = img
        return f.outputImage
    }
case .unsharpMask(let radius, let intensity):
    return applyCI(src) { img in
        let f = CIFilter.unsharpMask()
        f.inputImage = img
        f.radius = Float(radius)
        f.intensity = Float(intensity)
        return f.outputImage
    }
case .noiseReduction(let level, let sharpness):
    return applyCI(src) { img in
        let f = CIFilter.noiseReduction()
        f.inputImage = img
        f.noiseLevel = Float(level)
        f.sharpness = Float(sharpness)
        return f.outputImage
    }

```

Used in chunk 32.

The second group runs on the CPU. The Laplacian

$$\nabla^2 \approx \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

responds with both signs around an edge and zero on flat regions, so its absolute value is shown; otherwise half of every edge would be clipped to black. The emboss kernel's weights sum to 1, so flat regions keep their color and edges become a relief lit from the top left. Grayscale uses the Rec. 709 luma weights on the encoded values, $Y = 0.2126R + 0.7152G + 0.0722B$, and inversion replaces every component v with $1 - v$; both leave alpha alone.

34 *⟨Filters: CPU cases 34⟩* ≡

```

case .sobel:
    return sobel(src)
case .laplacian:
    // Absolute response so the edges are visible.
    return convolve3x3(src, kernel: [0, 1, 0, 1, -4, 1, 0, 1, 0], absolute: true)
case .emboss:
    return convolve3x3(src, kernel: [-2, -1, 0, -1, 1, 1, 0, 1, 2], absolute: false)
case .grayscale:
    return mapRGB(src) { c in
        let y = 0.2126 * c.r + 0.7152 * c.g + 0.0722 * c.b
    }

```

```

        return RGBA(r: y, g: y, b: y, a: c.a)
    }
    case .invert:
        return mapRGB(src) { c in RGBA(r: 1 - c.r, g: 1 - c.g, b: 1 - c.b, a: c.a) }
}
Used in chunk 32.

```

4.1.2 The Core Image bridge

By default Core Image color-manages everything: it converts inputs to a linear working space, filters there, and converts back. That is usually right, but here it would make the Core Image filters disagree with the CPU filters and with the resampler, which all work on encoded values (Chapter 2). The context is therefore created with color management switched off for both the working and the output space. Intermediate caching is off too, since every image is filtered once and then thrown away. A `CITexture` is expensive to create and safe to share between threads, so there is one for the whole program.

35 *⟨Filters: the Core Image context 35⟩* ≡

```

/// Works directly on encoded values (no color management) so results are consistent with the CPU
    ↪ filters.
private static let context = CITexture(options: [
    .workingColorSpace: NSNull(),
    .outputColorSpace: NSNull(),
    .cacheIntermediates: false,
])
Used in chunk 29.

```

Running a filter means wrapping the premultiplied floats as a `CITexture` in the `RGBAf` format (four 32-bit floats per pixel, hence 16 bytes per pixel in a row) with no color space, which again means “don’t convert”. Two details keep the image the same size and free of dark borders. First, the input is *clamped to its extent*, which extends the border pixels to infinity, so a blur near the edge averages real edge colors instead of transparent black. Second, the output is cropped back to the original rectangle, since a blur’s output would otherwise grow by its radius. The rendered floats are then converted back to straight alpha.

If a filter can’t be built, the source is returned unchanged rather than failing the operation.

36 *⟨Filters: running a Core Image filter on a buffer 36⟩* ≡

```

private static func applyCI(_ src: PixelBuffer, _ build: (CITexture) -> CITexture?) -> PixelBuffer {
    let w = src.width
    let h = src.height
    let premul = src.premultiplied()
    let data = premul.withUnsafeBytes { Data($0) }
    let input = CITexture(
        bitmapData: data, bytesPerRow: w * 16, size: CGSize(width: w, height: h),
        format: .RGBAf, colorSpace: nil
    )
    let extent = input.extent
    guard let output = build(input.clampedToExtent())?.cropped(to: extent) else { return src }
    var result = [Float](repeating: 0, count: w * h * 4)
    result.withUnsafeMutableBytes { raw in
        context.render(
            output, toBitmap: raw.baseAddress!, rowBytes: w * 16, bounds: extent,
            format: .RGBAf, colorSpace: nil
        )
    }
    return PixelBuffer.fromPremultiplied(width: w, height: h, result)
}
Used in chunk 29.

```

4.1.3 CPU filters

A color map applies a function to every pixel independently.

37 *⟨Filters: per-pixel color maps 37⟩* $\equiv$

```
private static func mapRGB(_ src: PixelBuffer, _ f: (RGBA) -> RGBA) -> PixelBuffer {
    var out = src
    for y in 0..

```

Used in chunk 29.

The 3×3 convolution computes, for each color channel,

$$\text{out}(x, y) = \sum_{k_y=-1}^1 \sum_{k_x=-1}^1 K(k_x, k_y) \text{in}(\text{clamp}(x + k_x), \text{clamp}(y + k_y)),$$

where clamping the coordinates repeats the border pixels, the same edge rule as the Core Image path. The kernel is given row by row. Results are clamped to $[0, 1]$, optionally after taking the absolute value, and alpha is copied through unchanged. Since these kernels are applied to straight colors, not premultiplied ones, they are meant for opaque images, which is what they are used on.

38 *⟨Filters: 3x3 convolution 38⟩* $\equiv$

```
/// 3x3 convolution on RGB with clamped edges; alpha is preserved.
static func convolve3x3(_ src: PixelBuffer, kernel k: [Float], absolute: Bool) -> PixelBuffer {
    var out = src
    let w = src.width
    let h = src.height
    src.data.withUnsafeBufferPointer { s in
        out.data.withUnsafeMutableBufferPointer { o in
            for y in 0..

```

Used in chunk 29.

The Sobel operator estimates the two components of the gradient with the kernels

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad G_y = G_x^T,$$

each a derivative in one direction combined with a small smoothing in the other. The output is the gradient magnitude $\sqrt{s_x^2 + s_y^2}$, per channel and clamped to 1. Both sums are accumulated in the same pass over the neighborhood, so this can't be written as two calls to the general convolution without reading every pixel twice.

39 *⟨Filters: Sobel gradient magnitude 39⟩* ≡

```
/// Per-channel Sobel gradient magnitude.
static func sobel(_ src: PixelBuffer) -> PixelBuffer {
    let gx: [Float] = [-1, 0, 1, -2, 0, 2, -1, 0, 1]
    let gy: [Float] = [-1, -2, -1, 0, 0, 0, 1, 2, 1]
    var out = src
    let w = src.width
    let h = src.height
    src.data.withUnsafeBufferPointer { s in
        out.data.withUnsafeMutableBufferPointer { o in
            for y in 0..

```

Used in chunk 29.

4.2 Operations

The app never calls the filters or the resampler directly. It works with `ImageOperation` values, which unify the two kinds of step behind one name and one `apply` function. That gives the session model a single thing to run in the background, and a single thing to record. A derived image stores the list of operations that produced it from the original, its *lineage*, and the inspector prints that list.

40 $\langle \text{Packages/LineScopeCore/Sources/LineScopeCore/ImageOperation.swift } 40 \rangle \equiv$

```
import Foundation
```

```
/// One step that derives a new image from an existing one.
```

```
public enum ImageOperation: Sendable, Equatable {
    case filter(FilterKind)
    case resample(ResampleMethod, scale: Double)
```

```
     $\langle \text{ImageOperation: display name } 41 \rangle$ 
```

```
     $\langle \text{ImageOperation: applying the step } 42 \rangle$ 
```

```
}
```

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/ImageOperation.swift.

41 $\langle \text{ImageOperation: display name } 41 \rangle \equiv$

```
public var displayName: String {
    switch self {
    case .filter(let f): f.displayName
    case .resample(let m, let s): "\((m.shortName) ×\((fmt(s))"
```

Used in chunk 40.

Applying an operation is a pure function from buffer to buffer. It touches no shared state, which is what makes it safe to run on a detached task.

42 $\langle \text{ImageOperation: applying the step } 42 \rangle \equiv$

```
public func apply(to buffer: PixelBuffer) -> PixelBuffer {
    switch self {
    case .filter(let f): Filters.apply(f, to: buffer)
    case .resample(let m, let s): Resampler.resample(buffer, scale: s, method: m)
    }
```

Used in chunk 40.

Chapter 5

Lines, Colors and Spectra

The analysis part of LineScope is a pipeline of three steps. The selection line is sampled in the pixel grid of one image; each sample is converted to the chosen color model and reduced to one channel; the resulting profile goes through a real FFT. This chapter follows that order: first the line and its sampler, then the color models, and last the spectrum.

5.1 The selection line

The line is shared by all the copies of an image, and the copies differ in size. So the line can't be stored in pixels. It is stored in *normalized* coordinates instead, where (0,0) is the top-left corner of the image and (1,1) the bottom-right corner, whatever the size. A resampled copy only scales the image, so the same normalized line falls on the same content in every copy. Converting to the pixels of an image of size $W \times H$ is one multiplication per axis: $x = uW$ and $y = vH$.

43 `<Packages/LineScopeCore/Sources/LineScopeCore/LineSampler.swift 43> ≡`

`import Foundation`

`<LineSampler: normalized point 44>`

`<LineSampler: line segment 45>`

`<LineSampler: sample record 46>`

`<LineSampler: sampler 47>`

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/LineSampler.swift.

A normalized point is a plain pair of doubles. The views need to keep the point inside the image while it is dragged, so it can clamp itself to the unit square.

44 `<LineSampler: normalized point 44> ≡`

/// A point relative to the image extent: (0,0) is the top-left corner, (1,1) the bottom-right corner.

```
public struct NormalizedPoint: Sendable, Hashable {
    public var x: Double
    public var y: Double

    public init(x: Double, y: Double) {
        self.x = x
        self.y = y
    }

    public func clamped() -> NormalizedPoint {
        NormalizedPoint(x: min(max(x, 0), 1), y: min(max(y, 0), 1))
    }
}
```

Used in chunk 43.

The segment has a direction: position 0 of the profile is at `start` and position 1 at `end`. The initial line crosses the full width at mid-height. Its length depends on the image it is measured in. With $\Delta u = u_1 - u_0$ and $\Delta v = v_1 - v_0$,

$$L = \sqrt{(\Delta u W)^2 + (\Delta v H)^2}.$$

A copy at half size therefore has a line half as long, in its own pixels.

45 $\langle \text{LineSampler: line segment 45} \rangle \equiv$

```
/// The selection line, in normalized coordinates so it lands on the same content in every resampled
    ↪ copy.
public struct LineSegment: Sendable, Hashable {
    public var start: NormalizedPoint
    public var end: NormalizedPoint

    public init(start: NormalizedPoint, end: NormalizedPoint) {
        self.start = start
        self.end = end
    }

    /// Full-width horizontal line at mid-height.
    public static let initial = LineSegment(start: .init(x: 0, y: 0.5), end: .init(x: 1, y: 0.5))

    public func pixelLength(width: Int, height: Int) -> Double {
        let dx = (end.x - start.x) * Double(width)
        let dy = (end.y - start.y) * Double(height)
        return (dx * dx + dy * dy).squareRoot()
    }
}
```

Used in chunk 43.

5.2 Sampling along the line

The result of sampling keeps, for every sample, its position $t \in [0, 1]$ along the line and its color. It also records the line length and the spacing between samples, both in pixels. The spectrum needs the spacing to express frequencies in cycles per pixel.

46 $\langle \text{LineSampler: sample record 46} \rangle \equiv$

```
public struct LineSamples: Sendable {
    /// Position along the line, 0 at `start` and 1 at `end`.
    public var positions: [Double]
    public var colors: [RGBA]
    /// Line length in the sampled image's pixels.
    public var pixelLength: Double
    /// Distance between consecutive samples, in pixels ≈( 1).
    public var spacing: Double
}
```

Used in chunk 43.

The sampler has two parts: the walk along the segment, and the interpolation at one point.

47 $\langle \text{LineSampler: sampler 47} \rangle \equiv$

```
public enum LineSampler {
     $\langle \text{LineSampler: sample along the line 48} \rangle$ 

     $\langle \text{LineSampler: bilinear interpolation 49} \rangle$ 
}
```

Used in chunk 43.

How many samples should a line of length L get? The choice matters more than it seems. If every copy got the same number of samples, a half-size copy would be sampled twice per pixel. Its profile would look as smooth as the original's, and the loss of detail from shrinking would be hidden. So LineScope samples each copy in its *own* grid, about once per pixel:

$$n = \max(2, \lceil L \rceil + 1), \quad t_i = \frac{i}{n-1}, \quad i = 0, \dots, n-1,$$

so the spacing $L/(n-1)$ is at most one pixel, and very close to one for long lines. For the initial line of a W -pixel-wide image, $n = W + 1$ and the spacing is exactly one pixel. At least two samples keep a degenerate line from dividing by zero.

48 *⟨LineSampler: sample along the line 48⟩* ≡

```
/// Samples the line in the buffer's own pixel grid: `ceil(length) + 1` bilinear samples,
/// so a half-size copy yields about half as many samples.
public static func sample(_ buffer: PixelBuffer, along line: LineSegment) -> LineSamples {
    let w = Double(buffer.width)
    let h = Double(buffer.height)
    let length = line.pixelLength(width: buffer.width, height: buffer.height)
    let count = max(2, Int(length.rounded(.up)) + 1)
    let x0 = line.start.x * w, y0 = line.start.y * h
    let x1 = line.end.x * w, y1 = line.end.y * h

    var positions = [Double](repeating: 0, count: count)
    var colors = [RGBA](repeating: RGBA(r: 0, g: 0, b: 0, a: 0), count: count)
    for i in 0..

```

Used in chunk 47.

The sample points fall between pixel centers, so the color there must be interpolated. The convention used throughout LineScope is that pixel j covers $[j, j+1)$ and has its center at $j + \frac{1}{2}$. So a continuous position x lies between the centers of pixels $i = \lfloor x - \frac{1}{2} \rfloor$ and $i+1$, at fraction $t_x = (x - \frac{1}{2}) - i$, and the same holds for y . Bilinear interpolation mixes the four neighbors a, b, e, f (top-left, top-right, bottom-left, bottom-right) in two steps:

$$top = a + (b - a) t_x, \quad bottom = e + (f - e) t_x, \quad c = top + (bottom - top) t_y.$$

Indices outside the image are clamped to the border. Near an edge this repeats the border pixel, which is also what the resampler does. Bilinear interpolation is deliberately simple: it is a measuring instrument, and it must not add ringing of its own to the profiles it measures.

49 *⟨LineSampler: bilinear interpolation 49⟩* ≡

```
/// Bilinear interpolation at a continuous position (pixel centers at +0.5), edges clamped.
public static func bilinear(_ buffer: PixelBuffer, x: Double, y: Double) -> RGBA {
    let fx = x - 0.5
    let fy = y - 0.5
    let ix = Int(fx.rounded(.down))
    let iy = Int(fy.rounded(.down))
    let tx = Float(fx - Double(ix))
    let ty = Float(fy - Double(iy))
    let x0 = min(max(ix, 0), buffer.width - 1)
```

```

let x1 = min(max(ix + 1, 0), buffer.width - 1)
let y0 = min(max(iy, 0), buffer.height - 1)
let y1 = min(max(iy + 1, 0), buffer.height - 1)

let d = buffer.data
func mix(_ c: Int) -> Float {
    let a = d[(y0 * buffer.width + x0) * 4 + c]
    let b = d[(y0 * buffer.width + x1) * 4 + c]
    let e = d[(y1 * buffer.width + x0) * 4 + c]
    let f = d[(y1 * buffer.width + x1) * 4 + c]
    let top = a + (b - a) * tx
    let bottom = e + (f - e) * tx
    return top + (bottom - top) * ty
}
return RGBA(r: mix(0), g: mix(1), b: mix(2), a: mix(3))
}

```

Used in chunk 47.

5.3 Color models

A profile is one number per sample. The user picks a color model and one of its channels, and every sample is converted to it. The conversions work on the sRGB-encoded values as stored, the same values the filters see (Chapter 2).

50 `<Packages/LineScopeCore/Sources/LineScopeCore/ColorSpaces.swift 50> ≡`

```
import Foundation
```

```
<ColorSpaces: color model 51>
```

```
<ColorSpaces: conversions 52>
```

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/ColorSpaces.swift.

A color model knows the names of its channels and the range in which to plot each of them. Hue is an angle in degrees; every other channel lies in $[0, 1]$. Fixing the plot range, instead of fitting it to the data, is what makes two profiles comparable at a glance.

51 `<ColorSpaces: color model 51> ≡`

```

public enum ColorModel: String, CaseIterable, Identifiable, Sendable {
    case rgba
    case hsl
    case cmyk

    public var id: String { rawValue }

    public var displayName: String {
        switch self {
        case .rgba: "RGBA"
        case .hsl: "HSL"
        case .cmyk: "CMYK"
        }
    }

    public var channels: [String] {
        switch self {
        case .rgba: ["Red", "Green", "Blue", "Alpha"]
        case .hsl: ["Hue", "Saturation", "Lightness"]
        case .cmyk: ["Cyan", "Magenta", "Yellow", "Black"]
        }
    }
}

```

/// Plot range for a channel; hue is in degrees, everything else in 0...1.

```

public func range(ofChannel channel: Int) -> ClosedRange<Double> {
    self == .hsl && channel == 0 ? 0...360 : 0...1
}

```

Used in chunk 50.

The conversions come in pairs, color model to RGB and back. Only the forward direction is used by the app; the inverses exist so that the tests can check round trips (Chapter 7).

52 $\langle \text{ColorSpaces: conversions 52} \rangle \equiv$

```

public enum ColorConversion {
    // MARK: HSL (hue in degrees)

     $\langle \text{ColorSpaces: RGB to HSL 53} \rangle$ 

     $\langle \text{ColorSpaces: HSL to RGB 54} \rangle$ 

    // MARK: CMYK (naive device-independent conversion,  $K = 1 - \max(R, G, B)$ )

     $\langle \text{ColorSpaces: RGB to CMYK and back 55} \rangle$ 

    // MARK: Channel extraction

     $\langle \text{ColorSpaces: channel extraction 56} \rangle$ 
}

```

Used in chunk 50.

5.3.1 HSL

With $M = \max(R, G, B)$, $m = \min(R, G, B)$ and chroma $d = M - m$, the standard definitions are

$$L = \frac{M + m}{2}, \quad S = \frac{d}{1 - |2L - 1|}, \quad H = 60^\circ \times \begin{cases} \frac{G - B}{d} \bmod 6 & \text{if } M = R, \\ \frac{B - R}{d} + 2 & \text{if } M = G, \\ \frac{R - G}{d} + 4 & \text{if } M = B. \end{cases}$$

A gray has $d = 0$. Its hue is undefined, and it is reported as 0 with zero saturation. The remainder operation in Swift keeps the sign of its dividend, so a negative hue is brought back into $[0^\circ, 360^\circ)$ at the end. The saturation is clamped because rounding can push it a hair outside $[0, 1]$ when L is close to 0 or 1.

Hue is circular, so a profile across a red area can jump between 0° and 360° . The spectrum then sees a sharp step that isn't really in the image. That's a property of hue itself, and it's worth remembering when reading hue spectra.

53 $\langle \text{ColorSpaces: RGB to HSL 53} \rangle \equiv$

```

public static func hsl(from c: RGBA) -> (h: Float, s: Float, l: Float) {
    let maxV = max(c.r, c.g, c.b)
    let minV = min(c.r, c.g, c.b)
    let l = (maxV + minV) / 2
    let d = maxV - minV
    guard d > 1e-7 else { return (0, 0, l) }
    let s = d / (1 - abs(2 * l - 1))
    var h: Float
    if maxV == c.r {
        h = ((c.g - c.b) / d).truncatingRemainder(dividingBy: 6)
    } else if maxV == c.g {
        h = (c.b - c.r) / d + 2
    } else {

```

```

        h = (c.r - c.g) / d + 4
    }
    h *= 60
    if h < 0 { h += 360 }
    return (h, min(max(s, 0), 1), l)
}

```

Used in chunk 52.

The inverse builds the color from its chroma $C = (1 - |2L - 1|)S$. With $H' = H/60^\circ$, the second-largest component is $X = C(1 - |H' \bmod 2 - 1|)$. The sextant $\lfloor H' \rfloor$ decides which of R, G, B receive C, X and 0. Finally the lightness offset $m = L - C/2$ is added to all three.

54 $\langle \text{ColorSpaces: HSL to RGB 54} \rangle \equiv$

```

public static func rgb(h: Float, s: Float, l: Float, alpha: Float = 1) -> RGBA {
    let c = (1 - abs(2 * l - 1)) * s
    let hp = h / 60
    let x = c * (1 - abs(hp.truncatingRemainder(dividingBy: 2) - 1))
    let (r, g, b): (Float, Float, Float)
    switch hp {
    case ..<1: (r, g, b) = (c, x, 0)
    case ..<2: (r, g, b) = (x, c, 0)
    case ..<3: (r, g, b) = (0, c, x)
    case ..<4: (r, g, b) = (0, x, c)
    case ..<5: (r, g, b) = (x, 0, c)
    default: (r, g, b) = (c, 0, x)
    }
    let m = l - c / 2
    return RGBA(r: r + m, g: g + m, b: b + m, a: alpha)
}

```

Used in chunk 52.

5.3.2 CMYK

Real CMYK depends on inks and paper, and needs an ICC profile. LineScope uses the naive conversion instead, which is enough to see how a filter treats the “amount of black” or of one ink:

$$K = 1 - \max(R, G, B), \quad C = \frac{1 - R - K}{1 - K}, \quad M = \frac{1 - G - K}{1 - K}, \quad Y = \frac{1 - B - K}{1 - K}.$$

Pure black ($K = 1$) would divide by zero, so it is returned as $(0, 0, 0, 1)$. The inverse is $R = (1 - C)(1 - K)$, and likewise for G and B .

55 $\langle \text{ColorSpaces: RGB to CMYK and back 55} \rangle \equiv$

```

public static func cmyk(from c: RGBA) -> (c: Float, m: Float, y: Float, k: Float) {
    let k = 1 - max(c.r, c.g, c.b)
    guard k < 1 - 1e-7 else { return (0, 0, 0, 1) }
    let d = 1 - k
    return ((1 - c.r - k) / d, (1 - c.g - k) / d, (1 - c.b - k) / d, k)
}

public static func rgb(c: Float, m: Float, y: Float, k: Float, alpha: Float = 1) -> RGBA {
    RGBA(r: (1 - c) * (1 - k), g: (1 - m) * (1 - k), b: (1 - y) * (1 - k), a: alpha)
}

```

Used in chunk 52.

The session calls this once per sample: it converts the color and picks the channel by its index, in the order given by `ColorModel.channels`.

56 *⟨ColorSpaces: channel extraction 56⟩* ≡

```
public static func value(of color: RGBA, model: ColorModel, channel: Int) -> Float {
    switch model {
    case .rgba:
        return [color.r, color.g, color.b, color.a][channel]
    case .hsl:
        let v = hsl(from: color)
        return [v.h, v.s, v.l][channel]
    case .cmyk:
        let v = cmyk(from: color)
        return [v.c, v.m, v.y, v.k][channel]
    }
}
```

Used in chunk 52.

5.4 The spectrum

The last step turns a profile of n samples into an amplitude spectrum. It uses the FFT of Apple’s Accelerate framework (vDSP).

57 *⟨Packages/LineScopeCore/Sources/LineScopeCore/Spectrum.swift 57⟩* ≡

```
import Accelerate
import Foundation
```

⟨Spectrum: window functions 58⟩

⟨Spectrum: result 59⟩

⟨Spectrum: analyzer 60⟩

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/Spectrum.swift.

The DFT treats its input as one period of a periodic signal. The two ends of a profile rarely match, so the implied periodic signal has a jump, and that jump leaks energy into every frequency. A window tapers the ends to zero and trades a little frequency resolution for much less leakage. LineScope offers no window (“rectangular”) and the Hann window [5],

$$w_i = \frac{1}{2} - \frac{1}{2} \cos \frac{2\pi i}{n-1}, \quad i = 0, \dots, n-1.$$

58 *⟨Spectrum: window functions 58⟩* ≡

```
public enum WindowFunction: String, CaseIterable, Identifiable, Sendable {
    case none
    case hann

    public var id: String { rawValue }
    public var displayName: String { self == .none ? "Rectangular" : "Hann" }
}
```

Used in chunk 57.

The result gives each bin’s amplitude and two frequency axes. *Cycles per pixel* are measured in the grid of the sampled image, where the Nyquist limit is $\frac{1}{2}$. *Cycles per line* count whole periods over the length of the line. The same image content has the same cycles-per-line frequency in every copy, so that axis is the one for comparing an original with its resampled copies. On that axis, each copy’s Nyquist limit sits at half its own sample count.

The peak is searched among the non-DC bins. The mean usually dominates bin 0 and is rarely what one wants to find.

59 *⟨Spectrum: result 59⟩* ≡

```
public struct SpectrumResult: Sendable {
  /// Bin frequencies in cycles per pixel of the sampled image (Nyquist ≈ 0.5).
  public var cyclesPerPixel: [Double]
  /// Bin frequencies in cycles over the whole line; comparable across resampled copies.
  public var cyclesPerLine: [Double]
  /// Single-sided amplitude: a sine of amplitude A peaks at ≈ A.
  public var amplitudes: [Double]
  public var fftSize: Int
  public var sampleCount: Int

  public static let empty = SpectrumResult(cyclesPerPixel: [], cyclesPerLine: [], amplitudes: [],
    ↪ fftSize: 0, sampleCount: 0)

  /// Index of the largest non-DC bin.
  public var peakIndex: Int? {
    guard amplitudes.count > 1 else { return nil }
    return amplitudes.indices.dropFirst().max { amplitudes[$0] < amplitudes[$1] }
  }
}
```

Used in chunk 57.

The analyzer is one function with five steps.

60 *⟨Spectrum: analyzer 60⟩* ≡

```
public enum SpectrumAnalyzer {
  /// Real FFT of a profile, zero-padded to the next power of two.
  /// - Parameter spacing: distance between samples, in pixels.
  public static func compute(
    _ values: [Float], spacing: Double, window: WindowFunction, removeMean: Bool
  ) -> SpectrumResult {
    ⟨Spectrum: choose the FFT size 61⟩

    ⟨Spectrum: condition the signal 62⟩
    ⟨Spectrum: zero-pad 63⟩

    ⟨Spectrum: packed real FFT 64⟩

    ⟨Spectrum: single-sided amplitudes 65⟩

    ⟨Spectrum: frequency axes 66⟩
  }
}
```

Used in chunk 57.

The radix-2 FFT of vDSP needs a power-of-two length $N = 2^{\lceil \log_2 n \rceil}$. A profile with fewer than two samples has no spectrum.

61 *⟨Spectrum: choose the FFT size 61⟩* ≡

```
let n = values.count
guard n >= 2 else { return .empty }
let log2n = vDSP_Length(Int(ceil(log2(Double(n))))))
let fftSize = 1 << Int(log2n)
let half = fftSize / 2
```

Used in chunk 60.

Removing the mean is optional. Without it, bin 0 holds the average brightness, which in dB scale flattens everything else. The window is applied after the mean is removed, so it tapers the fluctuations and not the offset. Its sum $\sum w_i$ is kept for normalization: with no window it is simply n .

62 *⟨Spectrum: condition the signal 62⟩* ≡

```

var signal = values
if removeMean {
    let mean = vDSP.mean(signal)
    signal = vDSP.add(-mean, signal)
}
var windowSum = Float(n)
if window == .hann {
    // Symmetric Hann over the n real samples.
    var w = [Float](repeating: 0, count: n)
    for i in 0..

```

Used in chunk 60.

The n conditioned samples are followed by $N - n$ zeros. Zero padding adds no information. It interpolates the spectrum more finely, so that peaks are drawn at about the right place.

63 *⟨Spectrum: zero-pad 63⟩* ≡

```

var padded = [Float](repeating: 0, count: fftSize)
padded.replaceSubrange(0..

```

Used in chunk 60.

vDSP computes a real FFT of length N as a complex FFT of length $N/2$. First vDSP_ctoz reads the real array as $N/2$ complex numbers: even samples become real parts and odd samples imaginary parts, in split form. Then vDSP_fft_zrip transforms it in place.

The output is *packed*. For $1 \leq k < N/2$, `real[k]` and `imag[k]` hold $2 \operatorname{Re} X_k$ and $2 \operatorname{Im} X_k$. The two purely real bins that have no partner share the first slot: `real[0]` holds $2X_0$ (the DC term) and `imag[0]` holds $2X_{N/2}$ (the Nyquist term). The factor 2 is vDSP's scaling convention for the forward real transform.

64 *⟨Spectrum: packed real FFT 64⟩* ≡

```

var real = [Float](repeating: 0, count: half)
var imag = [Float](repeating: 0, count: half)
guard let setup = vDSP_create_fftsetup(log2n, FFTRadix(kFFTRadix2)) else { return .empty }
defer { vDSP_destroy_fftsetup(setup) }

real.withUnsafeMutableBufferPointer { rp in
    imag.withUnsafeMutableBufferPointer { ip in
        var split = DSPSplitComplex(realp: rp.baseAddress!, imagp: ip.baseAddress!)
        padded.withUnsafeBytes { raw in
            vDSP_ctoz(raw.bindMemory(to: DSPComplex.self).baseAddress!, 2, &split, 1, vDSP_Length(half))
        }
        vDSP_fft_zrip(setup, &split, 1, log2n, FFTDirection(FFT_FORWARD))
    }
}

```

Used in chunk 60.

The amplitudes are made *single-sided* and normalized so they can be read as the amplitude of a sinusoid. A real signal $A \cos(2\pi fi + \varphi)$ puts $|X_k| = \frac{A}{2} \sum w_i$ into bin k and the same into its mirror bin $N - k$, which isn't stored. Therefore

$$A_k = \frac{2|X_k|}{\sum_i w_i} \quad (0 < k < N/2), \quad A_0 = \frac{|X_0|}{\sum_i w_i}, \quad A_{N/2} = \frac{|X_{N/2}|}{\sum_i w_i},$$

since DC and Nyquist have no mirror. Dividing by the window sum rather than by n corrects for the energy the window removed. With it, a sine of amplitude 1 reads about 1 with either window. The halving in the code undoes vDSP's factor 2.

65 *⟨Spectrum: single-sided amplitudes 65⟩* ≡

```
// vDSP's packed output is scaled by 2; DC is in real[0] and Nyquist in imag[0].
var amplitudes = [Double](repeating: 0, count: half + 1)
let norm = Double(max(windowSum, 1e-12))
amplitudes[0] = abs(Double(real[0]) / 2) / norm
amplitudes[half] = abs(Double(imag[0]) / 2) / norm
for k in 1..

```

Used in chunk 60.

Bin k of an N -point transform of samples spaced δ pixels apart has frequency $k/(N\delta)$ cycles per pixel, which reaches $1/(2\delta) \approx \frac{1}{2}$ at $k = N/2$. The line spans $(n - 1)\delta$ pixels, so the same bin has $k(n - 1)/N$ cycles per line.

66 *⟨Spectrum: frequency axes 66⟩* ≡

```
let lineLength = spacing * Double(n - 1)
var cpp = [Double](repeating: 0, count: half + 1)
var cpl = [Double](repeating: 0, count: half + 1)
for k in 0...half {
    cpp[k] = Double(k) / (Double(fftSize) * spacing)
    cpl[k] = cpp[k] * lineLength
}
return SpectrumResult(
    cyclesPerPixel: cpp, cyclesPerLine: cpl, amplitudes: amplitudes,
    fftSize: fftSize, sampleCount: n
)
```

Used in chunk 60.

Chapter 6

Test Patterns

Natural photographs make poor test inputs for a resampler. Their spectrum is unknown, so it's hard to tell whether a ripple in a profile comes from the image or from the filter. Synthetic patterns have known spectra, and each one stresses one property: a chirp sweeps all frequencies, an edge shows ringing, an impulse shows the point-spread function. LineScope can generate eleven such patterns. The generator lives in the core package, so the tests can use it too.

6.1 The catalog

67 *<Packages/LineScopeCore/Sources/LineScopeCore/TestPattern.swift 67> ≡*

```
import Foundation
```

```
<TestPattern: pattern kinds 68>
```

```
<TestPattern: specification 73>
```

```
<TestPattern: generator 74>
```

```
<TestPattern: SplitMix64 81>
```

Root chunk, tangled to Packages/LineScopeCore/Sources/LineScopeCore/TestPattern.swift.

A kind is an enumeration case. It is Codable because a pattern window is restored after a relaunch from its specification alone (Chapter 9).

68 *<TestPattern: pattern kinds 68> ≡*

```
/// Synthetic images commonly used to evaluate resampling and filtering.
```

```
public enum TestPatternKind: String, CaseIterable, Identifiable, Codable, Sendable {
```

```
  <TestPattern: the cases 69>
```

```
  public var id: String { rawValue }
```

```
  <TestPattern: display names 70>
```

```
  <TestPattern: what each pattern is for 71>
```

```
  <TestPattern: the tunable parameter 72>
```

```
}
```

Used in chunk 67.

69 $\langle \text{TestPattern: the cases 69} \rangle \equiv$

```
case zonePlate
case linearChirp
case sineGrating
case squareGrating
case checkerboard
case stepEdge
case slantedEdge
case impulses
case siemensStar
case whiteNoise
case hueRamp
```

Used in chunk 68.

70 $\langle \text{TestPattern: display names 70} \rangle \equiv$

```
public var displayName: String {
    switch self {
        case .zonePlate: "Zone Plate"
        case .linearChirp: "Linear Chirp"
        case .sineGrating: "Sine Grating"
        case .squareGrating: "Square-Wave Bars"
        case .checkerboard: "Checkerboard"
        case .stepEdge: "Step Edge"
        case .slantedEdge: "Slanted Edge"
        case .impulses: "Impulse Grid"
        case .siemensStar: "Siemens Star"
        case .whiteNoise: "White Noise"
        case .hueRamp: "Hue / Lightness Ramp"
    }
}
```

Used in chunk 68.

The generator window shows one sentence about each pattern. These sentences are the short version of this chapter, and they say which filter behavior each pattern reveals.

71 $\langle \text{TestPattern: what each pattern is for 71} \rangle \equiv$

/// What the pattern is good for, shown in the generator window.

```
public var purpose: String {
    switch self {
        case .zonePlate: "Circular chirp: frequency grows with radius up to the chosen maximum. Aliasing
        ↪ appears as false ring centers."
        case .linearChirp: "Horizontal sweep from 0 to the chosen frequency. Put the line across it to read
        ↪ a filter's frequency response."
        case .sineGrating: "Single pure frequency along x. The spectrum should show one peak."
        case .squareGrating: "Bars with odd harmonics. Shows how filters treat harmonics near Nyquist."
        case .checkerboard: "Two-dimensional square wave. Small cells alias badly under nearest-neighbor."
        case .stepEdge: "One hard vertical edge. Reveals ringing (sinc, Lanczos) versus blur (tent,
        ↪ Mitchell)."
        case .slantedEdge: "Edge tilted a few degrees, the standard target for measuring sharpness (MTF)."
        case .impulses: "Isolated single pixels. Each dot shows the filter's point-spread function."
        case .siemensStar: "Radial spokes: spatial frequency rises toward the center, in every orientation."
        case .whiteNoise: "Uniform random values with a flat spectrum. The profile's spectrum shows the
        ↪ filter's transfer curve."
        case .hueRamp: "Hue across x, lightness down y. Use it with the HSL and CMYK channels."
    }
}
```

Used in chunk 68.

Every pattern has at most one parameter. Keeping it to one number keeps the user interface simple, and a specification stays a small value that is easy to compare and store. The parameter's name, default and range are given here, next to the patterns, so the generator window needs no knowledge of what the numbers mean.

72 $\langle \text{TestPattern: the tunable parameter 72} \rangle \equiv$

/// The single tunable parameter, or nil if the pattern has none.

```
public var parameter: (name: String, defaultValue: Double, range: ClosedRange<Double>)? {
    switch self {
        case .zonePlate, .linearChirp: ("Max frequency (cycles/px)", 0.5, 0.01...1)
        case .sineGrating: ("Frequency (cycles/px)", 0.1, 0.001...1)
        case .squareGrating: ("Period (px)", 8, 2...512)
        case .checkerboard: ("Cell size (px)", 8, 1...512)
        case .slantedEdge: ("Angle (degrees)", 5, -45...45)
        case .impulses: ("Spacing (px)", 32, 2...1024)
        case .siemensStar: ("Spokes", 36, 4...360)
        case .whiteNoise: ("Seed", 1, 0...1_000_000)
        case .stepEdge, .hueRamp: nil
    }
}
```

Used in chunk 68.

6.2 Specifications

A specification is everything needed to generate a pattern: kind, size and parameter. Because it is Hashable and Codable, it can identify a window directly (it is its own id), and SwiftUI can store it to restore that window. A missing parameter takes the kind's default.

The title names the window. It shows the parameter only when it tells something: the seed of white noise is interesting only if it isn't the default.

73 $\langle \text{TestPattern: specification 73} \rangle \equiv$

```
public struct TestPatternSpec: Codable, Hashable, Identifiable, Sendable {
    public var kind: TestPatternKind
    public var width: Int
    public var height: Int
    public var parameter: Double

    public var id: Self { self }

    public init(kind: TestPatternKind, width: Int = 512, height: Int = 512, parameter: Double? = nil) {
        self.kind = kind
        self.width = width
        self.height = height
        self.parameter = parameter ?? kind.parameter?.defaultValue ?? 0
    }

    public var title: String {
        var t = "\(kind.displayName) \((width)×(height)"
        if let p = kind.parameter, kind != .whiteNoise || parameter != p.defaultValue {
            t += " (\(fmt(parameter)))"
        }
        return t
    }
}
```

Used in chunk 67.

6.3 The generator

Generation is one function: set up a canvas, then fill it according to the kind.

74 $\langle \text{TestPattern: generator 74} \rangle \equiv$

```
public enum TestPatternGenerator {
    public static func make(_ spec: TestPatternSpec) -> PixelBuffer {
         $\langle \text{TestPattern: set up the canvas 75} \rangle$ 

         $\langle \text{TestPattern: gray-level helper 76} \rangle$ 

        switch spec.kind {
             $\langle \text{TestPattern: chirps and gratings 77} \rangle$ 
             $\langle \text{TestPattern: edges and impulses 78} \rangle$ 
             $\langle \text{TestPattern: Siemens star 79} \rangle$ 
             $\langle \text{TestPattern: noise and color ramp 80} \rangle$ 
        }
        return buf
    }
}
```

Used in chunk 67.

The center (c_x, c_y) is the geometric center of the image, which for an even size lies on a pixel corner.

75 $\langle \text{TestPattern: set up the canvas 75} \rangle \equiv$

```
let w = max(1, spec.width)
let h = max(1, spec.height)
let p = spec.parameter
var buf = PixelBuffer(width: w, height: h)
let cx = Double(w) / 2
let cy = Double(h) / 2
```

Used in chunk 74.

Most patterns are gray levels given by a function $f(x, y)$. The helper evaluates it at pixel centers $(x + \frac{1}{2}, y + \frac{1}{2})$, the convention of the whole program. Evaluating at corners would shift every pattern by half a pixel against the line sampler. Point sampling like this is deliberate: no pattern is antialiased, so patterns with frequencies near or above the Nyquist limit already alias in the original. Those are exactly the cases that test a filter.

76 $\langle \text{TestPattern: gray-level helper 76} \rangle \equiv$

```
func gray(_ f: (Double, Double) -> Double) {
    for y in 0..

```

Used in chunk 74.

6.3.1 Chirps and gratings

A *zone plate* is a circular chirp,

$$f(r) = \frac{1}{2} + \frac{1}{2} \cos(\pi k r^2),$$

where r is the distance from the center. Its phase is $\phi(r) = \pi k r^2$, so its local frequency is $\frac{1}{2\pi} \frac{d\phi}{dr} = kr$ cycles per pixel, growing linearly with the radius. The constant k is chosen so that the frequency reaches the parameter p at $r = \max(w, h)/2$, the ends of the default horizontal line. Along that line a zone plate thus sweeps every frequency from 0 to p and back. When a copy can't represent the high end, the excess folds back and shows as false ring centers: the classic picture of aliasing.

The *linear chirp* is the one-dimensional version, a sweep across x only. Its phase $\pi p x^2 / W$ has instantaneous frequency $p x / W$, from 0 at the left edge to p at the right. Read along a horizontal line, its profile is the frequency response of the filter, with frequency on the horizontal axis.

The *sine grating* has a single frequency p , so its spectrum should show a single peak. The *square-wave bars* contain the odd harmonics 1, 3, 5, ... of 1/period, with amplitudes falling as $1/n$. The *checkerboard* is their two-dimensional product.

77 $\langle \text{TestPattern: chirps and gratings 77} \rangle \equiv$

```
case .zonePlate:
    // Local frequency k·r reaches p at r = max(w,h)/2 (the left/right edges of the midline).
    let k = p / (Double(max(w, h)) / 2)
    gray { x, y in
        let r2 = (x - cx) * (x - cx) + (y - cy) * (y - cy)
        return 0.5 + 0.5 * cos(Double.pi * k * r2)
    }
case .linearChirp:
    // Phase  $\pi 2 \cdot p \cdot x^2 / (2W)$ : instantaneous frequency  $p \cdot x / W$ .
    gray { x, _ in 0.5 + 0.5 * cos(Double.pi * p * x * x / Double(w)) }
case .sineGrating:
    gray { x, _ in 0.5 + 0.5 * sin(2 * Double.pi * p * x) }
case .squareGrating:
    let period = max(p, 1)
    gray { x, _ in (x - 0.5).truncatingRemainder(dividingBy: period) < period / 2 ? 1 : 0 }
case .checkerboard:
    let cell = max(Int(p.rounded()), 1)
    gray { x, y in ((Int(x) / cell) + (Int(y) / cell)) % 2 == 0 ? 1 : 0 }

Used in chunk 74.
```

6.3.2 Edges and impulses

A *step edge* is the input that separates the kernels most clearly. An interpolating kernel with negative lobes (Catmull-Rom, Lanczos, sinc) overshoots on both sides of the step and rings, and a truncated sinc rings the longest. A non-negative kernel (tent, area) only blurs, and Mitchell-Netravali sits in between [3]. The levels 0.1 and 0.9 leave room for overshoot to show instead of being clipped at 0 and 1.

A *slanted edge* is the same edge tilted by α degrees. Because the edge crosses the pixel grid at a slight angle, successive rows sample it at different sub-pixel phases, which is why it is the standard target for measuring sharpness. A pixel is on the bright side when $(x - c_x) \cos \alpha + (y - c_y) \sin \alpha \geq 0$.

The *impulse grid* puts single bright pixels on a black ground, far enough apart that their responses don't overlap. After a filter, each dot is the kernel's two-dimensional point-spread function, sampled.

78 $\langle \text{TestPattern: edges and impulses 78} \rangle \equiv$

```
case .stepEdge:
    gray { x, _ in x < cx ? 0.1 : 0.9 }
case .slantedEdge:
    // Dark/light halves split by a line through the center tilted from vertical.
    let a = p * Double.pi / 180
    gray { x, y in ((x - cx) * cos(a) + (y - cy) * sin(a)) < 0 ? 0.1 : 0.9 }
case .impulses:
    let s = max(Int(p.rounded()), 1)
    let off = s / 2
    gray { x, y in (Int(x) % s == off && Int(y) % s == off) ? 1 : 0 }

Used in chunk 74.
```

6.3.3 The Siemens star

A *Siemens star* has s bright and s dark spokes, given by the sign of $\sin(s\theta)$. At radius r the circumference $2\pi r$ holds s periods, so the tangential frequency is $s/(2\pi r)$ cycles per pixel. It rises toward the center without limit, and it does so in every direction, which the one-dimensional patterns can't show. Outside the inscribed circle the image is mid-gray.

79 $\langle \text{TestPattern: Siemens star 79} \rangle \equiv$

```
case .siemensStar:
  let spokes = max(p.rounded(), 2)
  let radius = Double(min(w, h)) / 2
  gray { x, y in
    let dx = x - cx, dy = y - cy
    guard dx * dx + dy * dy <= radius * radius else { return 0.5 }
    let theta = atan2(dy, dx)
    return sin(spokes * theta) >= 0 ? 1 : 0
  }
```

Used in chunk 74.

6.3.4 Noise and color

White noise has, on average, a flat spectrum. After filtering, the spectrum of a profile is therefore the filter's transfer function times a constant, up to random fluctuations. The values must be reproducible, so that a restored window shows the same noise. So they come from a seeded generator rather than the system's random source. The top 53 bits of each 64-bit output divided by 2^{53} give a uniform double in $[0, 1)$, with every representable value equally likely.

The *hue/lightness ramp* is the only colored pattern. Hue runs across x at full saturation, and lightness falls from 1 at the top to 0 at the bottom. It is meant for the HSL and CMYK channels of the profile. The conversion can land a rounding error outside $[0, 1]$, so every component is clamped.

80 $\langle \text{TestPattern: noise and color ramp 80} \rangle \equiv$

```
case .whiteNoise:
  var rng = SplitMix64(seed: UInt64(max(p, 0)))
  for y in 0..

```

Used in chunk 74.

6.4 SplitMix64

SplitMix64 is a tiny, fast generator with good statistical quality for a problem like this. Its state advances by a fixed odd constant (the “golden gamma” $\lfloor 2^{64}/\varphi \rfloor$), and each state is scrambled by two xor-shift-multiply rounds. The wrapping operators $\&+=$ and $\&*$ give the arithmetic modulo 2^{64} that the algorithm needs; the ordinary operators would trap on overflow.

81 $\langle \text{TestPattern: SplitMix64 } 81 \rangle \equiv$

/// Small deterministic generator so noise patterns are reproducible from their seed.

```
struct SplitMix64 {
    private var state: UInt64

    init(seed: UInt64) { state = seed }

    mutating func next() -> UInt64 {
        state &+= 0x9E37_79B9_7F4A_7C15
        var z = state
        z = (z ^ (z >> 30)) &* 0xBF58_476D_1CE4_E5B9
        z = (z ^ (z >> 27)) &* 0x94D0_49BB_1331_11EB
        return z ^ (z >> 31)
    }
}
```

Used in chunk 67.

Chapter 7

Tests

The core package has one test file. It doesn't try to cover every line. Each test pins down a *property* that the earlier chapters claim, one the rest of the program depends on and that a careless change could break without anyone seeing it. If the FFT normalization were off by a factor of two, for instance, every spectrum would still look plausible. Only a test that knows the right answer catches it. Run the tests with `swift test` in the package directory, or one of them with `swift test --filter SpectrumTests/testSinePeak`.

82 $\langle \text{Packages/LineScopeCore/Tests/LineScopeCoreTests/LineScopeCoreTests.swift } 82 \rangle \equiv$

```
import XCTest
@testable import LineScopeCore
```

$\langle \text{Tests: gradient fixture } 83 \rangle$

$\langle \text{Tests: spectrum } 84 \rangle$

$\langle \text{Tests: resampling } 85 \rangle$

$\langle \text{Tests: color conversions } 90 \rangle$

$\langle \text{Tests: line sampler } 91 \rangle$

$\langle \text{Tests: Core Graphics bridge and filters } 92 \rangle$

$\langle \text{Tests: test patterns } 93 \rangle$

Root chunk, tangled to Packages/LineScopeCore/Tests/LineScopeCoreTests/LineScopeCoreTests.swift.

`@testable` lets the tests reach internal members, such as the kernel function `Resampler.kernel` and `PixelBuffer`'s premultiplication helpers.

Several tests need an image whose content is known at every pixel. A gradient does the job: red grows with x , green with y , and blue is constant. So the pixel at (x, y) is $(x/W, y/H, \frac{1}{4})$, and a test can predict what any resampling or sampling of it should give.

83 $\langle \text{Tests: gradient fixture } 83 \rangle \equiv$

```
private func gradient(width: Int, height: Int) -> PixelBuffer {
    var buf = PixelBuffer(width: width, height: height)
    for y in 0..
```

Used in chunk 82.

7.1 Spectrum

The first test feeds the analyzer a sine that completes exactly $k = 16$ periods in $n = 256$ samples. Since n is already a power of two, there is no padding and no leakage. All the energy must land in bin 16, with amplitude 1 under the single-sided normalization of Chapter 5, at frequency $16/256$ cycles per pixel. The last bin must sit at the Nyquist frequency $\frac{1}{2}$. The test proves the packing, the factor 2 and the frequency axis all at once.

The second test checks the other special bin. A constant 0.3 with the mean kept must show amplitude 0.3 in bin 0, which has no mirror image and so no factor 2. Its 100 samples also check that the FFT size is rounded up to 128.

84 $\langle \text{Tests: spectrum 84} \rangle \equiv$

```
final class SpectrumTests: XCTestCase {
  func testSinePeak() {
    let n = 256
    let k = 16
    let values = (0..

```

Used in chunk 82.

7.2 Resampling

The resampling tests check four properties.

85 $\langle \text{Tests: resampling 85} \rangle \equiv$

```
final class ResamplingTests: XCTestCase {
   $\langle \text{Tests: kernel shapes 86} \rangle$ 

   $\langle \text{Tests: identity at scale one 87} \rangle$ 

   $\langle \text{Tests: area average halves 88} \rangle$ 

   $\langle \text{Tests: constants stay constant 89} \rangle$ 
}
```

Used in chunk 82.

First, the kernels themselves. An *interpolating* kernel k has $k(0) = 1$ and $k(j) = 0$ at every nonzero integer j , so that resampling at the original positions returns the original samples. The tent, Catmull-Rom, Lanczos and sinc kernels all have this property. Mitchell-Netravali doesn't. It is *approximating*: with $B = C = \frac{1}{3}$, the formula of Chapter 3 gives

$$k(0) = \frac{6 - 2B}{6} = \frac{8}{9}, \quad k(1) = \frac{-B - 6C + 6B + 30C - 12B - 48C + 8B + 24C}{6} = \frac{B}{6} = \frac{1}{18},$$

and $k(2) = 0$ where the support ends. The value $k(\frac{1}{2}) = \frac{9}{16}$ of Catmull-Rom checks the cubic piece away from the integers. The last assertion checks that the sinc really is truncated at its radius.

86 $\langle$ Tests: kernel shapes 86 $\rangle \equiv$

```
func testKernelShapes() {
    // Interpolating kernels are 1 at 0 and 0 at nonzero integers.
    for method in [ResampleMethod.tent, .catmullRom, .lanczos3, .sinc] {
        XCTAssertEqual(Resampler.kernel(method, 0), 1, accuracy: 1e-12, "\(method)")
        for k in 1...3 {
            XCTAssertEqual(Resampler.kernel(method, Double(k)), 0, accuracy: 1e-12, "\(method) at \(k)")
        }
    }
    // Mitchell-Netravali (B = C = 1/3) is approximating: k(0) = 8/9, k(1) = 1/18.
    XCTAssertEqual(Resampler.kernel(.mitchell, 0), 8.0 / 9, accuracy: 1e-12)
    XCTAssertEqual(Resampler.kernel(.mitchell, 1), 1.0 / 18, accuracy: 1e-12)
    XCTAssertEqual(Resampler.kernel(.mitchell, 2), 0, accuracy: 1e-12)
    XCTAssertEqual(Resampler.kernel(.catmullRom, 0.5), 0.5625, accuracy: 1e-12)
    // Sinc is truncated at its radius.
    XCTAssertEqual(Resampler.kernel(.sinc, Resampler.sincRadius + 0.5), 0)
}
```

Used in chunk 85.

Second, the consequence in whole images: at scale 1 every interpolating method must return the input unchanged. This test also exercises the pixel-center convention. With an off-by-half error in the mapping $(i + \frac{1}{2})/s$, the output would come out shifted and the test would fail. Mitchell is excluded, since by the previous test it slightly smooths even at scale 1.

87 $\langle$ Tests: identity at scale one 87 $\rangle \equiv$

```
func testIdentityAtScaleOne() {
    let src = gradient(width: 17, height: 9)
    // Mitchell is approximating (slightly smoothing), so it is not an identity at x1.
    for method in ResampleMethod.allCases where method != .mitchell {
        let out = Resampler.resample(src, scale: 1, method: method)
        XCTAssertEqual(out.width, 17)
        XCTAssertEqual(out.height, 9)
        for (a, b) in zip(out.data, src.data) {
            XCTAssertEqual(a, b, accuracy: 1e-5, "\(method)")
        }
    }
}
```

Used in chunk 85.

Third, the area average at scale $\frac{1}{2}$ must average exact pairs of pixels: $\frac{0+1}{2} = 0.5$ and $\frac{0.2+0.6}{2} = 0.4$. This checks the footprint-coverage weights.

88 $\langle$ Tests: area average halves 88 $\rangle \equiv$

```
func testAreaHalvesAverages() {
    var src = PixelBuffer(width: 4, height: 1)
    let values: [Float] = [0, 1, 0.2, 0.6]
    for (x, v) in values.enumerated() { src[x, 0] = RGBA(r: v, g: v, b: v) }
    let out = Resampler.resample(src, scale: 0.5, method: .area)
    XCTAssertEqual(out.width, 2)
    XCTAssertEqual(out[0, 0].r, 0.5, accuracy: 1e-5)
    XCTAssertEqual(out[1, 0].r, 0.4, accuracy: 1e-5)
}
```

Used in chunk 85.

Fourth, weights must sum to one for every output pixel, at every scale and with every method, including at the borders where the kernel is truncated and renormalized. The simplest observable consequence is that a constant image stays constant. The scales mix downscaling, upscaling, and a non-integer ratio.

89 *⟨Tests: constants stay constant 89⟩* ≡

```
func testConstantImageStaysConstant() {
    let src = PixelBuffer(width: 13, height: 7, fill: RGBA(r: 0.4, g: 0.5, b: 0.6))
    for method in ResampleMethod.allCases {
        for scale in [0.3, 0.5, 2.0, 3.7] {
            let out = Resampler.resample(src, scale: scale, method: method)
            for i in stride(from: 0, to: out.data.count, by: 4) {
                XCTAssertEqual(out.data[i], 0.4, accuracy: 1e-4, "\\(method) \\(scale)")
            }
        }
    }
}
```

Used in chunk 85.

7.3 Color conversions

Each conversion is checked by a round trip, color model and back, over a saturated color, the primaries, a nearly pure blue and a gray. The gray is the degenerate case where hue is undefined. One absolute value pins the hue convention: pure blue is at 240°. The CMYK round trip includes black and white, the two ends where K is 1 and 0.

90 *⟨Tests: color conversions 90⟩* ≡

```
final class ColorTests: XCTestCase {
    func testHSLRoundTrip() {
        for c in [RGBA(r: 0.2, g: 0.7, b: 0.4), RGBA(r: 1, g: 0, b: 0), RGBA(r: 0.1, g: 0.1, b: 0.9),
        ↪ RGBA(r: 0.5, g: 0.5, b: 0.5)] {
            let hsl = ColorConversion.hsl(from: c)
            let back = ColorConversion.rgb(h: hsl.h, s: hsl.s, l: hsl.l)
            XCTAssertEqual(back.r, c.r, accuracy: 1e-5)
            XCTAssertEqual(back.g, c.g, accuracy: 1e-5)
            XCTAssertEqual(back.b, c.b, accuracy: 1e-5)
        }
        XCTAssertEqual(ColorConversion.hsl(from: RGBA(r: 0, g: 0, b: 1)).h, 240, accuracy: 1e-4)
    }

    func testCMYKRoundTrip() {
        for c in [RGBA(r: 0.2, g: 0.7, b: 0.4), RGBA(r: 0, g: 0, b: 0), RGBA(r: 1, g: 1, b: 1)] {
            let v = ColorConversion.cmyk(from: c)
            let back = ColorConversion.rgb(c: v.c, m: v.m, y: v.y, k: v.k)
            XCTAssertEqual(back.r, c.r, accuracy: 1e-5)
            XCTAssertEqual(back.g, c.g, accuracy: 1e-5)
            XCTAssertEqual(back.b, c.b, accuracy: 1e-5)
        }
    }
}
```

Used in chunk 82.

7.4 Line sampling

The line sampler has two promises. First, the number of samples follows the image's own grid: $[L] + 1$ gives 201 samples on a 200-pixel-wide image and 101 on its half-size copy, with a spacing of exactly one pixel. This is what makes a shrunk copy show less detail in its profile. Second, the samples come from the right place. A vertical line at $u = 0.505$ passes through the center of pixel column 50 of a 100-pixel image, so red is exactly 50/100 along the whole line while green increases.

91 *⟨Tests: line sampler 91⟩* ≡

```
final class LineSamplerTests: XCTestCase {
    func testSampleCountScalesWithResample() {
        let src = gradient(width: 200, height: 100)
        let half = Resampler.resample(src, scale: 0.5, method: .tent)
        let a = LineSampler.sample(src, along: .initial)
        let b = LineSampler.sample(half, along: .initial)
        XCTAssertEqual(a.colors.count, 201)
        XCTAssertEqual(b.colors.count, 101)
        XCTAssertEqual(a.spacing, 1, accuracy: 1e-9)
    }

    func testSamplesFollowContent() {
        let src = gradient(width: 100, height: 100)
        let line = LineSegment(start: .init(x: 0.505, y: 0.1), end: .init(x: 0.505, y: 0.9))
        let s = LineSampler.sample(src, along: line)
        // Vertical line through pixel column 50: red is constant, green increases.
        XCTAssertEqual(s.colors.first!.r, 0.5, accuracy: 1e-4)
        XCTAssertEqual(s.colors.last!.r, 0.5, accuracy: 1e-4)
        XCTAssertEqual(s.colors.first!.g, s.colors.last!.g)
    }
}
```

Used in chunk 82.

7.5 Core Graphics and filters

The conversion to an 8-bit `CGImage` and back may lose at most one quantization step, $\frac{1}{255}$, and must keep the orientation and the size. A flipped row order would change every value of the gradient, far more than the tolerance.

The filters are checked less strictly. They come from different sources (Core Image and the CPU kernels), and the property that the rest of the program depends on is that they *keep the image size*: the normalized line relies on it. One qualitative check makes sure the Core Image path doesn't scramble the image: a blurred horizontal gradient still increases from left to right.

92 *⟨Tests: Core Graphics bridge and filters 92⟩* ≡

```
final class CGBridgeTests: XCTestCase {
    func testCGImageRoundTrip() throws {
        let src = gradient(width: 32, height: 16)
        let cg = try XCTUnwrap(src.makeCGImage())
        let back = try XCTUnwrap(PixelBuffer(cgImage: cg))
        XCTAssertEqual(back.width, 32)
        XCTAssertEqual(back.height, 16)
        for (a, b) in zip(back.data, src.data) {
            XCTAssertEqual(a, b, accuracy: 1.0 / 255 + 1e-4)
        }
    }

    func testFiltersKeepSize() {
        let src = gradient(width: 24, height: 12)
        let filters: [FilterKind] = [.gaussianBlur(radius: 2), .boxBlur(radius: 2), .median,
        ↪ .unsharpMask(radius: 2, intensity: 0.5),
        ↪ .noiseReduction(level: 0.02, sharpness: 0.4), .sobel, .laplacian,
        ↪ .emboss, .grayscale, .invert]
        for f in filters {
            let out = Filters.apply(f, to: src)
            XCTAssertEqual(out.width, 24, f.displayName)
            XCTAssertEqual(out.height, 12, f.displayName)
        }
        let blurred = Filters.apply(.gaussianBlur(radius: 1), to: src)
        // Horizontal gradient stays roughly monotone and in range after blur.
    }
}
```

```

        XCTAssertLessThan(blurred[2, 6].r, blurred[20, 6].r)
    }
}

```

Used in chunk 82.

7.6 Test patterns

Every pattern must honor its requested size and stay in $[0, 1]$. The range check caught a real bug: the hue ramp produced tiny negative components from rounding in the HSL conversion, which is why that pattern clamps (Chapter 6).

The sine grating test is an end-to-end check of the whole analysis pipeline: generate a grating of 0.125 cycles per pixel, sample it along the default line, and take the Hann-windowed spectrum. The peak must be within one bin of 0.125. Here generator, sampler and analyzer must all agree about pixel centers and frequency units.

The last test checks two exact facts: the checkerboard cells alternate at the right place, and the same seed gives the same noise, which window restoration relies on.

93 $\langle$ Tests: test patterns 93 $\rangle \equiv$

```

final class TestPatternTests: XCTestCase {
    func testAllPatternsGenerateRequestedSize() {
        for kind in TestPatternKind.allCases {
            let buf = TestPatternGenerator.make(TestPatternSpec(kind: kind, width: 64, height: 48))
            XCTAssertEqual(buf.width, 64, kind.displayName)
            XCTAssertEqual(buf.height, 48, kind.displayName)
            XCTAssertTrue(buf.data.allSatisfy { $0 >= 0 && $0 <= 1 }, kind.displayName)
        }
    }

    func testSineGratingPeaksAtItsFrequency() {
        let spec = TestPatternSpec(kind: .sineGrating, width: 256, height: 8, parameter: 0.125)
        let samples = LineSampler.sample(TestPatternGenerator.make(spec), along: .initial)
        let values = samples.colors.map(\.r)
        let s = SpectrumAnalyzer.compute(values, spacing: samples.spacing, window: .hann, removeMean:
↳ true)
        let peak = try! XCTUnwrap(s.peakIndex)
        XCTAssertEqual(s.cyclesPerPixel[peak], 0.125, accuracy: 1.0 / Double(s.fftSize) + 1e-9)
    }

    func testCheckerboardAndNoiseDeterminism() {
        let board = TestPatternGenerator.make(TestPatternSpec(kind: .checkerboard, width: 8, height: 8,
↳ parameter: 2))
        XCTAssertEqual(board[0, 0].r, 1)
        XCTAssertEqual(board[2, 0].r, 0)
        XCTAssertEqual(board[2, 2].r, 1)
        let a = TestPatternGenerator.make(TestPatternSpec(kind: .whiteNoise, width: 16, height: 16,
↳ parameter: 7))
        let b = TestPatternGenerator.make(TestPatternSpec(kind: .whiteNoise, width: 16, height: 16,
↳ parameter: 7))
        XCTAssertEqual(a, b)
    }
}

```

Used in chunk 82.

Chapter 8

Documents and the Session Model

With the numeric core in place, we turn to the app. This chapter describes the data the user interface is built on: the scenes that make up the app, the document that brings an image in, the small value types that describe images, zones and panes, and the `Session`, the one object per window that holds everything the views show.

8.1 The app and its scenes

A SwiftUI app is a list of *scenes*, each a kind of window. `LineScope` has four.

Image documents

One window per opened image file, supplied by `DocumentGroup` in its *viewing* form: the app opens documents but never creates or saves them, so there is no New or Save command to disable.

Test patterns

One window per generated pattern (Chapter 6). A `WindowGroup` keyed by a `TestPatternSpec` value opens a new window for each spec; because the spec is `Codable`, macOS can restore these windows after a relaunch.

Pattern generator

A single window for choosing a pattern and its parameters.

Inspector

A single window describing the image in focus, placed at the top right.

The menus of Chapter 10 are attached to the document scene, which puts them in the app's menu bar.

94 `<LineScope/LineScopeApp.swift 94> ≡`

```
import LineScopeCore
import SwiftUI

@main
struct LineScopeApp: App {
    var body: some Scene {
        <LineScopeApp: document windows 95>

        <LineScopeApp: test-pattern windows 96>

        <LineScopeApp: pattern generator window 97>

        <LineScopeApp: inspector window 98>
    }
}
```

Root chunk, tangled to LineScope/LineScopeApp.swift.

A document window gets the document and a name to show and export under. The file name falls back to “Untitled” for documents without a URL.

```
95  <LineScopeApp: document windows 95> ≡
    DocumentGroup(viewing: ImageDocument.self) { file in
        MainView(document: file.document, fileName: file.fileURL?.lastPathComponent ?? "Untitled")
    }
    .defaultSize(width: 1100, height: 900)
    .commands { LineScopeCommands() }
```

Used in chunk 94.

The window group passes a binding to an optional spec, which is nil only in the moment before SwiftUI has decoded a restored window’s value; nothing is shown then.

```
96  <LineScopeApp: test-pattern windows 96> ≡
    WindowGroup("Test Pattern", for: TestPatternSpec.self) { $spec in
        if let spec {
            PatternWindow(spec: spec)
        }
    }
    .defaultSize(width: 1100, height: 900)
```

Used in chunk 94.

```
97  <LineScopeApp: pattern generator window 97> ≡
    Window("New Test Pattern", id: TestPatternGeneratorView.windowID) {
        TestPatternGeneratorView()
    }
    .windowResizability(.contentSize)
```

Used in chunk 94.

```
98  <LineScopeApp: inspector window 98> ≡
    Window("Inspector", id: InspectorView.windowID) {
        InspectorView()
    }
    .defaultSize(width: 340, height: 760)
    .defaultPosition(.topTrailing)
```

Used in chunk 94.

8.2 The image document

A document is decoded once, when it is opened, into three things: the float `PixelBuffer` all analysis works on, a `CGImage` made from that buffer for display, and a record of facts about the file. Displaying the buffer’s own rendering, not the decoded file image, guarantees that what the user sees is exactly the data being analyzed.

```
99  <LineScope/Document/ImageDocument.swift 99> ≡
    import ImageIO
    import LineScopeCore
    import SwiftUI
    import UniformTypeIdentifiers
```

<ImageDocument: source information 100>

<ImageDocument: the document type 101>

Root chunk, tangled to LineScope/Document/ImageDocument.swift.

The facts are gathered before the conversion to 8-bit sRGB can hide them (Chapter 2): the original bit depth, color space and alpha. The inspector shows them, so a user who opened a 16-bit or CMYK file knows it was converted.

100 *<ImageDocument: source information 100> ≡*

/// Facts about the file as decoded, before conversion to the working float buffer.

```
struct SourceInfo {
    var fileType: String
    var pixelWidth: Int
    var pixelHeight: Int
    var bitsPerComponent: Int
    var bitsPerPixel: Int
    var colorSpaceName: String
    var hasAlpha: Bool
    var dpi: Double?
}
```

Used in chunk 99.

The document accepts any image type ImageIO can read. It is a value type holding immutable data, which is all FileDocument needs for viewing.

101 *<ImageDocument: the document type 101> ≡*

*/// Read-only image document. Pixels are decoded once into a `PixelBuffer`; EXIF orientation is ignored
/// on purpose so analysis runs on the stored pixel grid.*

```
struct ImageDocument: FileDocument {
    static var readableContentTypes: [UTType] { [.image] }

    let buffer: PixelBuffer
    let displayImage: CGImage
    let info: SourceInfo
}
```

<ImageDocument: reading a file 102>

<ImageDocument: a generated test pattern 105>

<ImageDocument: refusing to write 106>

```
}
```

Used in chunk 99.

Reading takes the first image of the file, converts it and renders it back. Any failure along the chain is reported as a corrupt file, which makes the system show its standard error alert.

One decision is visible only by its absence: the EXIF orientation tag is *not* applied. A photo taken in portrait may therefore appear rotated. Applying the tag would mean rotating or mirroring the pixel grid, and LineScope's purpose is to analyze the grid exactly as stored.

102 *<ImageDocument: reading a file 102> ≡*

```
init(configuration: ReadConfiguration) throws {
    <ImageDocument: decoding the first image 103>

    <ImageDocument: recording facts about the source 104>
}
```

Used in chunk 101.

103 *<ImageDocument: decoding the first image 103> ≡*

```
guard
  let data = configuration.file.regularFileContents,
  let source = CGImageSourceCreateWithData(data as CFData, nil),
  let cg = CGImageSourceCreateImageAtIndex(source, 0, nil),
  let buffer = PixelBuffer(cgImage: cg),
  let display = buffer.makeCGImage()
else { throw CocoaError(.fileReadCorruptFile) }
```

Used in chunk 102.

The file type comes from ImageIO’s type identifier, turned into a readable description such as “PNG image”. The color space name loses its `kCGColorSpace` prefix. An image has alpha unless its alpha info says the fourth channel is absent or to be skipped.

104 *<ImageDocument: recording facts about the source 104> ≡*

```
let props = CGImageSourceCopyPropertiesAtIndex(source, 0, nil) as? [CFString: Any]
let type = (CGImageSourceGetType(source) as String?).flatMap { UTType($0)?.localizedDescription }
let alpha = cg.alphaInfo
self.buffer = buffer
self.displayImage = display
self.info = SourceInfo(
  fileType: type ?? "Image",
  pixelWidth: cg.width,
  pixelHeight: cg.height,
  bitsPerComponent: cg.bitsPerComponent,
  bitsPerPixel: cg.bitsPerPixel,
  colorSpaceName: (cg.colorSpace?.name as String?).replacingOccurrences(of: "kCGColorSpace", with:
    ↪ "") ?? "Unknown",
  hasAlpha: !(alpha == .none || alpha == .noneSkipFirst || alpha == .noneSkipLast),
  dpi: props?[kCGImagePropertyDPIWidth] as? Double
)
```

Used in chunk 102.

A generated pattern becomes a document too, so that it can be shown in exactly the same window as a file. Its facts describe the float buffer itself: 32 bits per component, 128 per pixel, and no alpha.

105 *<ImageDocument: a generated test pattern 105> ≡*

/// A generated test pattern, shown in the same analysis window as a file.

```
init?(pattern spec: TestPatternSpec) {
  let buffer = TestPatternGenerator.make(spec)
  guard let display = buffer.makeCGImage() else { return nil }
  self.buffer = buffer
  self.displayImage = display
  self.info = SourceInfo(
    fileType: "Generated test pattern",
    pixelWidth: buffer.width,
    pixelHeight: buffer.height,
    bitsPerComponent: 32,
    bitsPerPixel: 128,
    colorSpaceName: "sRGB (float)",
    hasAlpha: false,
    dpi: nil
  )
}
```

Used in chunk 101.

`FileDocument` requires a way to write, even for a viewer. It is never called, because the viewing document group offers no Save command; derived images leave the program through Export instead (Section 8.4.5).

106 *<ImageDocument: refusing to write 106> ≡*

```
func fileWrapper(configuration: WriteConfiguration) throws -> FileWrapper {
    throw CocoaError(.featureUnsupported)
}
```

Used in chunk 101.

8.3 Variants, zones and panes

The next file holds the small types the session is made of.

107 *<LineScope/Model/ImageVariant.swift 107> ≡*

```
import CoreGraphics
import Foundation
import LineScopeCore
```

<ImageVariant: the variant type 108>

<ImageVariant: zones 109>

<ImageVariant: panes 110>

<ImageVariant: the frequency axis 111>

<ImageVariant: operation requests 112>

Root chunk, tangled to LineScope/Model/ImageVariant.swift.

A *variant* is the original image or one copy derived from it. Besides its pixels and their rendering, it remembers its parent and the operation that produced it, and the whole *lineage*: the operations from the original down to this copy. Storing the lineage, rather than walking parent links, keeps a variant's history intact when an intermediate tab is closed. A variant's name, shown on its tab, is the lineage joined with arrows.

Variants are immutable. An operation never changes a variant; it creates a new one.

108 *<ImageVariant: the variant type 108> ≡*

```
/// The original image or one derived copy. Each copy records the chain of operations back to the
    ↪ original.
```

```
struct ImageVariant: Identifiable {
    let id = UUID()
    let buffer: PixelBuffer
    let cgImage: CGImage
    let parentID: UUID?
    let operation: ImageOperation?
    /// Operations applied to the original, in order. Empty for the original.
    let lineage: [ImageOperation]

    var isOriginal: Bool { operation == nil }

    var name: String {
        lineage.isEmpty ? "Original" : lineage.map(\.displayName).joined(separator: " - ")
    }
}
```

Used in chunk 107.

The window is divided into three horizontal *zones*. The raw values index the session's pane arrays.

```

109 <ImageVariant: zones 109> ≡
    /// Which kind of content a zone shows.
    enum Zone: Int, CaseIterable, Identifiable {
        case images, profile, spectrum

        var id: Int { rawValue }

        var title: String {
            switch self {
            case .images: "Images"
            case .profile: "Line Profile"
            case .spectrum: "Spectrum"
            }
        }
    }

```

Used in chunk 107.

Each zone can be split into several side-by-side *panes*, each showing one variant. A pane has its own identity, separate from the variant it shows, so SwiftUI keeps a pane's view state (such as its scroll position) when the pane switches to another variant.

```

110 <ImageVariant: panes 110> ≡
    /// One side-by-side pane inside a zone; it shows one variant at a time.
    struct PaneState: Identifiable, Equatable {
        let id = UUID()
        var variantID: UUID
    }

```

Used in chunk 107.

The spectrum can be plotted against two frequency scales, discussed with the analysis in Chapter 5: cycles per pixel of the variant itself, or cycles along the whole line, which is the same for every copy.

```

111 <ImageVariant: the frequency axis 111> ≡
    enum FrequencyAxis: String, CaseIterable, Identifiable {
        case cyclesPerPixel
        case cyclesPerLine

        var id: String { rawValue }
        var displayName: String { self == .cyclesPerPixel ? "cycles/pixel" : "cycles/line" }
    }

```

Used in chunk 107.

Some menu commands need parameters before they can run: a blur radius, a scale factor. Such a command doesn't run the operation; it stores an `OperationRequest` in the session, and the window presents a sheet for it. The request is `Identifiable` because SwiftUI's `sheet(item:)` requires it.

```

112 <ImageVariant: operation requests 112> ≡
    /// Operations that need parameters before running; shown as a sheet.
    enum OperationRequest: Identifiable {
        case gaussianBlur, boxBlur, unsharpMask, noiseReduction
        case resample(ResampleMethod)

        var id: String {
            switch self {
            case .gaussianBlur: "gaussian"
            case .boxBlur: "box"
            case .unsharpMask: "unsharp"
            case .noiseReduction: "noise"
            case .resample(let m): "resample-\(m.rawValue)"
            }
        }
    }

```

```

    }

    var title: String {
        switch self {
            case .gaussianBlur: "Gaussian Blur"
            case .boxBlur: "Box Blur"
            case .unsharpMask: "Sharpen (Unsharp Mask)"
            case .noiseReduction: "Noise Reduction"
            case .resample(let m): "Resample: \(m.displayName)"
        }
    }
}

```

Used in chunk 107.

8.4 The session

Every window has one `Session`. It owns the variants, the shared line, the analysis settings, the pane layout and the focus, and it performs operations. The views hold no state of their own beyond transient gestures; they read the session and write to it.

The class is marked `@Observable`. SwiftUI records which properties each view's body reads, and re-renders exactly those views when one of the properties changes. This is what keeps the line in sync everywhere: dragging it in one pane writes `line`, and every pane in all three zones read `line` while drawing, so all of them update in the same frame, with no notification code. The class is also `@MainActor`: all its state is touched only on the main thread, and background work has to hop back to it explicitly.

113 *<LineScope/Model/Session.swift 113> ≡*

```

import AppKit
import ImageIO
import LineScopeCore
import SwiftUI
import UniformTypeIdentifiers

```

<Session: profile and spectrum point types 114>

<Session: the session class 115>

<Session: the focused-value entry 129>

<Session: the active session 130>

Root chunk, tangled to LineScope/Model/Session.swift.

The views plot two kinds of series. A profile keeps the raw channel values (for the spectrum and the inspector's statistics), the samples they came from, and a list of points for the chart. The points have integer identifiers because Swift Charts identifies marks, and the sample index is a natural, stable key.

114 *<Session: profile and spectrum point types 114> ≡*

```

struct Profile {
    struct Point: Identifiable {
        let id: Int
        let x: Double
        let y: Double
    }

    var points: [Point]
    var values: [Float]
    var samples: LineSamples
}

```

```

struct SpectrumPoint: Identifiable {
  let id: Int
  let frequency: Double
  let magnitude: Double
}

```

Used in chunk 113.

115 $\langle$ Session: the session class 115 $\rangle \equiv$

/// All state for one document window: the variants, the shared line, and view settings.

```

@MainActor @Observable
final class Session {
   $\langle$ Session: stored state 116 $\rangle$ 

   $\langle$ Session: initialization 120 $\rangle$ 

   $\langle$ Session: looking up variants 121 $\rangle$ 

  // MARK: Panes

   $\langle$ Session: selecting and splitting panes 122 $\rangle$ 

  // MARK: Variants

   $\langle$ Session: closing a variant 123 $\rangle$ 

   $\langle$ Session: performing an operation in the background 124 $\rangle$ 

   $\langle$ Session: showing a new variant 125 $\rangle$ 

  // MARK: Analysis

   $\langle$ Session: computing a profile 126 $\rangle$ 

   $\langle$ Session: computing a spectrum 127 $\rangle$ 

  // MARK: Export

   $\langle$ Session: exporting the focused variant 128 $\rangle$ 
}

```

Used in chunk 113.

8.4.1 State

116 $\langle$ Session: stored state 116 $\rangle \equiv$

$\langle$ Session: identity, variants and the line 117 $\rangle$

$\langle$ Session: analysis settings 118 $\rangle$

$\langle$ Session: panes, focus and pending work 119 $\rangle$

Used in chunk 115.

The variants can only be changed by the session's own methods, which maintain the invariant that index 0 is the original and is never removed.

The line is stored in *normalized* coordinates, $(u, v) \in [0, 1]^2$ relative to the image extent. A filtered copy has the same size as its source and a resampled copy is only scaled, so the same normalized line lands on the same content in every copy, however large it is. This is how a single

line can be “in the same position of all images, after considering filtering and resampling”: each pane converts it to its own pixel grid only when it draws or samples.

117 *⟨Session: identity, variants and the line 117⟩* ≡

```
let fileName: String
let sourceInfo: SourceInfo

private(set) var variants: [ImageVariant]
var line: LineSegment = .initial

Used in chunk 116.
```

The analysis settings are shared by all panes, deliberately: comparing the red channel of one copy with the hue of another, or a Hann-windowed spectrum with a rectangular one, would be misleading. The one subtle setting is the channel index. The color models have different numbers of channels (RGBA and CMYK have four, HSL three), so switching the model resets an index that would fall out of range.

118 *⟨Session: analysis settings 118⟩* ≡

```
var colorModel: ColorModel = .rgba {
  didSet { if channelIndex >= colorModel.channels.count { channelIndex = 0 } }
}
var channelIndex = 0
var window: WindowFunction = .hann
var removeMean = true
var logMagnitude = true
var frequencyAxis: FrequencyAxis = .cyclesPerPixel
var useDevicePixels = false

Used in chunk 116.
```

The *focused variant* is the one menu operations apply to and the inspector describes: the last one the user clicked, in any zone. Applying operations to it, rather than always to the original, is what lets operations chain: blur a copy, then downsample the blurred copy.

pendingRequest drives the parameter sheet, runningOperations the progress indicator, and errorMessage an alert.

119 *⟨Session: panes, focus and pending work 119⟩* ≡

```
/// Panes per zone, indexed by `Zone.rawValue`.
var panes: [[PaneState]]
/// The variant operations apply to and the inspector describes.
var focusedVariantID: UUID
var pendingRequest: OperationRequest?
private(set) var runningOperations = 0
var errorMessage: String?

Used in chunk 116.
```

A new session has just the original, focused, shown in one pane of each zone.

120 *⟨Session: initialization 120⟩* ≡

```
init(document: ImageDocument, fileName: String) {
  let original = ImageVariant(
    buffer: document.buffer, cgImage: document.displayImage,
    parentID: nil, operation: nil, lineage: []
  )
  self.fileName = fileName
  self.sourceInfo = document.info
  self.variants = [original]
  self.focusedVariantID = original.id
  self.panes = Zone.allCases.map { _ in [PaneState(variantID: original.id)] }
}

Used in chunk 115.
```

Looking a variant up by identifier is a linear search. A session holds a handful of variants, and an array keeps the tab order for free. If the focused variant has disappeared, the original stands in for it, so `focusedVariant` never fails.

121 *⟨Session: looking up variants 121⟩* ≡

```
var original: ImageVariant { variants[0] }

var focusedVariant: ImageVariant {
    variant(focusedVariantID) ?? original
}

func variant(_ id: UUID) -> ImageVariant? {
    variants.first { $0.id == id }
}

var channelName: String { colorModel.channels[channelIndex] }
Used in chunk 115.
```

8.4.2 Panes

Selecting a tab in a pane both shows that variant there and focuses it. Splitting a zone appends a pane showing the first variant not yet visible in that zone, which is usually what the user wants to compare with; if every variant is already shown, the new pane shows the focused one. Unsplitting removes the last pane, never the only one.

122 *⟨Session: selecting and splitting panes 122⟩* ≡

```
func select(_ variantID: UUID, zone: Zone, pane: Int) {
    panes[zone.rawValue][pane].variantID = variantID
    focusedVariantID = variantID
}

func split(_ zone: Zone) {
    let shown = Set(panes[zone.rawValue].map(\.variantID))
    let next = variants.first { !shown.contains($0.id) } ?? focusedVariant
    panes[zone.rawValue].append(PaneState(variantID: next.id))
}

func unsplit(_ zone: Zone) {
    guard panes[zone.rawValue].count > 1 else { return }
    panes[zone.rawValue].removeLast()
}
Used in chunk 115.
```

8.4.3 Variants and operations

Closing a variant removes its tab. The original can't be closed. Panes that showed the closed variant fall back to the original, and so does the focus. Variants derived from the closed one keep working, since each carries its own lineage.

123 *⟨Session: closing a variant 123⟩* ≡

```
func closeVariant(_ id: UUID) {
    guard let index = variants.firstIndex(where: { $0.id == id }), index > 0 else { return }
    variants.remove(at: index)
    for z in panes.indices {
        for p in panes[z].indices where panes[z][p].variantID == id {
            panes[z][p].variantID = original.id
        }
    }
    if focusedVariantID == id { focusedVariantID = original.id }
}
```

Used in chunk 115.

Performing an operation must not freeze the window: resampling a large photograph with a wide kernel takes a noticeable fraction of a second. The session captures the source variant and its buffer on the main actor, then starts a task. The task itself inherits the main actor, but the heavy call runs in a *detached* task at user-initiated priority, which runs on a background thread. This is safe because `apply` is a pure function of a `Sendable` buffer (Section 4.2). When the result comes back, the rest of the task runs on the main actor again: it renders the image, builds the variant with its extended lineage, appends it and shows it.

The source is captured *before* the task starts, so the operation applies to the variant that was focused when the user chose the command, even if the focus moves while it is running.

124 *⟨Session: performing an operation in the background 124⟩* ≡

/// Applies an operation to the focused variant in the background and adds the result as a new tab.

```
func perform(_ operation: ImageOperation) {
    let source = focusedVariant
    let input = source.buffer
    runningOperations += 1
    Task {
        let output = await Task.detached(priority: .userInitiated) {
            operation.apply(to: input)
        }.value
        runningOperations -= 1
        guard let cg = output.makeCGImage() else {
            errorMessage = "Could not render the result of \(operation.displayName)."
            return
        }
        let variant = ImageVariant(
            buffer: output, cgImage: cg, parentID: source.id,
            operation: operation, lineage: source.lineage + [operation]
        )
        variants.append(variant)
        show(variant.id)
    }
}
```

Used in chunk 115.

Where should a new variant appear? When a zone is split, the user is usually comparing: the left pane holds the baseline and the right pane the candidate. So the new variant goes into the *last* pane of each zone, which is also the only pane when the zone is not split. It becomes the focus, so the next operation chains from it.

125 *⟨Session: showing a new variant 125⟩* ≡

/// Shows a new variant in every zone: in the last pane when split (so the left pane keeps the comparison baseline), otherwise in the only pane.

```
private func show(_ id: UUID) {
    for z in panes.indices {
        panes[z][panes[z].count - 1].variantID = id
    }
    focusedVariantID = id
}
```

Used in chunk 115.

8.4.4 Analysis

A profile samples the line in the variant's own pixel grid (Chapter 5) and extracts the chosen channel of the chosen color model from every sample. The chart's *x* coordinate is the normalized position along the line, not the sample index. A half-size copy has half as many samples, and with positions in `[0, 1]` its profile still lines up with the original's.

These functions are called from view bodies, so they run again on every change of the line or the settings. That is cheap: a few thousand bilinear samples and one FFT.

126 *⟨Session: computing a profile 126⟩* ≡

```
func profile(for variant: ImageVariant) -> Profile {
    let samples = LineSampler.sample(variant.buffer, along: line)
    let values = samples.colors.map {
        ColorConversion.value(of: $0, model: colorModel, channel: channelIndex)
    }
    let points = values.indices.map { Profile.Point(id: $0, x: samples.positions[$0], y:
        ↪ Double(values[$0])) }
    return Profile(points: points, values: values, samples: samples)
}
```

Used in chunk 115.

The spectrum of a profile is computed with the current window and mean setting. The sample spacing is guarded away from zero, which it would be for a line of zero length. For plotting, each bin gets its frequency on the chosen axis, and its amplitude, optionally in decibels,

$$A_{\text{dB}} = \max(20 \log_{10} A, -100).$$

The floor keeps empty bins, whose logarithm would be $-\infty$, on the chart.

127 *⟨Session: computing a spectrum 127⟩* ≡

```
func spectrum(for profile: Profile) -> SpectrumResult {
    SpectrumAnalyzer.compute(
        profile.values, spacing: max(profile.samples.spacing, 1e-9),
        window: window, removeMean: removeMean
    )
}

func spectrumPoints(_ s: SpectrumResult) -> [SpectrumPoint] {
    let freqs = frequencyAxis == .cyclesPerPixel ? s.cyclesPerPixel : s.cyclesPerLine
    return s.amplitudes.indices.map { i in
        let a = s.amplitudes[i]
        return SpectrumPoint(
            id: i, frequency: freqs[i],
            magnitude: logMagnitude ? max(20 * log10(max(a, 1e-12)), Self.dBFloor) : a
        )
    }
}
```

```
static let dBFloor = -100.0
```

Used in chunk 115.

8.4.5 Export

Exporting writes the focused variant as a PNG. The proposed name combines the file's base name with the variant's lineage, with the arrows and slashes that can't appear in a file name replaced. The image written is the variant's display image, which is exactly what is on screen: 8-bit, sRGB.

128 *⟨Session: exporting the focused variant 128⟩* ≡

```
func exportFocused() {
    let variant = focusedVariant
    let panel = NSSavePanel()
    panel.allowedContentTypes = [.png]
    let base = (fileName as NSString).deletingPathExtension
    let suffix = variant.isOriginal ? "original" : variant.name
        .replacingOccurrences(of: " → ", with: "_")
        .replacingOccurrences(of: "/", with: "-")
    panel.nameFieldStringValue = "\(base)-\(suffix).png"
```

```

guard panel.runModal() == .OK, let url = panel.url else { return }
guard
    let dest = CGImageDestinationCreateWithURL(url as CFURL, UTType.png.identifier as CFString, 1,
    ↪ nil)
else {
    errorMessage = "Could not create \(url.lastPathComponent)."
    return
}
CGImageDestinationAddImage(dest, variant.cgImage, nil)
if !CGImageDestinationFinalize(dest) {
    errorMessage = "Could not write \(url.lastPathComponent)."
}
}

```

Used in chunk 115.

8.4.6 Finding the session from outside the window

Menus must act on the session of the window the user is working in. SwiftUI's *focused values* provide exactly this: a window publishes its session, and the menu commands read the value of the key window. The entry is declared with the `@Entry` macro.

129 *<Session: the focused-value entry 129> ≡*

```

extension FocusedValues {
    @Entry var session: Session?
}

```

Used in chunk 113.

Focused values fail in one case, and it matters here: the inspector. While the user interacts with the inspector window, *it* is the key window, and it has no session, so a focused value would be `nil` just when the inspector needs it. The app therefore also keeps track of the session of the most recently key *document* window, in a shared observable object. Document windows update it as they become key (Chapter 9), the inspector reads it, and the menus use it as a fallback.

130 *<Session: the active session 130> ≡*

```

/// The session of the most recently key document window, for the inspector window (which is itself
/// key while being used, so focused values would not reach it).
@MainActor @Observable
final class ActiveSession {
    static let shared = ActiveSession()
    var session: Session?
}

```

Used in chunk 113.

Chapter 9

Views

The views are deliberately thin. Every number they show comes from the `Session` of Chapter 8: a view asks it for the variants, the shared line, a profile or a spectrum, and draws the answer. What the views do own is *interaction*: which pane shows which variant, how a drag on an image turns into a new line, and how the controls in each zone header write back into the session. Because the session is an observable object, writing the line from one pane is enough to redraw every image, profile and spectrum in the window.

The chapter follows the window from the outside in: the window and its three zones, the panes and their tabs, the image canvas where the line is drawn, the two charts, and then the auxiliary windows (the parameter sheet, the inspector and the test-pattern windows).

9.1 The document window

A document window is three zones stacked vertically: images, line profile, spectrum. The window owns one `Session`, created from the document when the window opens.

131 *<LineScope/Views/MainView.swift 131> ≡*

```
import LineScopeCore
import SwiftUI

struct MainView: View {
    <MainView: the session is created once per window 132>

    var body: some View {
        <MainView: three zones stacked vertically 133>
        <MainView: publishing the session to menus and inspector 134>
        <MainView: sheets, alerts and the progress indicator 135>
    }
}
```

<MainView.swift: the window observer 136>

Root chunk, tangled to LineScope/Views/MainView.swift.

SwiftUI may call a view's initializer many times, but a `@State` property is initialized only the first time; later values passed to `State(initialValue:)` are ignored. That is exactly the lifetime we want: one session per window, surviving every redraw.

132 *<MainView: the session is created once per window 132> ≡*

```
@State private var session: Session

init(document: ImageDocument, fileName: String) {
    _session = State(initialValue: Session(document: document, fileName: fileName))
}
```

Used in chunk 131.

The zones sit in a `VSplitView`, so the user can drag the dividers between them. The ideal heights give the images most of the room at first; the minimum sizes keep every zone's header and at least a sliver of its content visible.

133 *⟨MainView: three zones stacked vertically 133⟩* ≡

```
VSplitView {
    ZoneView(zone: .images, session: session)
        .frame(minHeight: 180, idealHeight: 420)
    ZoneView(zone: .profile, session: session)
        .frame(minHeight: 140, idealHeight: 220)
    ZoneView(zone: .spectrum, session: session)
        .frame(minHeight: 140, idealHeight: 220)
}
.frame(minWidth: 720, minHeight: 560)
```

Used in chunk 131.

Two other parts of the app need to find this window's session. The menus (Chapter 10) read it as a *focused scene value*, which SwiftUI provides for the key window. The inspector cannot do that. It is a window of its own, and while the user works in it *it* is the key window, so the focused value would vanish just when it is needed. Instead, the document window reports itself to the shared `ActiveSession` whenever it becomes key, and withdraws when it closes (but only if it is still the active one, since another window may have become key in between).

134 *⟨MainView: publishing the session to menus and inspector 134⟩* ≡

```
.focusedSceneValue(\.session, session)
.background(WindowKeyObserver(
    onBecomeKey: { ActiveSession.shared.session = session },
    onClose: {
        if ActiveSession.shared.session === session { ActiveSession.shared.session = nil }
    }
))
```

Used in chunk 131.

The session also drives three transient pieces of UI. Setting its `pendingRequest` presents the parameter sheet (Section 9.6). Setting its `errorMessage` shows an alert, and dismissing the alert clears the message again through the hand-made binding. While background operations run, a small spinner appears in the toolbar.

135 *⟨MainView: sheets, alerts and the progress indicator 135⟩* ≡

```
.sheet(item: $session.pendingRequest) { request in
    ParameterSheet(request: request, session: session)
}
.alert(
    "Operation failed",
    isPresented: Binding(get: { session.errorMessage != nil }, set: { if !$0 { session.errorMessage =
        ↪ nil } }),
    presenting: session.errorMessage
) { _ in
    Button("OK") {}
} message: { Text($0) }
.toolbar {
    ToolbarItem(placement: .status) {
        if session.runningOperations > 0 {
            ProgressView().controlSize(.small)
        }
    }
}
```

Used in chunk 131.

SwiftUI has no notification for “this window became key”. So we place an invisible AppKit view in the background of the window’s content. An `NSView` knows its `NSWindow`, and through it can observe the window’s notifications. The representable just forwards the two callbacks.

136 *⟨MainView.swift: the window observer 136⟩* ≡

```
/// Reports when the hosting window becomes key or closes.
struct WindowKeyObserver: NSViewRepresentable {
    var onBecomeKey: () -> Void
    var onClose: () -> Void

    func makeNSView(context: Context) -> ObserverView { ObserverView() }

    func updateNSView(_ view: ObserverView, context: Context) {
        view.onBecomeKey = onBecomeKey
        view.onClose = onClose
    }

    ⟨WindowKeyObserver: the observing NSView 137⟩
}
```

Used in chunk 131.

The view learns its window in `viewDidMoveToWindow`, which also runs if the view ever moves to another window, so old observers are removed first. Notifications are delivered on the main queue. The closures are nonisolated as far as the compiler knows, so `MainActor.assumeIsolated` states what is true at run time. A window that is *already* key when the view arrives won’t send `didBecomeKeyNotification` again, so that case is reported directly, one run-loop turn later, after SwiftUI has finished installing the view.

137 *⟨WindowKeyObserver: the observing NSView 137⟩* ≡

```
final class ObserverView: NSView {
    var onBecomeKey: () -> Void = {}
    var onClose: () -> Void = {}
    private var tokens: [NSObjectProtocol] = []

    override func viewDidMoveToWindow() {
        super.viewDidMoveToWindow()
        tokens.forEach(NotificationCenter.default.removeObserver)
        tokens = []
        guard let window else { return }
        let center = NotificationCenter.default
        tokens.append(center.addObserver(forName: NSWindow.didBecomeKeyNotification, object: window,
        ↪ queue: .main) { [weak self] _ in
            MainActor.assumeIsolated { self?.onBecomeKey() }
        })
        tokens.append(center.addObserver(forName: NSWindow.willCloseNotification, object: window, queue:
        ↪ .main) { [weak self] _ in
            MainActor.assumeIsolated { self?.onClose() }
        })
        if window.isKeyWindow {
            DispatchQueue.main.async { [weak self] in self?.onBecomeKey() }
        }
    }

    deinit {
        tokens.forEach(NotificationCenter.default.removeObserver)
    }
}
```

Used in chunk 136.

9.2 Zones

A zone is a header bar followed by one or more panes side by side. The panes of a zone are listed in `session.panes`, indexed by the zone’s raw value, so splitting a zone is only a matter of appending to that list. `ZoneView` is `@Bindable` because its header controls write straight into the session.

138 *<LineScope/Views/ZoneView.swift 138> ≡*

```
import LineScopeCore
import SwiftUI

/// One horizontal zone: a header with its controls, and one or more side-by-side panes.
struct ZoneView: View {
    let zone: Zone
    @Bindable var session: Session

    <ZoneView: header above side-by-side panes 139>

    <ZoneView: the header row 140>

    <ZoneView: controls specific to each zone 141>

    <ZoneView: split and unsplit buttons 145>

    <ZoneView: describing the line 146>
}

Root chunk, tangled to LineScope/Views/ZoneView.swift.
```

The panes are laid out in an `HSplitView`, so their widths can be adjusted by hand. Each pane gets its index, which is how it finds its own entry in the session’s pane list. The `ForEach` is keyed by the pane’s identity rather than its index, so removing the last pane doesn’t rebuild the others.

139 *<ZoneView: header above side-by-side panes 139> ≡*

```
var body: some View {
    VStack(spacing: 0) {
        header
            .padding(.horizontal, 10)
            .padding(.vertical, 5)
            .background(.bar)
        Divider()
        HSplitView {
            ForEach(Array(session.panes[zone.rawValue].enumerated()), id: \.element.id) { index, _ in
                PaneView(zone: zone, paneIndex: index, session: session)
                    .frame(minWidth: 200)
            }
        }
    }
}

Used in chunk 138.
```

Every header has the same shape: the zone’s title on the left, then its own controls, then the split buttons on the right.

140 *<ZoneView: the header row 140> ≡*

```
private var header: some View {
    HStack(spacing: 12) {
        Text(zone.title).font(.headline)
        Spacer(minLength: 8)
        controls
        Divider().frame(height: 16)
        splitButtons
    }
    .controlSize(.small)
```

```
}
```

Used in chunk 138.

The controls differ by zone.

141 *<ZoneView: controls specific to each zone 141> ≡*

```
@ViewBuilder
private var controls: some View {
    switch zone {
    case .images:
        <ZoneView: image-zone controls 142>
    case .profile:
        <ZoneView: profile-zone controls 143>
    case .spectrum:
        <ZoneView: spectrum-zone controls 144>
    }
}
```

Used in chunk 138.

The image zone shows where the line is and offers to put it back in its initial place: the full width of the image, at mid-height.

142 *<ZoneView: image-zone controls 142> ≡*

```
Text(lineDescription)
    .font(.caption.monospacedDigit())
    .foregroundColor(.secondary)
Button("Reset Line") { session.line = .initial }
```

Used in chunk 141.

The profile zone chooses the color model and the channel. They are settings of the *session*, not of a pane, so every profile and every spectrum in the window shows the same channel. Comparisons between variants stay meaningful. The list of channels depends on the model (RGBA has four, HSL three), so the second picker is rebuilt from `colorModel.channels`.

143 *<ZoneView: profile-zone controls 143> ≡*

```
Picker("Color space", selection: $session.colorModel) {
    ForEach(ColorModel.allCases) { Text($0.displayName).tag($0) }
}
.pickerStyle(.segmented)
.fixedSize()
Picker("Channel", selection: $session.channelIndex) {
    ForEach(Array(session.colorModel.channels.enumerated()), id: \.offset) { i, name in
        Text(name).tag(i)
    }
}
.pickerStyle(.segmented)
.fixedSize()
```

Used in chunk 141.

The spectrum zone controls how the profile is transformed. There is the window function (a Hann window [5] reduces the leakage caused by the finite line), the unit of the frequency axis, whether to subtract the mean first (otherwise the DC term dwarfs everything else), and whether amplitudes are shown in decibels.

144 *<ZoneView: spectrum-zone controls 144> ≡*

```
Picker("Window", selection: $session.window) {
    ForEach(WindowFunction.allCases) { Text($0.displayName).tag($0) }
}
.fixedSize()
Picker("Axis", selection: $session.frequencyAxis) {
    ForEach(FrequencyAxis.allCases) { Text($0.displayName).tag($0) }
}
.fixedSize()
Toggle("Remove mean", isOn: $session.removeMean)
Toggle("dB", isOn: $session.logMagnitude)
```

Used in chunk 141.

Splitting adds a pane and unsplitting removes the last one; a zone always keeps at least one. The same actions are in the View menu.

145 *<ZoneView: split and unsplit buttons 145> ≡*

```
private var splitButtons: some View {
    HStack(spacing: 4) {
        Button {
            session.split(zone)
        } label: {
            Image(systemName: "rectangle.split.2x1")
        }
        .help("Split: add a pane to \$(zone.title)")
        Button {
            session.unsplit(zone)
        } label: {
            Image(systemName: "rectangle")
        }
        .help("Unsplit: remove the last pane")
        .disabled(session.panes[zone.rawValue].count < 2)
    }
    .labelsHidden()
}
```

Used in chunk 138.

The line is stored in normalized coordinates, with (0,0) at the top-left corner and (1,1) at the bottom-right corner of every image. So the same numbers describe it in all variants, whatever their size. Three decimals are finer than a pixel for any image up to a thousand pixels wide.

146 *<ZoneView: describing the line 146> ≡*

```
private var lineDescription: String {
    let l = session.line
    func p(_ v: Double) -> String { v.formatted(.number.precision(.fractionLength(3))) }
    return "(\(p(l.start.x)), \(p(l.start.y))) -> (\(p(l.end.x)), \(p(l.end.y)))"
}
```

Used in chunk 138.

9.3 Panes and tabs

Each pane shows one variant, chosen with a strip of tabs that lists all of them. The tab strips of all panes list the same variants in the same order: the order in which they were made. What a pane shows depends on its zone: the image, its profile, or its spectrum.

147 *<LineScope/Views/PaneView.swift 147>* ≡

```
import SwiftUI

/// A tab strip listing every variant, plus the zone's content for the selected one.
struct PaneView: View {
    let zone: Zone
    let paneIndex: Int
    let session: Session

    <PaneView: which variant this pane shows 148>

    <PaneView: tab strip above the zone's content 149>
}

<PaneView.swift: the tab strip 150>

Root chunk, tangled to LineScope/Views/PaneView.swift.
```

A pane's selection is looked up rather than stored in the view, so it survives redraws and can be changed by the session. When a variant is closed, the session moves every pane that showed it back to the original. The fallbacks cover the brief moments during an update when a pane index or a variant no longer exists: the original is always there.

148 *<PaneView: which variant this pane shows 148>* ≡

```
private var selectedID: UUID {
    let panes = session.panes[zone.rawValue]
    return paneIndex < panes.count ? panes[paneIndex].variantID : session.original.id
}

Used in chunk 147.
```

Selecting a tab also makes that variant the *focused* one, the target of the next menu operation.

149 *<PaneView: tab strip above the zone's content 149>* ≡

```
var body: some View {
    let variant = session.variant(selectedID) ?? session.original
    VStack(spacing: 0) {
        TabStrip(session: session, selectedID: variant.id) { id in
            session.select(id, zone: zone, pane: paneIndex)
        }
        Divider()
        Group {
            switch zone {
            case .images: ImageCanvas(variant: variant, session: session)
            case .profile: ProfileChart(variant: variant, session: session)
            case .spectrum: SpectrumChart(variant: variant, session: session)
            }
        }
        .frame(maxWidth: .infinity, maxHeight: .infinity)
    }
}

Used in chunk 147.
```

The tab strip scrolls horizontally when there are more variants than fit. Each tab is a plain button whose label is drawn by hand. Its context menu offers the per-variant commands.

150 *<PaneView.swift: the tab strip 150>* ≡

```
private struct TabStrip: View {
    let session: Session
    let selectedID: UUID
    let onSelect: (UUID) -> Void

    var body: some View {
        ScrollView(.horizontal, showsIndicators: false) {
```

```

HStack(spacing: 2) {
  ForEach(session.variants) { v in
    let selected = v.id == selectedID
    Button {
      onSelect(v.id)
    } label: {
      <TabStrip: a tab's label 151>
    }
    .buttonStyle(.plain)
    .help(v.name)
    .contextMenu {
      <TabStrip: a tab's context menu 152>
    }
  }
}
.padding(.horizontal, 6)
.padding(.vertical, 3)
}
}
}

```

Used in chunk 147.

A tab shows the variant's name (its chain of operations) and its pixel size, which is the quickest way to tell a resampled copy from a filtered one. Two kinds of state are marked: a tinted background for the variant this pane shows, and a small dot for the window's focused variant. They differ as soon as a zone is split.

151 <TabStrip: a tab's label 151> ≡

```

HStack(spacing: 4) {
  if v.id == session.focusedVariantID {
    Circle().fill(Color.accentColor).frame(width: 5, height: 5)
  }
  Text(v.name).lineLimit(1)
  Text("\${v.buffer.width}x${v.buffer.height}")
    .foregroundStyle(.secondary)
}
.font(.caption)
.padding(.horizontal, 8)
.padding(.vertical, 3)
.background(
  RoundedRectangle(cornerRadius: 5)
    .fill(selected ? Color.accentColor.opacity(0.22) : Color.clear)
)
.contentShape(Rectangle())

```

Used in chunk 150.

Export works on the focused variant, so the tab is selected first. The original can't be closed: every other variant descends from it.

152 *<TabStrip: a tab’s context menu 152> ≡*

```
Button("Export ...Image") {
    onSelect(v.id)
    session.exportFocused()
}
Button("Close Tab") { session.closeVariant(v.id) }
    .disabled(v.isOriginal)
```

Used in chunk 150.

9.4 The image canvas

The image zone must show each variant *at its true scale*: one pixel of the image is one point on the screen. A copy shrunk to half size then really looks half as big, and its blockiness or blur is seen as it is, not hidden by the display’s own interpolation. On a Retina display a point is two device pixels, so by default each image pixel is a 2×2 block. The View menu’s “Actual Device Pixels” option maps one image pixel to one device pixel instead.

On top of the image lies the selection line, which the user can drag.

153 *<LineScope/Views/ImageCanvas.swift 153> ≡*

```
import AppKit
import LineScopeCore
import SwiftUI

/// Shows a variant at its true scale (1 image pixel = 1 point, or 1 device pixel) with scrolling,
/// and lets the user drag the shared selection line.
struct ImageCanvas: View {
    let variant: ImageVariant
    @Bindable var session: Session
    @Environment(\.displayScale) private var displayScale
    @State private var drag: DragState?

    <ImageCanvas: the state of a drag 154>

    <ImageCanvas: drawing the image at true scale 155>

    <ImageCanvas: the line gesture 156>

    <ImageCanvas: hit testing the line 157>

    <ImageCanvas: updating the line during a drag 158>

    <ImageCanvas: Shift snapping 160>

    <ImageCanvas: coordinate conversions 161>
}

<ImageCanvas.swift: the line overlay 162>
```

Root chunk, tangled to LineScope/Views/ImageCanvas.swift.

A drag does one of four things, depending on where it starts. It moves the start point, moves the end point, moves the whole segment, or draws a new segment from scratch. The line as it was when the drag began is kept, because every update is computed from that initial line and the gesture’s total movement, never from the previous update. Small errors therefore cannot pile up over a long drag.

154 *⟨ImageCanvas: the state of a drag 154⟩* ≡

```
private struct DragState {
    enum Mode { case start, end, whole, draw }
    var mode: Mode
    var initial: LineSegment
}
```

Used in chunk 153.

`Image(decorative:scale:)` takes the scale of the bitmap. A scale of 1 draws a pixel per point, and the display's scale factor draws a pixel per device pixel. In both cases `size` is the image's extent in points, and interpolation is turned off so enlarged pixels stay square.

The image and its overlay are given exactly that size, and the gesture is attached to that fixed-size frame, *inside* the scroll view. As a result the gesture's coordinates are coordinates in the image. They don't depend on scrolling or on the empty margin around a small image, and dividing by `size` gives normalized coordinates directly. The outer frame, as large as the pane, only centers a small image in it. A tap focuses the variant without touching the line.

155 *⟨ImageCanvas: drawing the image at true scale 155⟩* ≡

```
var body: some View {
    let scale = session.useDevicePixels ? displayScale : 1
    let size = CGSize(width: CGFloat(variant.buffer.width) / scale, height:
        ↪ CGFloat(variant.buffer.height) / scale)
    GeometryReader { geo in
        ScrollView([.horizontal, .vertical]) {
            ZStack {
                Image(decorative: variant.cgImage, scale: scale)
                    .interpolation(.none)
                LineOverlay(line: session.line)
                    .allowsHitTesting(false)
            }
            .frame(width: size.width, height: size.height)
            .contentShape(Rectangle())
            .gesture(lineGesture(size: size))
            .simultaneousGesture(TapGesture().onEnded { session.focusedVariantID = variant.id })
            .frame(minWidth: geo.size.width, minHeight: geo.size.height)
        }
    }
    .background(Color(nsColor: .underPageBackgroundColor))
}
```

Used in chunk 153.

The drag decides its mode on its first change and remembers it until it ends. A minimum distance of two points keeps a plain click from being taken as a zero-length new line. Writing `session.line` is all it takes to move the line in every pane of every zone.

156 *⟨ImageCanvas: the line gesture 156⟩* ≡

```
private func lineGesture(size: CGSize) -> some Gesture {
    DragGesture(minimumDistance: 2)
        .onChanged { g in
            if drag == nil {
                drag = DragState(mode: hitTest(g.startLocation, size: size), initial: session.line)
                session.focusedVariantID = variant.id
            }
            guard let drag else { return }
            session.line = updatedLine(drag, gesture: g, size: size)
        }
        .onEnded { _ in drag = nil }
}
```

Used in chunk 153.

Hit testing works in points, so the targets are the same size whatever the image's scale. The endpoints come first and are a little more generous (9 points) than the segment (6 points), so an

endpoint can always be grabbed even where it touches the segment. A drag that starts anywhere else draws a new line.

157 *⟨ImageCanvas: hit testing the line 157⟩* ≡

```
private func hitTest(_ p: CGPoint, size: CGSize) -> DragState.Mode {
    let a = point(session.line.start, size)
    let b = point(session.line.end, size)
    if distance(p, a) <= 9 { return .start }
    if distance(p, b) <= 9 { return .end }
    if distanceToSegment(p, a, b) <= 6 { return .whole }
    return .draw
}
```

Used in chunk 153.

Moving an endpoint or drawing a new line places a point where the pointer is. That point has been clamped to the image, so a line can reach the image's edge but never pass it.

158 *⟨ImageCanvas: updating the line during a drag 158⟩* ≡

```
private func updatedLine(_ drag: DragState, gesture g: DragGesture.Value, size: CGSize) -> LineSegment {
    let shift = NSEvent.modifierFlags.contains(.shift)
    let current = normalized(g.location, size)
    var line = drag.initial
    switch drag.mode {
    case .start:
        line.start = snapped(current, anchor: line.end, size: size, enabled: shift)
    case .end:
        line.end = snapped(current, anchor: line.start, size: size, enabled: shift)
    case .draw:
        line.start = normalized(g.startLocation, size)
        line.end = snapped(current, anchor: line.start, size: size, enabled: shift)
    case .whole:
        ⟨ImageCanvas: moving the whole segment without leaving the image 159⟩
    }
    return line
}
```

Used in chunk 153.

Moving the whole segment is different. Clamping each endpoint separately would squash the segment against the edge and change its length and angle. Instead the *offset* is clamped, so that both endpoints stay inside. If the initial endpoints have x coordinates x_s and x_e , the horizontal offset Δx must satisfy

$$-\min(x_s, x_e) \leq \Delta x \leq 1 - \max(x_s, x_e),$$

and the same holds vertically. Within that range the segment keeps its shape exactly; at the limit it slides along the edge.

159 *⟨ImageCanvas: moving the whole segment without leaving the image 159⟩* ≡

```
// Move both endpoints, limiting the offset so neither leaves the image.
var dx = g.translation.width / size.width
var dy = g.translation.height / size.height
let i = drag.initial
dx = min(max(dx, -min(i.start.x, i.end.x)), 1 - max(i.start.x, i.end.x))
dy = min(max(dy, -min(i.start.y, i.end.y)), 1 - max(i.start.y, i.end.y))
line.start = NormalizedPoint(x: i.start.x + dx, y: i.start.y + dy)
line.end = NormalizedPoint(x: i.end.x + dx, y: i.end.y + dy)
```

Used in chunk 158.

Holding Shift makes the segment exactly horizontal or vertical, which is what a profile along a row or a column needs. Which of the two is decided by the larger displacement *in pixels*, not in normalized units. In a wide image a normalized step of 0.1 across is much longer than 0.1 down, and comparing normalized values would snap the wrong way.

160 *<ImageCanvas: Shift snapping 160> ≡*

```

/// With Shift held, constrains the segment to horizontal or vertical (in pixel space).
private func snapped(_ p: NormalizedPoint, anchor: NormalizedPoint, size: CGSize, enabled: Bool) ->
    NormalizedPoint {
    guard enabled else { return p }
    let dx = abs(p.x - anchor.x) * size.width
    let dy = abs(p.y - anchor.y) * size.height
    return dx >= dy ? NormalizedPoint(x: p.x, y: anchor.y) : NormalizedPoint(x: anchor.x, y: p.y)
}

```

Used in chunk 153.

The remaining helpers convert between points and normalized coordinates, and measure distances. The distance to a segment projects the point onto the segment's line, with the parameter t clamped to $[0, 1]$ so the nearest point stays between the endpoints. A degenerate segment is treated as a point.

161 *<ImageCanvas: coordinate conversions 161> ≡*

```

private func normalized(_ p: CGPoint, _ size: CGSize) -> NormalizedPoint {
    NormalizedPoint(x: p.x / size.width, y: p.y / size.height).clamped()
}

private func point(_ n: NormalizedPoint, _ size: CGSize) -> CGPoint {
    CGPoint(x: n.x * size.width, y: n.y * size.height)
}

private func distance(_ a: CGPoint, _ b: CGPoint) -> CGFloat {
    hypot(a.x - b.x, a.y - b.y)
}

private func distanceToSegment(_ p: CGPoint, _ a: CGPoint, _ b: CGPoint) -> CGFloat {
    let ab = CGPoint(x: b.x - a.x, y: b.y - a.y)
    let len2 = ab.x * ab.x + ab.y * ab.y
    guard len2 > 0 else { return distance(p, a) }
    let t = min(max(((p.x - a.x) * ab.x + (p.y - a.y) * ab.y) / len2, 0), 1)
    return distance(p, CGPoint(x: a.x + ab.x * t, y: a.y + ab.y * t))
}

```

Used in chunk 153.

The overlay is a Canvas the size of the image, so normalized coordinates become points by a single multiplication. The line has to be visible on any image, light or dark. So it is drawn twice: a wide dark stroke with a thin yellow stroke on top. The two handles differ on purpose. The filled one is the start, position 0 on the profile's horizontal axis, and the hollow one is the end. Without that, a profile read from right to left would be easy to misread.

162 *<ImageCanvas.swift: the line overlay 162> ≡*

```

/// Draws the selection line: a filled handle at the start (position 0) and a hollow one at the end.
private struct LineOverlay: View {
    let line: LineSegment

    var body: some View {
        Canvas { ctx, size in
            let a = CGPoint(x: line.start.x * size.width, y: line.start.y * size.height)
            let b = CGPoint(x: line.end.x * size.width, y: line.end.y * size.height)
            var path = Path()
            path.move(to: a)
            path.addLine(to: b)
            ctx.stroke(path, with: .color(.black.opacity(0.7)), lineWidth: 3)
            ctx.stroke(path, with: .color(.yellow), lineWidth: 1.25)

            let r: CGFloat = 5
            let startDot = Path(ellipseIn: CGRect(x: a.x - r, y: a.y - r, width: 2 * r, height: 2 * r))
            ctx.fill(startDot, with: .color(.yellow))
        }
    }
}

```

```

        ctx.stroke(startDot, with: .color(.black.opacity(0.7)), lineWidth: 1)
        let endDot = Path(ellipseIn: CGRect(x: b.x - r, y: b.y - r, width: 2 * r, height: 2 * r))
        ctx.fill(endDot, with: .color(.black.opacity(0.35)))
        ctx.stroke(endDot, with: .color(.yellow), lineWidth: 1.5)
    }
}
}

```

Used in chunk 153.

9.5 Charts

Both charts use Swift Charts' vectorized `LinePlot`, which draws thousands of points quickly enough to follow a drag. Each chart computes its data in `body`, from the session. Because the session is observable, any change to the line, the channel or the spectrum settings redraws the chart.

163 *<LineScope/Views/Charts.swift 163> ≡*

```

import Charts
import LineScopeCore
import SwiftUI

```

<Charts.swift: the profile chart 164>

<Charts.swift: the spectrum chart 166>

Root chunk, tangled to LineScope/Views/Charts.swift.

The profile's horizontal axis is the position along the line, from 0 at the start handle to 1 at the end. The number of samples depends on the variant: an image at half size gives half as many samples along the same line. So the axis label reports the count. Using normalized position lets profiles of differently sized copies line up side by side.

The vertical axis is *fixed* to the channel's natural range: `[0, 1]` for most channels, and `[0, 360]` degrees for hue. If each chart chose its own range, a faint ripple and a full-contrast square wave would look alike. With a shared, fixed domain, the charts of different variants can be compared by eye.

164 *<Charts.swift: the profile chart 164> ≡*

/// Zone 2: the selected channel sampled along the line in this variant's own pixel grid.

```

struct ProfileChart: View {
    let variant: ImageVariant
    let session: Session

    var body: some View {
        let profile = session.profile(for: variant)
        let channel = session.channelName
        Chart {
            LinePlot(profile.points, x: .value("Position", \.x), y: .value(channel, \.y))
                .foregroundStyle(channelColor)
                .lineStyle(StrokeStyle(lineWidth: 1.2))
        }
        .chartXScale(domain: 0...1)
        .chartYScale(domain: session.colorModel.range(ofChannel: session.channelIndex))
        .chartXAxisLabel("Position along line (\(profile.samples.colors.count) samples)")
        .chartYAxisLabel(channel)
        .padding(10)
    }
}

```

<ProfileChart: a color for each channel 165>

```

}

```

Used in chunk 163.

The curve takes the color of its channel, so a glance tells which channel is plotted. Channels without a natural hue (alpha, lightness, black) use the primary text color.

165 *<ProfileChart: a color for each channel 165> ≡*

```
private var channelColor: Color {
    switch (session.colorModel, session.channelIndex) {
    case (.rgba, 0): .red
    case (.rgba, 1): .green
    case (.rgba, 2): .blue
    case (.cmyk, 0): .cyan
    case (.cmyk, 1): .pink
    case (.cmyk, 2): .yellow
    case (.hsl, 0): .purple
    case (.hsl, 1): .orange
    default: .primary
    }
}
```

Used in chunk 164.

The spectrum chart transforms the same profile. Its horizontal axis ends at the Nyquist frequency, the highest frequency the samples can represent, and a dashed rule marks it. In cycles per pixel, Nyquist is about $\frac{1}{2}$ for every variant, since the samples are about one pixel apart. In cycles per line it is half the number of samples, so it moves: a copy at half size has its Nyquist limit at half the frequency. Energy that the original has above that limit either disappears (a good low-pass filter) or folds back below it (aliasing). The rule shows at once which side of the limit a feature is on.

166 *<Charts.swift: the spectrum chart 166> ≡*

/// Zone 3: amplitude spectrum of the zone-2 profile.

```
struct SpectrumChart: View {
    let variant: ImageVariant
    let session: Session

    var body: some View {
        let profile = session.profile(for: variant)
        let spectrum = session.spectrum(for: profile)
        let points = session.spectrumPoints(spectrum)
        let nyquist = nyquist(spectrum)
        Chart {
            LinePlot(points, x: .value("Frequency", \.frequency), y: .value("Amplitude", \.magnitude))
                .foregroundColor(Color.accentColor)
                .lineStyle(StrokeStyle(lineWidth: 1.2))
            if let nyquist {
                RuleMark(x: .value("Nyquist", nyquist))
                    .foregroundColor(.secondary)
                    .lineStyle(StrokeStyle(lineWidth: 1, dash: [4, 3]))
                    .annotation(position: .leading, alignment: .top) {
                        Text("Nyquist").font(.caption2).foregroundColor(.secondary)
                    }
            }
        }
        .chartXScale(domain: 0...max(nyquist ?? 0.5, 1e-9))
        .chartYScale(domain: yDomain(points))
        .chartXAxisLabel("Frequency (\(session.frequencyAxis.displayName)), FFT size
        ↪ \((spectrum.fftSize))")
        .chartYAxisLabel(session.logMagnitude ? "Amplitude (dB)" : "Amplitude")
        .padding(10)
    }
}
```

<SpectrumChart: the Nyquist frequency on the current axis 167>

<SpectrumChart: the amplitude range 168>

```
}
```

Used in chunk 163.

The last bin of the one-sided spectrum is the Nyquist bin, so its frequency, in whichever unit the axis uses, is the limit. An empty spectrum (a line too short to sample) has no limit.

167 *<SpectrumChart: the Nyquist frequency on the current axis 167> ≡*

```
private func nyquist(_ s: SpectrumResult) -> Double? {
    guard s.fftSize > 0 else { return nil }
    switch session.frequencyAxis {
    case .cyclesPerPixel: return s.cyclesPerPixel.last
    case .cyclesPerLine:  return s.cyclesPerLine.last
    }
}
```

Used in chunk 166.

In decibels, the vertical range is fixed at the bottom at -100 dB, the floor to which the session clips tiny amplitudes. The top is the next multiple of 10 dB above the peak (at least 0), so charts of different variants mostly share one scale. On the linear scale the peak sets the range, with 5% of headroom.

168 *<SpectrumChart: the amplitude range 168> ≡*

```
private func yDomain(_ points: [SpectrumPoint]) -> ClosedRange<Double> {
    let top = points.map(\.magnitude).max() ?? 1
    if session.logMagnitude {
        return -100...max(0, (top / 10).rounded(.up) * 10)
    }
    return 0...max(top * 1.05, 1e-6)
}
```

Used in chunk 166.

9.6 The parameter sheet

Operations that take parameters (the blurs, sharpening, noise reduction and every resampling method) ask for them in a sheet before running. The menu only sets the session's `pendingRequest`; the window presents this sheet for it; and Apply hands a finished `ImageOperation` to the session, which runs it on the focused variant in the background.

169 *<LineScope/Views/ParameterSheet.swift 169> ≡*

```
import LineScopeCore
import SwiftUI

/// Collects parameters for a filter or resample, then runs it on the focused variant.
struct ParameterSheet: View {
    let request: OperationRequest
    let session: Session
    @Environment(\.dismiss) private var dismiss

    <ParameterSheet: parameters and their defaults 170>

    <ParameterSheet: layout 171>

    <ParameterSheet: fields for each request 172>

    <ParameterSheet: a numeric field 173>

    <ParameterSheet: validating a resample 174>

    <ParameterSheet: building the operation 175>
}
```

Root chunk, tangled to LineScope/Views/ParameterSheet.swift.

One sheet serves every request, so it simply holds all the parameters any of them might need, with defaults that give a visible but moderate effect. A scale of 0.5 is the most instructive default for resampling: halving an image is where aliasing first shows.

170 *⟨ParameterSheet: parameters and their defaults 170⟩* ≡

```
@State private var radius = 2.0
@State private var intensity = 0.5
@State private var noiseLevel = 0.02
@State private var sharpness = 0.4
@State private var scale = 0.5
```

```
private static let maxOutputSide = 16_384
```

Used in chunk 169.

The sheet names the variant it will act on. Operations chain, so the source is often not the original, and the user should be able to see which variant it is. Return applies and Escape cancels.

171 *⟨ParameterSheet: layout 171⟩* ≡

```
var body: some View {
    let source = session.focusedVariant
    VStack(alignment: .leading, spacing: 14) {
        Text(request.title).font(.headline)
        Text("Applies to: \(source.name) \(source.buffer.width)×\(source.buffer.height)")
            .font(.caption)
            .foregroundColor(.secondary)

        Form { fields(source: source) }

        HStack {
            Spacer()
            Button("Cancel", role: .cancel) { dismiss() }
                .keyboardShortcut(.cancelAction)
            Button("Apply") {
                session.perform(operation)
                dismiss()
            }
                .keyboardShortcut(.defaultAction)
                .disabled(!isValid(source: source))
        }
    }
    .padding(20)
    .frame(width: 380)
}
```

Used in chunk 169.

Each request shows only its own fields. For resampling, the sheet shows the resulting size as the scale is typed, computed with the same rounding the resampler will use (`Resampler.outputSize`). What the user sees is exactly what the operation will produce.

172 *⟨ParameterSheet: fields for each request 172⟩* ≡

```
@ViewBuilder
private func fields(source: ImageVariant) -> some View {
    switch request {
    case .gaussianBlur, .boxBlur:
        number("Radius (px)", $radius, range: 0...100)
    case .unsharpMask:
        number("Radius (px)", $radius, range: 0...100)
        number("Intensity", $intensity, range: 0...10)
    case .noiseReduction:
        number("Noise level", $noiseLevel, range: 0...0.1)
        number("Sharpness", $sharpness, range: 0...2)
    }
```

```

case .resample:
    LabeledContent("Scale factor") {
        HStack {
            TextField("Scale", value: $scale, format: .number.precision(.fractionLength(0...4)))
                .frame(width: 80)
            Menu("Presets") {
                ForEach([0.25, 0.5, 0.75, 1.5, 2, 4], id: \.self) { s in
                    Button("×\($s.formatted())") { scale = s }
                }
            }
            .fixedSize()
        }
    }
    LabeledContent("Result") {
        Text("\($Resampler.outputSize(for: source.buffer.width, scale: scale)) ×
↪ \($Resampler.outputSize(for: source.buffer.height, scale: scale)) px")
            .monospacedDigit()
    }
}

```

Used in chunk 169.

A numeric parameter is a slider for exploring together with a text field for exact values. Both are bound to the same state.

173 *⟨ParameterSheet: a numeric field 173⟩* ≡

```

private func number(_ label: String, _ value: Binding<Double>, range: ClosedRange<Double>) -> some View {
    LabeledContent(label) {
        HStack {
            Slider(value: value, in: range)
            TextField(label, value: value, format: .number.precision(.fractionLength(0...3)))
                .frame(width: 60)
                .labelsHidden()
        }
    }
}

```

Used in chunk 169.

Only resampling can be asked for something unreasonable: a zero or negative scale, or an output so large it would exhaust memory. With four `Float` components per pixel, a 16384×16384 image already takes 4 GiB, so that is the limit.

174 *⟨ParameterSheet: validating a resample 174⟩* ≡

```

private func isValid(source: ImageVariant) -> Bool {
    guard case .resample = request else { return true }
    let w = Resampler.outputSize(for: source.buffer.width, scale: scale)
    let h = Resampler.outputSize(for: source.buffer.height, scale: scale)
    return scale > 0 && max(w, h) <= Self.maxOutputSide
}

```

Used in chunk 169.

Finally the request and the collected values become an operation, a plain value that can run anywhere, record itself in a variant's lineage, and name the new tab.

175 *⟨ParameterSheet: building the operation 175⟩* ≡

```

private var operation: ImageOperation {
    switch request {
        case .gaussianBlur: .filter(.gaussianBlur(radius: radius))
        case .boxBlur: .filter(.boxBlur(radius: radius))
        case .unsharpMask: .filter(.unsharpMask(radius: radius, intensity: intensity))
        case .noiseReduction: .filter(.noiseReduction(level: noiseLevel, sharpness: sharpness))
        case .resample(let method): .resample(method, scale: scale)
    }
}

```

}

Used in chunk 169.

9.7 The inspector

The inspector is a separate window that describes the focused variant of the most recently active document window. As explained with `MainView`, it can't find that window through focused values. The inspector *is* the key window while it is being read or scrolled, and then the focused value would be empty. It observes `ActiveSession.shared` instead, which document windows update when they become key.

176 *<LineScope/Views/InspectorView.swift 176>* $\equiv$

```
import LineScopeCore
import SwiftUI

/// Separate window describing the focused variant of the most recently active document window.
struct InspectorView: View {
    static let windowID = "inspector"

    private var active = ActiveSession.shared

    var body: some View {
        Group {
            if let session = active.session {
                InspectorContent(session: session)
            } else {
                ContentUnavailableView("No Image", systemImage: "photo", description: Text("Open an
↔ image to inspect it."))
            }
        }
        .frame(minWidth: 300, idealWidth: 340, minHeight: 320, idealHeight: 760)
    }
}
```

<InspectorView.swift: the inspector's content 177>

Root chunk, tangled to LineScope/Views/InspectorView.swift.

The content is a grouped form of read-only rows. It computes the same profile and spectrum as the charts, so its numbers always agree with what the zones show.

177 *<InspectorView.swift: the inspector's content 177>* $\equiv$

```
private struct InspectorContent: View {
    let session: Session

    var body: some View {
        let variant = session.focusedVariant
        let profile = session.profile(for: variant)
        let spectrum = session.spectrum(for: profile)
        let info = session.sourceInfo
        let original = session.original.buffer

        Form {
            <InspectorContent: the file section 178>

            <InspectorContent: the focused-image section 179>

            <InspectorContent: the operation-chain section 180>

            <InspectorContent: the line section 181>
        }
    }
}
```

```

    }
    .formStyle(.grouped)
}

<InspectorContent: row and number helpers 183>
}

Used in chunk 176.

```

The file section describes the image as it was stored, before it was converted to the float buffer all processing uses. A 16-bit or CMYK file shows that here, even though every variant is processed in the same format.

178 *<InspectorContent: the file section 178> ≡*

```

Section("File") {
    row("Name", session.fileName)
    row("Type", info.fileType)
    row("Stored size", "\((info.pixelWidth) × \((info.pixelHeight) px)")
    row("Bit depth", "\((info.bitsPerComponent) bpc, \((info.bitsPerPixel) bpp)")
    row("Color space", info.colorSpaceName)
    row("Alpha", info.hasAlpha ? "Yes" : "No")
    if let dpi = info.dpi { row("Resolution", "\((num(dpi, 0)) dpi)") }
}

Used in chunk 177.

```

For the focused variant, the scale is given separately for each axis. Rounding to whole pixels makes a nominal $\times 0.5$ of an odd-sized image slightly different horizontally and vertically.

179 *<InspectorContent: the focused-image section 179> ≡*

```

Section("Focused Image") {
    row("Tab", variant.name)
    row("Size", "\((variant.buffer.width) × \((variant.buffer.height) px)")
    row(
        "Scale vs. original",
        "×\((num(Double(variant.buffer.width) / Double(original.width), 4)) ·
        ↪ ×\((num(Double(variant.buffer.height) / Double(original.height), 4))"
    )
    row("Working format", "RGBA Float32, sRGB-encoded")
}

Used in chunk 177.

```

The operation chain lists, in order, every step from the original to this variant. This is the full history, even though each step was applied to whatever variant was focused at the time.

180 *<InspectorContent: the operation-chain section 180> ≡*

```

Section("Operation Chain") {
    Text("Original").foregroundColor(.secondary)
    ForEach(Array(variant.lineage.enumerated()), id: \.offset) { i, op in
        Text("\((i + 1). \((op.displayName)")
    }
}

Used in chunk 177.

```

The line is shared in normalized coordinates, but here it is shown in *this* variant's pixels. Its length, sample count and spacing follow, so you can see directly that a resampled copy is sampled more coarsely along the same line.

181 *<InspectorContent: the line section 181> ≡*

```

Section("Line") {
    let w = Double(variant.buffer.width), h = Double(variant.buffer.height)
    let l = session.line
    row("Start", "\(\num(l.start.x * w, 1)), \(\num(l.start.y * h, 1))) px")
    row("End", "\(\num(l.end.x * w, 1)), \(\num(l.end.y * h, 1))) px")
    row("Length", "\(\num(profile.samples.pixelLength, 1)) px")
    row("Samples", "\(\num(profile.values.count), spacing \(\num(profile.samples.spacing, 3)) px")
}

```

Used in chunk 177.

Last come summary statistics of the profile and the strongest frequency in its spectrum, given in both units of the frequency axis. With the mean removed, that peak is the dominant periodicity along the line: for a grating, its frequency.

182 *<InspectorContent: the channel-statistics section 182> ≡*

```

Section("\(\session.colorModel.displayName) · \(\session.channelName)") {
    let v = profile.values
    row("Min", num(Double(v.min() ?? 0), 4))
    row("Max", num(Double(v.max() ?? 0), 4))
    row("Mean", num(Double(v.reduce(0, +)) / Double(max(v.count, 1)), 4))
    if let k = spectrum.peakIndex {
        row("Peak frequency", "\(\num(spectrum.cyclesPerPixel[k], 4)) c/px ·
        ↪ \(\num(spectrum.cyclesPerLine[k], 2)) c/line")
        row("Peak amplitude", num(spectrum.amplitudes[k], 4))
    }
}

```

Used in chunk 177.

Values are selectable so they can be copied, and numbers are formatted with a fixed number of decimals in the user's locale.

183 *<InspectorContent: row and number helpers 183> ≡*

```

private func row(_ label: String, _ value: String) -> some View {
    LabeledContent(label) {
        Text(value).textSelection(.enabled).multilineTextAlignment(.trailing)
    }
}

private func num(_ v: Double, _ digits: Int) -> String {
    v.formatted(.number.precision(.fractionLength(digits)))
}

```

Used in chunk 177.

9.8 Test-pattern windows

A generated test pattern (Chapter 6) is shown in exactly the same analysis window as an image file. It is only a different way of producing the `ImageDocument`. Two views are needed: the window for a pattern, and a generator window where a pattern is designed before it is created.

184 *<LineScope/Views/TestPatternViews.swift 184> ≡*

```

import LineScopeCore
import SwiftUI

```

<TestPatternViews.swift: the pattern analysis window 185>

<TestPatternViews.swift: the generator window 186>

Root chunk, tangled to LineScope/Views/TestPatternViews.swift.

A pattern window receives only the pattern's `TestPatternSpec`, a small value that SwiftUI can store and restore with the window. The image must be generated from it, and that must happen *once*. Doing it in `init` or `body` would regenerate a possibly large image on every redraw, and a fresh document would also mean a fresh session, losing all derived variants. So the document lives in `@State` and is filled by a task keyed on the spec, which runs once when the window appears. The generation itself runs detached, off the main thread, while a progress indicator holds the window's place.

185 *⟨TestPatternViews.swift: the pattern analysis window 185⟩* ≡

/// Analysis window for a generated pattern. Generation runs once per window, not on every view update.

```
struct PatternWindow: View {
    let spec: TestPatternSpec
    @State private var document: ImageDocument?

    var body: some View {
        Group {
            if let document {
                MainView(document: document, fileName: spec.title)
            } else {
                ProgressView("Generating \(spec.kind.displayName)...")
                    .frame(minWidth: 720, minHeight: 560)
            }
        }
        .navigationTitle(spec.title)
        .task(id: spec) {
            let spec = spec
            document = await Task.detached(priority: .userInitiated) { ImageDocument(pattern: spec)
            }
        }
    }
}
```

Used in chunk 184.

The generator window edits a spec: the kind of pattern, its size, and its one parameter. It shows a live preview, and Create opens a pattern window for it.

186 *⟨TestPatternViews.swift: the generator window 186⟩* ≡

/// Window for choosing a pattern, its size and parameter before creating it.

```
struct TestPatternGeneratorView: View {
    static let windowID = "pattern-generator"

    @Environment(\.openWindow) private var openWindow
    @State private var spec = TestPatternSpec(kind: .zonePlate)
    @State private var preview: CGImage?

    private static let sizes = [128, 256, 512, 1024, 2048]
    private static let maxSide = 8192
    private static let maxPreviewPixels = 2048 * 2048

    var body: some View {
        HStack(alignment: .top, spacing: 20) {
            Form {
                ⟨Generator window: pattern picker and description 187⟩

                ⟨Generator window: size fields 188⟩

                ⟨Generator window: parameter field 189⟩

                ⟨Generator window: the create button 190⟩
            }
            .formStyle(.grouped)
            .frame(width: 380)
        }
    }
}
```

```

        previewView
    }
    .padding(16)
    .task(id: spec) {
        <Generator window: rendering the preview 191>
    }
}

<Generator window: the preview box 192>

<Generator window: changing the pattern kind 193>

<Generator window: validation and clamping 194>
}

```

Used in chunk 184.

Each kind of pattern carries a sentence on what it is good for. Showing that under the picker turns the list into a small catalogue of test targets.

187 *<Generator window: pattern picker and description 187> ≡*

```

Picker("Pattern", selection: kindBinding) {
    ForEach(TestPatternKind.allCases) { Text($0.displayName).tag($0) }
}
Text(spec.kind.purpose)
    .font(.caption)
    .foregroundStyle(.secondary)
    .fixedSize(horizontal: false, vertical: true)

```

Used in chunk 186.

The size is typed or picked from square presets. Grouping separators are turned off, so a width reads 2048 and not 2,048.

188 *<Generator window: size fields 188> ≡*

```

LabeledContent("Size") {
    HStack {
        TextField("Width", value: $spec.width, format: .number.grouping(.never))
            .frame(width: 64)
        Text("×")
        TextField("Height", value: $spec.height, format: .number.grouping(.never))
            .frame(width: 64)
        Menu("Presets") {
            ForEach(Self.sizes, id: \.self) { s in
                Button("\($s) × \($s)") { spec.width = s; spec.height = s }
            }
        }
        .fixedSize()
    }
    .labelsHidden()
}

```

Used in chunk 186.

The parameter's name and range come from the pattern kind itself. Kinds without a parameter show nothing. The noise seed gets no slider: neighboring seeds are unrelated, so dragging through them means nothing.

189 *⟨Generator window: parameter field 189⟩* ≡

```

if let p = spec.kind.parameter {
    LabeledContent(p.name) {
        HStack {
            if spec.kind != .whiteNoise {
                Slider(value: $spec.parameter, in: p.range)
            }
            TextField(p.name, value: $spec.parameter, format: .number.precision(.fractionLength(0...4)))
                .frame(width: 70)
                .labelsHidden()
        }
    }
}

```

Used in chunk 186.

Create opens a new pattern window *by value*. SwiftUI finds the window group for `TestPatternSpec`, and creating the same spec twice brings the existing window forward instead of opening a duplicate. A typed parameter can lie outside the slider’s range, so it is clamped first.

190 *⟨Generator window: the create button 190⟩* ≡

```

HStack {
    Spacer()
    Button("Create") { openWindow(value: clamped(spec)) }
        .keyboardShortcut(.defaultAction)
        .disabled(!isValid)
}

```

Used in chunk 186.

The preview must show what the pattern will really look like. Generating a small version of the pattern would be wrong: a zone plate made at 256×256 is a different image from the center of one made at 1024×1024 , with different frequencies and different aliasing. So the preview generates the full pattern and crops its central 256×256 pixels, which are shown at 1:1. Huge patterns are not previewed at all.

The task is keyed on the spec, so SwiftUI cancels it whenever the spec changes. The short sleep debounces typing and slider drags: only a spec that stays unchanged for 80 ms is rendered.

191 *⟨Generator window: rendering the preview 191⟩* ≡

```

guard isValid, spec.width * spec.height <= Self.maxPreviewPixels else { preview = nil; return }
// Generate the real pattern and show its central pixels at 1:1, so aliasing looks as it will
// in the analysis window.
let s = clamped(spec)
try? await Task.sleep(for: .milliseconds(80))
guard !Task.isCancelled else { return }
preview = await Task.detached {
    guard let full = TestPatternGenerator.make(s).makeCGImage() else { return nil }
    let w = min(s.width, 256), h = min(s.height, 256)
    let rect = CGRect(x: (s.width - w) / 2, y: (s.height - h) / 2, width: w, height: h)
    return full.cropping(to: rect)
}.value

```

Used in chunk 186.

The preview box always has the same size, so the window doesn’t jump as specs change. As in the image canvas, interpolation is off: a preview pixel is an image pixel.

192 *⟨Generator window: the preview box 192⟩* ≡

```

private var previewView: some View {
    VStack(spacing: 6) {
        Group {
            if let preview {
                Image(decorative: preview, scale: 1).interpolation(.none)
            } else {

```

```

        Color.clear
    }
}
.frame(width: 256, height: 256)
.background(Color(nsColor: .underPageBackgroundColor))
.clipShape(RoundedRectangle(cornerRadius: 4))
Text(spec.width * spec.height <= Self.maxPreviewPixels
    ? "Preview: center 256 × 256 px at 1:1" : "Too large to preview")
    .font(.caption)
    .foregroundStyle(.secondary)
}
}

```

Used in chunk 186.

Parameters mean different things for different patterns: a frequency in cycles per pixel for a chirp, a period in pixels for bars, a number of spokes for the star. Carrying the value over when the kind changes would produce nonsense, such as a checkerboard with cells 0.5 pixels wide. So the custom binding rebuilds the spec, keeping the size and taking the new kind's default parameter.

193 *⟨Generator window: changing the pattern kind 193⟩* ≡

```

/// Resets the parameter to the new pattern's default when the kind changes.
private var kindBinding: Binding<TestPatternKind> {
    Binding(
        get: { spec.kind },
        set: { spec = TestPatternSpec(kind: $0, width: spec.width, height: spec.height) }
    )
}

```

Used in chunk 186.

Sizes are limited to 8192 pixels per side, and the parameter is limited to its kind's range.

194 *⟨Generator window: validation and clamping 194⟩* ≡

```

private var isValid: Bool {
    (1...Self.maxSide).contains(spec.width) && (1...Self.maxSide).contains(spec.height)
}

private func clamped(_ s: TestPatternSpec) -> TestPatternSpec {
    var s = s
    if let p = s.kind.parameter {
        s.parameter = min(max(s.parameter, p.range.lowerBound), p.range.upperBound)
    }
    return s
}

```

Used in chunk 186.

Chapter 10

Menus

Everything the user can do to an image goes through the menu bar: derive a copy with a filter or a resampler, create a test pattern, split a zone, export, and open the inspector. SwiftUI describes menus with a `Commands` value, attached to the app's scenes in `LineScopeApp` (Chapter 8).

195 *<LineScope/Commands/LineScopeCommands.swift 195> ≡*

```
import LineScopeCore
import SwiftUI

struct LineScopeCommands: Commands {
    <Commands: finding the target session 196>

    var body: some Commands {
        <Commands: File menu, test patterns 197>

        <Commands: File menu, export and close 198>

        <Commands: Filter menu 199>

        <Commands: Resample menu 200>

        <Commands: View menu 201>
    }
}
```

Root chunk, tangled to LineScope/Commands/LineScopeCommands.swift.

10.1 Which document does a menu act on?

Menus are global, but their actions are for one document: the one in the frontmost window. SwiftUI's way to connect the two is a *focused value*. Each document window publishes its `Session` with `focusedSceneValue`, and `@FocusedValue` here reads the value of whichever window is key.

That alone has one bad consequence. The inspector is a separate window, and while the user works in it, *it* is the key window. It publishes no session, so the focused value is `nil`, and every menu item would turn gray at the moment the inspector is showing the image one wants to filter. So the menus fall back to `ActiveSession.shared`, the session of the most recently key document window, which the document windows keep up to date (Chapter 9). When a document window is key, the two agree. When the inspector is key, the fallback names the document it describes.

196 *⟨Commands: finding the target session 196⟩* ≡

```
@FocusedValue(\.session) private var focusedSession
@Environment(\.openWindow) private var openWindow

/// Falls back to the last active document so the menus keep working while the inspector is key.
private var session: Session? { focusedSession ?? ActiveSession.shared.session }

Used in chunk 195.
```

Every action below starts with `session?..` With no document open, the optional chain makes an action do nothing, and a `.disabled(session == nil)` makes that visible.

10.2 The File menu

Test patterns don't need a document: choosing one opens a new window for it. The value passed to `openWindow(value:)` is the pattern's specification with default size and parameter. SwiftUI finds the `WindowGroup` that presents that type (Chapter 9). The `Custom...` item opens the generator window, where size and parameter can be set first.

197 *⟨Commands: File menu, test patterns 197⟩* ≡

```
CommandGroup(after: .newItem) {
    Menu("New Test Pattern") {
        ForEach(TestPatternKind.allCases) { kind in
            Button(kind.displayName) { openWindow(value: TestPatternSpec(kind: kind)) }
        }
        Divider()
        Button("...Custom") { openWindow(id: TestPatternGeneratorView.windowID) }
            .keyboardShortcut("n", modifiers: [.command, .option])
    }
}

Used in chunk 195.
```

Export and Close Tab both act on the *focused variant*: the tab last selected or clicked in any zone. The original can't be closed, because every other variant is derived from it and every pane falls back to it.

198 *⟨Commands: File menu, export and close 198⟩* ≡

```
CommandGroup(after: .saveItem) {
    Button("Export ...Image") { session?.exportFocused() }
        .keyboardShortcut("e", modifiers: [.command, .shift])
        .disabled(session == nil)
    Button("Close Tab") {
        if let session { session.closeVariant(session.focusedVariantID) }
    }
    .keyboardShortcut(.delete, modifiers: [.command])
    .disabled(session?.focusedVariant.isOriginal ?? true)
}

Used in chunk 195.
```

10.3 Filters and resampling

A menu item either runs its operation at once or asks for parameters first. Items that need parameters end in an ellipsis, as the macOS convention asks. They set the session's `pendingRequest`, and the document window reacts by presenting the parameter sheet. The menu never shows user interface itself. The others call `perform`, which runs the operation in the background on the focused variant and adds the result as a new tab. Since the source is whichever tab is focused, operations chain naturally: blur a copy, then shrink the blurred copy.

The filters are grouped as the people using them think of them: smoothing, then sharpening and edge detection, then point operations on the colors.

199 *⟨Commands: Filter menu 199⟩* ≡

```

CommandMenu("Filter") {
  Group {
    Button("Gaussian ...Blur") { session?.pendingRequest = .gaussianBlur }
    Button("Box ...Blur") { session?.pendingRequest = .boxBlur }
    Button("Median 3×3") { session?.perform(.filter(.median)) }
    Button("Noise ...Reduction") { session?.pendingRequest = .noiseReduction }
    Divider()
    Button("Sharpen (Unsharp Mask)...") { session?.pendingRequest = .unsharpMask }
    Button("Edge Detect (Sobel)") { session?.perform(.filter(.sobel)) }
    Button("Laplacian") { session?.perform(.filter(.laplacian)) }
    Button("Emboss") { session?.perform(.filter(.emboss)) }
    Divider()
    Button("Grayscale") { session?.perform(.filter(.grayscale)) }
    Button("Invert") { session?.perform(.filter(.invert)) }
  }
  .disabled(session == nil)
}

```

Used in chunk 195.

The Resample menu is generated from `ResampleMethod.allCases`. A method added to that enumeration therefore appears in the menu without any change here. Every method needs a scale factor, so every item opens the sheet.

200 *⟨Commands: Resample menu 200⟩* ≡

```

CommandMenu("Resample") {
  ForEach(ResampleMethod.allCases) { method in
    Button("\(method.displayName)...") { session?.pendingRequest = .resample(method) }
  }
  .disabled(session == nil)
}

```

Used in chunk 195.

10.4 The View menu

The View menu repeats the split and unsplit buttons of the zone headers, so they have menu items for keyboard users. It also holds the display option “Actual Device Pixels”. That option lives in the session, so the menu adapts it with a `Binding` whose getter and setter go through the optional session. The inspector gets the shortcut Command-Option-I, which is conventional for inspectors on macOS.

201 *⟨Commands: View menu 201⟩* ≡

```

CommandGroup(before: .toolbar) {
  ForEach(Zone.allCases) { zone in
    Button("Split \(zone.title)") { session?.split(zone) }
    .disabled(session == nil)
  }
  ForEach(Zone.allCases) { zone in
    Button("Unsplit \(zone.title)") { session?.unsplit(zone) }
    .disabled((session?.panes[zone.rawValue].count ?? 1) < 2)
  }
  Divider()
  Toggle("Actual Device Pixels", isOn: Binding(
    get: { session?.useDevicePixels ?? false },
    set: { session?.useDevicePixels = $0 }
  ))
  .disabled(session == nil)
  Button("Reset Line") { session?.line = .initial }
  .disabled(session == nil)
  Divider()
}

```

```
Button("Show Inspector") { openWindow(id: InspectorView.windowID) }  
    .keyboardShortcut("i", modifiers: [.command, .option])  
Divider()  
}
```

Used in chunk 195.

Part II

LineScope for Windows

Chapter 11

The Windows Port

Everything so far described LineScope as a macOS program. This part describes its second life: a native Windows version, written in C# for .NET 9 with the Windows Presentation Foundation (WPF) and the Windows Imaging Component (WIC) [8]. It has the same three zones, the same shared line, the same chained operations, split panes, test patterns and inspector. More importantly, it computes the same numbers.

11.1 Why a native port

LineScope is a tool for looking closely at pixels, and that purpose rules out the easy ways of running it on Windows. A cross-platform toolkit would put a layer of its own between the image and the screen, with its own scaling and color handling, exactly where the program must be precise. Rewriting the app in a web technology would have the same problem, and would lose the fast numeric loops. The Swift code itself can't be reused: SwiftUI, Core Image, vDSP and Swift Charts exist only on Apple platforms.

So the Windows version is a port in the old sense: the same design, rewritten in the native language and framework of the other platform. C# is close enough to Swift that most files translate almost line by line. It has value types, generics, closures, `async/await`, and pattern matching. WPF draws with Direct3D and knows about per-monitor DPI, so one image pixel can be put on exactly one screen pixel.

11.2 The solution

A .NET *solution* groups projects, much as the Xcode project groups a package and an app. The Windows solution has three, one for each part of the macOS code base:

macOS	Windows	Contents
Packages/LineScopeCore	Windows/LineScope.Core	numeric core, no user interface
Packages/LineScopeCore/Tests	Windows/LineScope.Core.Tests	unit tests (xUnit)
LineScope (the app)	Windows/LineScope.App	documents, session, windows, menus

The core project targets plain `net9.0` and references nothing from WPF, just as `LineScopeCore` imports no SwiftUI. The app project depends on it; the test project depends only on the core. Inside each project the files mirror the Swift ones: `PixelBuffer.swift` becomes `PixelBuffer.cs`, `Resampling.swift` becomes `Resampling.cs`, and so on, with the type and member names kept and only the capitalization adapted to .NET conventions (`TestPatternGenerator.make` becomes `TestPatternGenerator.Make`, the property `lineage` becomes `Lineage`).

Where the macOS version leaned on an Apple framework, the port needs a replacement. The table summarizes them; the chapters that follow explain each one where it occurs.

macOS	Windows
SwiftUI <code>DocumentGroup(viewing:)</code>	one <code>MainWindow</code> per document, opened by the Open dialog, drag and drop or the command line; the app quits with its last document window
Core Graphics and ImageIO decoding	WIC through <code>BitmapDecoder</code> , converted to premultiplied 8-bit BGRA, embedded ICC profiles converted to sRGB
Core Image filters	CPU implementations in <code>Filters.cs</code>
vDSP real FFT	an iterative radix-2 complex FFT, with the same normalization
Swift Charts	<code>ChartView</code> , drawn with WPF's <code>DrawingContext</code>
<code>@Observable</code> Session	a plain class with a <code>Changed</code> event carrying flags
<code>@FocusedValue</code> menus	a menu bar per window, bound to that window's session
points and Retina pixels	1 image pixel = 1 device-independent pixel (1/96 inch), and a per-monitor DPI-aware process

11.3 Same algorithms, different plumbing

One rule governs the whole port: *the algorithms stay identical; only the plumbing around them changes*. The resampling kernels and their widening when downscaling, the footprint weights of the area method, the bilinear line sampler with its $\lceil L \rceil + 1$ samples, the color conversions, the window functions and the single-sided amplitude normalization, the test-pattern formulas and the `SplitMix64` generator are all transcribed, not reinvented. The same inputs therefore give the same outputs, to floating-point precision, on both systems, and a spectrum measured on a Mac can be compared with one measured on a PC.

Two pieces could not be transcribed, because on macOS they were not `LineScope`'s code at all. The FFT came from vDSP, and the port brings its own; since only the transform changes and not what is done with its output, the spectra still agree. The Gaussian, box, median, unsharp-mask and noise reduction filters came from Core Image, whose exact kernels are not documented; the port reimplements them on the CPU following Core Image's documented behavior, so their results are close to, but not bit-identical with, the macOS ones. Chapter 12 gives the details.

Everything above the core is plumbing, and there the port follows the conventions of Windows rather than imitating the Mac: a menu bar in every window, keyboard shortcuts with `Ctrl`, the standard file dialogs, and an application that ends when its last document window closes.

11.4 How to read this part

This part assumes the first. It concentrates on what the port *changes* and why, and refers back to the macOS chapters for the theory. Where a C# file is a faithful transcription of a Swift file, its chunks appear with only a short note and a reference to the chapter that explains them; where it differs, the difference is discussed. The chunk names all begin with “Win”, so the index keeps the two versions apart.

The chapters follow the order of the first part.

Chapter 12

The numeric core, `LineScope.Core`: the pixel buffer, resampling, filters, sampling, color models, the spectrum with its FFT, and the test patterns. It is the counterpart of Chapters 2 to 6.

Chapter 13

The xUnit tests, the counterpart of Chapter 7, with new tests for the FFT and the CPU filters.

Chapter 14

Documents, variants and the session, the counterpart of Chapter 8.

Chapter 15

The windows, zones, panes, image canvas, charts, dialogs and the inspector, and also the menus, which live in each window. It is the counterpart of Chapters 9 and 10.

The rest of this chapter covers the build files and the application object, the counterpart of the scenes in `LineScopeApp` (Chapter 8).

11.5 Building

With the .NET 9 SDK installed, three commands in the `Windows` directory build the program, run it (optionally on an image given on the command line), and run the tests:

```
dotnet build LineScope.sln
dotnet run --project LineScope.App [-- image.png]
dotnet test LineScope.Core.Tests
```

Visual Studio 2022 can open the same solution; nothing in the build depends on it.

11.5.1 The solution file

The solution file lists the three projects and maps the solution's configurations onto theirs. Its format is older than .NET and not meant to be written by hand: it is maintained with `dotnet sln add`. The long identifier `FAE04EC0-...` is the project-type code for C#; the other three are the projects' own. All six solution configurations (Debug and Release, each for Any CPU, x64 and x86) build the projects as Any CPU, since the code is pure managed code and runs on any processor. The file starts with an empty line, as Visual Studio writes it.

202 $\langle Windows/LineScope.sln\ 202 \rangle \equiv$

```
Microsoft Visual Studio Solution File, Format Version 12.00
# Visual Studio Version 17
VisualStudioVersion = 17.0.31903.59
MinimumVisualStudioVersion = 10.0.40219.1
Project("{FAE04EC0-301F-11D3-BF4B-00C04F79EFBC}") = "LineScope.Core",
    ↪ "LineScope.Core\LineScope.Core.csproj", "{D9919667-696E-456F-AB2B-5F9D8C0D562E}"
EndProject
Project("{FAE04EC0-301F-11D3-BF4B-00C04F79EFBC}") = "LineScope.Core.Tests",
    ↪ "LineScope.Core.Tests\LineScope.Core.Tests.csproj", "{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}"
EndProject
Project("{FAE04EC0-301F-11D3-BF4B-00C04F79EFBC}") = "LineScope.App",
    ↪ "LineScope.App\LineScope.App.csproj", "{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}"
EndProject
Global
    GlobalSection(SolutionConfigurationPlatforms) = preSolution
        Debug|Any CPU = Debug|Any CPU
        Debug|x64 = Debug|x64
        Debug|x86 = Debug|x86
        Release|Any CPU = Release|Any CPU
        Release|x64 = Release|x64
        Release|x86 = Release|x86
    EndGlobalSection
    GlobalSection(ProjectConfigurationPlatforms) = postSolution
        {D9919667-696E-456F-AB2B-5F9D8C0D562E}.Debug|Any CPU.ActiveCfg = Debug|Any CPU
        {D9919667-696E-456F-AB2B-5F9D8C0D562E}.Debug|Any CPU.Build.0 = Debug|Any CPU
        {D9919667-696E-456F-AB2B-5F9D8C0D562E}.Debug|x64.ActiveCfg = Debug|Any CPU
        {D9919667-696E-456F-AB2B-5F9D8C0D562E}.Debug|x64.Build.0 = Debug|Any CPU
        {D9919667-696E-456F-AB2B-5F9D8C0D562E}.Debug|x86.ActiveCfg = Debug|Any CPU
        {D9919667-696E-456F-AB2B-5F9D8C0D562E}.Debug|x86.Build.0 = Debug|Any CPU
```

```

{D9919667-696E-456F-AB2B-5F9D8C0D562E}.Release|Any CPU.ActiveCfg = Release|Any CPU
{D9919667-696E-456F-AB2B-5F9D8C0D562E}.Release|Any CPU.Build.0 = Release|Any CPU
{D9919667-696E-456F-AB2B-5F9D8C0D562E}.Release|x64.ActiveCfg = Release|Any CPU
{D9919667-696E-456F-AB2B-5F9D8C0D562E}.Release|x64.Build.0 = Release|Any CPU
{D9919667-696E-456F-AB2B-5F9D8C0D562E}.Release|x86.ActiveCfg = Release|Any CPU
{D9919667-696E-456F-AB2B-5F9D8C0D562E}.Release|x86.Build.0 = Release|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Debug|Any CPU.ActiveCfg = Debug|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Debug|Any CPU.Build.0 = Debug|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Debug|x64.ActiveCfg = Debug|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Debug|x64.Build.0 = Debug|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Debug|x86.ActiveCfg = Debug|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Debug|x86.Build.0 = Debug|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Release|Any CPU.ActiveCfg = Release|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Release|Any CPU.Build.0 = Release|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Release|x64.ActiveCfg = Release|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Release|x64.Build.0 = Release|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Release|x86.ActiveCfg = Release|Any CPU
{8A2D2E5C-6DEC-48FF-9E22-F98D41D13F1D}.Release|x86.Build.0 = Release|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Debug|Any CPU.ActiveCfg = Debug|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Debug|Any CPU.Build.0 = Debug|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Debug|x64.ActiveCfg = Debug|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Debug|x64.Build.0 = Debug|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Debug|x86.ActiveCfg = Debug|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Debug|x86.Build.0 = Debug|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Release|Any CPU.ActiveCfg = Release|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Release|Any CPU.Build.0 = Release|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Release|x64.ActiveCfg = Release|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Release|x64.Build.0 = Release|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Release|x86.ActiveCfg = Release|Any CPU
{91401FF6-44A6-4F7B-9D1D-241E0FE1896B}.Release|x86.Build.0 = Release|Any CPU
EndGlobalSection
GlobalSection(SolutionProperties) = preSolution
    HideSolutionNode = FALSE
EndGlobalSection
EndGlobal

```

Root chunk, tangled to Windows/LineScope.sln.

11.5.2 The app project

The app's project file is short, because the SDK supplies the defaults. `WinExe` builds a windowed program without a console. The target `net9.0-windows` and `UseWPF` bring in WPF. `Nullable` turns on the compiler's null-state analysis, which gives C# much of the safety of Swift's optionals, and `ImplicitUsings` imports the common namespaces (`System`, `System.Linq`, `System.Collections.Generic`, `System.Threading.Tasks`) in every file. The assembly is named `LineScope`, so the program is `LineScope.exe`, and it references the core project.

The core's project file, with its one notable setting (optimization even in Debug, the counterpart of the Swift package's `-O`), is in Chapter 12; the test project is in Chapter 13.

203 `<Windows/LineScope.App/LineScope.App.csproj 203> ≡`

```

<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net9.0-windows</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UseWPF>true</UseWPF>
    <RootNamespace>LineScope.App</RootNamespace>
    <AssemblyName>LineScope</AssemblyName>
    <ApplicationManifest>app.manifest</ApplicationManifest>
    <Version>0.1</Version>
  </PropertyGroup>

```

```

    <Product>LineScope</Product>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\LineScope.Core\LineScope.Core.csproj" />
  </ItemGroup>

</Project>

```

Root chunk, tangled to Windows/LineScope.App/LineScope.App.csproj.

11.5.3 The application manifest

The manifest matters more than its size suggests. View > Actual Device Pixels promises one image pixel per screen pixel. A process that doesn't declare itself DPI-aware is drawn by Windows at 96 dots per inch and then stretched as a bitmap to the monitor's scale factor, blurring exactly the pixels the user is studying. Declaring *per-monitor* awareness tells Windows that WPF handles scaling itself, and keeps doing so when a window moves to a monitor with a different scale. The `dpiAwareness` element with `PerMonitorV2` is read by Windows 10 (version 1703) and later; the older `dpiAware` element, `true/PM`, is the fallback for earlier systems. The `supportedOS` identifier declares compatibility with Windows 10 and 11, so the program gets their behavior rather than a compatibility mode.

204 `<Windows/LineScope.App/app.manifest 204> ≡`

```

<?xml version="1.0" encoding="utf-8"?>
<assembly manifestVersion="1.0" xmlns="urn:schemas-microsoft-com:asm.v1">
  <assemblyIdentity version="0.1.0.0" name="LineScope.app"/>
  <application xmlns="urn:schemas-microsoft-com:asm.v3">
    <windowsSettings>
      <!-- Per-monitor DPI awareness, so "Actual Device Pixels" maps one image pixel to one screen
           pixel. -->
      <dpiAware xmlns="http://schemas.microsoft.com/SMI/2005/WindowsSettings">true/PM</dpiAware>
      <dpiAwareness
        ↪ xmlns="http://schemas.microsoft.com/SMI/2016/WindowsSettings">PerMonitorV2</dpiAwareness>
    </windowsSettings>
  </application>
  <compatibility xmlns="urn:schemas-microsoft-com:compatibility.v1">
    <application>
      <supportedOS Id="{8e0f7a12-bfb3-4fe8-b9a5-48fd50a15a9a}"/>
    </application>
  </compatibility>
</assembly>

```

Root chunk, tangled to Windows/LineScope.App/app.manifest.

11.5.4 Theme information

The last build file comes from the WPF project template, unchanged. It tells WPF where to look for theme-specific resources. LineScope defines no custom controls with themed templates, so the attribute only states that there are no theme dictionaries.

205 `<Windows/LineScope.App/AssemblyInfo.cs 205> ≡`

```

using System.Windows;

[assembly:ThemeInfo(
    ResourceDictionaryLocation.None,           //where theme specific resource dictionaries are located
                                              //(used if a resource is not found in the page,
                                              // or application resource dictionaries)
    ResourceDictionaryLocation.SourceAssembly //where the generic resource dictionary is located
                                              //(used if a resource is not found in the page,
                                              // app, or any theme specific resource dictionaries)
)]

```

```
)}
```

Root chunk, tangled to Windows/LineScope.App/AssemblyInfo.cs.

11.6 The application object

A WPF program is an `Application` object, usually described in two halves: a XAML file for its declarative part and a C# file for its code. All LineScope's windows are built in C# (Chapter 15), so `App.xaml` is the only XAML in the program and it does little.

It sets two things. The first is the shutdown mode. By default a WPF application ends when its main window, the first one opened, closes. LineScope has no main window: it has any number of document windows plus two utility windows, the inspector and the pattern generator. With `OnExplicitShutdown` the application ends only when its code says so, which it does when the last *document* window closes. An inspector left open doesn't keep the program alive, and closing the first document while others are open doesn't end it. The second is a pair of text styles shared by the views: a small gray caption and a semibold section header. The caption uses the system's gray-text brush as a `DynamicResource`, so it follows the Windows colors, including high-contrast themes.

There is no `StartupUri`: the first window is created in code.

206 `<Windows/LineScope.App/App.xaml 206> ≡`

```
<Application x:Class="LineScope.App.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    ShutdownMode="OnExplicitShutdown">
  <Application.Resources>
    <Style x:Key="CaptionText" TargetType="TextBlock">
      <Setter Property="FontSize" Value="11" />
      <Setter Property="Foreground" Value="{DynamicResource {x:Static
↪ SystemColors.GrayTextBrushKey}}" />
    </Style>
    <Style x:Key="SectionHeader" TargetType="TextBlock">
      <Setter Property="FontWeight" Value="SemiBold" />
      <Setter Property="Margin" Value="0,12,0,4" />
    </Style>
  </Application.Resources>
</Application>
```

Root chunk, tangled to Windows/LineScope.App/App.xaml.

The code half plays the role of `LineScopeApp` and its four scenes (Chapter 8). `SwiftUI` created and tracked the windows of a scene by itself; here the application object does it.

207 `<Windows/LineScope.App/App.xaml.cs 207> ≡`

```
using System.IO;
using System.Windows;
using LineScope.App.Model;
using LineScope.App.Views;
using LineScope.Core;
using Microsoft.Win32;

namespace LineScope.App;

/// <summary>
/// Application entry: one <see cref="MainWindow"/> per document (image file or test pattern), plus the
↪ shared
/// inspector and test-pattern generator windows. The app quits when the last document window closes.
/// </summary>
public partial class App : Application
{
  <Win App: shared state 208>
```

```

    <Win App: startup 209>
    <Win App: creating a document window 210>
    <Win App: the Open dialog 211>
    <Win App: opening a file 212>
    <Win App: opening a test pattern 213>
    <Win App: the inspector and the pattern generator 214>
}

```

Root chunk, tangled to Windows/LineScope.App/App.xaml.cs.

The application keeps the list of open document windows and the two single utility windows, which are `null` while closed. `Current` hides the base class's property of the same name with one that has the right type, so views can write `App.Current.OpenFile(...)` without a cast. The file filter lists the formats WIC decodes out of the box, plus HEIF and WebP, which it decodes when the Windows codec extensions are installed; “All files” lets the user try anything else.

208 <Win App: shared state 208> ≡

```

public static new App Current => (App)Application.Current;

private readonly List<MainWindow> _documentWindows = [];
private InspectorWindow? _inspector;
private TestPatternWindow? _generator;

public const string OpenFilter =
    ↪ "Images|*.png;*.jpg;*.jpeg;*.jpe;*.jfif;*.tif;*.tiff;*.bmp;*.dib;*.gif;*.ico;*.wdp;*.jxr;*.heic;*.heif;*.webp;*.d
    ↪ files|*.*";

```

Used in chunk 207.

At startup, every command-line argument that names an existing file is opened in a window of its own. This is how `LineScope` is used from Explorer’s “Open with” and from a shortcut onto which images are dropped. Without files the app shows one empty document window. On macOS the document group shows an Open panel instead; an empty window with the menu bar and a hint is the more familiar Windows behavior (it offers Open and New Test Pattern buttons), and it gives the user a window to drop files on.

209 <Win App: startup 209> ≡

```

protected override void OnStartup(StartupEventArgs e)
{
    base.OnStartup(e);
    var files = e.Args.Where(File.Exists).ToList();
    if (files.Count == 0)
    {
        NewDocumentWindow().Show();
        return;
    }
    foreach (var path in files) OpenFile(path, reuse: null);
}

```

Used in chunk 207.

Every document window, whether it will hold a file or a pattern, is created here, so that the application can count them. When the count reaches zero the application shuts down, which is the counterpart of the explicit shutdown mode above.

210 *⟨Win App: creating a document window 210⟩* ≡

```

public MainWindow NewDocumentWindow()
{
    var window = new MainWindow();
    _documentWindows.Add(window);
    window.Closed += (_, _) =>
    {
        _documentWindows.Remove(window);
        if (_documentWindows.Count == 0) Shutdown();
    };
    return window;
}

```

Used in chunk 207.

File > Open allows several files at once. The first one goes into the window the command came from if that window is still empty, so opening a file right after launch doesn't leave an empty window behind; the others get new windows. Drag and drop onto a window calls `OpenFile` the same way (Chapter 15).

211 *⟨Win App: the Open dialog 211⟩* ≡

```

/// <summary>Shows the Open dialog and opens every chosen file; the first one goes into <paramref
    ↪ name="reuse"/> if it is empty.</summary>
public void ShowOpenDialog(MainWindow? reuse)
{
    var dialog = new OpenFileDialog { Filter = OpenFilter, Multiselect = true, Title = "Open Image" };
    if (dialog.ShowDialog(reuse) != true) return;
    foreach (var path in dialog.FileNames)
    {
        OpenFile(path, reuse);
        reuse = null;
    }
}

```

Used in chunk 207.

Opening a file must not freeze the interface, because decoding a large photograph and converting it to floats takes a noticeable moment. The window is shown at once with a loading message, and the decoding runs on a thread-pool thread with `Task.Run`. `ImageDocument.Load` is written to be safe off the UI thread (Chapter 14). The `await` resumes on the UI thread, where the window receives the document and builds its session.

The method is `async void`, the form C# reserves for fire-and-forget event handlers: nobody awaits it, so it catches its own exceptions. A file that can't be decoded produces a message box. The window that was opened for it is then closed, unless it was the empty window being reused or it is the only window left, in which case closing it would end the application.

212 *⟨Win App: opening a file 212⟩* ≡

```

public async void OpenFile(string path, MainWindow? reuse)
{
    var window = reuse is { HasDocument: false } ? reuse : NewDocumentWindow();
    window.ShowLoading($"Opening {Path.GetFileName(path)}...");
    if (!window.IsVisible) window.Show();
    window.Activate();
    try
    {
        var document = await Task.Run(() => ImageDocument.Load(path));
        window.LoadDocument(document, Path.GetFileName(path));
    }
    catch (Exception ex)
    {
        window.ShowEmpty();
        MessageBox.Show(window, $"{Path.GetFileName(path)} couldn't be opened.\n\n{ex.Message}",
            ↪ "LineScope",

```

```

        MessageBoxButton.OK, MessageBoxImage.Warning);
    // Keep the empty window if it is the only one, so the app doesn't quit.
    if (reuse != window && _documentWindows.Count > 1) window.Close();
}
}

```

Used in chunk 207.

A test pattern gets the same kind of window as a file, exactly as on macOS, where `ImageDocument(pattern:)` let a pattern be shown by the same `MainView`. Generating a large pattern also takes time, so it runs in the background too. A pattern window isn't restored after a relaunch as it is on macOS; Windows has no such mechanism for ordinary programs.

213 *⟨Win App: opening a test pattern 213⟩* ≡

```

/// <summary>Generates a test pattern in the background and shows it in a new analysis window.</summary>
public async void OpenPattern(TestPatternSpec spec)
{
    var window = NewDocumentWindow();
    window.Title = spec.Title;
    window.ShowLoading($"Generating {spec.Kind.DisplayName()}...");
    window.Show();
    var document = await Task.Run(() => ImageDocument.FromPattern(spec));
    window.LoadDocument(document, spec.Title);
}

```

Used in chunk 207.

The inspector and the pattern generator are single windows, like SwiftUI's window scenes. Asking for one that is open brings it to the front; asking for one that was closed creates it again. The inspector shows whichever session is active (Chapter 14), so it needs no owner. The generator opens a little below and to the right of the window it was requested from, so it doesn't appear somewhere unrelated on a multi-monitor desktop.

214 *⟨Win App: the inspector and the pattern generator 214⟩* ≡

```

public void ShowInspector()
{
    if (_inspector is null)
    {
        _inspector = new InspectorWindow();
        _inspector.Closed += (_, _) => _inspector = null;
        _inspector.Show();
    }
    _inspector.Activate();
}

public void ShowPatternGenerator(Window? owner)
{
    if (_generator is null)
    {
        _generator = new TestPatternWindow();
        if (owner is not null)
        {
            _generator.WindowStartupLocation = WindowStartupLocation.Manual;
            _generator.Left = owner.Left + 60;
            _generator.Top = owner.Top + 60;
        }
        _generator.Closed += (_, _) => _generator = null;
        _generator.Show();
    }
    _generator.Activate();
}

```

Used in chunk 207.

Chapter 12

The Numeric Core in C#

The Windows port of LineScope is written in C# for .NET 9, with a WPF user interface. Like the macOS program it is split in two, and this chapter covers the first half: `LineScope.Core`, a class library with no user interface that ports the Swift package `LineScopeCore` file by file. `PixelBuffer.swift` becomes `PixelBuffer.cs`, `Resampling.swift` becomes `Resampling.cs`, and so on, and the types keep their names in PascalCase.

The mathematics is not repeated here. It was derived in the chapters on pixels (Chapter 2), resampling (Chapter 3), filters (Chapter 4), analysis (Chapter 5) and test patterns (Chapter 6), and it holds unchanged. This chapter concentrates on the three places where the port had to differ, because the macOS frameworks it relied on don't exist on Windows:

- the bridge to 8-bit images, which is premultiplied BGRA instead of Core Graphics' RGBA (Section 12.2.2);
- the filters that Core Image supplied, now written out on the CPU (Section 12.4);
- the FFT that vDSP supplied, now a hand-written radix-2 transform (Section 12.7.1).

Everywhere else the rule is the opposite: the port is kept *identical* to the Swift code. The resampling kernels and weights, the line sampler, the color conversions, the normalization of the spectrum, the pattern formulas and the SplitMix64 generator compute the same numbers, so a profile or a spectrum on Windows matches the one on macOS to floating-point precision. Where C# and Swift differ in some small default (rounding, remainders, integer overflow), the C# code says explicitly what the Swift code meant. Those spots are pointed out as they come.

12.1 The project file

The library is an SDK-style project. It targets plain `net9.0`, not `net9.0-windows`: nothing in the core touches WPF or the Windows Imaging Component, so it builds and its tests run on any platform .NET supports. All the Windows-specific code lives in the app.

215 `< Windows/LineScope.Core/LineScope.Core.csproj 215> ≡`

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <Win Core project: target and language options 216>
    <Win Core project: optimizing Debug builds 217>
  </PropertyGroup>

  <ItemGroup>
    <InternalsVisibleTo Include="LineScope.Core.Tests" />
  </ItemGroup>

</Project>
```

Root chunk, tangled to Windows/LineScope.Core/LineScope.Core.csproj.

Implicit usings give every file `System`, `System.Linq`, `System.Collections.Generic` and `System.Threading.Tasks` (where `Parallel` lives), which is why most files below start directly with their namespace. Nullable reference types are on, so a `PixelBuffer?` that may be missing is marked as such, the counterpart of a Swift optional. The `InternalsVisibleTo` item plays the role of Swift’s `@testable` import: the test project can reach internal members such as the contribution tables of the resampler and the FFT.

216 *⟨Win Core project: target and language options 216⟩* ≡

```
<TargetFramework>net9.0</TargetFramework>
<ImplicitUsings>enable</ImplicitUsings>
<Nullable>enable</Nullable>
<RootNamespace>LineScope.Core</RootNamespace>
```

Used in chunk 215.

The Swift package asks for `-O` even in Debug builds, because unoptimized pixel loops are tens of times slower, as the introduction explains. The same holds for .NET: a Debug build normally tells the JIT compiler not to optimize, so that a debugger can inspect every local variable. `Optimize` set unconditionally undoes that for this one assembly. Debugging the core becomes a little harder, and the app stays usable in Debug.

217 *⟨Win Core project: optimizing Debug builds 217⟩* ≡

```
<!-- Pixel loops are unusably slow unoptimized; optimize even in Debug (like the Swift package's -O). -->
<Optimize>true</Optimize>
```

Used in chunk 215.

12.2 Pixels

The representation is the one of Chapter 2: interleaved RGBA, 32-bit floats in $[0, 1]$, row 0 at the top, sRGB-encoded values with straight alpha. Only the type that holds it changes.

218 *⟨Windows/LineScope.Core/PixelBuffer.cs 218⟩* ≡

```
namespace LineScope.Core;
```

⟨Win PixelBuffer: the color sample Rgba 219⟩

⟨Win PixelBuffer: the class 220⟩

Root chunk, tangled to Windows/LineScope.Core/PixelBuffer.cs.

A color sample is a `readonly` record struct. The positional syntax declares the four components, a constructor with an optional alpha, and value equality in one line, which is what the Swift struct RGBA spelled out by hand.

219 *⟨Win PixelBuffer: the color sample Rgba 219⟩* ≡

```
/// <summary>A color sample with straight (non-premultiplied) alpha, sRGB-encoded components in
    ↪ ...01.</summary>
public readonly record struct Rgba(float R, float G, float B, float A = 1);
```

Used in chunk 218.

12.2.1 A class treated as a value

In Swift, `PixelBuffer` is a struct over a copy-on-write array: passing it around is free, and mutating a copy never affects the original. C# has nothing like copy-on-write. A struct holding a `float` array would copy only the reference to the array, so two “copies” would share their pixels and the value semantics would be an illusion. The port therefore makes the buffer a `sealed` class and

adopts a convention instead: *treat instances as values*. Every operation returns a new buffer and leaves its input alone; code that wants to change pixels first calls `Clone`. The session can then hand the same buffer to a background task and to a view without locks, exactly as with the Swift struct.

220 *⟨Win PixelBuffer: the class 220⟩* ≡

```
/// <summary>
/// Interleaved RGBA Float32 image. Row 0 is the top row.
/// Components are sRGB-encoded (not linearized) with straight alpha, so values match what is displayed.
/// Treat instances as values: operations return new buffers instead of mutating their input.
/// </summary>
public sealed class PixelBuffer : IEquatable<PixelBuffer>
{
    ⟨Win PixelBuffer: properties and constructors 221⟩

    ⟨Win PixelBuffer: pixel indexer 222⟩

    public PixelBuffer Clone() => new(Width, Height, (float[])Data.Clone());

    ⟨Win PixelBuffer: premultiplication 223⟩

    ⟨Win PixelBuffer: premultiplied BGRA bridge 224⟩

    ⟨Win PixelBuffer: value equality 226⟩
}
```

Used in chunk 218.

The designated constructor checks the same invariant as the Swift precondition, but throws an `ArgumentException`, the .NET way to reject bad arguments. The filling constructor chains to it, and chaining has a trap: the array allocated in the argument list is evaluated *before* the validating constructor runs, and a negative length would throw an `OverflowException` with a less helpful message. Clamping the dimensions to zero first lets the size check report the error. A second constructor without a color stands in for Swift's default argument, since a default parameter value in C# must be a compile-time constant and a `Rgba` isn't one.

221 *⟨Win PixelBuffer: properties and constructors 221⟩* ≡

```
public int Width { get; }
public int Height { get; }
public float[] Data { get; }

public PixelBuffer(int width, int height, float[] data)
{
    if (width <= 0 || height <= 0 || data.Length != width * height * 4)
        throw new ArgumentException("Invalid pixel buffer dimensions.");
    Width = width;
    Height = height;
    Data = data;
}

public PixelBuffer(int width, int height) : this(width, height, new Rgba(0, 0, 0, 1)) { }

public PixelBuffer(int width, int height, Rgba fill)
: this(width, height, new float[Math.Max(width, 0) * Math.Max(height, 0) * 4])
{
    for (int i = 0; i < Data.Length; i += 4)
    {
        Data[i] = fill.R;
        Data[i + 1] = fill.G;
        Data[i + 2] = fill.B;
        Data[i + 3] = fill.A;
    }
}
```

```
}
```

Used in chunk 220.

The Swift two-argument subscript becomes an indexer with the same index formula. The `Data` property has no setter, but the array it returns is mutable; that is what lets the indexer's setter, and the hot loops elsewhere, write into it. C# arrays are bounds-checked like Swift's, but the JIT compiler removes many of the checks in simple loops, so the port indexes `Data` directly where Swift needed unsafe buffer pointers.

222 *⟨Win PixelBuffer: pixel indexer 222⟩* ≡

```
public Rgba this[int x, int y]
{
    get
    {
        int i = (y * Width + x) * 4;
        return new Rgba(Data[i], Data[i + 1], Data[i + 2], Data[i + 3]);
    }
    set
    {
        int i = (y * Width + x) * 4;
        Data[i] = value.R;
        Data[i + 1] = value.G;
        Data[i + 2] = value.B;
        Data[i + 3] = value.A;
    }
}
```

Used in chunk 220.

Premultiplication is Section 2.3 line for line, with `Math.Clamp` for `min(max(...))`. One difference is deliberate. Swift's `fromPremultiplied` copies its argument before converting (the copy is free until written, and then it's a real copy). The C# version converts *in place* and wraps the same array in the new buffer, saving one image-sized allocation per filter or resampling. This is safe because every caller passes an array it has just created and never touches again; the method is `internal`, so no one else can call it.

223 *⟨Win PixelBuffer: premultiplication 223⟩* ≡

// Premultiplication (used by resampling and the blur filters, which work on premultiplied data)

```
internal float[] Premultiplied()
{
    var output = (float[])Data.Clone();
    for (int i = 0; i < output.Length; i += 4)
    {
        float a = output[i + 3];
        output[i] *= a;
        output[i + 1] *= a;
        output[i + 2] *= a;
    }
    return output;
}

/// <summary>Un-premultiplies (in place) and clamps to ...01.</summary>
internal static PixelBuffer FromPremultiplied(int width, int height, float[] p)
{
    for (int i = 0; i < p.Length; i += 4)
    {
        float a = Math.Clamp(p[i + 3], 0, 1);
        p[i + 3] = a;
        if (a > 1e-6f)
        {
            p[i] = Math.Clamp(p[i] / a, 0, 1);
            p[i + 1] = Math.Clamp(p[i + 1] / a, 0, 1);
        }
    }
}
```

```

        p[i + 2] = Math.Clamp(p[i + 2] / a, 0, 1);
    }
    else
    {
        p[i] = 0;
        p[i + 1] = 0;
        p[i + 2] = 0;
    }
}
return new PixelBuffer(width, height, p);
}

```

Used in chunk 220.

12.2.2 The 8-bit bridge

On macOS the buffer converts itself to and from a `CGImage`, and Core Graphics does the decoding and color conversion. On Windows those jobs belong to the Windows Imaging Component (WIC) and to WPF, and both live in the app. The core only exchanges *bytes* with them, which keeps it free of any UI dependency.

The byte layout is the one WPF draws natively: `Pbgra32`, four bytes per pixel in the order blue, green, red, alpha, with the colors premultiplied by alpha. (Read as a little-endian 32-bit integer, that is `ARGB`, the traditional Windows pixel.) The app asks WIC to convert every decoded image to this format, whatever its source layout, just as the Swift decoder draws every `CGImage` into an 8-bit premultiplied bitmap; and it wraps the bytes of a rendered buffer into a bitmap in this format without further conversion. Compared with the Swift code, only the order of the color bytes differs: red is at offset 2 and blue at offset 0.

Decoding scales the bytes to `[0, 1]` and un-premultiplies, which is again where 16-bit sources lose their extra precision.

224 *<Win PixelBuffer: premultiplied BGRA bridge 224> ≡*

*// 8-bit bridging (the Windows counterpart of the Core Graphics bridge). The byte layout is
// premultiplied BGRA, which is what WIC's and WPF's Pbgra32 pixel format expects.*

/// <summary>Converts decoded 8-bit premultiplied BGRA pixels to a float buffer.</summary>

```

public static PixelBuffer FromPremultipliedBgra8(int width, int height, byte[] bytes)
{
    if (bytes.Length < width * height * 4) throw new ArgumentException("Too few bytes.");
    var p = new float[width * height * 4];
    for (int i = 0; i < p.Length; i += 4)
    {
        p[i] = bytes[i + 2] / 255f;
        p[i + 1] = bytes[i + 1] / 255f;
        p[i + 2] = bytes[i] / 255f;
        p[i + 3] = bytes[i + 3] / 255f;
    }
    return FromPremultiplied(width, height, p);
}

```

/// <summary>Renders the buffer as 8-bit premultiplied BGRA for display or export.</summary>

```

public byte[] ToPremultipliedBgra8()
{
    var bytes = new byte[Data.Length];
    for (int i = 0; i < Data.Length; i += 4)
    {
        float a = Math.Clamp(Data[i + 3], 0, 1);
        bytes[i + 2] = ToByte(Math.Clamp(Data[i], 0, 1) * a);
        bytes[i + 1] = ToByte(Math.Clamp(Data[i + 1], 0, 1) * a);
        bytes[i] = ToByte(Math.Clamp(Data[i + 2], 0, 1) * a);
        bytes[i + 3] = ToByte(a);
    }
}

```

```
    return bytes;
}
```

⟨Win PixelBuffer: rounding to a byte 225⟩

Used in chunk 220.

Rounding to a byte is the first of the small defaults that differ between the languages. Swift’s `rounded()` rounds halves away from zero; .NET’s `Math.Round` rounds halves to the nearest *even* integer unless told otherwise. A value of 126.5 would become 127 on macOS and 126 on Windows. Asking for `MidpointRounding.AwayFromZero` makes the exported bytes agree.

225 ⟨Win PixelBuffer: rounding to a byte 225⟩ ≡

```
private static byte ToByte(float v) => (byte)MathF.Round(v * 255, MidpointRounding.AwayFromZero);
```

Used in chunk 224.

Swift synthesizes `Equatable` for the struct. A class compares by reference unless told otherwise, so the port implements `IEquatable` by hand: same size and the same floats, compared as spans. The hash code combines only the dimensions. That is cheap and consistent with `Equals` (equal buffers have equal sizes), and buffers are compared in tests, not used as dictionary keys.

226 ⟨Win PixelBuffer: value equality 226⟩ ≡

```
public bool Equals(PixelBuffer? other) =>
    other is not null && Width == other.Width && Height == other.Height &&
    ↪ Data.AsSpan().SequenceEqual(other.Data);

public override bool Equals(object? obj) => Equals(obj as PixelBuffer);

public override int GetHashCode() => GetHashCode.Combine(Width, Height, Data.Length);
```

Used in chunk 220.

12.3 Resampling

The resampler is the part of the port that must agree with macOS most closely, since comparing resampling methods is what `LineScope` is for. Kernels, supports, pixel-center convention, trimming and normalization are therefore exactly those of Chapter 3. The one real change is that both passes now run in parallel.

227 ⟨Windows/LineScope.Core/Resampling.cs 227⟩ ≡

```
namespace LineScope.Core;

public enum ResampleMethod { Nearest, Tent, CatmullRom, Mitchell, Lanczos3, Sinc, Area }
```

⟨Win Resampling: method names as extension methods 228⟩

⟨Win Resampling: the Resampler class 229⟩

Root chunk, tangled to Windows/LineScope.Core/Resampling.cs.

A C# enumeration is only a set of named integers; it can’t have properties as a Swift enum can. The menu names therefore become extension methods in a companion static class, so that `method.DisplayName()` reads almost like Swift’s `method.displayName`. All replaces `CaseIterable`. The switch expressions end in a discard arm, because the compiler can’t prove that an enum variable holds one of the named values (any integer can be cast to it); here the discard stands for the last case.

228 *⟨Win Resampling: method names as extension methods 228⟩* ≡

```
public static class ResampleMethodInfo
{
    public static readonly ResampleMethod[] All = Enum.GetValues<ResampleMethod>();

    public static string DisplayName(this ResampleMethod m) => m switch
    {
        ResampleMethod.Nearest => "Nearest Neighbor",
        ResampleMethod.Tent => "Tent (Bilinear)",
        ResampleMethod.CatmullRom => "Catmull-Rom (Bicubic)",
        ResampleMethod.Mitchell => "Mitchell-Netravali",
        ResampleMethod.Lanczos3 => "Lanczos-3",
        ResampleMethod.Sinc => $"Sinc (truncated, r={{(int)Resampler.SincRadius}})",
        _ => "Area Average",
    };

    public static string ShortName(this ResampleMethod m) => m switch
    {
        ResampleMethod.Nearest => "Nearest",
        ResampleMethod.Tent => "Tent",
        ResampleMethod.CatmullRom => "Catmull-Rom",
        ResampleMethod.Mitchell => "Mitchell",
        ResampleMethod.Lanczos3 => "Lanczos",
        ResampleMethod.Sinc => "Sinc",
        _ => "Area",
    };
}
```

Used in chunk 227.

The caseless Swift enum used as a namespace becomes a static class, and its Markdown documentation becomes an XML comment with the same text.

229 *⟨Win Resampling: the Resampler class 229⟩* ≡

```
/// <summary>
/// Separable resampling with explicit, textbook kernels.
/// </summary>
/// <remarks>
/// Coordinates follow the pixel-center convention: input pixel <c>j</c> covers <c>[j, j+1)</c> and its
/// ↪ center is <c>j + 0.5</c>.
/// Output pixel <c>i</c> maps to input position <c>(i + 0.5) / scale</c>.
/// <list type="bullet">
/// <item><c>Nearest</c> picks one source pixel and does no prefiltering, so it aliases when
/// ↪ downscaling.</item>
/// <item>The kernel methods (tent, Catmull-Rom, Mitchell, Lanczos, sinc) widen their kernel by
/// ↪ <c>1/scale</c>
/// when downscaling (antialiased).</item>
/// <item><c>Area</c> weights source pixels by how much of the output pixel's footprint they
/// ↪ cover.</item>
/// </list>
/// </remarks>
public static class Resampler
{
    ⟨Win Resampler: sinc radius and output size 230⟩

    ⟨Win Resampler: resampling by a scale factor 231⟩

    ⟨Win Resampler: resampling to a given size 232⟩

    // Kernels

    ⟨Win Resampler: contributions of input pixels 235⟩

    ⟨Win Resampler: trimming and normalizing weights 239⟩
}
```

⟨Win Resampler: the kernels 240⟩
}

Used in chunk 227.

The output size rounds the scaled size with `MidpointRounding.AwayFromZero`, for the reason given with `ToByte` above: halving an image 5 pixels wide must give 3 pixels on both platforms, not 2 on one of them.

230 *⟨Win Resampler: sinc radius and output size 230⟩* ≡

/// <summary>Half-width of the truncated sinc kernel, in source pixels (before widening).</summary>
public const double SincRadius = 8.0;

public static int OutputSize(**int** size, **double** scale) =>
 Math.Max(1, (**int**)Math.Round(size * scale, MidpointRounding.AwayFromZero));

Used in chunk 229.

Overloading replaces Swift's argument labels: `Resample(src, scale, method)` and `Resample(src, width, height, method)` are told apart by their parameter types.

231 *⟨Win Resampler: resampling by a scale factor 231⟩* ≡

public static PixelBuffer Resample(PixelBuffer src, **double** scale, ResampleMethod method) =>
 Resample(src, OutputSize(src.Width, scale), OutputSize(src.Height, scale), method);

Used in chunk 229.

232 *⟨Win Resampler: resampling to a given size 232⟩* ≡

public static PixelBuffer Resample(PixelBuffer src, **int** width, **int** height, ResampleMethod method)
{

var premul = src.Premultiplied();
 var xContrib = Contributions(src.Width, width, method);
 var yContrib = Contributions(src.Height, height, method);

⟨Win Resampler: horizontal pass 233⟩

⟨Win Resampler: vertical pass 234⟩

return PixelBuffer.FromPremultiplied(width, height, output);

}

Used in chunk 229.

The two passes are the loops of Chapter 3, with the outer loop over rows replaced by `Parallel.For`. Each iteration of the horizontal pass reads one source row and writes one row of the intermediate image, so the iterations share nothing but read-only data, and they can run on all cores without locks. Parallelism doesn't change the result: every output component is still the same sum, accumulated in the same order, so the image is bit-identical to that of a serial loop.

233 *⟨Win Resampler: horizontal pass 233⟩* ≡

// Horizontal pass: (src.Width x src.Height) -> (width x src.Height)

var tmp = **new float**[width * src.Height * 4];
Parallel.For(0, src.Height, y =>
{
 int srcRow = y * src.Width * 4;
 int dstRow = y * width * 4;
 for (**int** x = 0; x < width; x++)
 {
 var c = xContrib[x];
 float r = 0, g = 0, b = 0, a = 0;
 for (**int** k = 0; k < c.Weights.Length; k++)
 {
 float w = c.Weights[k];

```

        int si = srcRow + (c.Start + k) * 4;
        r += premul[si] * w;
        g += premul[si + 1] * w;
        b += premul[si + 2] * w;
        a += premul[si + 3] * w;
    }
    int di = dstRow + x * 4;
    tmp[di] = r;
    tmp[di + 1] = g;
    tmp[di + 2] = b;
    tmp[di + 3] = a;
}
});

```

Used in chunk 232.

The vertical pass is parallel over *output* rows. Each one accumulates several input rows, and input rows are shared between neighboring output rows, but they are only read; every output row is written by exactly one iteration. C# arrays start out zeroed, so the accumulation needs no explicit initialization, just as the Swift array created with `repeating: 0`.

234 *⟨Win Resampler: vertical pass 234⟩* ≡

```

// Vertical pass: (width x src.Height) -> (width x height)
var output = new float[width * height * 4];
int rowStride = width * 4;
Parallel.For(0, height, y =>
{
    var c = yContrib[y];
    int dstRow = y * rowStride;
    for (int k = 0; k < c.Weights.Length; k++)
    {
        float w = c.Weights[k];
        int srcRow = (c.Start + k) * rowStride;
        for (int i = 0; i < rowStride; i++)
            output[dstRow + i] += tmp[srcRow + i] * w;
    }
});

```

Used in chunk 232.

A Contribution becomes a readonly record struct, and the table an array filled by index rather than appended to. The rounding helpers translate one to one: `rounded(.down)` is `Math.Floor` and `rounded(.up)` is `Math.Ceiling`, both followed by a cast to `int`.

235 *⟨Win Resampler: contributions of input pixels 235⟩* ≡

```

internal readonly record struct Contribution(int Start, float[] Weights);

internal static Contribution[] Contributions(int inSize, int outSize, ResampleMethod method)
{
    double scale = (double)outSize / inSize;
    var result = new Contribution[outSize];

    for (int i = 0; i < outSize; i++)
    {
        double center = (i + 0.5) / scale;
        switch (method)
        {
            {
                ⟨Win Resampler: nearest-neighbor contribution 236⟩
                ⟨Win Resampler: area-average contribution 237⟩
                ⟨Win Resampler: kernel contribution 238⟩
            }
        }
    }
    return result;
}

```

Used in chunk 229.

The three cases are those of Chapter 3. Each is wrapped in braces, so that the local names *first*, *last* and *weights* of one case don't collide with those of the next; in C# all the cases of a switch share one scope. The kernel methods fall into the default arm.

236 *⟨Win Resampler: nearest-neighbor contribution 236⟩* ≡

```
case ResampleMethod.Nearest:
{
    int j = Math.Clamp((int)Math.Floor(center), 0, inSize - 1);
    result[i] = new Contribution(j, [1]);
    break;
}
```

Used in chunk 235.

The Swift code guards its loops with `if last >= first`, because a closed range `first...last` with `last < first` traps at run time. A C# `for` loop simply runs zero times, so the guard disappears and an empty weight list goes to the fallback in `Normalized`, as before.

237 *⟨Win Resampler: area-average contribution 237⟩* ≡

```
case ResampleMethod.Area:
{
    // Output pixel footprint in input coordinates, exact coverage weights.
    double half = 0.5 / scale;
    double lo = center - half;
    double hi = center + half;
    int first = Math.Max(0, (int)Math.Floor(lo));
    int last = Math.Min(inSize - 1, (int)Math.Ceiling(hi) - 1);
    var weights = new List<float>();
    for (int j = first; j <= last; j++)
    {
        double overlap = Math.Min(hi, j + 1) - Math.Max(lo, j);
        weights.Add((float)Math.Max(0, overlap));
    }
    result[i] = Normalized(first, weights, center, inSize);
    break;
}
```

Used in chunk 235.

238 *⟨Win Resampler: kernel contribution 238⟩* ≡

```
default:
{
    double filterScale = Math.Max(1, 1 / scale);
    double support = KernelSupport(method) * filterScale;
    int first = Math.Max(0, (int)Math.Floor(center - support));
    int last = Math.Min(inSize - 1, (int)Math.Ceiling(center + support));
    var weights = new List<float>();
    for (int j = first; j <= last; j++)
    {
        double x = (j + 0.5 - center) / filterScale;
        weights.Add((float)Kernel(method, x));
    }
    result[i] = Normalized(first, weights, center, inSize);
    break;
}
```

Used in chunk 235.

Trimming and normalizing are unchanged, including the arithmetic type: the sum of the weights is a float, accumulated from left to right as Swift's `reduce` does. The test is written as `!(Math.Abs(sum) > 1e-12)` rather than `Math.Abs(sum) <= 1e-12` so that it keeps the meaning of Swift's guard `abs(sum) > 1e-12` else when the sum is not a number: a NaN fails every comparison, and both versions then take the fallback.

239 *⟨Win Resampler: trimming and normalizing weights 239⟩* ≡

```
private static Contribution Normalized(int start, List<float> weights, double center, int inSize)
{
    // Trim zero weights at both ends to keep inner loops short.
    int first = 0;
    int last = weights.Count - 1;
    while (first <= last && weights[first] == 0) first++;
    while (last >= first && weights[last] == 0) last--;
    int fallback = Math.Clamp((int)Math.Floor(center), 0, inSize - 1);
    if (first > last) return new Contribution(fallback, [1]);
    var trimmed = weights.GetRange(first, last - first + 1);
    float sum = 0;
    foreach (float w in trimmed) sum += w;
    if (!Math.Abs(sum) > 1e-12) return new Contribution(fallback, [1]);
    return new Contribution(start + first, trimmed.Select(w => w / sum).ToArray());
}
```

Used in chunk 229.

The kernels are Chapter 3's formulas, transcribed: the box, the tent, the BC-splines of Mitchell and Netravali with $(B, C) = (0, \frac{1}{2})$ and $(\frac{1}{3}, \frac{1}{3})$, Lanczos-3 and the truncated sinc. They are evaluated in double and only then converted to float, in the same places as in Swift.

240 *⟨Win Resampler: the kernels 240⟩* ≡

```
internal static double KernelSupport(ResampleMethod method) => method switch
{
    ResampleMethod.Nearest or ResampleMethod.Area => 0.5,
    ResampleMethod.Tent => 1,
    ResampleMethod.CatmullRom or ResampleMethod.Mitchell => 2,
    ResampleMethod.Lanczos3 => 3,
    _ => SincRadius,
};

private static double Sinc(double x)
{
    if (Math.Abs(x) < 1e-9) return 1;
    double px = Math.PI * x;
    return Math.Sin(px) / px;
}

/// <summary>Mitchell-Netravali / Keys family: (B, C) = (1/3, 1/3) is Mitchell, (0, 1/2) is
    ↪ Catmull-Rom.</summary>
private static double BcSpline(double ax, double b, double c)
{
    if (ax < 1)
        return ((12 - 9 * b - 6 * c) * ax * ax * ax + (-18 + 12 * b + 6 * c) * ax * ax + (6 - 2 * b)) /
            ↪ 6;
    if (ax < 2)
        return ((-b - 6 * c) * ax * ax * ax + (6 * b + 30 * c) * ax * ax
            + (-12 * b - 48 * c) * ax + (8 * b + 24 * c)) / 6;
    return 0;
}

internal static double Kernel(ResampleMethod method, double x)
{
    double ax = Math.Abs(x);
    return method switch
    {
        ResampleMethod.Nearest or ResampleMethod.Area => ax < 0.5 ? 1 : 0,
        ResampleMethod.Tent => Math.Max(0, 1 - ax),
        ResampleMethod.CatmullRom => BcSpline(ax, 0, 0.5),
        ResampleMethod.Mitchell => BcSpline(ax, 1.0 / 3, 1.0 / 3),
        ResampleMethod.Lanczos3 => ax < 3 ? Sinc(ax) * Sinc(ax / 3) : 0,
        // Ideal low-pass, truncated (rectangular window): expect ringing near edges.
    }
}
```

```

        _ => ax < SincRadius ? Sinc(ax) : 0,
    };
}

```

Used in chunk 229.

12.4 Filters

Half of the filters of Chapter 4 were never LineScope’s own code: the Gaussian and box blurs, the median, the unsharp mask and the noise reduction were Core Image built-ins, and Windows has no Core Image. The port writes all five out on the CPU. The other half (Sobel, Laplacian, emboss, grayscale, invert) was already CPU code and is ported directly.

241 *⟨Windows/LineScope.Core/Filters.cs 241⟩* ≡

```
namespace LineScope.Core;
```

⟨Win Filters: the FilterKind records 242⟩

⟨Win Filters: the Filters class 245⟩

Root chunk, tangled to Windows/LineScope.Core/Filters.cs.

12.4.1 Filter values

A Swift enumeration with associated values has no direct C# equivalent. The idiom that comes closest is an abstract record with one nested sealed record per case. Records provide the value equality that `Equatable` gave, including for the parameters, and a switch on the type with positional patterns takes the place of case `.gaussianBlur(let r)`. Parameterless cases like `Median` are records without fields: any two instances compare equal. The hierarchy is open as far as the compiler knows, so each switch needs a discard arm that is never reached.

242 *⟨Win Filters: the FilterKind records 242⟩* ≡

```
public abstract record FilterKind
```

```
{
```

⟨Win Filters: one record per filter 243⟩

⟨Win Filters: display names 244⟩

```
}
```

Used in chunk 241.

243 *⟨Win Filters: one record per filter 243⟩* ≡

```
public sealed record GaussianBlur(double Radius) : FilterKind;
```

```
public sealed record BoxBlur(double Radius) : FilterKind;
```

```
public sealed record Median : FilterKind;
```

```
public sealed record UnsharpMask(double Radius, double Intensity) : FilterKind;
```

```
public sealed record NoiseReduction(double Level, double Sharpness) : FilterKind;
```

```
public sealed record Sobel : FilterKind;
```

```
public sealed record Laplacian : FilterKind;
```

```
public sealed record Emboss : FilterKind;
```

```
public sealed record Grayscale : FilterKind;
```

```
public sealed record Invert : FilterKind;
```

Used in chunk 242.

The names are those of the Swift version, with the parameters formatted by `Fmt.Number` (Section 12.5.1), the port’s counterpart of the helper `fmt`.

244 *⟨Win Filters: display names 244⟩* ≡

```
public string DisplayName => this switch
{
    GaussianBlur g => $"Gaussian r={Fmt.Number(g.Radius)}",
    BoxBlur b => $"Box r={Fmt.Number(b.Radius)}",
    Median => "Median 3×3",
    UnsharpMask u => $"Sharpen r={Fmt.Number(u.Radius)} i={Fmt.Number(u.Intensity)}",
    NoiseReduction n => $"Denoise {Fmt.Number(n.Level)}/{Fmt.Number(n.Sharpness)}",
    Sobel => "Sobel",
    Laplacian => "Laplacian",
    Emboss => "Emboss",
    Grayscale => "Grayscale",
    Invert => "Invert",
    _ => "",
};
```

Used in chunk 242.

12.4.2 The filter class

The class's documentation comment is the specification of the replacements, and the rest of this section explains it item by item. Its layout mirrors the Swift file: dispatch, then the filters that replace Core Image, then the CPU filters that were already there.

245 *⟨Win Filters: the Filters class 245⟩* ≡

```
/// <summary>
/// All filters run on the CPU and work directly on encoded values (no color management), with clamped
/// ↪ edges.
/// On macOS the blur, median, unsharp-mask and noise-reduction filters came from Core Image; here they
/// ↪ are
/// explicit implementations of the same operations:
/// <list type="bullet">
/// <item>Gaussian: separable kernel with  $\sigma$  = radius, truncated at  $\sigma 3$  (Core Image's radius is  $\sigma$ ).</item>
/// <item>Box: separable box covering  $-(r+\frac{1}{2}), r+\frac{1}{2}]$ , with fractional coverage at the ends.</item>
/// <item>Median: 3×3 per-channel median.</item>
/// <item>Unsharp mask:  $\text{src} + \text{intensity} \cdot (\text{src} - \text{Gaussian}(\text{src}, \text{radius}))$ .</item>
/// <item>Noise reduction: a threshold ("sigma") filter. Neighbors whose luminance differs from the
/// ↪ center by less
/// than the noise level are averaged; pixels on edges (a neighbor above the threshold) are sharpened
/// ↪ instead,
/// by <c>sharpness</c> times a 3×3 unsharp mask. This follows Core Image's documented behavior, not its
/// ↪ exact kernel.</item>
/// </list>
/// The blurs work on premultiplied data, as Core Image did.
/// </summary>
```

```
public static class Filters
{
```

⟨Win Filters: dispatching a filter 246⟩

⟨Win Filters: running on premultiplied data 247⟩

// Separable filters on premultiplied RGBA

⟨Win Filters: separable convolution 248⟩

⟨Win Filters: the Gaussian kernel 249⟩

⟨Win Filters: the box kernel 250⟩

⟨Win Filters: the two blurs 251⟩

⟨Win Filters: unsharp mask 252⟩

```

    <Win Filters: 3x3 median 253>

    <Win Filters: noise reduction 254>

    // CPU filters on straight RGB

    <Win Filters: per-pixel color maps 257>

    <Win Filters: 3x3 convolution 258>

    <Win Filters: Sobel gradient magnitude 259>
}

Used in chunk 241.

```

Dispatch is one switch expression. The five former Core Image filters go through Premultiplied; the rest are unchanged from Chapter 4, kernels included.

246 <Win Filters: dispatching a filter 246> ≡

```

public static PixelBuffer Apply(FilterKind filter, PixelBuffer src) => filter switch
{
    FilterKind.GaussianBlur g => Premultiplied(src, p => GaussianBlur(p, src.Width, src.Height,
        ↪ g.Radius)),
    FilterKind.BoxBlur b => Premultiplied(src, p => BoxBlur(p, src.Width, src.Height, b.Radius)),
    FilterKind.Median => Premultiplied(src, p => Median3x3(p, src.Width, src.Height)),
    FilterKind.UnsharpMask u => Premultiplied(src, p => UnsharpMask(p, src.Width, src.Height, u.Radius,
        ↪ u.Intensity)),
    FilterKind.NoiseReduction n => Premultiplied(src, p => NoiseReduction(p, src.Width, src.Height,
        ↪ n.Level, n.Sharpness)),
    FilterKind.Sobel => Sobel(src),
    // Absolute response so the edges are visible.
    FilterKind.Laplacian => Convolve3x3(src, [0, 1, 0, 1, -4, 1, 0, 1, 0], absolute: true),
    FilterKind.Emboss => Convolve3x3(src, [-2, -1, 0, -1, 1, 1, 0, 1, 2], absolute: false),
    FilterKind.Grayscale => MapRgb(src, c =>
    {
        float y = 0.2126f * c.R + 0.7152f * c.G + 0.0722f * c.B;
        return new Rgba(y, y, y, c.A);
    }),
    FilterKind.Invert => MapRgb(src, c => new Rgba(1 - c.R, 1 - c.G, 1 - c.B, c.A)),
    _ => src,
};

```

Used in chunk 245.

This helper is what is left of the Core Image bridge. The Swift `applyCI` premultiplied the buffer, wrapped it as a `CIImage` without a color space, filtered, and un-premultiplied the result. Here the filter is simply a function from premultiplied floats to premultiplied floats. Both halves of the Core Image setup that mattered survive in spirit: there is no color management, because nothing here knows about color spaces, and the edges are clamped, as `clampedToExtent` did, by the index clamping in every loop below.

247 <Win Filters: running on premultiplied data 247> ≡

```

private static PixelBuffer Premultiplied(PixelBuffer src, Func<float[], float[]> body) =>
    PixelBuffer.FromPremultiplied(src.Width, src.Height, body(src.Premultiplied()));

```

Used in chunk 245.

12.4.3 Separable blurs

Both blurs convolve with a symmetric one-dimensional kernel, first along the rows and then along the columns, like the resampler (Chapter 3). Unlike the resampler, the image keeps its size, and

a tap that falls outside the image reads the nearest border pixel. The two passes are parallel over rows, and each pass writes a fresh array, so no iteration reads what another one writes.

248 *Win Filters: separable convolution* 248) $\equiv$

*/// <summary>Convolves every channel with a symmetric 1-D kernel (index <radius> is the center)
 ↳ along x, then y.</summary>*

```
internal static float[] ConvolveSeparable(float[] src, int w, int h, float[] kernel)
{
    int r = kernel.Length / 2;
    var tmp = new float[src.Length];
    Parallel.For(0, h, y =>
    {
        for (int x = 0; x < w; x++)
        {
            float a0 = 0, a1 = 0, a2 = 0, a3 = 0;
            for (int k = -r; k <= r; k++)
            {
                int xx = Math.Clamp(x + k, 0, w - 1);
                int i = (y * w + xx) * 4;
                float wt = kernel[k + r];
                a0 += src[i] * wt; a1 += src[i + 1] * wt; a2 += src[i + 2] * wt; a3 += src[i + 3] * wt;
            }
            int o = (y * w + x) * 4;
            tmp[o] = a0; tmp[o + 1] = a1; tmp[o + 2] = a2; tmp[o + 3] = a3;
        }
    });
    var output = new float[src.Length];
    Parallel.For(0, h, y =>
    {
        for (int x = 0; x < w; x++)
        {
            float a0 = 0, a1 = 0, a2 = 0, a3 = 0;
            for (int k = -r; k <= r; k++)
            {
                int yy = Math.Clamp(y + k, 0, h - 1);
                int i = (yy * w + x) * 4;
                float wt = kernel[k + r];
                a0 += tmp[i] * wt; a1 += tmp[i + 1] * wt; a2 += tmp[i + 2] * wt; a3 += tmp[i + 3] * wt;
            }
            int o = (y * w + x) * 4;
            output[o] = a0; output[o + 1] = a1; output[o + 2] = a2; output[o + 3] = a3;
        }
    });
    return output;
}
```

Used in chunk 245.

Apple documents the radius of `CIGaussianBlur` as the standard deviation of the Gaussian, so the port takes $\sigma = r$. The kernel

$$g_i = \frac{e^{-i^2/2\sigma^2}}{\sum_{j=-R}^R e^{-j^2/2\sigma^2}}, \quad |i| \leq R = \max(1, \lceil 3\sigma \rceil),$$

is truncated at 3σ , where the Gaussian has fallen to about 1% of its peak and the omitted tails hold less than 0.3% of its mass; dividing by the sum restores the rest, so a flat image stays flat. A radius below 10^{-3} is the identity.

249 *⟨Win Filters: the Gaussian kernel 249⟩* ≡

```

internal static float[] GaussianKernel(double sigma)
{
    if (sigma < 1e-3) return [1];
    int r = Math.Max(1, (int)Math.Ceiling(3 * sigma));
    var k = new float[2 * r + 1];
    double sum = 0;
    for (int i = -r; i <= r; i++)
    {
        double v = Math.Exp(-(i * i) / (2 * sigma * sigma));
        k[i + r] = (float)v;
        sum += v;
    }
    for (int i = 0; i < k.Length; i++) k[i] = (float)(k[i] / sum);
    return k;
}

```

Used in chunk 245.

The box blur must accept fractional radii, since the parameter sheet does. The port treats the box as the continuous interval $[-(r + \frac{1}{2}), r + \frac{1}{2}]$ and weights each tap i by the part of its pixel $[i - \frac{1}{2}, i + \frac{1}{2}]$ that the interval covers. This is the area-averaging rule of Chapter 3 applied to a filter. With $r = 1$ the box covers three whole pixels, with $r = 0$ it is the identity, and with $r = 1.5$ the weights are $\frac{1}{4}(\frac{1}{2}, 1, 1, 1, \frac{1}{2})$, so the result changes smoothly as the radius does.

250 *⟨Win Filters: the box kernel 250⟩* ≡

```

internal static float[] BoxKernel(double radius)
{
    double half = Math.Max(radius, 0) + 0.5;
    int r = (int)Math.Ceiling(half - 0.5);
    var k = new float[2 * r + 1];
    double sum = 0;
    for (int i = -r; i <= r; i++)
    {
        double overlap = Math.Min(i + 0.5, half) - Math.Max(i - 0.5, -half);
        k[i + r] = (float)Math.Max(0, overlap);
        sum += k[i + r];
    }
    for (int i = 0; i < k.Length; i++) k[i] = (float)(k[i] / sum);
    return k;
}

```

Used in chunk 245.

251 *⟨Win Filters: the two blurs 251⟩* ≡

```

private static float[] GaussianBlur(float[] p, int w, int h, double radius) =>
    ConvolveSeparable(p, w, h, GaussianKernel(radius));

private static float[] BoxBlur(float[] p, int w, int h, double radius) =>
    ConvolveSeparable(p, w, h, BoxKernel(radius));

```

Used in chunk 245.

12.4.4 Sharpening, median and noise reduction

The unsharp mask is the formula of Chapter 4, $out = src + intensity \times (src - G_r * src)$, with the Gaussian just defined. It is applied to the three premultiplied color components; alpha is copied, so sharpening never makes an opaque image partly transparent. Overshoot is left in the result and clamped when the buffer is un-premultiplied.

252 *⟨Win Filters: unsharp mask 252⟩* ≡

```
private static float[] UnsharpMask(float[] p, int w, int h, double radius, double intensity)
{
    var blurred = GaussianBlur(p, w, h, radius);
    float amount = (float)intensity;
    var output = new float[p.Length];
    for (int i = 0; i < p.Length; i += 4)
    {
        for (int c = 0; c < 3; c++)
            output[i + c] = p[i + c] + amount * (p[i + c] - blurred[i + c]);
        output[i + 3] = p[i + 3];
    }
    return output;
}
```

Used in chunk 245.

The median takes, for each channel separately, the middle one of the nine values in the 3×3 neighborhood. Sorting nine floats is fast enough; the only care taken is not to allocate the nine-element window once per pixel. The overload of `Parallel.For` with thread-local state gives every worker thread its own window, created once by the `localInit` delegate and passed from one row to the next.

253 *⟨Win Filters: 3x3 median 253⟩* ≡

```
private static float[] Median3x3(float[] p, int w, int h)
{
    var output = new float[p.Length];
    Parallel.For(0, h, () => new float[9], (y, _, window) =>
    {
        for (int x = 0; x < w; x++)
        {
            int o = (y * w + x) * 4;
            for (int c = 0; c < 4; c++)
            {
                int n = 0;
                for (int ky = -1; ky <= 1; ky++)
                {
                    int yy = Math.Clamp(y + ky, 0, h - 1);
                    for (int kx = -1; kx <= 1; kx++)
                    {
                        int xx = Math.Clamp(x + kx, 0, w - 1);
                        window[n++] = p[(yy * w + xx) * 4 + c];
                    }
                }
                Array.Sort(window);
                output[o + c] = window[4];
            }
        }
        return window;
    }, _ => { });
    return output;
}
```

Used in chunk 245.

Core Image’s noise reduction is documented only in words: changes in luminance smaller than the noise level count as noise and are smoothed, larger ones count as edges and are sharpened by the sharpness amount. The port implements that description as a *threshold* (or “sigma”) filter on the 3×3 neighborhood. It first computes the Rec. 709 luma of every pixel, with the grayscale filter’s weights.

254 *⟨Win Filters: noise reduction 254⟩* $\equiv$

```
private static float[] NoiseReduction(float[] p, int w, int h, double level, double sharpness)
{
    float threshold = (float)level;
    float amount = (float)sharpness;
    var luma = new float[w * h];
    for (int i = 0; i < luma.Length; i++)
        luma[i] = 0.2126f * p[i * 4] + 0.7152f * p[i * 4 + 1] + 0.0722f * p[i * 4 + 2];

    var output = new float[p.Length];
    Parallel.For(0, h, y =>
    {
        for (int x = 0; x < w; x++)
        {
            ⟨Win Filters: noise reduction, scanning the neighborhood 255⟩
            ⟨Win Filters: noise reduction, smoothing or sharpening 256⟩
        }
    });
    return output;
}
```

Used in chunk 245.

One pass over the nine neighbors gathers two sums: m , of all of them, and s , of those whose luma is within the threshold of the center's. A neighbor outside the threshold marks the pixel as lying on an edge.

255 *⟨Win Filters: noise reduction, scanning the neighborhood 255⟩* $\equiv$

```
int ci = y * w + x;
float center = luma[ci];
bool edge = false;
float s0 = 0, s1 = 0, s2 = 0, s3 = 0;
float m0 = 0, m1 = 0, m2 = 0, m3 = 0;
int count = 0;
for (int ky = -1; ky <= 1; ky++)
{
    int yy = Math.Clamp(y + ky, 0, h - 1);
    for (int kx = -1; kx <= 1; kx++)
    {
        int xx = Math.Clamp(x + kx, 0, w - 1);
        int ni = yy * w + xx;
        int i = ni * 4;
        m0 += p[i]; m1 += p[i + 1]; m2 += p[i + 2]; m3 += p[i + 3];
        if (MathF.Abs(luma[ni] - center) < threshold)
        {
            s0 += p[i]; s1 += p[i + 1]; s2 += p[i + 2]; s3 += p[i + 3];
            count++;
        }
        else
        {
            edge = true;
        }
    }
}
```

Used in chunk 254.

An edge pixel is sharpened by an unsharp mask whose blur is the 3×3 mean, $out = p + sharpness \times (p - \bar{m})$, with alpha kept. Any other pixel is replaced by the mean of the neighbors within the threshold. Because smoothing happens only when *no* neighbor crossed the threshold, that mean is in practice over all nine; the count is kept so that the division is safe whatever the test says. A noise level of 0 makes every pixel an edge, so the filter becomes a pure sharpener, and a large level turns it into a 3×3 box blur.

256 *⟨Win Filters: noise reduction, smoothing or sharpening 256⟩* ≡

```
int o = ci * 4;
if (edge || count == 0)
{
    // Edge: unsharp mask against the 3×3 mean.
    output[o] = p[o] + amount * (p[o] - m0 / 9);
    output[o + 1] = p[o + 1] + amount * (p[o + 1] - m1 / 9);
    output[o + 2] = p[o + 2] + amount * (p[o + 2] - m2 / 9);
    output[o + 3] = p[o + 3];
}
else
{
    output[o] = s0 / count;
    output[o + 1] = s1 / count;
    output[o + 2] = s2 / count;
    output[o + 3] = s3 / count;
}
```

Used in chunk 254.

12.4.5 Why the results differ from macOS

For resampling, the port can promise the same numbers as macOS because both sides run the same explicit algorithm. For these five filters it can't. Core Image documents what its filters do, not how: how many taps its Gaussian uses and how it handles large radii, the exact shape of its box, the kernel behind its noise reduction. It also runs on the GPU, often with reduced intermediate precision. The C# filters follow the documented behavior with the simplest exact algorithm, so a Gaussian blur of radius 2 looks the same on both systems, but profiles compared pixel by pixel differ slightly, and noise reduction may differ visibly. A comparison *between* methods on one machine is unaffected, and that is what LineScope is for. The Windows README.md lists the same caveats for users.

12.4.6 CPU filters

The remaining filters are the CPU code of Chapter 4, with the same kernels, clamping and alpha handling. Two things change. The row loops are parallel. And the copy that receives the output is made with `Clone`: in Swift, `var out = src` copied the struct and copy-on-write duplicated the pixels at the first write, but with a class the same line would alias the input and the filter would overwrite the image it is reading.

257 *⟨Win Filters: per-pixel color maps 257⟩* ≡

```
private static PixelBuffer MapRgb(PixelBuffer src, Func<Rgba, Rgba> f)
{
    var output = src.Clone();
    for (int y = 0; y < src.Height; y++)
        for (int x = 0; x < src.Width; x++)
            output[x, y] = f(src[x, y]);
    return output;
}
```

Used in chunk 245.

258 *⟨Win Filters: 3×3 convolution 258⟩* ≡

```
/// <summary>3×3 convolution on RGB with clamped edges; alpha is preserved.</summary>
internal static PixelBuffer Convolve3x3(PixelBuffer src, float[] k, bool absolute)
{
    var output = src.Clone();
    int w = src.Width, h = src.Height;
    var s = src.Data;
    var o = output.Data;
```

```

Parallel.For(0, h, y =>
{
    for (int x = 0; x < w; x++)
    {
        float a0 = 0, a1 = 0, a2 = 0;
        for (int ky = -1; ky <= 1; ky++)
        {
            int yy = Math.Clamp(y + ky, 0, h - 1);
            for (int kx = -1; kx <= 1; kx++)
            {
                int xx = Math.Clamp(x + kx, 0, w - 1);
                float wgt = k[(ky + 1) * 3 + (kx + 1)];
                int i = (yy * w + xx) * 4;
                a0 += s[i] * wgt;
                a1 += s[i + 1] * wgt;
                a2 += s[i + 2] * wgt;
            }
        }
        if (absolute) { a0 = MathF.Abs(a0); a1 = MathF.Abs(a1); a2 = MathF.Abs(a2); }
        int j = (y * w + x) * 4;
        o[j] = Math.Clamp(a0, 0, 1);
        o[j + 1] = Math.Clamp(a1, 0, 1);
        o[j + 2] = Math.Clamp(a2, 0, 1);
    }
});
return output;
}

```

Used in chunk 245.

The Swift code accumulated the three channels in tuples; C# uses three named locals, which the JIT keeps in registers just as well.

259 *Win Filters: Sobel gradient magnitude 259* ≡

/// <summary>Per-channel Sobel gradient magnitude.</summary>

```

internal static PixelBuffer Sobel(PixelBuffer src)
{
    float[] gx = [-1, 0, 1, -2, 0, 2, -1, 0, 1];
    float[] gy = [-1, -2, -1, 0, 0, 0, 1, 2, 1];
    var output = src.Clone();
    int w = src.Width, h = src.Height;
    var s = src.Data;
    var o = output.Data;
    Parallel.For(0, h, y =>
    {
        for (int x = 0; x < w; x++)
        {
            float sx0 = 0, sx1 = 0, sx2 = 0, sy0 = 0, sy1 = 0, sy2 = 0;
            for (int ky = -1; ky <= 1; ky++)
            {
                int yy = Math.Clamp(y + ky, 0, h - 1);
                for (int kx = -1; kx <= 1; kx++)
                {
                    int xx = Math.Clamp(x + kx, 0, w - 1);
                    int ki = (ky + 1) * 3 + (kx + 1);
                    int i = (yy * w + xx) * 4;
                    sx0 += s[i] * gx[ki]; sy0 += s[i] * gy[ki];
                    sx1 += s[i + 1] * gx[ki]; sy1 += s[i + 1] * gy[ki];
                    sx2 += s[i + 2] * gx[ki]; sy2 += s[i + 2] * gy[ki];
                }
            }
            int j = (y * w + x) * 4;
            o[j] = MathF.Min(MathF.Sqrt(sx0 * sx0 + sy0 * sy0), 1);
            o[j + 1] = MathF.Min(MathF.Sqrt(sx1 * sx1 + sy1 * sy1), 1);
            o[j + 2] = MathF.Min(MathF.Sqrt(sx2 * sx2 + sy2 * sy2), 1);
        }
    }
}

```

```
    }
  });
  return output;
}
```

Used in chunk 245.

12.5 Operations and number formatting

`ImageOperation` unifies filters and resampling exactly as in Section 4.2, and it is built with the same record idiom as `FilterKind`.

260 `<Windows/LineScope.Core/ImageOperation.cs 260> ≡`

```
namespace LineScope.Core;

/// <summary>One step that derives a new image from an existing one.</summary>
public abstract record ImageOperation
{
    public sealed record Filter(FilterKind Kind) : ImageOperation;
    public sealed record Resample(ResampleMethod Method, double Scale) : ImageOperation;

    <Win ImageOperation: display name 261>

    <Win ImageOperation: applying the step 262>
}
```

Root chunk, tangled to Windows/LineScope.Core/ImageOperation.cs.

261 `<Win ImageOperation: display name 261> ≡`

```
public string DisplayName => this switch
{
    Filter f => f.Kind.DisplayName,
    Resample r => $"{r.Method.ShortName()} ×{Fmt.Number(r.Scale)}",
    _ => "",
};
```

Used in chunk 260.

Applying an operation is still a pure function from buffer to buffer. Since buffers are treated as values (Section 12.2), the result can be computed on a worker thread while the views keep drawing the input.

262 `<Win ImageOperation: applying the step 262> ≡`

```
public PixelBuffer Apply(PixelBuffer buffer) => this switch
{
    Filter f => Filters.Apply(f.Kind, buffer),
    Resample r => Resampler.Resample(buffer, r.Scale, r.Method),
    _ => buffer,
};
```

Used in chunk 260.

12.5.1 Formatting numbers

The Swift helper `fmt` lives in `Filters.swift` and uses `FormatStyle`, which formats for the user's locale. The port gives formatting its own small class, `Fmt`, used by the core's names and by the app's labels alike.

263 *⟨Windows/LineScope.Core/Formatting.cs 263⟩* ≡

```
using System.Globalization;

namespace LineScope.Core;

public static class Fmt
{
    ⟨Win Fmt: at most a few decimals 264⟩

    ⟨Win Fmt: a fixed number of decimals 265⟩
}

Root chunk, tangled to Windows/LineScope.Core/Formatting.cs.
```

Number reproduces `.precision(.fractionLength(0...3))` with a custom format string. In `#,0.###`, the comma asks for digit grouping, `0` forces at least one integer digit, and each `#` after the point is an optional decimal, so trailing zeros vanish: `2` becomes “2” and `0.125` becomes “0.125”. With no decimals allowed the format is just `#,0`.

The culture is passed explicitly. `CultureInfo.CurrentCulture` follows the regional settings of Windows, so with a Spanish locale a tab reads “Gaussian r=0,5” and a large size “1.024”, as the same user would expect from any other program; macOS’s formatter behaves the same way. Passing the culture, instead of relying on the implicit default, also documents that the choice is intended: the parameter dialog parses what the user types with the same culture (and accepts a period as well), so what is displayed can be typed back. How the last digit is rounded is left to each platform’s formatter.

264 *⟨Win Fmt: at most a few decimals 264⟩* ≡

```
/// <summary>Locale-aware number with grouping and at most <paramref name="maxFraction"/>
    ↪ decimals.</summary>
public static string Number(double v, int maxFraction = 3) =>
    v.ToString(maxFraction > 0 ? "#,0." + new string('#', maxFraction) : "#,0",
    ↪ CultureInfo.CurrentCulture);

Used in chunk 263.
```

The standard format `N` with a digit count gives grouping and exactly that many decimals, for readouts that always show the same number of decimals.

265 *⟨Win Fmt: a fixed number of decimals 265⟩* ≡

```
/// <summary>Locale-aware number with exactly <paramref name="digits"/> decimals.</summary>
public static string Fixed(double v, int digits) =>
    v.ToString("N" + digits, CultureInfo.CurrentCulture);

Used in chunk 263.
```

12.6 Lines and colors

The line sampler and the color conversions are the measuring instruments of LineScope, and they are ported without any change of algorithm (Chapter 5). The Swift structs become record structs, which again bring value equality for free.

266 *⟨Windows/LineScope.Core/LineSampler.cs 266⟩* ≡

```
namespace LineScope.Core;

⟨Win LineSampler: normalized point 267⟩

⟨Win LineSampler: line segment 268⟩

⟨Win LineSampler: sample record 269⟩

⟨Win LineSampler: sampler 270⟩
```

Root chunk, tangled to Windows/LineScope.Core/LineSampler.cs.

267 *⟨Win LineSampler: normalized point 267⟩* ≡

*/// <summary>A point relative to the image extent: (0,0) is the top-left corner, (1,1) the bottom-right
↪ corner.</summary>*

```
public readonly record struct NormalizedPoint(double X, double Y)
{
    public NormalizedPoint Clamped() => new(Math.Clamp(X, 0, 1), Math.Clamp(Y, 0, 1));
}
```

Used in chunk 266.

The Swift point and segment had mutable fields; these are `readonly`: moving an end point makes a new value, typically with a `with` expression, instead of assigning to a field.

268 *⟨Win LineSampler: line segment 268⟩* ≡

*/// <summary>The selection line, in normalized coordinates so it lands on the same content in every
↪ resampled copy.</summary>*

```
public readonly record struct LineSegment(NormalizedPoint Start, NormalizedPoint End)
{
    /// <summary>Full-width horizontal line at mid-height.</summary>
    public static readonly LineSegment Initial = new(new NormalizedPoint(0, 0.5), new NormalizedPoint(1,
        ↪ 0.5));

    public double PixelLength(int width, int height)
    {
        double dx = (End.X - Start.X) * width;
        double dy = (End.Y - Start.Y) * height;
        return Math.Sqrt(dx * dx + dy * dy);
    }
}
```

Used in chunk 266.

The sample record holds arrays and is a class (a plain record), since it is produced once per redraw and only read afterwards. Its field comments move into param tags.

269 *⟨Win LineSampler: sample record 269⟩* ≡

*/// <param name="Positions">Position along the line, 0 at Start and 1 at End.</param>
/// <param name="Colors">Sampled colors.</param>
/// <param name="PixelLength">Line length in the sampled image's pixels.</param>
/// <param name="Spacing">Distance between consecutive samples, in pixels ≈(1).</param>*

```
public sealed record LineSamples(double[] Positions, RGBA[] Colors, double PixelLength, double Spacing);
```

Used in chunk 266.

270 *⟨Win LineSampler: sampler 270⟩* ≡

```
public static class LineSampler
{
    ⟨Win LineSampler: sample along the line 271⟩

    ⟨Win LineSampler: bilinear interpolation 272⟩
}
```

Used in chunk 266.

The count of samples is $\max(2, \lceil L \rceil + 1)$ as before, with `Math.Ceiling` for `rounded(.up)`.

271 *⟨Win LineSampler: sample along the line 271⟩* ≡

```

/// <summary>
/// Samples the line in the buffer's own pixel grid: <c>ceil(length) + 1</c> bilinear samples,
/// so a half-size copy yields about half as many samples.
/// </summary>
public static LineSamples Sample(PixelBuffer buffer, LineSegment line)
{
    double w = buffer.Width, h = buffer.Height;
    double length = line.PixelLength(buffer.Width, buffer.Height);
    int count = Math.Max(2, (int)Math.Ceiling(length) + 1);
    double x0 = line.Start.X * w, y0 = line.Start.Y * h;
    double x1 = line.End.X * w, y1 = line.End.Y * h;

    var positions = new double[count];
    var colors = new Rgba[count];
    for (int i = 0; i < count; i++)
    {
        double t = (double)i / (count - 1);
        positions[i] = t;
        colors[i] = Bilinear(buffer, x0 + (x1 - x0) * t, y0 + (y1 - y0) * t);
    }
    return new LineSamples(positions, colors, length, length / (count - 1));
}

```

Used in chunk 270.

The bilinear interpolation keeps Swift's nested function `mix` as a C# local function, which captures the four indices and the two fractions in the same way. The fractions are converted to float before mixing, as in Swift, so the interpolated values agree bit for bit.

272 *⟨Win LineSampler: bilinear interpolation 272⟩* ≡

```

/// <summary>Bilinear interpolation at a continuous position (pixel centers at +0.5), edges
///   ↪ clamped.</summary>
public static Rgba Bilinear(PixelBuffer buffer, double x, double y)
{
    double fx = x - 0.5;
    double fy = y - 0.5;
    int ix = (int)Math.Floor(fx);
    int iy = (int)Math.Floor(fy);
    float tx = (float)(fx - ix);
    float ty = (float)(fy - iy);
    int x0 = Math.Clamp(ix, 0, buffer.Width - 1);
    int x1 = Math.Clamp(ix + 1, 0, buffer.Width - 1);
    int y0 = Math.Clamp(iy, 0, buffer.Height - 1);
    int y1 = Math.Clamp(iy + 1, 0, buffer.Height - 1);

    var d = buffer.Data;
    int w = buffer.Width;
    float Mix(int c)
    {
        float a = d[(y0 * w + x0) * 4 + c];
        float b = d[(y0 * w + x1) * 4 + c];
        float e = d[(y1 * w + x0) * 4 + c];
        float f = d[(y1 * w + x1) * 4 + c];
        float top = a + (b - a) * tx;
        float bottom = e + (f - e) * tx;
        return top + (bottom - top) * ty;
    }
    return new Rgba(Mix(0), Mix(1), Mix(2), Mix(3));
}

```

Used in chunk 270.

12.6.1 Color models

The color models follow the pattern of `ResampleMethod`: a bare enumeration, and a companion class of extension methods for the channel names and plot ranges. A value tuple (Min, Max) stands in for Swift's `ClosedRange`.

273 *<Windows/LineScope.Core/ColorSpaces.cs 273> ≡*

```
namespace LineScope.Core;
```

```
public enum ColorModel { Rgba, Hsl, Cmyk }
```

<Win ColorSpaces: names, channels and plot ranges 274>

```
public static class ColorConversion
```

```
{
```

```
    <Win ColorSpaces: HSL 275>
```

```
    <Win ColorSpaces: CMYK 276>
```

```
    <Win ColorSpaces: channel extraction 277>
```

```
}
```

Root chunk, tangled to Windows/LineScope.Core/ColorSpaces.cs.

274 *<Win ColorSpaces: names, channels and plot ranges 274> ≡*

```
public static class ColorModelInfo
```

```
{
```

```
    public static readonly ColorModel[] All = [ColorModel.Rgba, ColorModel.Hsl, ColorModel.Cmyk];
```

```
    public static string DisplayName(this ColorModel m) => m switch
```

```
    {
```

```
        ColorModel.Rgba => "RGBA",
```

```
        ColorModel.Hsl => "HSL",
```

```
        _ => "CMYK",
```

```
    };
```

```
    public static string[] Channels(this ColorModel m) => m switch
```

```
    {
```

```
        ColorModel.Rgba => ["Red", "Green", "Blue", "Alpha"],
```

```
        ColorModel.Hsl => ["Hue", "Saturation", "Lightness"],
```

```
        _ => ["Cyan", "Magenta", "Yellow", "Black"],
```

```
    };
```

```
    /// <summary>Plot range for a channel; hue is in degrees, everything else in ..01.</summary>
```

```
    public static (double Min, double Max) RangeOfChannel(this ColorModel m, int channel) =>
```

```
        m == ColorModel.Hsl && channel == 0 ? (0, 360) : (0, 1);
```

```
}
```

Used in chunk 273.

The HSL formulas are those of Chapter 5, and three details keep them exactly equal. The C# remainder operator `%` on floats truncates like Swift's `truncatingRemainder(dividingBy:)`, keeping the sign of the dividend, so the correction of a negative hue is still needed and still sufficient. The gray test is written `!(d > 1e-7f)` to behave like Swift's guard for every input, NaN included. And the sextant switch uses relational patterns, `< 1 =>`, which are tried in order exactly like Swift's case `..1:`.

275 *⟨Win ColorSpaces: HSL 275⟩* ≡*// HSL (hue in degrees)*

```

public static (float H, float S, float L) Hsl(Rgba c)
{
    float maxV = Math.Max(c.R, Math.Max(c.G, c.B));
    float minV = Math.Min(c.R, Math.Min(c.G, c.B));
    float l = (maxV + minV) / 2;
    float d = maxV - minV;
    if (!(d > 1e-7f)) return (0, 0, l);
    float s = d / (1 - MathF.Abs(2 * l - 1));
    float h;
    if (maxV == c.R) h = ((c.G - c.B) / d) % 6;
    else if (maxV == c.G) h = (c.B - c.R) / d + 2;
    else h = (c.R - c.G) / d + 4;
    h *= 60;
    if (h < 0) h += 360;
    return (h, Math.Clamp(s, 0, 1), l);
}

public static Rgba FromHsl(float h, float s, float l, float alpha = 1)
{
    float c = (1 - MathF.Abs(2 * l - 1)) * s;
    float hp = h / 60;
    float x = c * (1 - MathF.Abs(hp % 2 - 1));
    (float r, float g, float b) = hp switch
    {
        < 1 => (c, x, 0f),
        < 2 => (x, c, 0f),
        < 3 => (0f, c, x),
        < 4 => (0f, x, c),
        < 5 => (x, 0f, c),
        _ => (c, 0f, x),
    };
    float m = l - c / 2;
    return new Rgba(r + m, g + m, b + m, alpha);
}

```

Used in chunk 273.

The Swift inverses were both called `rgb` and told apart by their argument labels. C# overloads can't be distinguished by parameter names alone, and both take four floats, so the inverses are named `FromHsl` and `FromCmyk`.

276 *⟨Win ColorSpaces: CMYK 276⟩* ≡*// CMYK (naive device-independent conversion, K = 1 - max(R,G,B))*

```

public static (float C, float M, float Y, float K) Cmyk(Rgba c)
{
    float k = 1 - Math.Max(c.R, Math.Max(c.G, c.B));
    if (!(k < 1 - 1e-7f)) return (0, 0, 0, 1);
    float d = 1 - k;
    return ((1 - c.R - k) / d, (1 - c.G - k) / d, (1 - c.B - k) / d, k);
}

public static Rgba FromCmyk(float c, float m, float y, float k, float alpha = 1) =>
    new((1 - c) * (1 - k), (1 - m) * (1 - k), (1 - y) * (1 - k), alpha);

```

Used in chunk 273.

Swift picked the channel by indexing a temporary array. The port uses switch expressions, which allocate nothing; this function runs once per sample for every redraw of a profile.

277 *⟨Win ColorSpaces: channel extraction 277⟩* ≡

// Channel extraction

```
public static float Value(Rgba color, ColorModel model, int channel)
{
    switch (model)
    {
        case ColorModel.Rgba:
            return channel switch { 0 => color.R, 1 => color.G, 2 => color.B, _ => color.A };
        case ColorModel.Hsl:
            var v = Hsl(color);
            return channel switch { 0 => v.H, 1 => v.S, _ => v.L };
        default:
            var k = Cmyk(color);
            return channel switch { 0 => k.C, 1 => k.M, 2 => k.Y, _ => k.K };
    }
}
```

Used in chunk 273.

12.7 The spectrum

The spectrum keeps everything of Chapter 5 that defines the numbers on the screen: the zero padding to a power of two, the optional mean removal, the symmetric Hann window, the single-sided amplitudes divided by the window sum, and the two frequency axes. What changes is the transform underneath. vDSP isn't available, and rather than take a dependency for one function, the port carries its own FFT.

278 *⟨Windows/LineScope.Core/Spectrum.cs 278⟩* ≡

using System.Numerics;

namespace LineScope.Core;

⟨Win Spectrum: window functions 279⟩

⟨Win Spectrum: result 280⟩

⟨Win Spectrum: analyzer 281⟩

⟨Win Spectrum: the radix-2 FFT 287⟩

Root chunk, tangled to Windows/LineScope.Core/Spectrum.cs.

279 *⟨Win Spectrum: window functions 279⟩* ≡

public enum WindowFunction { None, Hann }

```
public static class WindowFunctionInfo
{
```

```
    public static readonly WindowFunction[] All = [WindowFunction.None, WindowFunction.Hann];
```

```
    public static string DisplayName(this WindowFunction w) => w == WindowFunction.None ? "Rectangular"
        ↪ : "Hann";
```

```
}
```

Used in chunk 278.

The peak search is written as a loop that replaces the best bin only when a later one is strictly larger. That is how Swift's `max(by:)` resolves ties, so both versions report the *first* of several equal peaks.

280 *⟨Win Spectrum: result 280⟩* ≡

```

/// <param name="CyclesPerPixel">Bin frequencies in cycles per pixel of the sampled image (Nyquist ≈
    ↪ 0.5).</param>
/// <param name="CyclesPerLine">Bin frequencies in cycles over the whole line; comparable across
    ↪ resampled copies.</param>
/// <param name="Amplitudes">Single-sided amplitude: a sine of amplitude A peaks at ≈ A.</param>
public sealed record SpectrumResult(
    double[] CyclesPerPixel, double[] CyclesPerLine, double[] Amplitudes, int FftSize, int SampleCount)
{
    public static readonly SpectrumResult Empty = new([], [], [], 0, 0);

    /// <summary>Index of the largest non-DC bin.</summary>
    public int? PeakIndex
    {
        get
        {
            if (Amplitudes.Length <= 1) return null;
            int best = 1;
            for (int i = 2; i < Amplitudes.Length; i++)
                if (Amplitudes[best] < Amplitudes[i]) best = i;
            return best;
        }
    }
}

```

Used in chunk 278.

The analyzer has the five steps of the Swift version; the zero padding is now folded into the transform step.

281 *⟨Win Spectrum: analyzer 281⟩* ≡

```

public static class SpectrumAnalyzer
{
    /// <summary>Real FFT of a profile, zero-padded to the next power of two.</summary>
    /// <param name="spacing">Distance between samples, in pixels.</param>
    public static SpectrumResult Compute(float[] values, double spacing, WindowFunction window, bool
        ↪ removeMean)
    {
        ⟨Win Spectrum: choose the FFT size 282⟩

        ⟨Win Spectrum: condition the signal 283⟩

        ⟨Win Spectrum: complex FFT of the padded signal 284⟩

        ⟨Win Spectrum: single-sided amplitudes 285⟩

        ⟨Win Spectrum: frequency axes 286⟩
    }
}

```

Used in chunk 278.

282 *⟨Win Spectrum: choose the FFT size 282⟩* ≡

```

int n = values.Length;
if (n < 2) return SpectrumResult.Empty;
int log2n = (int)Math.Ceiling(Math.Log2(n));
int fftSize = 1 << log2n;
int half = fftSize / 2;

```

Used in chunk 281.

Mean removal and windowing are written as loops where Swift called `vdSP.mean`, `vdSP.add`, `vdSP.multiply` and `vdSP.sum`. The formulas are the same, but a sum computed in a different order (`vdSP`'s is vectorized, LINQ's `Sum` accumulates in double) can differ in its last bits. This is the

one place where a profile's spectrum may differ from macOS at all, and only at the level of float rounding.

283 *⟨Win Spectrum: condition the signal 283⟩* ≡

```
var signal = (float[])values.Clone();
if (removeMean)
{
    float mean = signal.Sum() / n;
    for (int i = 0; i < n; i++) signal[i] -= mean;
}
float windowSum = n;
if (window == WindowFunction.Hann)
{
    // Symmetric Hann over the n real samples.
    windowSum = 0;
    for (int i = 0; i < n; i++)
    {
        float w = 0.5f - 0.5f * MathF.Cos(2 * MathF.PI * i / (n - 1));
        signal[i] *= w;
        windowSum += w;
    }
}
```

Used in chunk 281.

The conditioned samples become the real parts of a complex array of length N ; the rest of the array, and every imaginary part, is zero. That is the zero padding, and it happens by itself, because a new array of `Complex` is zero-initialized.

Swift's version used vDSP's *packed real* FFT, which transforms the N real samples as $N/2$ complex numbers and unpacks the result. The port transforms the full complex array instead. This does about twice the arithmetic, since half of the input is known to be zero, but a profile has at most a few thousand samples and the transform takes microseconds; the simpler code wins. `Complex` is also double precision, where vDSP worked in `float`, so the port's transform is if anything slightly more accurate.

284 *⟨Win Spectrum: complex FFT of the padded signal 284⟩* ≡

```
var buffer = new Complex[fftSize];
for (int i = 0; i < n; i++) buffer[i] = signal[i];
Fft.Forward(buffer);
```

Used in chunk 281.

Here the two normalizations meet. vDSP's packed output held $2X_k$ for $0 < k < N/2$, and squeezed the two real bins into one slot, $2X_0$ in the real part and $2X_{N/2}$ in the imaginary part of element 0; the Swift code therefore unpacked element 0 and divided everything by 2. The complex FFT returns the plain, unscaled $X_k = \sum_j x_j e^{-2\pi i j k / N}$, with X_0 at index 0 and $X_{N/2}$ at index $N/2$, each in its own place. Both the unpacking and the halving disappear, and what remains is literally the formula of Chapter 5:

$$A_0 = \frac{|X_0|}{\sum_i w_i}, \quad A_k = \frac{2|X_k|}{\sum_i w_i} \quad (0 < k < N/2), \quad A_{N/2} = \frac{|X_{N/2}|}{\sum_i w_i}.$$

The bins above $N/2$ are the complex conjugates of those below, because the input is real, and are ignored.

285 *⟨Win Spectrum: single-sided amplitudes 285⟩* ≡

```
var amplitudes = new double[half + 1];
double norm = Math.Max(windowSum, 1e-12);
amplitudes[0] = buffer[0].Magnitude / norm;
amplitudes[half] = buffer[half].Magnitude / norm;
for (int k = 1; k < Math.Max(half, 1); k++)
    amplitudes[k] = 2 * buffer[k].Magnitude / norm;
```

Used in chunk 281.

286 *⟨Win Spectrum: frequency axes 286⟩* ≡

```
double lineLength = spacing * (n - 1);
var cpp = new double[half + 1];
var cpl = new double[half + 1];
for (int k = 0; k <= half; k++)
{
    cpp[k] = k / (fftSize * spacing);
    cpl[k] = cpp[k] * lineLength;
}
return new SpectrumResult(cpp, cpl, amplitudes, fftSize, n);
```

Used in chunk 281.

12.7.1 The FFT

The transform is the iterative radix-2 algorithm of Cooley and Tukey [7], in its decimation-in-time form. Its idea is to split a DFT of length N into the DFTs of the even-indexed and the odd-indexed samples, E_k and O_k , each of length $N/2$:

$$X_k = E_k + W_N^k O_k, \quad X_{k+N/2} = E_k - W_N^k O_k, \quad W_N = e^{-2\pi i/N}, \quad 0 \leq k < N/2.$$

Applied recursively down to length 1, this costs $O(N \log N)$ operations instead of the $O(N^2)$ of the definition. The iterative version runs the recursion bottom up and in place: first it reorders the input so that every sub-transform finds its samples next to each other, then it combines neighbors into transforms of length 2, 4, 8, ..., N .

287 *⟨Win Spectrum: the radix-2 FFT 287⟩* ≡

```
/// <summary>In-place iterative radix-2 Cooley-Tukey FFT (replaces vDSP on macOS).</summary>
internal static class Fft
{
    /// <summary>Forward DFT,  $X[k] = \sum x[j] \cdot e^{-\pi i (2i \cdot jk/N)}$ , unscaled. The length must be a power of
    ↪ two.</summary>
    public static void Forward(Complex[] a)
    {
        int n = a.Length;
        if (n <= 1) return;
        if ((n & (n - 1)) != 0) throw new ArgumentException("FFT length must be a power of two.");

        ⟨Win Fft: bit-reversal permutation 288⟩

        ⟨Win Fft: butterflies 289⟩
    }
}
```

Used in chunk 278.

Splitting into even and odd samples at every level means that, after all the splits, sample j ends up at the position whose binary digits are those of j reversed. For $N = 8$, sample $1 = 001_2$ goes to position $100_2 = 4$, and sample $3 = 011_2$ to $110_2 = 6$. The loop walks $i = 1, \dots, N - 1$ while keeping j equal to the bit reversal of i . Incrementing a reversed number means adding 1 at its *top* bit and carrying downward: clear the leading ones until the first zero, then set that zero. The inner `for` clears the ones, and the `j ^= bit` after it sets the zero. Each pair is swapped once, when $i < j$; positions with $i = j$ stay put.

288 $\langle \text{Win Fft: bit-reversal permutation 288} \rangle \equiv$

```
// Bit-reversal permutation.
for (int i = 1, j = 0; i < n; i++)
{
    int bit = n >> 1;
    for (; (j & bit) != 0; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) (a[i], a[j]) = (a[j], a[i]);
}
```

Used in chunk 287.

Each stage combines pairs of transforms of length $len/2$, lying next to each other, into one transform of length len , by the *butterfly* above: u is E_k , v is $W^k O_k$, and they are replaced by $u + v$ and $u - v$. The twiddle factor $w = W_{len}^k$ starts at 1 for every block and is advanced by one multiplication with $W_{len} = e^{-2\pi i/len}$ per step, instead of calling `Math.Cos` and `Math.Sin` for every k . The recurrence accumulates rounding error proportional to the block length, which in double precision stays far below the float precision of the input for any length LineScope uses.

The output is unscaled, as the documentation comment states, and that is what the amplitude normalization above assumes. The tests check the transform against a direct evaluation of the DFT's definition.

289 $\langle \text{Win Fft: butterflies 289} \rangle \equiv$

```
for (int len = 2; len <= n; len <= 1)
{
    double angle = -2 * Math.PI / len;
    var wLen = new Complex(Math.Cos(angle), Math.Sin(angle));
    int halfLen = len >> 1;
    for (int i = 0; i < n; i += len)
    {
        Complex w = Complex.One;
        for (int k = 0; k < halfLen; k++)
        {
            Complex u = a[i + k];
            Complex v = a[i + k + halfLen] * w;
            a[i + k] = u + v;
            a[i + k + halfLen] = u - v;
            w *= wLen;
        }
    }
}
```

Used in chunk 287.

12.8 Test patterns

The test patterns are ported formula for formula from Chapter 6, and the same specification produces the same image on both systems, noise included. The file has the same parts as the Swift one, plus a record for the pattern parameter.

290 $\langle \text{Windows/LineScope.Core/TestPattern.cs 290} \rangle \equiv$

```
namespace LineScope.Core;
```

$\langle \text{Win TestPattern: pattern kinds 291} \rangle$

$\langle \text{Win TestPattern: the parameter record 292} \rangle$

$\langle \text{Win TestPattern: names, purposes and parameters 293} \rangle$

$\langle \text{Win TestPattern: specification 297} \rangle$

⟨Win TestPattern: generator 298⟩

⟨Win TestPattern: SplitMix64 305⟩

Root chunk, tangled to Windows/LineScope.Core/TestPattern.cs.

291 *⟨Win TestPattern: pattern kinds 291⟩* ≡

/// <summary>Synthetic images commonly used to evaluate resampling and filtering.</summary>

```
public enum TestPatternKind
{
    ZonePlate,
    LinearChirp,
    SineGrating,
    SquareGrating,
    Checkerboard,
    StepEdge,
    SlantedEdge,
    Impulses,
    SiemensStar,
    WhiteNoise,
    HueRamp,
}
```

Used in chunk 290.

Swift described a parameter with a named tuple whose range was a `ClosedRange`. The port gives it a small record with a `Clamp` method, which is what the range was used for.

292 *⟨Win TestPattern: the parameter record 292⟩* ≡

/// <summary>The single tunable parameter of a pattern.</summary>

```
public sealed record PatternParameter(string Name, double DefaultValue, double Min, double Max)
{
    public double Clamp(double v) => Math.Clamp(v, Min, Max);
}
```

Used in chunk 290.

The names, the one-sentence purposes and the parameters are the Swift tables, as extension methods.

293 *⟨Win TestPattern: names, purposes and parameters 293⟩* ≡

```
public static class TestPatternKindInfo
{
    public static readonly TestPatternKind[] All = Enum.GetValues<TestPatternKind>();

    ⟨Win TestPattern: display names 294⟩

    ⟨Win TestPattern: what each pattern is for 295⟩

    ⟨Win TestPattern: the tunable parameter 296⟩
}
```

Used in chunk 290.

294 *⟨Win TestPattern: display names 294⟩* ≡

```
public static string DisplayName(this TestPatternKind k) => k switch
{
    TestPatternKind.ZonePlate => "Zone Plate",
    TestPatternKind.LinearChirp => "Linear Chirp",
    TestPatternKind.SineGrating => "Sine Grating",
    TestPatternKind.SquareGrating => "Square-Wave Bars",
    TestPatternKind.Checkerboard => "Checkerboard",
    TestPatternKind.StepEdge => "Step Edge",
    TestPatternKind.SlantedEdge => "Slanted Edge",
    TestPatternKind.Impulses => "Impulse Grid",
}
```

```

TestPatternKind.SiemensStar => "Siemens Star",
TestPatternKind.WhiteNoise => "White Noise",
_ => "Hue / Lightness Ramp",
};

```

Used in chunk 293.

295 *⟨Win TestPattern: what each pattern is for 295⟩* ≡

```

/// <summary>What the pattern is good for, shown in the generator window.</summary>
public static string Purpose(this TestPatternKind k) => k switch
{
    TestPatternKind.ZonePlate => "Circular chirp: frequency grows with radius up to the chosen maximum.
    ↪ Aliasing appears as false ring centers.",
    TestPatternKind.LinearChirp => "Horizontal sweep from 0 to the chosen frequency. Put the line across
    ↪ it to read a filter's frequency response.",
    TestPatternKind.SineGrating => "Single pure frequency along x. The spectrum should show one peak.",
    TestPatternKind.SquareGrating => "Bars with odd harmonics. Shows how filters treat harmonics near
    ↪ Nyquist.",
    TestPatternKind.Checkerboard => "Two-dimensional square wave. Small cells alias badly under
    ↪ nearest-neighbor.",
    TestPatternKind.StepEdge => "One hard vertical edge. Reveals ringing (sinc, Lanczos) versus blur
    ↪ (tent, Mitchell).",
    TestPatternKind.SlantedEdge => "Edge tilted a few degrees, the standard target for measuring
    ↪ sharpness (MTF).",
    TestPatternKind.Impulses => "Isolated single pixels. Each dot shows the filter's point-spread
    ↪ function.",
    TestPatternKind.SiemensStar => "Radial spokes: spatial frequency rises toward the center, in every
    ↪ orientation.",
    TestPatternKind.WhiteNoise => "Uniform random values with a flat spectrum. The profile's spectrum
    ↪ shows the filter's transfer curve.",
    _ => "Hue across x, lightness down y. Use it with the HSL and CMYK channels.",
};

```

Used in chunk 293.

296 *⟨Win TestPattern: the tunable parameter 296⟩* ≡

```

/// <summary>The single tunable parameter, or null if the pattern has none.</summary>
public static PatternParameter? Parameter(this TestPatternKind k) => k switch
{
    TestPatternKind.ZonePlate or TestPatternKind.LinearChirp => new("Max frequency (cycles/px)", 0.5,
    ↪ 0.01, 1),
    TestPatternKind.SineGrating => new("Frequency (cycles/px)", 0.1, 0.001, 1),
    TestPatternKind.SquareGrating => new("Period (px)", 8, 2, 512),
    TestPatternKind.Checkerboard => new("Cell size (px)", 8, 1, 512),
    TestPatternKind.SlantedEdge => new("Angle (degrees)", 5, -45, 45),
    TestPatternKind.Impulses => new("Spacing (px)", 32, 2, 1024),
    TestPatternKind.SiemensStar => new("Spokes", 36, 4, 360),
    TestPatternKind.WhiteNoise => new("Seed", 1, 0, 1_000_000),
    _ => null,
};

```

Used in chunk 293.

The specification is a readonly record struct, so it compares and hashes by value, as the Swift Hashable struct did. Its positional constructor takes all four values. A second constructor supplies the defaults and a nullable parameter, which becomes the kind's default, like Swift's `parameter: Double? = nil`; a call with a double for the last argument picks the positional constructor, one without it the second. `Clamped` is new in the port; it forces the parameter into its range with `with`, which copies a record and changes one field.

297 *⟨Win TestPattern: specification 297⟩* ≡

```
public readonly record struct TestPatternSpec(TestPatternKind Kind, int Width, int Height, double
    ↪ Parameter)
{
    public TestPatternSpec(TestPatternKind kind, int width = 512, int height = 512, double? parameter =
        ↪ null)
        : this(kind, width, height, parameter ?? kind.Parameter()?.DefaultValue ?? 0) { }

    public string Title
    {
        get
        {
            var t = $"{Kind.DisplayName()} {Width}x{Height}";
            var p = Kind.Parameter();
            if (p is not null && (Kind != TestPatternKind.WhiteNoise || Parameter != p.DefaultValue))
                t += $" ({Fmt.Number(Parameter)})";
            return t;
        }
    }

    /// <summary>The spec with its parameter clamped to the kind's range.</summary>
    public TestPatternSpec Clamped()
    {
        var p = Kind.Parameter();
        return p is null ? this : this with { Parameter = p.Clamp(Parameter) };
    }
}
```

Used in chunk 290.

12.8.1 The generator

The generator has the Swift structure: set up the canvas, define the gray-level helper, switch on the kind. The helper `RoundInt` at the end of the class is the port's spelling of Swift's `Int(p.rounded())`. Without `MidpointRounding.AwayFromZero`, a checkerboard with cells of 2.5 pixels would get cells of 2 on Windows and 3 on macOS.

298 *⟨Win TestPattern: generator 298⟩* ≡

```
public static class TestPatternGenerator
{
    public static PixelBuffer Make(TestPatternSpec spec)
    {
        ⟨Win TestPattern: set up the canvas 299⟩

        ⟨Win TestPattern: gray-level helper 300⟩

        switch (spec.Kind)
        {
            ⟨Win TestPattern: chirps and gratings 301⟩
            ⟨Win TestPattern: edges and impulses 302⟩
            ⟨Win TestPattern: Siemens star 303⟩
            ⟨Win TestPattern: noise and color ramp 304⟩
        }
        return buf;
    }

    private static int RoundInt(double v) => (int)Math.Round(v, MidpointRounding.AwayFromZero);
}
```

Used in chunk 290.

Writing `w / 2.0` instead of `w / 2` matters in C#: with two integers the division would truncate, and the center of an odd-sized image would move by half a pixel.

299 *⟨Win TestPattern: set up the canvas 299⟩* ≡

```
int w = Math.Max(1, spec.Width);
int h = Math.Max(1, spec.Height);
double p = spec.Parameter;
var buf = new PixelBuffer(w, h);
double cx = w / 2.0;
double cy = h / 2.0;
```

Used in chunk 298.

The helper still evaluates f at pixel centers, but over rows in parallel. That is safe because every pattern function is pure and each iteration writes only its own row.

300 *⟨Win TestPattern: gray-level helper 300⟩* ≡

```
void Gray(Func<double, double, double> f)
{
    Parallel.For(0, h, y =>
    {
        for (int x = 0; x < w; x++)
        {
            // Evaluate at pixel centers.
            float v = (float)Math.Clamp(f(x + 0.5, y + 0.5), 0, 1);
            buf[x, y] = new Rgba(v, v, v);
        }
    });
}
```

Used in chunk 298.

The cases are the Swift ones. Each case that declares locals gets braces, for the scoping reason given with the resampler. The square grating's % on doubles truncates exactly like truncatingRemainder, and the casts (int)x truncate like Swift's Int(x).

301 *⟨Win TestPattern: chirps and gratings 301⟩* ≡

```
case TestPatternKind.ZonePlate:
{
    // Local frequency k·r reaches p at r = max(w,h)/2 (the left/right edges of the midline).
    double k = p / (Math.Max(w, h) / 2.0);
    Gray((x, y) =>
    {
        double r2 = (x - cx) * (x - cx) + (y - cy) * (y - cy);
        return 0.5 + 0.5 * Math.Cos(Math.PI * k * r2);
    });
    break;
}
case TestPatternKind.LinearChirp:
    // Phase  $\pi^2 \cdot p \cdot x^2 / (2W)$ : instantaneous frequency  $p \cdot x / W$ .
    Gray((x, _) => 0.5 + 0.5 * Math.Cos(Math.PI * p * x * x / w));
    break;
case TestPatternKind.SineGrating:
    Gray((x, _) => 0.5 + 0.5 * Math.Sin(2 * Math.PI * p * x));
    break;
case TestPatternKind.SquareGrating:
{
    double period = Math.Max(p, 1);
    Gray((x, _) => (x - 0.5) % period < period / 2 ? 1 : 0);
    break;
}
case TestPatternKind.Checkerboard:
{
    int cell = Math.Max(RoundInt(p), 1);
    Gray((x, y) => (((int)x / cell) + ((int)y / cell)) % 2 == 0 ? 1 : 0);
    break;
}
```

Used in chunk 298.

302 *⟨Win TestPattern: edges and impulses 302⟩* ≡

```

case TestPatternKind.StepEdge:
    Gray((x, _) => x < cx ? 0.1 : 0.9);
    break;
case TestPatternKind.SlantedEdge:
{
    // Dark/light halves split by a line through the center tilted from vertical.
    double a = p * Math.PI / 180;
    Gray((x, y) => ((x - cx) * Math.Cos(a) + (y - cy) * Math.Sin(a)) < 0 ? 0.1 : 0.9);
    break;
}
case TestPatternKind.Impulses:
{
    int s = Math.Max(RoundInt(p), 1);
    int off = s / 2;
    Gray((x, y) => ((int)x % s == off && (int)y % s == off) ? 1 : 0);
    break;
}

```

Used in chunk 298.

The Swift guard that keeps the star inside its circle becomes an early return with the test inverted.

303 *⟨Win TestPattern: Siemens star 303⟩* ≡

```

case TestPatternKind.SiemensStar:
{
    double spokes = Math.Max(Math.Round(p, MidpointRounding.AwayFromZero), 2);
    double radius = Math.Min(w, h) / 2.0;
    Gray((x, y) =>
    {
        double dx = x - cx, dy = y - cy;
        if (dx * dx + dy * dy > radius * radius) return 0.5;
        double theta = Math.Atan2(dy, dx);
        return Math.Sin(spokes * theta) >= 0 ? 1 : 0;
    });
    break;
}

```

Used in chunk 298.

White noise is the one gray pattern that is *not* generated in parallel. Its value at a pixel is the k -th output of the generator, where k counts pixels in reading order, so the loops must run in that order to reproduce the Swift image from the same seed. The conversion to $[0, 1)$ is the same: the top 53 bits divided by 2^{53} , written `1UL << 53` so the shift is done on a 64-bit unsigned integer.

304 *⟨Win TestPattern: noise and color ramp 304⟩* ≡

```

case TestPatternKind.WhiteNoise:
{
    var rng = new SplitMix64((ulong)Math.Max(p, 0));
    for (int y = 0; y < h; y++)
        for (int x = 0; x < w; x++)
        {
            float v = (float)((rng.Next() >> 11) / (double)(1UL << 53));
            buf[x, y] = new Rgba(v, v, v);
        }
    break;
}
case TestPatternKind.HueRamp:
    for (int y = 0; y < h; y++)
        for (int x = 0; x < w; x++)
        {
            float hue = (float)((x + 0.5) / w * 360);
            float light = (float)(1 - (y + 0.5) / h);
            var c = ColorConversion.FromHsl(Math.Min(hue, 359.999f), 1, light);

```

```

        buf[x, y] = new Rgba(Math.Clamp(c.R, 0, 1), Math.Clamp(c.G, 0, 1), Math.Clamp(c.B, 0, 1));
    }
    break;

```

Used in chunk 298.

12.8.2 SplitMix64

The generator is SplitMix64 with the same constants. Swift needs the wrapping operators `&+=` and `&*`, because its ordinary arithmetic traps on overflow. C# arithmetic wraps by default, but a project can turn on overflow checking for the whole assembly, and then the multiplications would throw. The unchecked block states that wrapping modulo 2^{64} is the intended arithmetic, whatever the project settings.

The struct uses a primary constructor to take the seed. It is mutable, and `rng` above is a local variable, so each call to `Next` advances the one generator rather than a copy.

305 *⟨Win TestPattern: SplitMix64 305⟩* ≡

/// <summary>Small deterministic generator so noise patterns are reproducible from their seed.</summary>

```

internal struct SplitMix64(ulong seed)
{
    private ulong _state = seed;

    public ulong Next()
    {
        unchecked
        {
            _state += 0x9E37_79B9_7F4A_7C15;
            ulong z = _state;
            z = (z ^ (z >> 30)) * 0xBF58_476D_1CE4_E5B9;
            z = (z ^ (z >> 27)) * 0x94D0_49BB_1331_11EB;
            return z ^ (z >> 31);
        }
    }
}

```

Used in chunk 290.

Chapter 13

Tests in C#

The tests of the core are ported along with the core. Their purpose is the one stated in Chapter 7: each test pins down a property the rest of the program depends on. For the port there is a second purpose. Since the algorithms are meant to be identical on both systems, a test that passes in Swift and in C# with the same inputs, expected values and tolerances is evidence that the transcription is faithful. So every XCTest test has a C# twin with the same numbers. The port also adds tests of its own, for the two pieces that are new: the FFT and the CPU filters that replace Core Image.

13.1 The test project

The tests use xUnit, the most common test framework for .NET. The project references the test SDK (which lets `dotnet test` find and run the tests), xUnit itself, and its runner for Visual Studio and the command line. It references the core project, not the app, so the tests run without WPF. The `Using` item imports the `xunit` namespace into every file, the way `import XCTest` does in Swift.

The Swift tests used `@testable import` to reach internal members. The C# counterpart is on the other side: the core's project file grants `InternalsVisibleTo` to this test assembly (Chapter 12), which lets the tests call the internal `Fft` class and the filter kernels.

306 `<Windows/LineScope.Core.Tests/LineScope.Core.Tests.csproj 306> ≡`

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <IsPackable>false</IsPackable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\LineScope.Core\LineScope.Core.csproj" />
  </ItemGroup>

  <ItemGroup>
    <Using Include="Xunit" />
  </ItemGroup>

</Project>
```

Root chunk, tangled to Windows/LineScope.Core.Tests/LineScope.Core.Tests.csproj.

13.2 The test file

The file keeps the structure of the Swift one: a class per area, a method per property. In xUnit a test is a public method marked `[Fact]`, and the class needs no base class. The method names drop Swift's test prefix, which XCTest needed to discover them. A single test is run with a filter on its full name:

```
dotnet test LineScope.Core.Tests --filter "FullyQualifiedName~SpectrumTests.SinePeak"
```

307 *< Windows/LineScope.Core.Tests/LineScopeCoreTests.cs 307 >* $\equiv$

```
using LineScope.Core;
```

```
namespace LineScope.Core.Tests;
```

```
< Win Tests: helpers 308 >
```

```
< Win Tests: spectrum 309 >
```

```
< Win Tests: resampling 311 >
```

```
< Win Tests: color conversions 316 >
```

```
< Win Tests: line sampler 317 >
```

```
< Win Tests: bitmap bridge 318 >
```

```
< Win Tests: filters 319 >
```

```
< Win Tests: test patterns 325 >
```

Root chunk, tangled to Windows/LineScope.Core.Tests/LineScopeCoreTests.cs.

The gradient fixture is the one of Chapter 7: pixel (x, y) is $(x/W, y/H, \frac{1}{4})$, opaque. xUnit has no counterpart of XCTest's `XCTAssertEqual(_:_:accuracy:)`, so a small helper provides it: `Near` fails unless the two values differ by at most the accuracy, and the optional label says which case failed when a test loops over methods or bins.

308 *< Win Tests: helpers 308 >* $\equiv$

```
internal static class Helpers
```

```
{
```

```
    public static PixelBuffer Gradient(int width, int height)
```

```
    {
```

```
        var buf = new PixelBuffer(width, height);
```

```
        for (int y = 0; y < height; y++)
```

```
            for (int x = 0; x < width; x++)
```

```
                buf[x, y] = new Rgba((float)x / width, (float)y / height, 0.25f, 1);
```

```
        return buf;
```

```
    }
```

```
    public static void Near(double expected, double actual, double accuracy, string? label = null) =>
```

```
        Assert.True(Math.Abs(expected - actual) <= accuracy,
```

```
            $"{label} expected {expected} ± {accuracy}, got {actual}");
```

```
}
```

Used in chunk 307.

13.3 Spectrum

The first two tests are the Swift ones: a sine of $k = 16$ periods in $n = 256$ samples must put an amplitude of 1 in bin 16, at $16/256$ cycles per pixel, with the last bin at Nyquist; and a constant 0.3 with the mean kept must give amplitude 0.3 in bin 0, with 100 samples padded to an FFT of 128. They check the normalization and the frequency axis, which the port keeps from vDSP's conventions, and they pass unchanged.

309 *⟨Win Tests: spectrum 309⟩* ≡

```
public class SpectrumTests
{
    [Fact]
    public void SinePeak()
    {
        const int n = 256, k = 16;
        var values = Enumerable.Range(0, n).Select(i => (float)Math.Sin(2 * Math.PI * k * i /
        ↪ n)).ToArray();
        var s = SpectrumAnalyzer.Compute(values, 1, WindowFunction.None, removeMean: true);
        Assert.Equal(256, s.FftSize);
        Assert.Equal(k, s.PeakIndex);
        Helpers.Near(1, s.Amplitudes[k], 1e-3);
        Helpers.Near((double)k / n, s.CyclesPerPixel[k], 1e-9);
        Helpers.Near(0.5, s.CyclesPerPixel[^1], 1e-9);
    }

    [Fact]
    public void DCAmplitude()
    {
        var s = SpectrumAnalyzer.Compute(Enumerable.Repeat(0.3f, 100).ToArray(), 1, WindowFunction.None,
        ↪ removeMean: false);
        Assert.Equal(128, s.FftSize);
        Helpers.Near(0.3, s.Amplitudes[0], 1e-4);
    }
}

⟨Win Tests: the FFT against a naive DFT 310⟩
}
```

Used in chunk 307.

These tests would not catch every mistake in a hand-written FFT. A pure sine concentrates its energy in one bin, and several classic errors, such as a wrong twiddle factor in a later stage, leave a sine's peak in the right place while scrambling the other bins. The macOS version didn't need more, since vDSP is a trusted library; the port's own FFT (Chapter 12) does.

The new test compares the FFT with the *definition* of the discrete Fourier transform,

$$X_k = \sum_{j=0}^{N-1} x_j e^{-2\pi i j k / N}, \quad k = 0, \dots, N-1,$$

computed directly, in $O(N^2)$ operations, for $N = 64$ random real samples. The random generator is seeded, so the test is repeatable. Every bin must agree to 10^{-9} . In double precision the rounding error of both computations is around 10^{-14} for sums of this size, so the tolerance is generous for a correct FFT and far too tight for a wrong one, whose errors are of the order of the values themselves. The inputs are real because the analyzer only ever transforms real profiles.

310 *⟨Win Tests: the FFT against a naive DFT 310⟩* ≡

```
[Fact]
public void FftMatchesNaiveDft()
{
    var rng = new Random(3);
    var x = Enumerable.Range(0, 64).Select(_ => new System.Numerics.Complex(rng.NextDouble(),
    ↪ 0)).ToArray();
    var fast = (System.Numerics.Complex[])x.Clone();
    Fft.Forward(fast);
    for (int k = 0; k < x.Length; k++)
    {
        System.Numerics.Complex sum = 0;
        for (int j = 0; j < x.Length; j++)
            sum += x[j] * System.Numerics.Complex.FromPolarCoordinates(1, -2 * Math.PI * j * k /
    ↪ x.Length);
        Helpers.Near(0, (sum - fast[k]).Magnitude, 1e-9, $"bin {k}");
    }
}
```

Used in chunk 309.

13.4 Resampling

The four resampling tests are transcriptions of the Swift ones, and Chapter 7 derives their expected values: the interpolating kernels are 1 at 0 and 0 at the other integers, Mitchell-Netravali gives $k(0) = \frac{8}{9}$ and $k(1) = \frac{1}{18}$, Catmull-Rom gives $k(\frac{1}{2}) = \frac{9}{16}$, and the sinc is truncated at its radius; every method but Mitchell is the identity at scale 1; the area method averages exact pairs at scale $\frac{1}{2}$; and a constant image stays constant at every scale. The kernel function is internal, as in Swift.

311 *⟨Win Tests: resampling 311⟩* ≡

```
public class ResamplingTests
{
    ⟨Win Tests: kernel shapes 312⟩

    ⟨Win Tests: identity at scale one 313⟩

    ⟨Win Tests: area average halves 314⟩

    ⟨Win Tests: constants stay constant 315⟩
}
```

Used in chunk 307.

312 *⟨Win Tests: kernel shapes 312⟩* ≡

```
[Fact]
public void KernelShapes()
{
    // Interpolating kernels are 1 at 0 and 0 at nonzero integers.
    foreach (var method in new[] { ResampleMethod.Tent, ResampleMethod.CatmullRom,
    ↪ ResampleMethod.Lanczos3, ResampleMethod.Sinc })
    {
        Helpers.Near(1, Resampler.Kernel(method, 0), 1e-12, method.ToString());
        for (int k = 1; k <= 3; k++)
            Helpers.Near(0, Resampler.Kernel(method, k), 1e-12, $"{method} at {k}");
    }

    // Mitchell-Netravali (B = C = 1/3) is approximating: k(0) = 8/9, k(1) = 1/18.
    Helpers.Near(8.0 / 9, Resampler.Kernel(ResampleMethod.Mitchell, 0), 1e-12);
    Helpers.Near(1.0 / 18, Resampler.Kernel(ResampleMethod.Mitchell, 1), 1e-12);
    Helpers.Near(0, Resampler.Kernel(ResampleMethod.Mitchell, 2), 1e-12);
    Helpers.Near(0.5625, Resampler.Kernel(ResampleMethod.CatmullRom, 0.5), 1e-12);
}
```

```

// Sinc is truncated at its radius.
Assert.Equal(0, Resampler.Kernel(ResampleMethod.Sinc, Resampler.SincRadius + 0.5));
}

Used in chunk 311.

```

`ResampleMethodInfo.All` stands in for Swift's `ResampleMethod.allCases`: a C# enumeration has no built-in list of its cases, so the core provides one.

313 *⟨Win Tests: identity at scale one 313⟩* ≡

```

[Fact]
public void IdentityAtScaleOne()
{
    var src = Helpers.Gradient(17, 9);
    // Mitchell is approximating (slightly smoothing), so it is not an identity at x1.
    foreach (var method in ResampleMethodInfo.All.Where(m => m != ResampleMethod.Mitchell))
    {
        var output = Resampler.Resample(src, 1, method);
        Assert.Equal(17, output.Width);
        Assert.Equal(9, output.Height);
        for (int i = 0; i < src.Data.Length; i++)
            Helpers.Near(src.Data[i], output.Data[i], 1e-5, method.ToString());
    }
}

Used in chunk 311.

```

314 *⟨Win Tests: area average halves 314⟩* ≡

```

[Fact]
public void AreaHalvesAverages()
{
    var src = new PixelBuffer(4, 1);
    float[] values = {0, 1, 0.2f, 0.6f};
    for (int x = 0; x < values.Length; x++) src[x, 0] = new Rgba(values[x], values[x], values[x]);
    var output = Resampler.Resample(src, 0.5, ResampleMethod.Area);
    Assert.Equal(2, output.Width);
    Helpers.Near(0.5, output[0, 0].R, 1e-5);
    Helpers.Near(0.4, output[1, 0].R, 1e-5);
}

Used in chunk 311.

```

315 *⟨Win Tests: constants stay constant 315⟩* ≡

```

[Fact]
public void ConstantImageStaysConstant()
{
    var src = new PixelBuffer(13, 7, new Rgba(0.4f, 0.5f, 0.6f));
    foreach (var method in ResampleMethodInfo.All)
        foreach (double scale in new[] { 0.3, 0.5, 2.0, 3.7 })
        {
            var output = Resampler.Resample(src, scale, method);
            for (int i = 0; i < output.Data.Length; i += 4)
                Helpers.Near(0.4, output.Data[i], 1e-4, $"{method} {scale}");
        }
}

Used in chunk 311.

```

13.5 Color conversions and line sampling

The color tests are the same round trips as in Swift, with the same colors, and the same absolute check that pure blue has hue 240°. The line sampler tests are the same too: 201 and 101 samples on a 200-pixel image and its half-size copy, and a vertical line at $u = 0.505$ that reads exactly the red of pixel column 50. The C# index-from-end operator `^1` replaces Swift's `last!`.

316 *⟨Win Tests: color conversions 316⟩* ≡

```

public class ColorTests
{
    [Fact]
    public void HslRoundTrip()
    {
        foreach (var c in new[] { new Rgba(0.2f, 0.7f, 0.4f), new Rgba(1, 0, 0), new Rgba(0.1f, 0.1f,
        ↪ 0.9f), new Rgba(0.5f, 0.5f, 0.5f) })
        {
            var hsl = ColorConversion.Hsl(c);
            var back = ColorConversion.FromHsl(hsl.H, hsl.S, hsl.L);
            Helpers.Near(c.R, back.R, 1e-5);
            Helpers.Near(c.G, back.G, 1e-5);
            Helpers.Near(c.B, back.B, 1e-5);
        }
        Helpers.Near(240, ColorConversion.Hsl(new Rgba(0, 0, 1)).H, 1e-4);
    }

    [Fact]
    public void CmykRoundTrip()
    {
        foreach (var c in new[] { new Rgba(0.2f, 0.7f, 0.4f), new Rgba(0, 0, 0), new Rgba(1, 1, 1) })
        {
            var v = ColorConversion.Cmyk(c);
            var back = ColorConversion.FromCmyk(v.C, v.M, v.Y, v.K);
            Helpers.Near(c.R, back.R, 1e-5);
            Helpers.Near(c.G, back.G, 1e-5);
            Helpers.Near(c.B, back.B, 1e-5);
        }
    }
}

```

Used in chunk 307.

317 *⟨Win Tests: line sampler 317⟩* ≡

```

public class LineSamplerTests
{
    [Fact]
    public void SampleCountScalesWithResample()
    {
        var src = Helpers.Gradient(200, 100);
        var half = Resampler.Resample(src, 0.5, ResampleMethod.Tent);
        var a = LineSampler.Sample(src, LineSegment.Initial);
        var b = LineSampler.Sample(half, LineSegment.Initial);
        Assert.Equal(201, a.Colors.Length);
        Assert.Equal(101, b.Colors.Length);
        Helpers.Near(1, a.Spacing, 1e-9);
    }

    [Fact]
    public void SamplesFollowContent()
    {
        var src = Helpers.Gradient(100, 100);
        var line = new LineSegment(new NormalizedPoint(0.505, 0.1), new NormalizedPoint(0.505, 0.9));
        var s = LineSampler.Sample(src, line);
        // Vertical line through pixel column 50: red is constant, green increases.
        Helpers.Near(0.5, s.Colors[0].R, 1e-4);
        Helpers.Near(0.5, s.Colors[^1].R, 1e-4);
        Assert.True(s.Colors[0].G < s.Colors[^1].G);
    }
}

```

Used in chunk 307.

13.6 The bitmap bridge

On macOS the bridge to the screen was a `CGImage`; here it is an array of 8-bit premultiplied BGRA bytes, the layout of WPF's `Pbgra32` format (Chapter 12). The round-trip test is the Swift one: converting to bytes and back may lose one quantization step, $\frac{1}{255}$, and must keep the size and the row order.

A second test checks what the round trip can't see: the *order* of the bytes. If red and blue were swapped in both directions, the round trip would still pass, and every image would show with its colors exchanged. So one orange pixel, (1, 0.5, 0) opaque, must become the bytes blue, green, red, alpha = (0, 128, 255, 255). The 128 also checks the rounding: $0.5 \cdot 255 = 127.5$ rounds up.

318 *<Win Tests: bitmap bridge 318> ≡*

```
public class BitmapBridgeTests
{
    [Fact]
    public void Bgra8RoundTrip()
    {
        var src = Helpers.Gradient(32, 16);
        var bytes = src.ToPremultipliedBgra8();
        var back = PixelBuffer.FromPremultipliedBgra8(32, 16, bytes);
        Assert.Equal(32, back.Width);
        Assert.Equal(16, back.Height);
        for (int i = 0; i < src.Data.Length; i++)
            Helpers.Near(src.Data[i], back.Data[i], 1.0 / 255 + 1e-4);
    }

    [Fact]
    public void Bgra8ChannelOrder()
    {
        var src = new PixelBuffer(1, 1, new Rgba(1, 0.5f, 0, 1));
        var bytes = src.ToPremultipliedBgra8();
        Assert.Equal(new byte[] { 0, 128, 255, 255 }, bytes);
    }
}
```

Used in chunk 307.

13.7 Filters

The filter tests grow the most, because the port reimplements the five Core Image filters on the CPU. On macOS there was little to test there: Core Image is Apple's, and the tests only had to show that LineScope called it correctly. Here the filters are LineScope's own code, so they get tests of their own.

319 *<Win Tests: filters 319> ≡*

```
public class FilterTests
{
    <Win Tests: every filter 320>

    <Win Tests: filters keep the size 321>

    <Win Tests: smoothing filters keep constants 322>

    <Win Tests: kernels are normalized 323>

    <Win Tests: the median and the noise filter 324>
}
```

Used in chunk 307.

The tests draw on one list of filters, one of each kind, with moderate parameters.

320 *⟨Win Tests: every filter 320⟩* ≡

```
private static readonly FilterKind[] AllFilters =
[
    new FilterKind.GaussianBlur(2), new FilterKind.BoxBlur(2), new FilterKind.Median(),
    new FilterKind.UnsharpMask(2, 0.5), new FilterKind.NoiseReduction(0.02, 0.4), new FilterKind.Sobel(),
    new FilterKind.Laplacian(), new FilterKind.Emboss(), new FilterKind.Grayscale(), new
    ↪ FilterKind.Invert(),
];
```

Used in chunk 319.

The first test is the Swift one, extended to every filter: the size must be kept, since the normalized line relies on it, and a blurred gradient must still increase from left to right. It also checks that every output stays in $[0, 1]$. That matters most for the Sobel, Laplacian and emboss kernels, whose sums can leave the range, and for the unsharp mask, which overshoots at edges by design; both must be brought back into range before the result is stored.

321 *⟨Win Tests: filters keep the size 321⟩* ≡

```
[Fact]
public void FiltersKeepSize()
{
    var src = Helpers.Gradient(24, 12);
    foreach (var f in AllFilters)
    {
        var output = Filters.Apply(f, src);
        Assert.Equal(24, output.Width);
        Assert.Equal(12, output.Height);
        Assert.All(output.Data, v => Assert.InRange(v, 0f, 1f));
    }
    var blurred = Filters.Apply(new FilterKind.GaussianBlur(1), src);
    // Horizontal gradient stays roughly monotone and in range after blur.
    Assert.True(blurred[2, 6].R < blurred[20, 6].R);
}
```

Used in chunk 319.

A smoothing filter is a weighted average whose weights sum to one, so it must leave a constant image unchanged; so must the unsharp mask, which adds a multiple of *src* − *blur*, zero for a constant, and the noise reduction, which averages equal neighbors. The test checks every pixel, the borders included. That tests the edge handling: the filters clamp coordinates at the border, so the kernel always sees real pixels, and a wrong clamp shows at once as a darker or lighter frame. Red and blue (0.3 and 0.9) are checked, which would also expose a channel mix-up.

322 *⟨Win Tests: smoothing filters keep constants 322⟩* ≡

```
[Fact]
public void SmoothingFiltersKeepConstantImages()
{
    var src = new PixelBuffer(9, 7, new Rgba(0.3f, 0.6f, 0.9f));
    foreach (var f in AllFilters.Where(f => f is FilterKind.GaussianBlur or FilterKind.BoxBlur or
    ↪ FilterKind.Median
        or FilterKind.UnsharpMask or FilterKind.NoiseReduction))
    {
        var output = Filters.Apply(f, src);
        for (int i = 0; i < output.Data.Length; i += 4)
        {
            Helpers.Near(0.3, output.Data[i], 1e-5, f.DisplayName);
            Helpers.Near(0.9, output.Data[i + 2], 1e-5, f.DisplayName);
        }
    }
}
```

Used in chunk 319.

The Gaussian and box kernels are tested directly. For every radius, including 0 and the fractional 0.4, the weights must sum to one. Two exact shapes pin the box convention: the box covers $[-(r + \frac{1}{2}), r + \frac{1}{2}]$, so radius 0 is the identity, a single weight 1, and radius 1 is a three-pixel box.

323 *⟨Win Tests: kernels are normalized 323⟩* ≡

```
[Fact]
public void KernelsAreNormalized()
{
    foreach (double r in new[] { 0.0, 0.4, 1, 2.5, 10 })
    {
        Helpers.Near(1, Filters.GaussianKernel(r).Sum(), 1e-5, $"gaussian {r}");
        Helpers.Near(1, Filters.BoxKernel(r).Sum(), 1e-5, $"box {r}");
    }
    Assert.Equal(new float[] { 1 }, Filters.BoxKernel(0));
    Assert.Equal(3, Filters.BoxKernel(1).Length);
}
```

Used in chunk 319.

The last two tests check the defining behavior of the two nonlinear filters. A 3×3 median removes an isolated impulse completely: among the nine values in the window only one is the bright pixel, and the median is the background, exactly. The noise reduction averages a neighbor only when its luminance differs from the center by less than the noise level. A bump of 0.005 under a level of 0.02 is therefore averaged away, to within 0.002 of the background, and, not being an edge, is not sharpened back.

324 *⟨Win Tests: the median and the noise filter 324⟩* ≡

```
[Fact]
public void MedianRemovesAnImpulse()
{
    var src = new PixelBuffer(5, 5, new Rgba(0.2f, 0.2f, 0.2f));
    src[2, 2] = new Rgba(1, 1, 1);
    var output = Filters.Apply(new FilterKind.Median(), src);
    Helpers.Near(0.2, output[2, 2].R, 1e-6);
}

[Fact]
public void NoiseReductionSmoothsSmallVariations()
{
    var src = new PixelBuffer(5, 5, new Rgba(0.5f, 0.5f, 0.5f));
    src[2, 2] = new Rgba(0.505f, 0.505f, 0.505f);
    var output = Filters.Apply(new FilterKind.NoiseReduction(0.02, 0.4), src);
    Assert.True(Math.Abs(output[2, 2].R - 0.5) < 0.002);
}
```

Used in chunk 319.

13.8 Test patterns

The first three pattern tests are the Swift ones: every pattern has the requested size and stays in $[0, 1]$; a sine grating of 0.125 cycles per pixel, sampled along the default line, has its spectral peak within one bin of 0.125, which tests generator, sampler and analyzer together; and the checkerboard and the seeded noise are exact and repeatable. Comparing two noise buffers with `Assert.Equal` works because `PixelBuffer` implements value equality over its pixels (Chapter 12), while the class itself is a reference type.

325 *⟨Win Tests: test patterns 325⟩* ≡

```
public class TestPatternTests
{
    [Fact]
    public void AllPatternsGenerateRequestedSize()
    {
        foreach (var kind in TestPatternKindInfo.All)
        {
            var buf = TestPatternGenerator.Make(new TestPatternSpec(kind, 64, 48));
            Assert.Equal(64, buf.Width);
            Assert.Equal(48, buf.Height);
            Assert.All(buf.Data, v => Assert.InRange(v, 0f, 1f));
        }
    }

    [Fact]
    public void SineGratingPeaksAtItsFrequency()
    {
        var spec = new TestPatternSpec(TestPatternKind.SineGrating, 256, 8, 0.125);
        var samples = LineSampler.Sample(TestPatternGenerator.Make(spec), LineSegment.Initial);
        var values = samples.Colors.Select(c => c.R).ToArray();
        var s = SpectrumAnalyzer.Compute(values, samples.Spacing, WindowFunction.Hann, removeMean: true);
        int peak = Assert.NotNull(s.PeakIndex);
        Helpers.Near(0.125, s.CyclesPerPixel[peak], 1.0 / s.FftSize + 1e-9);
    }

    [Fact]
    public void CheckerboardAndNoiseDeterminism()
    {
        var board = TestPatternGenerator.Make(new TestPatternSpec(TestPatternKind.Checkerboard, 8, 8,
↪ 2));
        Assert.Equal(1, board[0, 0].R);
        Assert.Equal(0, board[2, 0].R);
        Assert.Equal(1, board[2, 2].R);
        var a = TestPatternGenerator.Make(new TestPatternSpec(TestPatternKind.WhiteNoise, 16, 16, 7));
        var b = TestPatternGenerator.Make(new TestPatternSpec(TestPatternKind.WhiteNoise, 16, 16, 7));
        Assert.Equal(a, b);
    }

    ⟨Win Tests: SplitMix64 reference values 326⟩
}

```

Used in chunk 307.

The repeatability test only shows that a seed gives the same noise twice on the *same* system. The port promises more: the same seed must give the same noise on both systems, so that a noise pattern on Windows is the pattern on the Mac. That holds only if the C# SplitMix64 is bit for bit the algorithm of Chapter 6, with its unsigned 64-bit wrap-around arithmetic, which C# has to be told about with `ulong` and unchecked operations. The last test compares the first two outputs for seed 0 with the values of Vigna's reference implementation.

326 *⟨Win Tests: SplitMix64 reference values 326⟩* ≡

```
[Fact]
public void SplitMixMatchesReferenceSequence()
{
    // Reference values of SplitMix64 seeded with 0 (Vigna's reference implementation).
    var rng = new SplitMix64(0);
    Assert.Equal(0xE220A8397B1DCDAFUL, rng.Next());
    Assert.Equal(0x6E789E6AA1B965F4UL, rng.Next());
}

```

Used in chunk 325.

Chapter 14

Documents and the Session in C#

This chapter is the counterpart of Chapter 8. Its three files have the same roles as there: the document decodes a file once into a `PixelBuffer` and a display image; the variant, zone and pane types describe what the window shows; and the `Session` holds all the state of one window and performs the operations. Most of the logic is unchanged. Three things are not. Decoding goes through WIC instead of ImageIO. SwiftUI’s automatic observation is replaced by an explicit change event, which is the largest difference in the whole app. And the plumbing that found the right session for a menu shrinks, because on Windows every window has its own menus.

14.1 The image document

A macOS document was a value conforming to `FileDocument`, created by the document group. On Windows there is no document framework to conform to, so `ImageDocument` is a plain immutable class with two factory methods, one for a file and one for a test pattern. The application object calls them on a background thread (Chapter 11). Nor is there a `fileWrapper` to refuse: with no framework, there is no Save command to disable.

327 *<Windows/LineScope.App/Model/ImageDocument.cs 327> ≡*

```
using System.IO;
using System.Text;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using LineScope.Core;
```

```
namespace LineScope.App.Model;
```

<Win ImageDocument: source information 328>

```
/// <summary>
/// Read-only image document. Pixels are decoded once (with WIC) into a <see cref="PixelBuffer"/>. EXIF
///   ↳ orientation
/// is ignored on purpose so analysis runs on the stored pixel grid. Images with an embedded ICC profile
///   ↳ are
/// converted to sRGB, as on macOS.
/// </summary>
public sealed class ImageDocument
{
```

<Win ImageDocument: stored data 329>

<Win ImageDocument: reading a file 330>

<Win ImageDocument: a generated test pattern 334>

<Win ImageDocument: converting to sRGB 335>

```

    <Win ImageDocument: detecting alpha 336>

    <Win ImageDocument: the ICC profile's name 337>
}

<Win ImageDocument: rendering a buffer 338>
    Root chunk, tangled to Windows/LineScope.App/Model/ImageDocument.cs.

```

The facts about the source are the same as on macOS. A C# *positional record* declares them in one statement: the compiler generates the properties, a constructor and value equality.

328 <Win ImageDocument: source information 328> ≡

```

/// <summary>Facts about the file as decoded, before conversion to the working float buffer.</summary>
public sealed record SourceInfo(
    string FileType,
    int PixelWidth,
    int PixelHeight,
    int BitsPerComponent,
    int BitsPerPixel,
    string ColorSpaceName,
    bool HasAlpha,
    double? Dpi);

    Used in chunk 327.

```

The document holds the float buffer, a display bitmap and the facts. As on macOS, the display image is rendered *from the buffer*, not taken from the decoder, so what the user sees is exactly what is analyzed. The constructor is private, so a document can only come from the two factories.

329 <Win ImageDocument: stored data 329> ≡

```

public PixelBuffer Buffer { get; }
public BitmapSource DisplayImage { get; }
public SourceInfo Info { get; }

private ImageDocument(PixelBuffer buffer, SourceInfo info)
{
    Buffer = buffer;
    DisplayImage = Bitmaps.FromBuffer(buffer);
    Info = info;
}

    Used in chunk 327.

```

14.1.1 Decoding with WIC

Reading a file has the same three steps as in Chapter 8: decode the first image, bring it into the working format, and record what the file was.

330 <Win ImageDocument: reading a file 330> ≡

```

/// <summary>Decodes an image file. Safe to call off the UI thread.</summary>
public static ImageDocument Load(string path)
{
    <Win ImageDocument: decoding the first frame 331>

    <Win ImageDocument: converting to the working buffer 332>

    <Win ImageDocument: recording facts about the source 333>
}

    Used in chunk 327.

```

WPF's `BitmapDecoder` is a thin layer over WIC, which chooses the codec from the file's contents. Three options matter. `PreservePixelFormat` keeps the frame in its stored pixel format, so the facts below describe the file and not a conversion. `IgnoreImageCache` makes sure the file is really read again, not served from WPF's cache of recently opened images, which would show a stale image after the file was changed on disk. `OnLoad` decodes everything immediately, so the file stream can be closed when the method returns. A file with several frames, such as an animated GIF or a multi-page TIFF, contributes its first, like `CGImageSourceCreateImageAtIndex(source, 0, nil)`. Errors are exceptions, which the caller turns into a message.

331 *⟨Win ImageDocument: decoding the first frame 331⟩* ≡

```
using var stream = File.OpenRead(path);
var decoder = BitmapDecoder.Create(
    stream, BitmapCreateOptions.PreservePixelFormat | BitmapCreateOptions.IgnoreImageCache,
    ⇨ BitmapCacheOption.OnLoad);
if (decoder.Frames.Count == 0) throw new InvalidDataException("The file contains no image.");
BitmapFrame frame = decoder.Frames[0];
int w = frame.PixelWidth, h = frame.PixelHeight;
if (w <= 0 || h <= 0) throw new InvalidDataException("The image is empty.");

Used in chunk 330.
```

On macOS, `PixelBuffer(cgImage:)` drew the image into an 8-bit sRGB context, and Core Graphics converted the colors from the image's embedded profile on the way (Chapter 2). WIC doesn't convert colors unless asked, so the port asks explicitly: the frame is converted to sRGB and to premultiplied BGRA, the bytes are copied out with a stride of four bytes per pixel, and the core turns them into a float buffer (`FromPremultipliedBgra8`, Chapter 12). The result is the same sRGB-encoded, straight-alpha buffer the Mac produces.

332 *⟨Win ImageDocument: converting to the working buffer 332⟩* ≡

```
var profile = frame.ColorContexts is { Count: > 0 } contexts ? contexts[0] : null;
BitmapSource sRGB = ToSrgbPbgra32(frame, profile);
var bytes = new byte[w * h * 4];
sRGB.CopyPixels(bytes, w * 4, 0);
var buffer = PixelBuffer.FromPremultipliedBgra8(w, h, bytes);

Used in chunk 330.
```

Like the macOS version, the port ignores the EXIF orientation tag, for the reason given in Chapter 8: `LineScope` analyzes the pixel grid as stored. WIC never applies the tag on its own; the orientation is only metadata, and `LineScope` doesn't read it. Doing nothing is therefore the whole implementation.

The facts need some work, because WIC reports them differently. The file type comes from the codec's name: "PNG Decoder" becomes "PNG image". WIC gives the bits per pixel, and one *mask* per channel, so the bits per component are their quotient, for instance $24/3 = 8$ for `Bgr24`. An image has alpha if its format has an alpha channel or, for indexed images such as GIF and 8-bit PNG, if a palette entry is not fully opaque. The color space is named after the embedded profile, and an image without one is taken to be sRGB, as the web and Windows assume.

333 *⟨Win ImageDocument: recording facts about the source 333⟩* ≡

```
var format = frame.Format;
int channels = Math.Max(format.Masks.Count, 1);
string codec = decoder.CodecInfo?.FriendlyName ?? "Image";
var info = new SourceInfo(
    FileType: codec.Replace(" Decoder", "", StringComparison.OrdinalIgnoreCase).Trim() + " image",
    PixelWidth: w,
    PixelHeight: h,
    BitsPerComponent: format.BitsPerPixel / channels,
    BitsPerPixel: format.BitsPerPixel,
    ColorSpaceName: profile is null ? "sRGB (no embedded profile)" : ProfileDescription(profile) ??
    ⇨ "Embedded ICC profile",
    HasAlpha: HasAlphaChannel(format) || (frame.Palette?.Colors.Any(c => c.A < 255) ?? false),
```

```
Dpi: frame.DpiX > 0 ? frame.DpiX : null);
return new ImageDocument(buffer, info);
```

Used in chunk 330.

A pattern becomes a document exactly as on macOS, with the facts of a float buffer.

334 *Win ImageDocument: a generated test pattern 334* ≡

/// <summary>A generated test pattern, shown in the same analysis window as a file.</summary>

```
public static ImageDocument FromPattern(TestPatternSpec spec)
{
    var buffer = TestPatternGenerator.Make(spec);
    return new ImageDocument(buffer, new SourceInfo(
        FileType: "Generated test pattern",
        PixelWidth: buffer.Width,
        PixelHeight: buffer.Height,
        BitsPerComponent: 32,
        BitsPerPixel: 128,
        ColorSpaceName: "sRGB (float)",
        HasAlpha: false,
        Dpi: null));
}
```

Used in chunk 327.

14.1.2 Color management

The conversion uses WPF's `ColorConvertedBitmap`, which runs the frame through the Windows color system from the embedded profile to a destination context. A `ColorContext` made from a pixel format alone, here `Bgra32`, is the standard sRGB profile. Profiles found in the wild are sometimes broken or of a kind the color system doesn't support; then the conversion throws, and the stored values are used as they are, which means they are read as sRGB. A plain format conversion does that, and is also the path for images without a profile. Both results are frozen: a frozen WPF object is immutable and may be used from any thread, and this code runs on a background thread.

335 *Win ImageDocument: converting to sRGB 335* ≡

```
private static BitmapSource ToSrgbPbgra32(BitmapFrame frame, ColorContext? profile)
{
    if (profile is not null)
    {
        try
        {
            var converted = new ColorConvertedBitmap(frame, profile, new
        ↪ ColorContext(PixelFormats.Bgra32), PixelFormats.Pbgra32);
            converted.Freeze();
            return converted;
        }
        catch (Exception)
        {
            // Unsupported or broken profile: fall back to the stored values.
        }
    }
    var plain = new FormatConvertedBitmap(frame, PixelFormats.Pbgra32, null, 0);
    plain.Freeze();
    return plain;
}
```

Used in chunk 327.

WPF's pixel formats don't say whether they have alpha, so the formats with an alpha channel are listed. The premultiplied and straight variants both count.

336 *⟨Win ImageDocument: detecting alpha 336⟩* ≡

```
private static bool HasAlphaChannel(PixelFormat f) =>
    f == PixelFormats.Bgra32 || f == PixelFormats.Pbgra32 || f == PixelFormats.Rgba64 || f ==
    ↪ PixelFormats.Prgba64
    || f == PixelFormats.Rgba128Float || f == PixelFormats.Prgba128Float;
```

Used in chunk 327.

Core Graphics gives a color space a name, such as “Display P3”. WPF has no such property, so the port reads the name from the profile itself. An ICC profile starts with a 128-byte header, followed by a tag count and a table of 12-byte entries, each a four-letter signature, an offset and a size, all big-endian. The name is the tag with signature *desc*. Its contents come in two types. Version 2 profiles use the *text description* type, also marked *desc*: an ASCII length at offset 8 and the ASCII text at offset 12. Version 4 profiles use the *multi-localized Unicode* type, *mluc*: a list of records, one per language, the first of which gives the length and offset of a UTF-16 big-endian string at offsets 20 and 24. The code takes the first language. Every read is checked against the length of the data, and any failure leaves the generic name “Embedded ICC profile”; a name is not worth rejecting an image for.

337 *⟨Win ImageDocument: the ICC profile’s name 337⟩* ≡

```
/// <summary>Reads the profile's description tag ('desc' in ICC v2, 'mluc' in v4).</summary>
```

```
private static string? ProfileDescription(ColorContext profile)
{
    try
    {
        using var s = profile.OpenProfileStream();
        using var ms = new MemoryStream();
        s.CopyTo(ms);
        var d = ms.ToArray();
        if (d.Length < 132) return null;
        uint count = BE32(d, 128);
        for (int i = 0; i < count && 132 + i * 12 + 12 <= d.Length; i++)
        {
            int t = 132 + i * 12;
            if (Encoding.ASCII.GetString(d, t, 4) != "desc") continue;
            int off = (int)BE32(d, t + 4), size = (int)BE32(d, t + 8);
            if (off + size > d.Length || size < 12) return null;
            string type = Encoding.ASCII.GetString(d, off, 4);
            if (type == "desc")
            {
                int len = (int)BE32(d, off + 8);
                return Encoding.ASCII.GetString(d, off + 12, Math.Max(0, Math.Min(len, size -
                ↪ 12))).TrimEnd('\0');
            }
            if (type == "mluc" && size >= 28)
            {
                int recLen = (int)BE32(d, off + 20), recOff = (int)BE32(d, off + 24);
                if (off + recOff + recLen <= d.Length)
                    return Encoding.BigEndianUnicode.GetString(d, off + recOff, recLen).TrimEnd('\0');
            }
            return null;
        }
    }
    catch (Exception)
    {
    }
    return null;
}
```

```
private static uint BE32(byte[] d, int i) => (uint)(d[i] << 24 | d[i + 1] << 16 | d[i + 2] << 8 | d[i +
    ↪ 3]);
```

Used in chunk 327.

The last piece renders a buffer for display, and every variant uses it, not only the original. It plays the part of `makeCGImage`. The bitmap is set to 96 dots per inch, the value at which WPF maps one pixel to one device-independent unit, so an image is drawn at its true pixel size on a monitor at 100% scale, and the views can compensate for other scales (Chapter 15).

338 *⟨Win ImageDocument: rendering a buffer 338⟩* ≡

```
public static class Bitmaps
{
    /// <summary>Renders a buffer as a frozen 8-bit premultiplied BGRA bitmap at 96 dpi (1 pixel = 1
    ↪ DIP).</summary>
    public static BitmapSource FromBuffer(PixelBuffer buffer)
    {
        var bitmap = BitmapSource.Create(buffer.Width, buffer.Height, 96, 96, PixelFormats.Pbgra32, null,
            buffer.ToPremultipliedBgra8(), buffer.Width * 4);
        bitmap.Freeze();
        return bitmap;
    }
}
```

Used in chunk 327.

14.2 Variants, zones and panes

The small types of the session follow Chapter 8 closely. The differences come from the languages.

339 *⟨Windows/LineScope.App/Model/ImageVariant.cs 339⟩* ≡

```
using System.Windows.Media.Imaging;
using LineScope.Core;
```

```
namespace LineScope.App.Model;
```

⟨Win ImageVariant: the variant type 340⟩

⟨Win ImageVariant: zones 341⟩

⟨Win ImageVariant: panes 342⟩

⟨Win ImageVariant: the frequency axis 343⟩

⟨Win ImageVariant: operation requests 344⟩

Root chunk, tangled to Windows/LineScope.App/Model/ImageVariant.cs.

A variant is immutable, as on macOS, and keeps its lineage. C# 12's *primary constructor* lets the class take its values in its declaration. The Swift `UUID` is a `Guid` here, and the arrow-joined name is built with `string.Join`.

340 *⟨Win ImageVariant: the variant type 340⟩* ≡

```
/// <summary>The original image or one derived copy. Each copy records the chain of operations back to
    ↪ the original.</summary>
public sealed class ImageVariant(PixelBuffer buffer, BitmapSource bitmap, Guid? parentId,
    ↪ ImageOperation? operation,
    IReadOnlyList<ImageOperation> lineage)
{
    public Guid Id { get; } = Guid.NewGuid();
    public PixelBuffer Buffer { get; } = buffer;
    public BitmapSource Bitmap { get; } = bitmap;
    public Guid? ParentId { get; } = parentId;
    public ImageOperation? Operation { get; } = operation;
    /// <summary>Operations applied to the original, in order. Empty for the original.</summary>
    public IReadOnlyList<ImageOperation> Lineage { get; } = lineage;
```

```

public bool IsOriginal => Operation is null;

public string Name => Lineage.Count == 0 ? "Original" : string.Join(" → ", Lineage.Select(o =>
    ↪ o.DisplayName));
}

```

Used in chunk 339.

A C# enumeration is only a named integer: it can't have methods, and it doesn't list its cases. Each enumeration therefore has a static companion class, with the list of cases in `All`, the counterpart of `CaseIterable`, and extension methods that give it the Swift computed properties, here `Title`. The zone's integer value indexes the pane arrays, as its raw value did.

341 *⟨Win ImageVariant: zones 341⟩* ≡

```

/// <summary>Which kind of content a zone shows.</summary>
public enum Zone { Images, Profile, Spectrum }

public static class ZoneInfo
{
    public static readonly Zone[] All = [Zone.Images, Zone.Profile, Zone.Spectrum];

    public static string Title(this Zone z) => z switch
    {
        Zone.Images => "Images",
        Zone.Profile => "Line Profile",
        _ => "Spectrum",
    };
}

```

Used in chunk 339.

A pane is a *class*, where Swift had a struct. The Swift session changed a pane by assigning into its array, and observation noticed the assignment. Here the session changes the pane's `VariantId` in place and announces the change itself (below). The pane's own identifier is kept, for the views to recognize a pane across rebuilds.

342 *⟨Win ImageVariant: panes 342⟩* ≡

```

/// <summary>One side-by-side pane inside a zone; it shows one variant at a time.</summary>
public sealed class PaneState(Guid variantId)
{
    public Guid Id { get; } = Guid.NewGuid();
    public Guid VariantId { get; set; } = variantId;
}

```

Used in chunk 339.

343 *⟨Win ImageVariant: the frequency axis 343⟩* ≡

```

public enum FrequencyAxis { CyclesPerPixel, CyclesPerLine }

public static class FrequencyAxisInfo
{
    public static readonly FrequencyAxis[] All = [FrequencyAxis.CyclesPerPixel,
        ↪ FrequencyAxis.CyclesPerLine];

    public static string DisplayName(this FrequencyAxis a) => a == FrequencyAxis.CyclesPerPixel ?
        ↪ "cycles/pixel" : "cycles/line";
}

```

Used in chunk 339.

A Swift enumeration with an associated value, `resample(ResampleMethod)`, has no direct counterpart in C#. The usual substitute is a small closed hierarchy: an abstract record with one nested record per case, which a switch expression takes apart by type, and a `Resample` case that carries its method. The compiler can't prove that the switch covers every case, since another class could derive from the record, hence the empty default. The request needs no identifier: SwiftUI's

sheet(item:) required one, but the Windows parameter dialog is simply shown, modally, by the menu command (Chapter 15).

344 *⟨Win ImageVariant: operation requests 344⟩* ≡

```
/// <summary>Operations that need parameters before running; shown as a dialog.</summary>
public abstract record OperationRequest
{
    public sealed record GaussianBlur : OperationRequest;
    public sealed record BoxBlur : OperationRequest;
    public sealed record UnsharpMask : OperationRequest;
    public sealed record NoiseReduction : OperationRequest;
    public sealed record Resample(ResampleMethod Method) : OperationRequest;

    public string Title => this switch
    {
        GaussianBlur => "Gaussian Blur",
        BoxBlur => "Box Blur",
        UnsharpMask => "Sharpen (Unsharp Mask)",
        NoiseReduction => "Noise Reduction",
        Resample r => $"Resample: {r.Method.DisplayName()}",
        _ => "",
    };
}
```

Used in chunk 339.

14.3 The session

The session has the same state and the same methods as in Chapter 8, and the reasoning there, from the normalized line to the shared analysis settings and chaining through the focused variant, applies without change. What changes is how the views learn that something happened.

345 *⟨Windows/LineScope.App/Model/Session.cs 345⟩* ≡

```
using System.IO;
using System.Windows;
using System.Windows.Media.Imaging;
using LineScope.Core;
using Microsoft.Win32;
```

```
namespace LineScope.App.Model;
```

⟨Win Session: the profile record 346⟩

⟨Win Session: change flags 347⟩

```
/// <summary>All state for one document window: the variants, the shared line, and view
    ↪ settings.</summary>
```

```
public sealed class Session
{
```

⟨Win Session: identity, variants and events 348⟩

⟨Win Session: initialization 349⟩

⟨Win Session: raising a change 350⟩

⟨Win Session: the line 351⟩

⟨Win Session: color model and channel 352⟩

⟨Win Session: spectrum and display settings 353⟩

```

    <Win Session: panes, focus and running operations 354>

    <Win Session: looking up variants 355>

    // Panes

    <Win Session: selecting and splitting panes 356>

    // Variants

    <Win Session: closing a variant 357>

    <Win Session: performing an operation in the background 358>

    <Win Session: showing a new variant 359>

    // Analysis

    <Win Session: computing a profile 360>

    <Win Session: computing a spectrum 361>

    // Export

    <Win Session: exporting the focused variant 362>
}

<Win Session: the active session 363>
Root chunk, tangled to Windows/LineScope.App/Model/Session.cs.

```

A profile is simpler than on macOS. Swift Charts wanted a list of identifiable points; the port's chart view (Chapter 15) draws from plain arrays, so a profile holds the positions along the line, the channel values and the samples they came from.

346 <Win Session: the profile record 346> $\equiv$
public sealed record Profile(**double**[] X, **float**[] Values, LineSamples Samples);
Used in chunk 345.

14.3.1 Change notification

On macOS the session was `@observable`, and SwiftUI tracked which properties each view read and redrew exactly those views when one changed. WPF has a comparable mechanism, data binding with `INotifyPropertyChanged`, but it is designed for XAML bindings to individual properties, and the views of this port are code that draws: an image canvas, charts, tab strips. For them a coarser and more explicit scheme is simpler. The session raises one event, `Changed`, after every change, and the event says *what kind* of state changed, as a set of flags. Each view subscribes and redraws only if a flag it depends on is set. Dragging the line raises `Line`: the canvases redraw the line and the charts recompute their profile and spectrum, but the tab strips, which show only the variants, ignore it. Choosing a tab raises `Panes` and `Focus`, and the charts ignore the focus.

The flags are powers of two, so a change can combine them with `|`, and `[Flags]` tells .NET to treat the enumeration as a set, for instance when printing it. `Running` covers the count of background operations, which drives the progress indicator.

347 *⟨Win Session: change flags 347⟩* ≡

```
/// <summary>What changed in a <see cref="Session"/>, so views redraw only what they show.</summary>
[Flags]
```

```
public enum SessionChange
{
    None = 0,
    Line = 1,
    Settings = 2,
    Variants = 4,
    Panes = 8,
    Focus = 16,
    Running = 32,
    All = Line | Settings | Variants | Panes | Focus | Running,
}
```

Used in chunk 345.

Besides Changed there is an Error event. The Swift session stored an errorMessage for an alert to observe; here the window subscribes and shows a message box. Both events are raised on the UI thread only, as all of the session's state is touched only there. That was the meaning of @MainActor in Swift; in C# it is a convention that the code below keeps.

348 *⟨Win Session: identity, variants and events 348⟩* ≡

```
public string FileName { get; }
public SourceInfo SourceInfo { get; }

private readonly List<ImageVariant> _variants;
public IReadOnlyList<ImageVariant> Variants => _variants;

/// <summary>Raised on the UI thread after any state change.</summary>
public event Action<SessionChange>? Changed;
public event Action<string>? Error;
```

Used in chunk 345.

A new session has the original, focused, in one pane of each zone, as before.

349 *⟨Win Session: initialization 349⟩* ≡

```
public Session(ImageDocument document, string fileName)
{
    var original = new ImageVariant(document.Buffer, document.DisplayImage, null, null, []);
    FileName = fileName;
    SourceInfo = document.Info;
    _variants = [original];
    _focusedVariantId = original.Id;
    Panes = ZoneInfo.All.Select(_ => new List<PaneState> { new(original.Id) }).ToArray();
}
```

Used in chunk 345.

350 *⟨Win Session: raising a change 350⟩* ≡

```
private void Raise(SessionChange change) => Changed?.Invoke(change);
```

Used in chunk 345.

14.3.2 State

Every setting is a property with a private field, and its setter follows one pattern: if the value is really new, store it and raise the flag. The comparison matters. A slider or a mouse drag assigns the same value many times, and without the test each assignment would make every view redraw.

351 *⟨Win Session: the line 351⟩* ≡

```

private LineSegment _line = LineSegment.Initial;
public LineSegment Line
{
    get => _line;
    set { if (_line != value) { _line = value; Raise(SessionChange.Line); } }
}

```

Used in chunk 345.

The color model keeps the rule of Chapter 8: a channel index that would be out of range for the new model is reset. The setter assigns the *field* `_channelIndex`, not the property, so the two changes are announced by one event rather than two.

352 *⟨Win Session: color model and channel 352⟩* ≡

```

private ColorModel _colorModel = ColorModel.Rgba;
public ColorModel ColorModel
{
    get => _colorModel;
    set
    {
        if (_colorModel == value) return;
        _colorModel = value;
        if (_channelIndex >= value.Channels().Length) _channelIndex = 0;
        Raise(SessionChange.Settings);
    }
}

private int _channelIndex;
public int ChannelIndex
{
    get => _channelIndex;
    set { if (_channelIndex != value) { _channelIndex = value; Raise(SessionChange.Settings); } }
}

```

Used in chunk 345.

The other settings all raise `Settings`. Even `UseDevicePixels`, which only affects the image canvases, belongs here: it is a view setting shared by all panes, like the rest.

353 *⟨Win Session: spectrum and display settings 353⟩* ≡

```

private WindowFunction _window = WindowFunction.Hann;
public WindowFunction Window
{
    get => _window;
    set { if (_window != value) { _window = value; Raise(SessionChange.Settings); } }
}

private bool _removeMean = true;
public bool RemoveMean
{
    get => _removeMean;
    set { if (_removeMean != value) { _removeMean = value; Raise(SessionChange.Settings); } }
}

private bool _logMagnitude = true;
public bool LogMagnitude
{
    get => _logMagnitude;
    set { if (_logMagnitude != value) { _logMagnitude = value; Raise(SessionChange.Settings); } }
}

private FrequencyAxis _frequencyAxis = FrequencyAxis.CyclesPerPixel;
public FrequencyAxis FrequencyAxis
{

```

```

    get => _frequencyAxis;
    set { if (_frequencyAxis != value) { _frequencyAxis = value; Raise(SessionChange.Settings); } }
}

private bool _useDevicePixels;
public bool UseDevicePixels
{
    get => _useDevicePixels;
    set { if (_useDevicePixels != value) { _useDevicePixels = value; Raise(SessionChange.Settings); } }
}

```

Used in chunk 345.

The panes are an array of lists, indexed by zone, and the lists and panes are changed only by the session's methods below. `RunningOperations` can be read by anyone but changed only by the session. There is no `pendingRequest`: on macOS it drove a sheet from the session's state, while here the menu command shows the parameter dialog directly.

354 *〈Win Session: panes, focus and running operations 354〉* ≡

```

/// <summary>Panes per zone, indexed by <see cref="Zone"/>.</summary>
public List<PaneState>[] Panes { get; }

private Guid _focusedVariantId;
/// <summary>The variant operations apply to and the inspector describes.</summary>
public Guid FocusedVariantId
{
    get => _focusedVariantId;
    set { if (_focusedVariantId != value) { _focusedVariantId = value; Raise(SessionChange.Focus); } }
}

public int RunningOperations { get; private set; }

```

Used in chunk 345.

355 *〈Win Session: looking up variants 355〉* ≡

```

public ImageVariant Original => _variants[0];
public ImageVariant FocusedVariant => Variant(FocusedVariantId) ?? Original;
public ImageVariant? Variant(Guid id) => _variants.FirstOrDefault(v => v.Id == id);
public string ChannelName => ColorModel.Channels()[ChannelIndex];

```

Used in chunk 345.

14.3.3 Panes and variants

The pane methods are those of Chapter 8, with the change events made explicit. Each method changes what it needs and then raises one event that covers all of it. `Select`, for instance, assigns the focus through its field and raises `Panes` and `Focus` together, so the views update once, after the state is consistent. Raising in the middle, with the pane changed and the focus not yet, would let a view draw a state that never really existed.

356 *〈Win Session: selecting and splitting panes 356〉* ≡

```

public void Select(Guid variantId, Zone zone, int pane)
{
    Panes[(int)zone][pane].VariantId = variantId;
    _focusedVariantId = variantId;
    Raise(SessionChange.Panes | SessionChange.Focus);
}

public void Split(Zone zone)
{
    var shown = Panes[(int)zone].Select(p => p.VariantId).ToHashSet();
    var next = _variants.FirstOrDefault(v => !shown.Contains(v.Id)) ?? FocusedVariant;
    Panes[(int)zone].Add(new PaneState(next.Id));
}

```

```

        Raise(SessionChange.Panes);
    }

    public void Unsplit(Zone zone)
    {
        var panes = Panes[(int)zone];
        if (panes.Count <= 1) return;
        panes.RemoveAt(panes.Count - 1);
        Raise(SessionChange.Panes);
    }

```

Used in chunk 345.

357 *⟨Win Session: closing a variant 357⟩* ≡

```

public void CloseVariant(Guid id)
{
    int index = _variants.FindIndex(v => v.Id == id);
    if (index <= 0) return;
    _variants.RemoveAt(index);
    foreach (var zone in Panes)
        foreach (var pane in zone.Where(p => p.VariantId == id))
            pane.VariantId = Original.Id;
    if (_focusedVariantId == id) _focusedVariantId = Original.Id;
    Raise(SessionChange.Variants | SessionChange.Panes | SessionChange.Focus);
}

```

Used in chunk 345.

Performing an operation keeps the design of Chapter 8: capture the focused variant and its buffer first, compute in the background, and append the result on the UI thread. The C# tools are different. `Task.Run` runs the operation on the thread pool, the counterpart of the detached task. The `await` returns to the UI thread by itself, because WPF installs a *synchronization context* that sends continuations back to the dispatcher of the thread that awaited; this plays the role of the main actor. The method is `async void`, since a menu command starts it and nobody waits for it.

One step moves to the background: rendering the display bitmap. Swift made the `CGImage` on the main actor; here `Bitmaps.FromBuffer` runs on the worker thread too, which is allowed because it freezes the bitmap, and keeps the UI thread free of all per-pixel work. An exception, which the Swift operations couldn't throw, is reported through `Error`. Whatever happens, the `finally` block decrements the running count, so the progress indicator can't get stuck.

358 *⟨Win Session: performing an operation in the background 358⟩* ≡

*/// <summary>Applies an operation to the focused variant in the background and adds the result as a new
 ↪ tab.</summary>*

```

public async void Perform(ImageOperation operation)
{
    var source = FocusedVariant;
    var input = source.Buffer;
    RunningOperations++;
    Raise(SessionChange.Running);
    try
    {
        var (output, bitmap) = await Task.Run(() =>
        {
            var result = operation.Apply(input);
            return (result, Bitmaps.FromBuffer(result));
        });
        var variant = new ImageVariant(output, bitmap, source.Id, operation, [... source.Lineage,
        ↪ operation]);
        _variants.Add(variant);
        Show(variant.Id);
    }
    catch (Exception ex)
    {
    }
}

```

```

    {
        Error?.Invoke($"Could not compute {operation.DisplayName}: {ex.Message}");
    }
    finally
    {
        RunningOperations--;
        Raise(SessionChange.Running);
    }
}

```

Used in chunk 345.

The new variant goes into the last pane of each zone, for the reason given in Chapter 8. The index-from-end `^1` names that pane.

359 *⟨Win Session: showing a new variant 359⟩* ≡

```

/// <summary>
/// Shows a new variant in every zone: in the last pane when split (so the left pane keeps the
/// comparison baseline), otherwise in the only pane.
/// </summary>
private void Show(Guid id)
{
    foreach (var zone in Panes) zone[^1].VariantId = id;
    _focusedVariantId = id;
    Raise(SessionChange.Variants | SessionChange.Panes | SessionChange.Focus);
}

```

Used in chunk 345.

14.3.4 Analysis and export

The analysis is the same computation: sample the line in the variant's own grid, extract the channel, and take the spectrum with the shared settings, with the decibel floor of -100 . Only the return types are simpler: arrays and a tuple of arrays, instead of lists of chart points.

360 *⟨Win Session: computing a profile 360⟩* ≡

```

public Profile Profile(ImageVariant variant)
{
    var samples = LineSampler.Sample(variant.Buffer, Line);
    var values = samples.Colors.Select(c => ColorConversion.Value(c, ColorModel,
        ↪ ChannelIndex)).ToArray();
    return new Profile(samples.Positions, values, samples);
}

```

Used in chunk 345.

361 *⟨Win Session: computing a spectrum 361⟩* ≡

```

public SpectrumResult Spectrum(Profile profile) =>
    SpectrumAnalyzer.Compute(profile.Values, Math.Max(profile.Samples.Spacing, 1e-9), Window,
        ↪ RemoveMean);

/// <summary>Frequencies on the chosen axis and magnitudes (in dB when enabled).</summary>
public (double[] Frequencies, double[] Magnitudes) SpectrumPoints(SpectrumResult s)
{
    var freqs = FrequencyAxis == FrequencyAxis.CyclesPerPixel ? s.CyclesPerPixel : s.CyclesPerLine;
    var mags = s.Amplitudes.Select(a => LogMagnitude ? Math.Max(20 * Math.Log10(Math.Max(a, 1e-12)),
        ↪ DbFloor) : a).ToArray();
    return (freqs, mags);
}

public const double DbFloor = -100.0;

```

Used in chunk 345.

Export writes the focused variant's display bitmap as a PNG, which is again exactly what is on screen. Windows forbids more characters in file names than macOS does (among them \, :, *, ? and |), so instead of replacing only the slash, every character that `Path.GetInvalidFileNameChars` reports is replaced with a hyphen. The Save dialog is owned by the document window, so it is modal to it.

362 *⟨Win Session: exporting the focused variant 362⟩* ≡

```
public void ExportFocused(Window? owner)
{
    var variant = FocusedVariant;
    string baseName = Path.GetFileNameWithoutExtension(FileName);
    string suffix = variant.IsOriginal ? "original" : variant.Name.Replace(" → ", "_");
    foreach (char c in Path.GetInvalidFileNameChars()) suffix = suffix.Replace(c, '-');
    var dialog = new SaveFileDialog
    {
        Filter = "PNG image|*.png",
        DefaultExt = ".png",
        FileName = $"{baseName}--{suffix}.png",
        Title = "Export Image",
    };
    if (dialog.ShowDialog(owner) != true) return;
    try
    {
        var encoder = new PngBitmapEncoder();
        encoder.Frames.Add(BitmapFrame.Create(variant.Bitmap));
        using var stream = File.Create(dialog.FileName);
        encoder.Save(stream);
    }
    catch (Exception ex)
    {
        Error?.Invoke($"Could not write {Path.GetFileName(dialog.FileName)}: {ex.Message}");
    }
}
```

Used in chunk 345.

14.3.5 The active session

On macOS the menus found their session through a focused value, with the active session as a fallback for when the inspector was key. On Windows the first mechanism isn't needed: each document window has its own menu bar, bound to its own session (Chapter 15), so there is no focused-value entry to port. The inspector still needs the second. It is its own window, and while the user works in it, it is the active one.

So the port keeps `ActiveSession`. Document windows set it when they are activated and clear it when they close. A static class with its own `Changed` event replaces the observable singleton, and the inspector subscribes to it, then to the `Changed` event of the session it names.

363 *⟨Win Session: the active session 363⟩* ≡

```
/// <summary>
/// The session of the most recently activated document window, for the inspector window (which is itself
/// the active window while being used).
/// </summary>
public static class ActiveSession
{
    private static Session? _session;

    public static event Action? Changed;

    public static Session? Current
    {
        get => _session;
    }
}
```

```
    set { if (_session != value) { _session = value; Changed?.Invoke(); } }  
  }  
}
```

Used in chunk 345.

Chapter 15

Views and Menus in WPF

This chapter is the Windows counterpart of Chapters 9 and 10. The nine files in `Windows/LineScope.App/Views` mirror the SwiftUI views one by one: the document window with its zones, panes and tabs, the image canvas where the line is drawn, the two charts, and the three auxiliary windows (the parameter dialog, the test-pattern generator and the inspector). Their *behavior* is meant to be identical, and where it is, this chapter only points back to the Swift chapter that explains it. The prose concentrates on what the port had to change, and why.

Three changes run through the whole chapter.

- **Views are built in C#, without XAML.** Each view is a class deriving from a WPF element (`Window`, `DockPanel`, `ScrollViewer`, `FrameworkElement`) whose constructor creates its children. The code then reads much like the SwiftUI body it replaces, with the same names and the same numbers, and a reader can compare the two side by side. The only XAML in the application is `App.xaml` (Chapter 11).
- **Redrawing is explicit.** SwiftUI tracks which observable properties a body reads and redraws exactly those views. WPF has no such mechanism for plain C# objects. So the `Session` of Chapter 14 raises one event, `Changed`, with a set of `SessionChange` flags (`Line`, `Settings`, `Variants`, `Panes`, `Focus`, `Running`), and every view decides from the flags what it has to update. A line drag, the most frequent change by far, then redraws the line overlays and the charts and nothing else.
- **Menus belong to windows.** A Windows application has no global menu bar. Each document window carries its own, bound to its own session, which removes the whole question of Chapter 10 of which document a menu acts on.

15.1 Views without XAML

A few helpers are shared by several views. They live in `Ui.cs`, together with the command type the menus use.

364 `<Windows/LineScope.App/Views/Ui.cs 364> ≡`

```
using System.Globalization;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Controls.Primitives;
using System.Windows.Input;
using System.Windows.Media;
```

```
namespace LineScope.App.Views;
```

`<Win Ui.cs: a command for menu items 365>`

```
/// <summary>Small helpers for building the views in code.</summary>
public static class Ui
{
    <Win Ui: secondary text 366>

    <Win Ui: a segmented control 367>

    <Win Ui: parsing a number in either notation 436>

    <Win Ui: a presets button 434>
}

Root chunk, tangled to Windows/LineScope.App/Views/Ui.cs.
```

SwiftUI disables a menu item with `.disabled(session == nil)`, evaluated on every redraw. WPF asks an `ICommand` instead, through `CanExecute`. The `RelayCommand` wraps two closures, and hooks its `CanExecuteChanged` event to the `CommandManager`'s `RequerySuggested`. WPF raises that event after input and focus changes, and the main window raises it explicitly (`InvalidateRequerySuggested`) whenever its session changes. Every menu item is then re-queried without any bookkeeping per item. `Execute` checks `CanExecute` once more, because a keyboard shortcut can fire while the menu that would have shown the item as disabled is closed.

365 <Win Ui.cs: a command for menu items 365> ≡

```
/// <summary>An <see cref="ICommand"/> whose enabled state is re-queried by WPF's command
    ↪ manager.</summary>
public sealed class RelayCommand(Action execute, Func<bool>? canExecute = null) : ICommand
{
    public bool CanExecute(object? parameter) => canExecute?.Invoke() ?? true;
    public void Execute(object? parameter) { if (CanExecute(parameter)) execute(); }

    public event EventHandler? CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }
}

Used in chunk 364.
```

SwiftUI's `.foregroundColor(.secondary)` becomes the system's gray-text brush, which also follows the high-contrast themes of Windows.

366 <Win Ui: secondary text 366> ≡

```
public static Brush SecondaryText => SystemColors.GrayTextBrush;

public static TextBlock Caption(string text) => new()
{
    Text = text,
    FontSize = 11,
    Foreground = SecondaryText,
    VerticalAlignment = VerticalAlignment.Center,
    TextTrimming = TextTrimming.CharacterEllipsis,
};

Used in chunk 364.
```

WPF has no segmented control, and the profile zone needs two of them (the color model and the channel). A row of `ToggleButton`s behaves like one if checking a button unchecks the others and a checked button can't be unchecked by clicking it again. The negative left margin makes adjacent borders overlap, so the buttons read as one control. Reacting to `Checked` rather than to `Click` matters for accessibility: the keyboard and UI Automation toggle the button without clicking it.

367 *⟨Win Ui: a segmented control 367⟩* ≡

```

/// <summary>
/// A segmented control (like SwiftUI's segmented picker): mutually exclusive toggle buttons.
/// </summary>
public static StackPanel Segmented(IReadOnlyList<string> labels, int selected, Action<int> onSelect)
{
    var panel = new StackPanel { Orientation = Orientation.Horizontal, VerticalAlignment =
        ↪ VerticalAlignment.Center };
    var buttons = new List<ToggleButton>();
    for (int i = 0; i < labels.Count; i++)
    {
        int index = i;
        var b = new ToggleButton
        {
            Content = labels[i],
            IsChecked = i == selected,
            Padding = new Thickness(8, 1, 8, 1),
            Margin = new Thickness(i == 0 ? 0 : -1, 0, 0, 0),
            MinWidth = 36,
        };
        // Checked/Unchecked rather than Click, so keyboard and UI Automation toggles work too.
        b.Checked += (_, _) =>
        {
            foreach (var other in buttons.Where(o => o != b)) other.IsChecked = false;
            onSelect(index);
        };
        b.Unchecked += (_, _) =>
        {
            // Clicking the selected segment keeps it selected.
            if (buttons.All(o => o.IsChecked != true)) b.IsChecked = true;
        };
        buttons.Add(b);
        panel.Children.Add(b);
    }
    return panel;
}

```

Used in chunk 364.

15.2 The document window

MainWindow replaces MainView and, with its menu bar, LineScopeCommands as well. Unlike a SwiftUI document window, it can exist without a document: the application opens one empty window at start-up, which shows a start screen, and loading an image fills it (the life cycle is in Chapter 11). The window is a DockPanel with the menu at the top, a status bar at the bottom, and a ContentControl in between whose content is swapped between the start screen, a loading indicator and the three zones.

368 *⟨Windows/LineScope.App/Views/MainWindow.cs 368⟩* ≡

```

using System.Windows;
using System.Windows.Controls;
using System.Windows.Controls.Primitives;
using System.Windows.Input;
using LineScope.App.Model;
using LineScope.Core;
using LineSegment = LineScope.Core.LineSegment;

namespace LineScope.App.Views;

/// <summary>
/// One document window: the menu bar, the three zones (images, line profile, spectrum) stacked with
    ↪ splitters,

```

```

/// and a status bar. Without a document it shows a start screen.
/// </summary>
public sealed class MainWindow : Window
{
    <Win MainWindow: fields 369>

    <Win MainWindow: building the window 370>

    // Content states

    <Win MainWindow: the start screen and the loading state 371>

    <Win MainWindow: loading a document 372>

    <Win MainWindow: reacting to session changes 374>

    <Win MainWindow: dropping files 375>

    // Menus

    <Win MainWindow: the menu bar 376>

    <Win MainWindow: adding a menu item and its shortcut 381>

    <Win MainWindow: running and requesting operations 382>
}

Root chunk, tangled to Windows/LineScope.App/Views/MainWindow.cs.

```

The session is `null` until a document is loaded. The progress bar replaces the toolbar spinner of the Swift version, and the menu item for Actual Device Pixels is kept in a field because its check mark has to be updated from the session.

369 <Win MainWindow: fields 369> ≡

```

private readonly ContentControl _body = new();
private readonly StatusBarItem _status = new();
private readonly ProgressBar _progress = new() { Width = 90, Height = 12, IsIndeterminate = true,
    ↪ Visibility = Visibility.Collapsed };
private readonly List<ZoneView> _zones = [];
private MenuItem _devicePixels = null!;
private Session? _session;

public bool HasDocument => _session is not null;

Used in chunk 368.

```

The minimum size is the one of `MainView`, and the initial size is limited to the work area, so the window fits on a small laptop screen. `UseLayoutRounding` snaps element edges to whole device pixels, which keeps the image canvas sharp (Section 15.6).

The two window events replace the `WindowKeyObserver` of Chapter 9. WPF reports activation directly, so no invisible helper view is needed. `Activated` makes this window's session the `ActiveSession`, which only the inspector still reads, and `Closed` withdraws it if it is still the active one. The window also accepts files dropped on it.

370 <Win MainWindow: building the window 370> ≡

```

public MainWindow()
{
    Title = "LineScope";
    var work = SystemParameters.WorkArea;
    Width = Math.Min(1100, work.Width);
    Height = Math.Min(900, work.Height);
    MinWidth = 720;
    MinHeight = 560;
}

```

```

UseLayoutRounding = true;
AllowDrop = true;
Background = SystemColors.WindowBrush;

var root = new DockPanel();
var menu = BuildMenu();
DockPanel.SetDock(menu, Dock.Top);
root.Children.Add(menu);
var statusBar = new StatusBar { Items = { _status, new StatusBarItem { Content = _progress,
    ↪ HorizontalAlignment = HorizontalAlignment.Right } } };
DockPanel.SetDock(statusBar, Dock.Bottom);
root.Children.Add(statusBar);
root.Children.Add(_body);
Content = root;

Activated += (_, _) => { if (_session is not null) ActiveSession.Current = _session; };
Closed += (_, _) => { if (ActiveSession.Current == _session && _session is not null)
    ↪ ActiveSession.Current = null; };
Drop += OnDrop;
DragOver += (_, e) =>
{
    e.Effects = e.Data.GetDataPresent(DataFormats.FileDrop) ? DragDropEffects.Copy :
    ↪ DragDropEffects.None;
    e.Handled = true;
};
ShowEmpty();
}

```

Used in chunk 368.

The start screen offers the two ways to get something to analyze. The loading state is shown while an image is decoded or a pattern generated, the counterpart of the `ProgressView` of the Swift pattern window.

371 *⟨Win Main Window: the start screen and the loading state 371⟩* ≡

```

public void ShowEmpty()
{
    var open = new Button { Content = "Open ...Image", Padding = new Thickness(14, 4, 14, 4), Margin = new
    ↪ Thickness(0, 0, 8, 0) };
    open.Click += (_, _) => App.Current.ShowOpenDialog(this);
    var pattern = new Button { Content = "New Test ...Pattern", Padding = new Thickness(14, 4, 14, 4) };
    pattern.Click += (_, _) => App.Current.ShowPatternGenerator(this);
    _body.Content = new StackPanel
    {
        HorizontalAlignment = HorizontalAlignment.Center,
        VerticalAlignment = VerticalAlignment.Center,
        Children =
        {
            new TextBlock { Text = "LineScope", FontSize = 26, FontWeight = FontWeights.Light,
            ↪ HorizontalAlignment = HorizontalAlignment.Center },
            new TextBlock
            {
                Text = "Open an image (or drop one here) or create a test pattern.", Foreground =
                ↪ Ui.SecondaryText,
                Margin = new Thickness(0, 6, 0, 16), HorizontalAlignment = HorizontalAlignment.Center,
            },
            new StackPanel { Orientation = Orientation.Horizontal, HorizontalAlignment =
            ↪ HorizontalAlignment.Center, Children = { open, pattern } },
        },
    };
    _status.Content = "";
}

public void ShowLoading(string message)
{

```

```

_body.Content = new StackPanel
{
    HorizontalAlignment = HorizontalAlignment.Center,
    VerticalAlignment = VerticalAlignment.Center,
    Children =
    {
        new ProgressBar { Width = 160, Height = 6, IsIndeterminate = true, Margin = new Thickness(0,
↪ 0, 0, 10) },
        new TextBlock { Text = message, Foreground = Ui.SecondaryText, HorizontalAlignment =
↪ HorizontalAlignment.Center },
    },
};
}

```

Used in chunk 368.

Loading a document creates the window's one session, as the `@State` initializer of `MainView` does. The session reports failures through an `Error` event rather than an `errorMessage` property, and the window shows them in a message box owned by itself. If the window is already active when the document arrives, `Activated` won't fire again, so the session is made active here; this is the same corner case the Swift observer handles for a window that is already key.

372 *⟨Win MainWindow: loading a document 372⟩* ≡

```

public void LoadDocument(ImageDocument document, string fileName)
{
    var session = new Session(document, fileName);
    _session = session;
    Title = $"{fileName} - LineScope";
    session.Changed += OnSessionChanged;
    session.Error += message =>
        MessageBox.Show(this, message, "Operation failed", MessageBoxButton.OK, MessageBoxImage.Warning);

    ⟨Win MainWindow: three zones stacked with splitters 373⟩
    UpdateStatus();
    if (IsActive) ActiveSession.Current = session;
    CommandManager.InvalidateRequerySuggested();
}

```

Used in chunk 368.

WPF has no `VSplitView`. Its equivalent is a `Grid` whose rows are separated by thin rows holding a `GridSplitter`, which resizes the rows before and after it. The zones' rows use star sizes in the proportions of the Swift ideal heights (420 : 220 : 220), so they share the window in that ratio and keep it when the window is resized, and the minimum heights are the Swift ones.

373 *⟨Win MainWindow: three zones stacked with splitters 373⟩* ≡

```

var grid = new Grid();
_zones.Clear();
double[] ideal = [420, 220, 220];
double[] min = [180, 140, 140];
foreach (var zone in ZoneInfo.All)
{
    int i = (int)zone;
    if (i > 0)
    {
        grid.RowDefinitions.Add(new RowDefinition { Height = new GridLength(5) });
        var splitter = new GridSplitter
        {
            Height = 5, HorizontalAlignment = HorizontalAlignment.Stretch, VerticalAlignment =
↪ VerticalAlignment.Stretch,
            ResizeBehavior = GridResizeBehavior.PreviousAndNext, ResizeDirection =
↪ GridResizeDirection.Rows,
            Background = SystemColors.ControlDarkBrush,
        };
    }
}

```

```

        Grid.SetRow(splitter, grid.RowDefinitions.Count - 1);
        grid.Children.Add(splitter);
    }
    grid.RowDefinitions.Add(new RowDefinition { Height = new GridLength(ideal[i], GridUnitType.Star),
        ↪ MinHeight = min[i] });
    var view = new ZoneView(zone, session);
    Grid.SetRow(view, grid.RowDefinitions.Count - 1);
    grid.Children.Add(view);
    _zones.Add(view);
}
_body.Content = grid;

```

Used in chunk 372.

This is the root of the redrawing scheme. The window passes every change down to its zones, which pass it on to their panes, and each level looks only at the flags that concern it. The window itself updates the menu's check mark when the settings change, and the status bar (which names the focused variant, like the dot on its tab) only when an operation starts or ends, the focus moves, or a variant is added or closed. Finally, since the change may have enabled or disabled menu items, the command manager is asked to re-query them.

374 *⟨Win MainWindow: reacting to session changes 374⟩* ≡

```

private void OnSessionChanged(SessionChange change)
{
    foreach (var zone in _zones) zone.Update(change);
    if (change.HasFlag(SessionChange.Settings)) _devicePixels.IsChecked = _session?.UseDevicePixels ==
        ↪ true;
    if ((change & (SessionChange.Running | SessionChange.Focus | SessionChange.Variants)) != 0)
        ↪ UpdateStatus();
    CommandManager.InvalidateRequerySuggested();
}

private void UpdateStatus()
{
    if (_session is null) return;
    var v = _session.FocusedVariant;
    _status.Content = $"Focused: {v.Name} · {v.Buffer.Width}×{v.Buffer.Height} px";
    _progress.Visibility = _session.RunningOperations > 0 ? Visibility.Visible : Visibility.Collapsed;
}

```

Used in chunk 368.

The first dropped file goes into this window (which replaces its document if it has one) and any further files open windows of their own.

375 *⟨Win MainWindow: dropping files 375⟩* ≡

```

private void OnDrop(object sender, DragEventArgs e)
{
    if (e.Data.GetData(DataFormats.FileDrop) is not string[] files) return;
    MainWindow? reuse = this;
    foreach (var path in files)
    {
        App.Current.OpenFile(path, reuse);
        reuse = null;
    }
}

```

Used in chunk 368.

15.3 Menus and shortcuts

On macOS the menu bar is global, and Chapter 10 needs a focused value, with `ActiveSession` as a fallback while the inspector is key, to find the document a menu item acts on. On Windows

each `MainWindow` builds its own menu, and every action closes over *this* window's `_session`. There is nothing to look up. The inspector has no menu bar, so the case that required the fallback never arises. The local function `Has` plays the role of `.disabled(session == nil)`: it is the `CanExecute` of every item that needs a document.

The menus have the same items in the same order as the Swift ones. What macOS and SwiftUI supply by themselves has to be written out here: Open, Close Window and Exit in the File menu. Headers carry access keys (the letter after the underscore), so every item is reachable with Alt.

376 *⟨Win MainWindow: the menu bar 376⟩* ≡

```
private Menu BuildMenu()
{
    bool Has() => _session is not null;
    var menu = new Menu();

    ⟨Win MainWindow: the File menu 377⟩

    ⟨Win MainWindow: the Filter menu 378⟩

    ⟨Win MainWindow: the Resample menu 379⟩

    ⟨Win MainWindow: the View menu 380⟩

    return menu;
}
```

Used in chunk 368.

The shortcuts translate the macOS ones by replacing Command with Control and Option with Alt: Ctrl+O to open, Ctrl+Alt+N for a custom test pattern, Ctrl+Shift+E to export, Ctrl+W to close the window. Command-Delete (Close Tab) becomes Ctrl+Delete. Exit only displays Alt+F4, which the system already handles for every window. Test patterns open new windows, so their items don't depend on the session at all.

377 *⟨Win MainWindow: the File menu 377⟩* ≡

```
var file = TopMenu("_File");
Add(file, "..._Open", () => App.Current.ShowDialog(this), key: new KeyGesture(Key.O,
    ↪ ModifierKeys.Control));
var patterns = new MenuItem { Header = "New _Test Pattern" };
foreach (var kind in TestPatternKindInfo.All)
    Add(patterns, kind.DisplayName(), () => App.Current.OpenPattern(new TestPatternSpec(kind)));
patterns.Items.Add(new Separator());
Add(patterns, "..._Custom", () => App.Current.ShowPatternGenerator(this),
    key: new KeyGesture(Key.N, ModifierKeys.Control | ModifierKeys.Alt));
file.Items.Add(patterns);
file.Items.Add(new Separator());
Add(file, "_Export ...Image", () => _session!.ExportFocused(this), Has,
    new KeyGesture(Key.E, ModifierKeys.Control | ModifierKeys.Shift));
Add(file, "Close T_ab", () => _session!.CloseVariant(_session.FocusedVariantId),
    () => _session is { FocusedVariant.IsOriginal: false }, new KeyGesture(Key.Delete,
    ↪ ModifierKeys.Control));
file.Items.Add(new Separator());
Add(file, "_Close Window", Close, key: new KeyGesture(Key.W, ModifierKeys.Control));
Add(file, "E_xit", () => Application.Current.Shutdown(), gestureText: "Alt+F4");
menu.Items.Add(file);
```

Used in chunk 376.

The Filter and Resample menus are those of Chapter 10, grouped the same way. Items ending in an ellipsis Request parameters first; the others Perform their filter at once.

378 *⟨Win MainWindow: the Filter menu 378⟩* ≡

```

var filter = TopMenu("F_filter");
Add(filter, "_Gaussian ...Blur", () => Request(new OperationRequest.GaussianBlur(), Has);
Add(filter, "_Box ...Blur", () => Request(new OperationRequest.BoxBlur(), Has);
Add(filter, "_Median 3x3", () => Perform(new FilterKind.Median(), Has);
Add(filter, "_Noise ...Reduction", () => Request(new OperationRequest.NoiseReduction(), Has);
filter.Items.Add(new Separator());
Add(filter, "_Sharpen (Unsharp Mask)...", () => Request(new OperationRequest.UnsharpMask(), Has);
Add(filter, "_Edge Detect (Sobel)", () => Perform(new FilterKind.Sobel(), Has);
Add(filter, "_Laplacian", () => Perform(new FilterKind.Laplacian(), Has);
Add(filter, "Em_boss", () => Perform(new FilterKind.Emboss(), Has);
filter.Items.Add(new Separator());
Add(filter, "G_ayscale", () => Perform(new FilterKind.Grayscale(), Has);
Add(filter, "_Invert", () => Perform(new FilterKind.Invert(), Has);
menu.Items.Add(filter);

```

Used in chunk 376.

As in Swift, the Resample menu is generated from the list of methods.

379 *⟨Win MainWindow: the Resample menu 379⟩* ≡

```

var resample = TopMenu("_Resample");
foreach (var method in ResampleMethodInfo.All)
    Add(resample, method.DisplayName() + "...", () => Request(new OperationRequest.Resample(method), Has);
menu.Items.Add(resample);

```

Used in chunk 376.

The View menu repeats the zones' split buttons. Actual Device Pixels is a checkable item whose Checked and Unchecked events write the setting, the counterpart of the Binding through the optional session in Swift. Its check mark is read back when the menu opens and whenever the settings change, so it also follows changes made elsewhere. The inspector keeps the conventional shortcut, as Ctrl+Alt+I.

380 *⟨Win MainWindow: the View menu 380⟩* ≡

```

var view = TopMenu("_View");
foreach (var zone in ZoneInfo.All)
    Add(view, $"Split {zone.Title()}", () => _session!.Split(zone), Has);
foreach (var zone in ZoneInfo.All)
    Add(view, $"Unsplit {zone.Title()}", () => _session!.Unsplit(zone), () =>
        ↪ _session?.Panels[(int)zone].Count > 1);
view.Items.Add(new Separator());
_devicePixels = new MenuItem { Header = "Actual _Device Pixels", IsCheckable = true };
_devicePixels.Checked += (_, _) => { if (_session is not null) _session.UseDevicePixels = true; };
_devicePixels.Unchecked += (_, _) => { if (_session is not null) _session.UseDevicePixels = false; };
view.SubmenuOpened += (_, _) =>
{
    _devicePixels.IsEnabled = Has();
    _devicePixels.IsChecked = _session?.UseDevicePixels == true;
};
view.Items.Add(_devicePixels);
Add(view, "Reset _Line", () => _session!.Line = LineSegment.Initial, Has);
view.Items.Add(new Separator());
Add(view, "Show _Inspector", () => App.Current.ShowInspector(), key: new KeyGesture(Key.I,
    ↪ ModifierKeys.Control | ModifierKeys.Alt));
menu.Items.Add(view);

```

Used in chunk 376.

In WPF a menu item's InputGestureText is only a label; it does not make the shortcut work. So Add registers the same command twice: on the menu item, and as a KeyBinding on the window. Because the binding belongs to the window, the shortcut acts on this window's session and only while this window is active, which is exactly the scope of its menu. The label is generated from the gesture, in the user's language.

381 *⟨Win MainWindow: adding a menu item and its shortcut 381⟩* ≡

```
private static MenuItem TopMenu(string header) => new() { Header = header };

private void Add(MenuItem parent, string header, Action action, Func<bool>? canExecute = null,
    ↪ KeyGesture? key = null,
    string? gestureText = null)
{
    var command = new RelayCommand(action, canExecute);
    var item = new MenuItem { Header = header, Command = command };
    if (key is not null)
    {
        InputBindings.Add(new KeyBinding(command, key));
        item.InputGestureText =
            ↪ key.GetDisplayStringForCulture(System.Globalization.CultureInfo.CurrentUICulture);
    }
    else if (gestureText is not null)
    {
        item.InputGestureText = gestureText;
    }
    parent.Items.Add(item);
}
```

Used in chunk 368.

SwiftUI presents the parameter sheet in reaction to the session's pendingRequest. A modal WPF dialog is simpler: ShowDialog returns only when the dialog is closed, so the request is a local value, and the operation built by the dialog runs right after it. The session needs no pending-request state at all.

382 *⟨Win MainWindow: running and requesting operations 382⟩* ≡

```
private void Perform(FilterKind filter) => _session?.Perform(new ImageOperation.Filter(filter));

private void Request(OperationRequest request)
{
    if (_session is null) return;
    var dialog = new ParameterDialog(request, _session) { Owner = this };
    if (dialog.ShowDialog() == true && dialog.Operation is { } op) _session.Perform(op);
}
```

Used in chunk 368.

15.4 Zones

A zone is laid out as in Chapter 9: a header bar with the title on the left, the split buttons on the right and the zone's own controls next to them, above one or more panes side by side. The difference is in how it stays current. SwiftUI rebuilds ZoneView.body whenever something it reads changes; the WPF zone is built once and then patched by Update.

383 *⟨Windows/LineScope.App/Views/ZoneView.cs 383⟩* ≡

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using LineScope.App.Model;
using LineScope.Core;
using LineSegment = LineScope.Core.LineSegment;

namespace LineScope.App.Views;

/// <summary>One horizontal zone: a header with its controls, and one or more side-by-side
    ↪ panes.</summary>
public sealed class ZoneView : DockPanel
{
    {
```

```

    <Win ZoneView: fields 384>
    <Win ZoneView: the header row 385>
    <Win ZoneView: updating from session changes 386>
    <Win ZoneView: side-by-side panes with splitters 387>
    <Win ZoneView: controls specific to each zone 388>
    <Win ZoneView: a combo box and a check box 392>
    <Win ZoneView: split and unsplit buttons 393>
    <Win ZoneView: describing the line 394>
}

```

Root chunk, tangled to Windows/LineScope.App/Views/ZoneView.cs.

The fields keep the elements that later updates touch: the pane views, the line description, the Unsplit button, and the color model the controls were built for.

384 <Win ZoneView: fields 384> ≡

```

private readonly Zone _zone;
private readonly Session _session;
private readonly Grid _panesGrid = new();
private readonly List<PaneView> _panes = [];
private readonly StackPanel _controls = new() { Orientation = Orientation.Horizontal, VerticalAlignment
    ↪ = VerticalAlignment.Center };
private TextBlock? _lineText;
private Button _unsplit = null!;
private ColorModel _builtFor;

```

Used in chunk 383.

The header is a DockPanel with LastChildFill off. Children docked to the right are placed from the right edge inward in the order they are added, so the split buttons come first, then the divider, then the controls; the free space between the title and the controls plays the role of the SwiftUI Spacer.

385 <Win ZoneView: the header row 385> ≡

```

public ZoneView(Zone zone, Session session)
{
    _zone = zone;
    _session = session;

    var header = new DockPanel { Margin = new Thickness(10, 4, 10, 4), LastChildFill = false };
    var title = new TextBlock
    {
        Text = zone.Title(), FontWeight = FontWeights.SemiBold, FontSize = 13, VerticalAlignment =
        ↪ VerticalAlignment.Center,
    };
    SetDock(title, Dock.Left);
    header.Children.Add(title);
    var splitButtons = SplitButtons();
    SetDock(splitButtons, Dock.Right);
    header.Children.Add(splitButtons);
    var divider = new Border
    {
        Width = 1, Height = 16, Margin = new Thickness(10, 0, 10, 0), Background =
        ↪ SystemColors.ControlDarkBrush,
    };
    SetDock(divider, Dock.Right);
    header.Children.Add(divider);
}

```

```

SetDock(_controls, Dock.Right);
header.Children.Add(_controls);

var bar = new Border
{
    Child = header,
    Background = SystemColors.ControlBrush,
    BorderBrush = SystemColors.ControlDarkBrush,
    BorderThickness = new Thickness(0, 0, 0, 1),
};
SetDock(bar, Dock.Top);
Children.Add(bar);
Children.Add(_panesGrid);

BuildControls();
RebuildPanes();
}

```

Used in chunk 383.

The zone rebuilds its panes only when their *number* changed. A pane showing a different variant is updated in place by the pane itself. Its line description changes with the line. Its controls are rebuilt only when the color model changed, because only then does the list of channels change (RGBA has four, HSL three). Other settings leave the controls alone, which matters because a control that is rebuilt while the user operates it would lose its keyboard focus.

386 *⟨Win ZoneView: updating from session changes 386⟩* ≡

```

public void Update(SessionChange change)
{
    if (change.HasFlag(SessionChange.Panes) && _session.Panes[(int)_zone].Count != _panes.Count)
        RebuildPanes();
    foreach (var pane in _panes) pane.Update(change);
    if (change.HasFlag(SessionChange.Line) && _lineText is not null) _lineText.Text = LineDescription();
    if (change.HasFlag(SessionChange.Settings) && _zone == Zone.Profile && _builtFor !=
        ↪ _session.ColorModel)
        BuildControls();
    _unsplit.IsEnabled = _session.Panes[(int)_zone].Count > 1;
}

```

Used in chunk 383.

Split panes use the same device as the zones, turned by ninety degrees: a Grid of equal star columns separated by five-pixel columns with a vertical GridSplitter. The minimum width of 200 is the one PaneView has in the Swift HSplitView. Rebuilding the columns after a split gives every pane an equal share of the width again.

387 *⟨Win ZoneView: side-by-side panes with splitters 387⟩* ≡

```

private void RebuildPanes()
{
    _panesGrid.Children.Clear();
    _panesGrid.ColumnDefinitions.Clear();
    _panes.Clear();
    int count = _session.Panes[(int)_zone].Count;
    for (int i = 0; i < count; i++)
    {
        if (i > 0)
        {
            _panesGrid.ColumnDefinitions.Add(new ColumnDefinition { Width = new GridLength(5) });
            var splitter = new GridSplitter
            {
                Width = 5, HorizontalAlignment = HorizontalAlignment.Stretch, ResizeBehavior =
                ↪ GridResizeBehavior.PreviousAndNext,
                Background = SystemColors.ControlBrush,
            };

```

```

        Grid.SetColumn(splitter, _panesGrid.ColumnDefinitions.Count - 1);
        _panesGrid.Children.Add(splitter);
    }
    _panesGrid.ColumnDefinitions.Add(new ColumnDefinition { Width = new GridLength(1,
↪ GridUnitType.Star), MinWidth = 200 });
    var pane = new PaneView(_zone, i, _session);
    Grid.SetColumn(pane, _panesGrid.ColumnDefinitions.Count - 1);
    _panesGrid.Children.Add(pane);
    _panes.Add(pane);
}
}

```

Used in chunk 383.

The controls of each zone are those of Chapter 9, created in a switch instead of a `@ViewBuilder`. The SwiftUI bindings (`$session.window` and the like) become callbacks that assign the session's property; the session's setter raises `Changed` and the rest follows.

388 *⟨Win ZoneView: controls specific to each zone 388⟩* ≡

```

private void BuildControls()
{
    _controls.Children.Clear();
    _lineText = null;
    _builtFor = _session.ColorModel;
    switch (_zone)
    {
        case Zone.Images:
            ⟨Win ZoneView: image-zone controls 389⟩

        case Zone.Profile:
            ⟨Win ZoneView: profile-zone controls 390⟩

        case Zone.Spectrum:
            ⟨Win ZoneView: spectrum-zone controls 391⟩
    }
}

```

Used in chunk 383.

The line description is set in `Consolas`, the Windows stand-in for the monospaced digits of the Swift caption, so the text doesn't jitter while the line is dragged.

389 *⟨Win ZoneView: image-zone controls 389⟩* ≡

```

_lineText = new TextBlock
{
    Text = LineDescription(), FontFamily = new FontFamily("Consolas"), FontSize = 11.5,
    Foreground = Ui.SecondaryText, VerticalAlignment = VerticalAlignment.Center, Margin = new
↪ Thickness(0, 0, 12, 0),
};
_controls.Children.Add(_lineText);
var reset = new Button { Content = "Reset Line", Padding = new Thickness(8, 1, 8, 1) };
reset.Click += (_, _) => _session.Line = LineSegment.Initial;
_controls.Children.Add(reset);
break;

```

Used in chunk 388.

The two segmented pickers of the profile zone use `Ui.Segmented`; the channel labels come from the current color model.

390 *⟨Win ZoneView: profile-zone controls 390⟩* ≡

```

var models = Ui.Segmented(ColorModelInfo.All.Select(m => m.DisplayName()).ToList(),
    Array.IndexOf(ColorModelInfo.All, _session.ColorModel), i => _session.ColorModel =
        ⇨ ColorModelInfo.All[i]);
models.Margin = new Thickness(0, 0, 12, 0);
models.ToolTip = "Color space";
_controls.Children.Add(models);
var channels = Ui.Segmented(_session.ColorModel.Channels(), _session.ChannelIndex, i =>
    ⇨ _session.ChannelIndex = i);
channels.ToolTip = "Channel";
_controls.Children.Add(channels);
break;

```

Used in chunk 388.

The spectrum zone's pop-up pickers become combo boxes, and its toggles check boxes.

391 *⟨Win ZoneView: spectrum-zone controls 391⟩* ≡

```

_controls.Children.Add(Combo("Window", WindowFunctionInfo.All.Select(w => w.DisplayName()).ToList(),
    Array.IndexOf(WindowFunctionInfo.All, _session.Window), i => _session.Window =
        ⇨ WindowFunctionInfo.All[i]));
_controls.Children.Add(Combo("Axis", FrequencyAxisInfo.All.Select(a => a.DisplayName()).ToList(),
    Array.IndexOf(FrequencyAxisInfo.All, _session.FrequencyAxis), i => _session.FrequencyAxis =
        ⇨ FrequencyAxisInfo.All[i]));
_controls.Children.Add(Check("Remove mean", _session.RemoveMean, v => _session.RemoveMean = v));
_controls.Children.Add(Check("dB", _session.LogMagnitude, v => _session.LogMagnitude = v));
break;

```

Used in chunk 388.

A combo box reports a selected index of -1 while its items are being replaced, which must not reach the session.

392 *⟨Win ZoneView: a combo box and a check box 392⟩* ≡

```

private static StackPanel Combo(string label, List<string> items, int selected, Action<int> onSelect)
{
    var combo = new ComboBox { ItemsSource = items, SelectedIndex = selected, MinWidth = 90, Padding =
        ⇨ new Thickness(6, 1, 6, 1) };
    combo.SelectionChanged += (_, _) => { if (combo.SelectedIndex >= 0) onSelect(combo.SelectedIndex); };
    return new StackPanel
    {
        Orientation = Orientation.Horizontal,
        Margin = new Thickness(0, 0, 12, 0),
        Children =
        {
            new TextBlock { Text = label, VerticalAlignment = VerticalAlignment.Center, Margin = new
                ⇨ Thickness(0, 0, 5, 0) },
            combo,
        },
    };
}

private static CheckBox Check(string label, bool value, Action<bool> onChange)
{
    var box = new CheckBox { Content = label, IsChecked = value, VerticalAlignment =
        ⇨ VerticalAlignment.Center, Margin = new Thickness(0, 0, 12, 0) };
    box.Checked += (_, _) => onChange(true);
    box.Unchecked += (_, _) => onChange(false);
    return box;
}

```

Used in chunk 383.

The split buttons have text labels instead of SF Symbols, which don't exist on Windows. WPF hides the tooltip of a disabled element by default, so it is turned on explicitly for Unsplit: a grayed-out button should still say what it would do.

393 *<Win ZoneView: split and unsplit buttons 393> ≡*

```
private StackPanel SplitButtons()
{
    var split = new Button { Content = "Split", Padding = new Thickness(8, 1, 8, 1), ToolTip = $"Split:
    ↪ add a pane to {_zone.Title()}" };
    split.Click += (_, _) => _session.Split(_zone);
    _unsplit = new Button
    {
        Content = "Unsplit", Padding = new Thickness(8, 1, 8, 1), Margin = new Thickness(4, 0, 0, 0),
        ToolTip = "Unsplit: remove the last pane", IsEnabled = _session.Panes[(int)_zone].Count > 1,
    };
    ToolTipService.SetShowOnDisabled(_unsplit, true);
    _unsplit.Click += (_, _) => _session.Unsplit(_zone);
    return new StackPanel { Orientation = Orientation.Horizontal, VerticalAlignment =
    ↪ VerticalAlignment.Center, Children = { split, _unsplit } };
}
```

Used in chunk 383.

`Fmt.Fixed` (Chapter 14) formats with three decimals in the user's locale, like `.number.precision(.fractionLength(3))` in Swift.

394 *<Win ZoneView: describing the line 394> ≡*

```
private string LineDescription()
{
    var l = _session.Line;
    static string P(double v) => Fmt.Fixed(v, 3);
    return $"({P(l.Start.X)}, {P(l.Start.Y)}) → ({P(l.End.X)}, {P(l.End.Y)})";
}
```

Used in chunk 383.

15.5 Panes and tabs

A pane is a strip of tabs above the zone's content, as in Chapter 9. The content element (canvas or chart) is created once, and later only told which variant to show.

395 *<Windows/LineScope.App/Views/PaneView.cs 395> ≡*

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Shapes;
using LineScope.App.Model;

namespace LineScope.App.Views;

/// <summary>A tab strip listing every variant, plus the zone's content for the selected one.</summary>
public sealed class PaneView : DockPanel
{
    <Win PaneView: fields 396>

    <Win PaneView: tab strip above the zone's content 397>

    <Win PaneView: which variant this pane shows 398>

    <Win PaneView: updating from session changes 399>

    <Win PaneView: the tab strip 400>
}
```

Root chunk, tangled to Windows/LineScope.App/Views/PaneView.cs.

A pane knows its zone, its index in the zone and the session, as the Swift `PaneView` does, and in addition keeps its tab row and its content element.

396 *⟨Win PaneView: fields 396⟩* ≡

```
private readonly Zone _zone;
private readonly int _paneIndex;
private readonly Session _session;
private readonly StackPanel _tabs = new() { Orientation = Orientation.Horizontal, Margin = new
    ↳ Thickness(6, 3, 6, 3) };
private readonly FrameworkElement _content;
```

Used in chunk 395.

The tab strip is a `ScrollViewer` with its scroll bar hidden, like the SwiftUI `ScrollView` without indicators. A mouse without a horizontal wheel could not scroll it, so the vertical wheel is redirected to scroll sideways.

397 *⟨Win PaneView: tab strip above the zone's content 397⟩* ≡

```
public PaneView(Zone zone, int paneIndex, Session session)
{
    _zone = zone;
    _paneIndex = paneIndex;
    _session = session;
    LastChildFill = true;

    var strip = new ScrollViewer
    {
        HorizontalScrollBarVisibility = ScrollBarVisibility.Hidden,
        VerticalScrollBarVisibility = ScrollBarVisibility.Disabled,
        Content = _tabs,
        Focusable = false,
    };
    strip.PreviewMouseWheel += (_, e) =>
    {
        strip.ScrollToHorizontalOffset(strip.HorizontalOffset - e.Delta / 2.0);
        e.Handled = true;
    };
    SetDock(strip, Dock.Top);
    Children.Add(strip);
    var divider = new Border { Height = 1, Background = SystemColors.ControlDarkBrush, Opacity = 0.5 };
    SetDock(divider, Dock.Top);
    Children.Add(divider);

    var variant = SelectedVariant;
    _content = zone switch
    {
        Zone.Images => new ImageCanvas(session, variant),
        Zone.Profile => new ProfileChart(session, variant),
        _ => new SpectrumChart(session, variant),
    };
    Children.Add(_content);
    RebuildTabs();
}
```

Used in chunk 395.

The selection is looked up in the session with the same fallbacks as in Swift.

398 *⟨Win PaneView: which variant this pane shows 398⟩* ≡

```
private ImageVariant SelectedVariant
{
    get
    {
        var panes = _session.Panes[(int)_zone];
        var id = _paneIndex < panes.Count ? panes[_paneIndex].VariantId : _session.Original.Id;
```

```

        return _session.Variant(id) ?? _session.Original;
    }
}

```

Used in chunk 395.

Here the flags are sorted out in detail. The tabs depend on the list of variants, on the pane's selection and on the focus (the dot), so those three rebuild them and hand the content its variant; the canvas is only given a new variant if it actually changed, because that resets its bitmap. Then the content reacts to what it shows. The canvas redraws only its line overlay when the line moves, and resizes when the settings change (Actual Device Pixels is a setting). A chart depends on the line, on all settings (channel, window, axis, dB) and on its variant, and simply invalidates its visual: WPF calls `OnRender` again before the next frame.

399 *⟨Win PaneView: updating from session changes 399⟩* ≡

```

public void Update(SessionChange change)
{
    if ((change & (SessionChange.Variants | SessionChange.Panes | SessionChange.Focus)) != 0)
    {
        RebuildTabs();
        var variant = SelectedVariant;
        switch (_content)
        {
            case ImageCanvas canvas when canvas.Variant != variant: canvas.Variant = variant; break;
            case ProfileChart chart: chart.Variant = variant; break;
            case SpectrumChart chart: chart.Variant = variant; break;
        }
    }
    switch (_content)
    {
        case ImageCanvas canvas:
            if (change.HasFlag(SessionChange.Line)) canvas.RefreshLine();
            if (change.HasFlag(SessionChange.Settings)) canvas.UpdateSize();
            break;
        case ChartView chart:
            if ((change & (SessionChange.Line | SessionChange.Settings | SessionChange.Variants |
↪ SessionChange.Panes)) != 0)
                chart.InvalidateVisual();
            break;
    }
}

```

Used in chunk 395.

Tabs are few and cheap, so they are simply rebuilt.

400 *⟨Win PaneView: the tab strip 400⟩* ≡

```

private void RebuildTabs()
{
    _tabs.Children.Clear();
    var selectedId = SelectedVariant.Id;
    foreach (var v in _session.Variants)
        _tabs.Children.Add(Tab(v, v.Id == selectedId));
}

private Border Tab(ImageVariant v, bool selected)
{
    ⟨Win PaneView: a tab's label 401⟩

    ⟨Win PaneView: a tab's context menu 402⟩
    return tab;
}

```

Used in chunk 395.

A tab shows the same things as in Swift: the focus dot, the name and the pixel size, on a tinted rounded background when the pane shows this variant. The tint is the system highlight color at 22% opacity (alpha 56/255), the accent color's opacity in Swift. A click selects the variant, which also focuses it.

401 *⟨Win PaneView: a tab's label 401⟩* ≡

```
var row = new StackPanel { Orientation = Orientation.Horizontal };
if (v.Id == _session.FocusedVariantId)
{
    row.Children.Add(new Ellipse
    {
        Width = 5, Height = 5, Fill = SystemColors.HighlightBrush,
        Margin = new Thickness(0, 0, 4, 0), VerticalAlignment = VerticalAlignment.Center,
    });
}
row.Children.Add(new TextBlock { Text = v.Name, FontSize = 11.5, VerticalAlignment =
    ↪ VerticalAlignment.Center });
row.Children.Add(new TextBlock
{
    Text = $"{v.Buffer.Width}x{v.Buffer.Height}", FontSize = 11.5, Foreground = Ui.SecondaryText,
    Margin = new Thickness(4, 0, 0, 0), VerticalAlignment = VerticalAlignment.Center,
});

var highlight = SystemColors.HighlightColor;
var tab = new Border
{
    Child = row,
    Padding = new Thickness(8, 3, 8, 3),
    Margin = new Thickness(0, 0, 2, 0),
    CornerRadius = new CornerRadius(5),
    Background = selected
        ? new SolidColorBrush(Color.FromArgb(56, highlight.R, highlight.G, highlight.B))
        : Brushes.Transparent,
    ToolTip = v.Name,
    Cursor = Cursors.Hand,
};
tab.MouseLeftButtonUp += (_, _) => _session.Select(v.Id, _zone, _paneIndex);
```

Used in chunk 400.

The context menu is the Swift one. Export needs a window to own its save dialog, and the pane finds its own.

402 *⟨Win PaneView: a tab's context menu 402⟩* ≡

```
var menu = new ContextMenu();
var export = new MenuItem { Header = "Export ...Image" };
export.Click += (_, _) =>
{
    _session.Select(v.Id, _zone, _paneIndex);
    _session.ExportFocused(Window.GetWindow(this));
};
var close = new MenuItem { Header = "Close Tab", IsEnabled = !v.IsOriginal };
close.Click += (_, _) => _session.CloseVariant(v.Id);
menu.Items.Add(export);
menu.Items.Add(close);
tab.ContextMenu = menu;
```

Used in chunk 400.

15.6 The image canvas

The canvas has the same job as in Chapter 9: show a variant at its true scale and let the user drag the shared line, with the same four drag modes, the same hit-test radii and the same Shift

snapping. What changes is the unit of true scale, and the gesture, which WPF doesn't provide and which is therefore assembled from mouse events.

403 *< Windows/LineScope.App/Views/ImageCanvas.cs 403 >* ≡

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Media;
using LineScope.App.Model;
using LineScope.Core;
using LineSegment = LineScope.Core.LineSegment;

namespace LineScope.App.Views;

/// <summary>
/// Shows a variant at its true scale (1 image pixel = 1 DIP, or 1 device pixel) with scrolling,
/// and lets the user drag the shared selection line.
/// </summary>
public sealed class ImageCanvas : ScrollViewer
{
    < Win ImageCanvas: fields and the state of a drag 404 >

    < Win ImageCanvas: the scrolling surface 405 >

    < Win ImageCanvas: showing a variant 406 >

    < Win ImageCanvas: image pixels and DIPs 407 >

    // Mouse handling

    < Win ImageCanvas: the mouse handlers 408 >

    < Win ImageCanvas: hit testing the line 409 >

    < Win ImageCanvas: updating the line during a drag 410 >

    < Win ImageCanvas: Shift snapping 412 >

    < Win ImageCanvas: coordinate conversions 413 >
}

< Win ImageCanvas.cs: the line overlay 414 >
    Root chunk, tangled to Windows/LineScope.App/Views/ImageCanvas.cs.
```

The Swift `DragState` becomes three fields. `_pressPoint` is set while the button is down, `_dragMode` once the drag has been recognized, and `_dragInitial` is the line as it was then, from which every update is computed.

404 *< Win ImageCanvas: fields and the state of a drag 404 >* ≡

```
private readonly Session _session;
private readonly Grid _surface;
private readonly Image _image;
private readonly LineOverlay _overlay;
private ImageVariant _variant;

private enum DragMode { Start, End, Whole, Draw }
private Point? _pressPoint;
private DragMode? _dragMode;
private LineSegment _dragInitial;

    Used in chunk 403.
```

The structure is the SwiftUI one: a scroll view containing a fixed-size surface that holds

the image and the line overlay. The surface is centered, so a small image sits in the middle of the pane, and the mouse handlers are attached to the surface, so that positions are image coordinates regardless of scrolling and margins. Its transparent background makes all of it hit-testable, like `.contentShape(Rectangle())`. The image is stretched to exactly the surface's size with nearest-neighbor scaling, the WPF spelling of `.interpolation(.none)`.

Losing the mouse capture (to a message box, say, or Alt+Tab) ends a drag as surely as releasing the button. Loaded sizes the surface again once the canvas knows the monitor it is on.

405 *⟨Win ImageCanvas: the scrolling surface 405⟩* ≡

```
public ImageCanvas(Session session, ImageVariant variant)
{
    _session = session;
    _variant = variant;
    HorizontalScrollBarVisibility = ScrollBarVisibility.Auto;
    VerticalScrollBarVisibility = ScrollBarVisibility.Auto;
    Background = SystemColors.AppWorkspaceBrush;
    Focusable = false;

    _image = new Image { Stretch = Stretch.Fill };
    RenderOptions.SetBitmapScalingMode(_image, BitmapScalingMode.NearestNeighbor);
    _overlay = new LineOverlay { IsHitTestVisible = false };
    _surface = new Grid
    {
        HorizontalAlignment = HorizontalAlignment.Center,
        VerticalAlignment = VerticalAlignment.Center,
        Background = Brushes.Transparent,
        Cursor = Cursors.Cross,
        Children = { _image, _overlay },
    };
    Content = _surface;

    _surface.MouseLeftButtonDown += OnDown;
    _surface.MouseMove += OnMove;
    _surface.MouseLeftButtonUp += OnUp;
    _surface.LostMouseCapture += (_, _) => { _pressPoint = null; _dragMode = null; };

    Loaded += (_, _) => UpdateSize();
    Refresh();
}
```

Used in chunk 403.

Changing the variant replaces the bitmap and the size; a moving line only touches the overlay. The variant's `Bitmap` is the frozen 8-bit bridge of Chapter 14, made once per variant.

406 *⟨Win ImageCanvas: showing a variant 406⟩* ≡

```
public ImageVariant Variant
{
    get => _variant;
    set { _variant = value; Refresh(); }
}

public void Refresh()
{
    _image.Source = _variant.Bitmap;
    UpdateSize();
    _overlay.Line = _session.Line;
}

public void RefreshLine() => _overlay.Line = _session.Line;
```

Used in chunk 403.

WPF measures in *device-independent pixels* (DIPs) of 1/96 inch, the role points play on

macOS. At 100% display scaling a DIP is one device pixel; at 150% it is one and a half. By default one image pixel is one DIP, so an image looks the same physical size on every monitor, just as one pixel per point does on a Mac. With Actual Device Pixels, the surface is made smaller by the monitor's scale factor, which maps each image pixel onto exactly one device pixel. Together with layout rounding, this is what makes the 1:1 view really one to one.

The application is per-monitor DPI aware, so the factor can change while the window is open, when it is dragged to another monitor. `OnDpiChanged` resizes the surface then, where `SwiftUI` would update `displayScale`.

407 *⟨Win ImageCanvas: image pixels and DIPs 407⟩* ≡

```
protected override void OnDpiChanged(DpiScale oldDpi, DpiScale newDpi)
{
    base.OnDpiChanged(oldDpi, newDpi);
    UpdateSize();
}

private Size DisplaySize
{
    get
    {
        double scale = _session.UseDevicePixels ? VisualTreeHelper.GetDpi(this).DpiScaleX : 1;
        return new Size(_variant.Buffer.Width / scale, _variant.Buffer.Height / scale);
    }
}

public void UpdateSize()
{
    var size = DisplaySize;
    _surface.Width = size.Width;
    _surface.Height = size.Height;
}
```

Used in chunk 403.

The three handlers rebuild `DragGesture(minimumDistance: 2)` together with the tap gesture. Pressing the button captures the mouse, so the drag goes on even when the pointer leaves the image, and focuses the variant (the tap). Moving decides the mode once the pointer has traveled two DIPs from the press point, and from then on writes a new line into the session, which redraws every overlay and chart through `Changed`. The hit test uses the *press* point, as the Swift gesture uses `startLocation`, so the mode depends on where the drag began, not on where it was recognized.

408 *⟨Win ImageCanvas: the mouse handlers 408⟩* ≡

```
private void OnDown(object sender, MouseButtonEventArgs e)
{
    _pressPoint = e.GetPosition(_surface);
    _dragMode = null;
    _surface.CaptureMouse();
    _session.FocusedVariantId = _variant.Id;
    e.Handled = true;
}

private void OnMove(object sender, MouseEventArgs e)
{
    if (_pressPoint is not Point start || e.LeftButton != MouseButtonState.Pressed) return;
    var p = e.GetPosition(_surface);
    if (_dragMode is null)
    {
        if ((p - start).Length < 2) return;
        _dragMode = HitTest(start);
        _dragInitial = _session.Line;
    }
    _session.Line = UpdatedLine(_dragMode.Value, start, p);
}
```

```

}

private void OnUp(object sender, MouseButtonEventArgs e)
{
    _pressPoint = null;
    _dragMode = null;
    _surface.ReleaseMouseCapture();
}

```

Used in chunk 403.

The hit test is the Swift one: endpoints within 9 DIPs first, then the segment within 6, and a new line otherwise. Measuring in DIPs keeps the targets the same size on the screen at any image scale. WPF's `Point` and `Vector` have the arithmetic the Swift code wrote by hand, so the distances are just `(p - a).Length`.

409 *⟨Win ImageCanvas: hit testing the line 409⟩* ≡

```

private DragMode HitTest(Point p)
{
    var size = DisplaySize;
    var a = ToPoint(_session.Line.Start, size);
    var b = ToPoint(_session.Line.End, size);
    if ((p - a).Length <= 9) return DragMode.Start;
    if ((p - b).Length <= 9) return DragMode.End;
    if (DistanceToSegment(p, a, b) <= 6) return DragMode.Whole;
    return DragMode.Draw;
}

```

Used in chunk 403.

Updating the line follows the Swift chunk case by case. `LineSegment` is an immutable record here, so a changed endpoint is made with `with` instead of assigning to a `var` copy. The `Shift` key is read at every move from `Keyboard.Modifiers`, the counterpart of `NSEvent.modifierFlags`; pressing or releasing it in the middle of a drag takes effect at once.

410 *⟨Win ImageCanvas: updating the line during a drag 410⟩* ≡

```

private LineSegment UpdatedLine(DragMode mode, Point start, Point location)
{
    var size = DisplaySize;
    bool shift = Keyboard.Modifiers.HasFlag(ModifierKeys.Shift);
    var current = Normalized(location, size);
    var line = _dragInitial;
    switch (mode)
    {
        case DragMode.Start:
            line = line with { Start = Snapped(current, line.End, size, shift) };
            break;
        case DragMode.End:
            line = line with { End = Snapped(current, line.Start, size, shift) };
            break;
        case DragMode.Draw:
            var s = Normalized(start, size);
            line = new LineSegment(s, Snapped(current, s, size, shift));
            break;
        case DragMode.Whole:
            ⟨Win ImageCanvas: moving the whole segment without leaving the image 411⟩
    }
    return line;
}

```

Used in chunk 403.

Moving the whole segment clamps the offset, not the endpoints, with the bounds derived in Chapter 9. The translation is the pointer's distance from the press point.

411 *⟨Win ImageCanvas: moving the whole segment without leaving the image 411⟩* ≡

```
// Move both endpoints, limiting the offset so neither leaves the image.
var i = _dragInitial;
double dx = (location.X - start.X) / size.Width;
double dy = (location.Y - start.Y) / size.Height;
dx = Math.Min(Math.Max(dx, -Math.Min(i.Start.X, i.End.X)), 1 - Math.Max(i.Start.X, i.End.X));
dy = Math.Min(Math.Max(dy, -Math.Min(i.Start.Y, i.End.Y)), 1 - Math.Max(i.Start.Y, i.End.Y));
line = new LineSegment(new NormalizedPoint(i.Start.X + dx, i.Start.Y + dy),
    new NormalizedPoint(i.End.X + dx, i.End.Y + dy));
break;
```

Used in chunk 410.

Shift snapping compares the displacements in displayed pixels, for the reason given in Chapter 9: in a wide image equal normalized steps are unequal on the screen.

412 *⟨Win ImageCanvas: Shift snapping 412⟩* ≡

```
/// <summary>With Shift held, constrains the segment to horizontal or vertical (in pixel
    ↪ space).</summary>
private static NormalizedPoint Snapped(NormalizedPoint p, NormalizedPoint anchor, Size size, bool
    ↪ enabled)
{
    if (!enabled) return p;
    double dx = Math.Abs(p.X - anchor.X) * size.Width;
    double dy = Math.Abs(p.Y - anchor.Y) * size.Height;
    return dx >= dy ? new NormalizedPoint(p.X, anchor.Y) : new NormalizedPoint(anchor.X, p.Y);
}
```

Used in chunk 403.

The conversions and the distance to a segment are the Swift ones, with `Math.Clamp` for the clamped projection parameter t .

413 *⟨Win ImageCanvas: coordinate conversions 413⟩* ≡

```
private static NormalizedPoint Normalized(Point p, Size size) =>
    new NormalizedPoint(p.X / size.Width, p.Y / size.Height).Clamped();

private static Point ToPoint(NormalizedPoint n, Size size) => new(n.X * size.Width, n.Y * size.Height);

private static double DistanceToSegment(Point p, Point a, Point b)
{
    var ab = b - a;
    double len2 = ab.LengthSquared;
    if (len2 <= 0) return (p - a).Length;
    double t = Math.Clamp(((p.X - a.X) * ab.X + (p.Y - a.Y) * ab.Y) / len2, 0, 1);
    return (p - (a + ab * t)).Length;
}
```

Used in chunk 403.

The overlay replaces the SwiftUI Canvas with a `FrameworkElement` that draws in `OnRender`. It fills the surface, so again normalized coordinates become DIPs by one multiplication. The pens and brushes are those of the Swift overlay, with the opacities written as alpha bytes ($0.7 \approx 179/255$, $0.35 \approx 89/255$). They are created once and *frozen*: a frozen `Freezable` can't change, so WPF needn't watch it for changes, which makes it cheaper to render. Setting `Line` invalidates only this element, which is how a drag redraws the line without redrawing the image under it.

414 *⟨Win ImageCanvas.cs: the line overlay 414⟩* ≡

```
/// <summary>Draws the selection line: a filled handle at the start (position 0) and a hollow one at the
    ↪ end.</summary>
internal sealed class LineOverlay : FrameworkElement
{
    private static readonly Pen Shadow = Frozen(new Pen(new SolidColorBrush(Color.FromArgb(179, 0, 0,
        ↪ 0)), 3));
    private static readonly Pen Yellow = Frozen(new Pen(Brushes.Yellow, 1.25));
```

```

private static readonly Pen DotOutline = Frozen(new Pen(new SolidColorBrush(Color.FromArgb(179, 0,
    ↪ 0, 0)), 1));
private static readonly Pen EndOutline = Frozen(new Pen(Brushes.Yellow, 1.5));
private static readonly Brush EndFill = Frozen(new SolidColorBrush(Color.FromArgb(89, 0, 0, 0)));

private LineSegment _line = LineSegment.Initial;

public LineSegment Line
{
    get => _line;
    set { _line = value; InvalidateVisual(); }
}

protected override void OnRender(DrawingContext dc)
{
    double w = ActualWidth, h = ActualHeight;
    var a = new Point(_line.Start.X * w, _line.Start.Y * h);
    var b = new Point(_line.End.X * w, _line.End.Y * h);
    dc.DrawLine(Shadow, a, b);
    dc.DrawLine(Yellow, a, b);
    const double r = 5;
    dc.DrawEllipse(Brushes.Yellow, DotOutline, a, r, r);
    dc.DrawEllipse(EndFill, EndOutline, b, r, r);
}

private static T Frozen<T>(T f) where T : Freezable { f.Freeze(); return f; }
}

```

Used in chunk 403.

15.7 Charts

WPF has no counterpart of Swift Charts, and a charting package would be a large dependency for two line plots. So the port has its own minimal chart: one abstract `ChartView` that draws a single series with its grid, axes, titles and an optional vertical rule, and two subclasses that only say *what* to draw. The division is the one Swift Charts imposes: the Swift `ProfileChart` and `SpectrumChart` declare marks and scales, and the framework draws them.

415 *<Windows/LineScope.App/Views/Charts.cs 415> ≡*

```

using System.Globalization;
using System.Windows;
using System.Windows.Media;
using LineScope.App.Model;
using LineScope.Core;

namespace LineScope.App.Views;

```

<Win Charts.cs: what a chart draws 416>

<Win Charts.cs: the chart base class 417>

<Win Charts.cs: the profile chart 426>

<Win Charts.cs: the spectrum chart 428>

Root chunk, tangled to Windows/LineScope.App/Views/Charts.cs.

A `ChartData` gathers what the Swift modifiers set one by one: the points (as two arrays rather than an array of structs), the fixed domains of `.chartXScale` and `.chartYScale`, the two axis labels, the stroke, and the `RuleMark` with its annotation.

416 *<Win Charts.cs: what a chart draws 416> ≡*

```
/// <summary>What a <see cref="ChartView"/> draws: one line series, its axes and an optional vertical
    ↪ rule.</summary>
```

```
public sealed record ChartData(
    double[] X, double[] Y,
    double XMin, double XMax, double YMin, double YMax,
    string XLabel, string YLabel,
    Brush Stroke,
    double? RuleX = null, string? RuleLabel = null);
```

Used in chunk 415.

The data is requested in `OnRender`, so it is computed only when WPF actually draws, and at most once per frame however many changes arrived before it. `InvalidateVisual`, called by the pane (Section 15.5), is all it takes to redraw.

417 *<Win Charts.cs: the chart base class 417> ≡*

```
/// <summary>A minimal line chart (the counterpart of Swift Charts' LinePlot) drawn directly with
    ↪ WPF.</summary>
```

```
public abstract class ChartView : FrameworkElement
{
    private const double Pad = 10;

    protected abstract ChartData? GetData();

    protected override void OnRender(DrawingContext dc)
    {
        <Win ChartView: data, pens and text 418>

        <Win ChartView: laying out the plot area 419>

        <Win ChartView: grid, ticks and axes 420>

        <Win ChartView: axis titles 421>

        dc.PushClip(new RectangleGeometry(new Rect(plot.Left, plot.Top - 1, plot.Width, plot.Height +
        ↪ 2)));

        <Win ChartView: the Nyquist rule 422>

        <Win ChartView: the series as one polyline 423>
        dc.Pop();
    }

    <Win ChartView: nice tick values 424>

    <Win ChartView: tick labels 425>
}
```

Used in chunk 415.

The transparent rectangle makes the whole element hit-testable, so tooltips and context menus of the pane work over an empty chart too. Nothing is drawn when there is no data or no room. Text is measured with `FormattedText`, which needs the monitor's pixels per DIP to place glyphs on device pixels. A degenerate range would divide by zero below, hence the last check (written with `!(x > 0)` so that it also catches NaN).

418 *⟨Win ChartView: data, pens and text 418⟩* ≡

```
dc.DrawRectangle(Brushes.Transparent, null, new Rect(0, 0, ActualWidth, ActualHeight));
var data = GetData();
if (data is null || ActualWidth < 60 || ActualHeight < 50) return;

double dip = VisualTreeHelper.GetDpi(this).PixelsPerDip;
var textBrush = SystemColors.ControlTextBrush;
var secondary = SystemColors.GrayTextBrush;
var gridPen = new Pen(new SolidColorBrush(Color.FromArgb(40, 128, 128, 128)), 1);
var axisPen = new Pen(new SolidColorBrush(Color.FromArgb(110, 128, 128, 128)), 1);

FormattedText Text(string s, Brush brush, double size = 10.5) =>
    new(s, CultureInfo.CurrentCulture, FlowDirection.LeftToRight, new Typeface("Segoe UI"), size, brush,
        ↪ dip);

double yRange = data.YMax - data.YMin, xRange = data.XMax - data.XMin;
if (!(yRange > 0) || !(xRange > 0)) return;

    Used in chunk 417.
```

The plot area is what remains after the axis decorations. The left margin depends on the widest y tick label (-100 is wider than 0.5), so the y ticks are computed first, about one per 28 DIPs of height, from an estimate of the plot's height. The bottom margin holds the x tick labels and the x title. P_x and P_y map data to DIPs, with y growing upward.

419 *⟨Win ChartView: laying out the plot area 419⟩* ≡

```
// Y tick labels decide the left margin.
double approxPlotH = ActualHeight - 2 * Pad - 34;
var yTicks = Ticks(data.YMin, data.YMax, Math.Max(2, (int)(approxPlotH / 28)));
var yLabels = yTicks.Select(v => Text(TickLabel(v, yTicks), secondary)).ToList();
double yLabelWidth = yLabels.Count > 0 ? yLabels.Max(t => t.Width) : 0;
var yTitle = Text(data.YLabel, secondary, 11);

double left = Pad + yTitle.Height + 4 + yLabelWidth + 5;
double right = ActualWidth - Pad - 4;
double top = Pad;
var xTitle = Text(data.XLabel, secondary, 11);
double bottom = ActualHeight - Pad - xTitle.Height - 4 - 14;
if (right - left < 20 || bottom - top < 20) return;
var plot = new Rect(left, top, right - left, bottom - top);

double Px(double x) => plot.Left + (x - data.XMin) / xRange * plot.Width;
double Py(double y) => plot.Bottom - (y - data.YMin) / yRange * plot.Height;

    Used in chunk 417.
```

Grid lines are drawn at every tick, faintly, as Swift Charts does by default. The x ticks are spaced about 70 DIPs apart, and their labels are kept inside the element at the edges.

420 *⟨Win ChartView: grid, ticks and axes 420⟩* ≡

```
// Grid and ticks.
for (int i = 0; i < yTicks.Length; i++)
{
    double y = Py(yTicks[i]);
    dc.DrawLine(gridPen, new Point(plot.Left, y), new Point(plot.Right, y));
    var t = yLabels[i];
    dc.DrawText(t, new Point(plot.Left - 5 - t.Width, y - t.Height / 2));
}
var xTicks = Ticks(data.XMin, data.XMax, Math.Max(2, (int)(plot.Width / 70)));
foreach (double v in xTicks)
{
    double x = Px(v);
    dc.DrawLine(gridPen, new Point(x, plot.Top), new Point(x, plot.Bottom));
    dc.DrawLine(axisPen, new Point(x, plot.Bottom), new Point(x, plot.Bottom + 3));
    var t = Text(TickLabel(v, xTicks), secondary);
}
```

```

        double tx = Math.Clamp(x - t.Width / 2, 0, ActualWidth - t.Width);
        dc.DrawText(t, new Point(tx, plot.Bottom + 3));
    }
    dc.DrawLine(axisPen, new Point(plot.Left, plot.Bottom), new Point(plot.Right, plot.Bottom));
    dc.DrawLine(axisPen, new Point(plot.Left, plot.Top), new Point(plot.Left, plot.Bottom));

```

Used in chunk 417.

The y title is rotated a quarter turn counterclockwise about a point at the left margin, level with the plot's middle. Drawing it centered on that point before the rotation centers it vertically after it.

421 *Win ChartView: axis titles 421* ≡

```

// Axis titles.
dc.DrawText(xTitle, new Point(plot.Left + (plot.Width - xTitle.Width) / 2, plot.Bottom + 18));
dc.PushTransform(new RotateTransform(-90, Pad, plot.Top + plot.Height / 2));
dc.DrawText(yTitle, new Point(Pad - yTitle.Width / 2, plot.Top + plot.Height / 2));
dc.Pop();

```

Used in chunk 417.

The rule and the series are clipped to the plot area (with one DIP to spare so that a curve lying on the axis is not cut in half). The rule is the dashed RuleMark of Chapter 9, with the same 4,3 dash pattern and its label to the left of the line at the top.

422 *Win ChartView: the Nyquist rule 422* ≡

```

// Rule (Nyquist).
if (data.RuleX is double rx)
{
    var rulePen = new Pen(secondary, 1) { DashStyle = new DashStyle([4, 3], 0) };
    double x = Px(rx);
    dc.DrawLine(rulePen, new Point(x, plot.Top), new Point(x, plot.Bottom));
    if (data.RuleLabel is { } label)
    {
        var t = Text(label, secondary, 10);
        dc.DrawText(t, new Point(x - t.Width - 3, plot.Top + 1));
    }
}

```

Used in chunk 417.

The series must be fast enough to follow a drag with thousands of samples, which was the reason for LinePlot in Swift. Drawing one DrawLine per segment would create thousands of drawing objects. Instead, the whole curve is a single StreamGeometry, the lightest geometry WPF has, built with one PolyLineTo and frozen, then stroked once with round joins.

423 *Win ChartView: the series as one polyline 423* ≡

```

// Series.
int n = Math.Min(data.X.Length, data.Y.Length);
if (n >= 2)
{
    var geometry = new StreamGeometry();
    using (var ctx = geometry.Open())
    {
        ctx.BeginFigure(new Point(Px(data.X[0]), Py(data.Y[0])), false, false);
        var points = new List<Point>(n - 1);
        for (int i = 1; i < n; i++) points.Add(new Point(Px(data.X[i]), Py(data.Y[i])));
        ctx.PolyLineTo(points, true, true);
    }
    geometry.Freeze();
    dc.DrawGeometry(null, new Pen(data.Stroke, 1.2) { LineJoin = PenLineJoin.Round }, geometry);
}

```

Used in chunk 417.

Swift Charts chooses “nice” tick values by itself; here it is done by the classic rule. Divide the range by the wanted number of ticks, write the rough step as $r \cdot 10^n$ with $1 \leq r < 10$, and round

r up to 1, 2, 5 or 10. The ticks are then the multiples of the step inside the range. The small tolerances keep the last tick despite rounding errors and turn a value like 10^{-17} into an honest zero.

424 *⟨Win ChartView: nice tick values 424⟩* ≡

```
/// <summary>"Nice" tick values (steps of 1, 2 or 5 × 10) covering [min, max].</summary>
internal static double[] Ticks(double min, double max, int maxCount)
{
    double range = max - min;
    if (!(range > 0)) return [min];
    double rough = range / Math.Max(maxCount, 1);
    double mag = Math.Pow(10, Math.Floor(Math.Log10(rough)));
    double residual = rough / mag;
    double step = (residual <= 1 ? 1 : residual <= 2 ? 2 : residual <= 5 ? 5 : 10) * mag;
    var ticks = new List<double>();
    for (double v = Math.Ceiling(min / step) * step; v <= max + step * 1e-9; v += step)
        ticks.Add(Math.Abs(v) < step * 1e-9 ? 0 : v);
    return [.. ticks];
}
```

Used in chunk 417.

A label has as many decimals as the step needs: none for a step of 20, one for 0.2, two for 0.05.

425 *⟨Win ChartView: tick labels 425⟩* ≡

```
private static string TickLabel(double v, double[] ticks)
{
    double step = ticks.Length > 1 ? ticks[1] - ticks[0] : Math.Abs(v);
    int decimals = step > 0 ? Math.Clamp(-(int)Math.Floor(Math.Log10(step) + 1e-9), 0, 6) : 0;
    return Fmt.Number(v, decimals);
}
```

Used in chunk 417.

The profile chart is the Swift one: x from 0 to 1 along the line, y fixed to the channel's natural range, so charts of different variants can be compared by eye.

426 *⟨Win Charts.cs: the profile chart 426⟩* ≡

```
/// <summary>Zone 2: the selected channel sampled along the line in this variant's own pixel
    ↪ grid.</summary>
public sealed class ProfileChart(Session session, ImageVariant variant) : ChartView
{
    public ImageVariant Variant { get; set; } = variant;

    protected override ChartData GetData()
    {
        var profile = session.Profile(Variant);
        var (min, max) = session.ColorModel.RangeOfChannel(session.ChannelIndex);
        return new ChartData(
            profile.X, profile.Values.Select(v => (double)v).ToArray(), 0, 1, min, max,
            $"Position along line ({profile.Samples.Colors.Length} samples)", session.ChannelName,
            ↪ ChannelBrush());
    }
}
```

⟨Win ProfileChart: a color for each channel 427⟩

}

Used in chunk 415.

The channel colors are the RGB values of the macOS system colors that the Swift chart names (`.red`, `.green`, `.blue`, `.cyan`, `.pink`, `.yellow`, `.purple`, `.orange`), so both versions color a channel alike. Channels without a natural hue use the text color.

427 *⟨Win ProfileChart: a color for each channel 427⟩* ≡

```

private Brush ChannelBrush() => (session.ColorModel, session.ChannelIndex) switch
{
    (ColorModel.Rgba, 0) => Rgb(0xFF, 0x3B, 0x30),
    (ColorModel.Rgba, 1) => Rgb(0x28, 0xCD, 0x41),
    (ColorModel.Rgba, 2) => Rgb(0x00, 0x7A, 0xFF),
    (ColorModel.Cmyk, 0) => Rgb(0x00, 0xC0, 0xE8),
    (ColorModel.Cmyk, 1) => Rgb(0xFF, 0x2D, 0x55),
    (ColorModel.Cmyk, 2) => Rgb(0xFF, 0xCC, 0x00),
    (ColorModel.Hsl, 0) => Rgb(0xAF, 0x52, 0xDE),
    (ColorModel.Hsl, 1) => Rgb(0xFF, 0x95, 0x00),
    _ => SystemColors.ControlTextBrush,
};

private static SolidColorBrush Rgb(byte r, byte g, byte b)
{
    var brush = new SolidColorBrush(Color.FromRgb(r, g, b));
    brush.Freeze();
    return brush;
}

```

Used in chunk 426.

The spectrum chart gathers in one method what the Swift version spreads over its helpers. The x axis ends at the Nyquist frequency, the frequency of the last bin in the current unit, and the rule marks it (why this matters is explained in Chapter 9). In decibels the y range runs from the session's floor of -100 dB up to the next multiple of 10 dB above the peak, and at least to 0 dB; on the linear scale it runs from 0 to the peak plus 5% headroom. Both ranges are fixed by rules rather than fitted to the data, so the spectra of different variants mostly share one scale. An empty spectrum (FftSize of 0 , a line too short to sample) draws nothing. As in Swift, the x label gives the FFT size.

428 *⟨Win Charts.cs: the spectrum chart 428⟩* ≡

```

/// <summary>Zone 3: amplitude spectrum of the zone-2 profile.</summary>
public sealed class SpectrumChart(Session session, ImageVariant variant) : ChartView
{
    public ImageVariant Variant { get; set; } = variant;

    protected override ChartData? GetData()
    {
        var profile = session.Profile(Variant);
        var spectrum = session.Spectrum(profile);
        if (spectrum.FftSize == 0) return null;
        var (freqs, mags) = session.SpectrumPoints(spectrum);
        double nyquist = freqs[^1];
        double top = mags.Length > 0 ? mags.Max() : 1;
        var (yMin, yMax) = session.LogMagnitude
            ? (Session.DbFloor, Math.Max(0, Math.Ceiling(top / 10) * 10))
            : (0.0, Math.Max(top * 1.05, 1e-6));
        return new ChartData(
            freqs, mags, 0, Math.Max(nyquist, 1e-9), yMin, yMax,
            $"Frequency ({session.FrequencyAxis.DisplayName()}) FFT size {spectrum.FftSize}",
            session.LogMagnitude ? "Amplitude (dB)" : "Amplitude",
            SystemColors.HighlightBrush, nyquist, "Nyquist");
    }
}

```

Used in chunk 415.

15.8 The parameter dialog

The sheet of Section 9.6 becomes a modal dialog. Its contract is narrower: it doesn't run anything, it only builds an `Operation`, and the main window runs it (Section 15.3). The parameters, defaults, ranges and limits are the Swift ones.

```
429 <Windows/LineScope.App/Views/ParameterDialog.cs 429> ≡
using System.Windows;
using System.Windows.Controls;
using System.Windows.Documents;
using System.Windows.Media;
using LineScope.App.Model;
using LineScope.Core;

namespace LineScope.App.Views;

/// <summary>Collects parameters for a filter or resample; the caller runs <see cref="Operation"/> on
    ↪ the focused variant.</summary>
public sealed class ParameterDialog : Window
{
    <Win ParameterDialog: parameters and their defaults 430>

    <Win ParameterDialog: layout 431>

    <Win ParameterDialog: fields for each request 432>

    <Win ParameterDialog: a numeric field 435>

    <Win ParameterDialog: a labeled row 437>

    <Win ParameterDialog: validating a resample 438>

    <Win ParameterDialog: building the operation 439>
}

Root chunk, tangled to Windows/LineScope.App/Views/ParameterDialog.cs.
```

The source variant is captured when the dialog opens. It cannot change while the dialog is modal, and the Swift sheet names the same variant.

```
430 <Win ParameterDialog: parameters and their defaults 430> ≡
private const int MaxOutputSide = 16_384;

private readonly OperationRequest _request;
private readonly ImageVariant _source;
private readonly Button _apply;
private TextBlock? _result;

private double _radius = 2.0;
private double _intensity = 0.5;
private double _noiseLevel = 0.02;
private double _sharpness = 0.4;
private double _scale = 0.5;

public ImageOperation? Operation { get; private set; }

Used in chunk 429.
```

The layout is the sheet's: title, the variant it applies to, the fields, and Cancel and Apply. `IsCancel` and `IsDefault` give Escape and Enter their usual meaning, like the Swift keyboard shortcuts. Setting `DialogResult` closes the dialog and makes `ShowDialog` return true.

431 *⟨Win ParameterDialog: layout 431⟩* ≡

```

public ParameterDialog(OperationRequest request, Session session)
{
    _request = request;
    _source = session.FocusedVariant;
    Title = request.Title;
    SizeToContent = SizeToContent.Height;
    Width = 420;
    ResizeMode = ResizeMode.NoResize;
    WindowStartupLocation = WindowStartupLocation.CenterOwner;
    ShowInTaskbar = false;
    UseLayoutRounding = true;

    var fields = new Grid { Margin = new Thickness(0, 10, 0, 16) };
    fields.ColumnDefinitions.Add(new ColumnDefinition { Width = GridLength.Auto });
    fields.ColumnDefinitions.Add(new ColumnDefinition { Width = new GridLength(1, GridUnitType.Star) });
    BuildFields(fields);

    var cancel = new Button { Content = "Cancel", IsCancel = true, MinWidth = 80, Margin = new
        ↪ Thickness(0, 0, 8, 0) };
    _apply = new Button { Content = "Apply", IsDefault = true, MinWidth = 80 };
    _apply.Click += (_, _) =>
    {
        Operation = BuildOperation();
        DialogResult = true;
    };

    Content = new StackPanel
    {
        Margin = new Thickness(20),
        Children =
        {
            new TextBlock { Text = request.Title, FontWeight = FontWeights.SemiBold, FontSize = 14 },
            new TextBlock
            {
                Text = $"Applies to: {_source.Name} ({_source.Buffer.Width}x{_source.Buffer.Height})",
                FontSize = 11, Foreground = Ui.SecondaryText, TextWrapping = TextWrapping.Wrap, Margin =
                ↪ new Thickness(0, 4, 0, 0),
            },
            fields,
            new StackPanel { Orientation = Orientation.Horizontal, HorizontalAlignment =
                ↪ HorizontalAlignment.Right, Children = { cancel, _apply } },
        },
    };
    Validate();
}

```

Used in chunk 429.

Each request shows its own fields, as in Swift.

432 *⟨Win ParameterDialog: fields for each request 432⟩* ≡

```

private void BuildFields(Grid grid)
{
    switch (_request)
    {
        case OperationRequest.GaussianBlur or OperationRequest.BoxBlur:
            Number(grid, "Radius (px)", _radius, 0, 100, v => _radius = v);
            break;
        case OperationRequest.UnsharpMask:
            Number(grid, "Radius (px)", _radius, 0, 100, v => _radius = v);
            Number(grid, "Intensity", _intensity, 0, 10, v => _intensity = v);
            break;
        case OperationRequest.NoiseReduction:
            Number(grid, "Noise level", _noiseLevel, 0, 0.1, v => _noiseLevel = v);
    }
}

```

```

        Number(grid, "Sharpness", _sharpness, 0, 2, v => _sharpness = v);
        break;
    case OperationRequest.Resample:
        ⟨Win ParameterDialog: the scale field and the resulting size 433⟩
    }
}

```

Used in chunk 429.

The scale is a text field with a presets button, and below it the size the resample will produce. Text that doesn't parse, or a scale that isn't positive, is marked with a red border and turns the scale into NaN, which `Validate` rejects. The result uses tabular figures, so the numbers don't shift sideways as they are typed.

433 ⟨Win ParameterDialog: the scale field and the resulting size 433⟩ ≡

```

var box = new TextBox { Text = Fmt.Number(_scale, 4), Width = 80, VerticalContentAlignment =
    ↪ VerticalAlignment.Center };
box.TextChanged += (_, _) =>
{
    bool ok = Ui.TryParse(box.Text, out double v) && v > 0;
    if (ok) _scale = v; else _scale = double.NaN;
    box.BorderBrush = ok ? SystemColors.ControlDarkBrush : Brushes.Red;
    Validate();
};
var presets = Ui.PresetsButton(new[] { 0.25, 0.5, 0.75, 1.5, 2, 4 }
    .Select(s => ($"×{Fmt.Number(s)}"), (Action)(() => box.Text = Fmt.Number(s, 4))));
AddRow(grid, "Scale factor", new StackPanel { Orientation = Orientation.Horizontal, Children = { box,
    ↪ presets } });
_result = new TextBlock { VerticalAlignment = VerticalAlignment.Center, FontFamily = new
    ↪ FontFamily("Segoe UI") };
Typography.SetNumeralAlignment(_result, FontNumeralAlignment.Tabular);
AddRow(grid, "Result", _result);
break;

```

Used in chunk 432.

WPF has no drop-down button, so `Ui.PresetsButton` opens a context menu below an ordinary button. The parameter dialog uses it for scales, the pattern window for sizes.

434 ⟨Win Ui: a presets button 434⟩ ≡

```

/// <summary>A button that opens a drop-down menu of presets.</summary>
public static Button PresetsButton(IEnumerable<string Label, Action Action> items)
{

```

```

    var menu = new ContextMenu();
    foreach (var (label, action) in items)
    {
        var mi = new MenuItem { Header = label };
        mi.Click += (_, _) => action();
        menu.Items.Add(mi);
    }
    var button = new Button { Content = "Presets ▼", Padding = new Thickness(8, 1, 8, 1), Margin = new
        ↪ Thickness(6, 0, 0, 0) };
    button.Click += (_, _) =>
    {
        menu.PlacementTarget = button;
        menu.Placement = PlacementMode.Bottom;
        menu.IsOpen = true;
    };
    return button;
}

```

Used in chunk 364.

A numeric parameter is a slider next to a text field, as in the sheet. SwiftUI binds both to one `@State`; here each control updates the other, and the syncing flag stops the echo (the slider

moving the text, which moves the slider again). The slider shows the value clamped to its range, but a typed value outside the range is kept, as in Swift.

435 *⟨Win ParameterDialog: a numeric field 435⟩* ≡

```
private void Number(Grid grid, string label, double initial, double min, double max, Action<double> set)
{
    var slider = new Slider { Minimum = min, Maximum = max, Value = initial, VerticalAlignment =
        ↪ VerticalAlignment.Center };
    var box = new TextBox { Text = Fmt.Number(initial), Width = 64, Margin = new Thickness(8, 0, 0, 0),
        ↪ VerticalContentAlignment = VerticalAlignment.Center };
    bool syncing = false;
    slider.ValueChanged += (_, e) =>
    {
        if (syncing) return;
        syncing = true;
        set(e.NewValue);
        box.Text = Fmt.Number(e.NewValue);
        syncing = false;
    };
    box.TextChanged += (_, _) =>
    {
        if (syncing) return;
        bool ok = Ui.TryParse(box.Text, out double v) && v >= 0;
        box.BorderBrush = ok ? SystemColors.ControlDarkBrush : Brushes.Red;
        if (!ok) return;
        syncing = true;
        set(v);
        slider.Value = Math.Clamp(v, min, max);
        syncing = false;
    };
    var cell = new DockPanel();
    DockPanel.SetDock(box, Dock.Right);
    cell.Children.Add(box);
    cell.Children.Add(slider);
    AddRow(grid, label, cell);
}
```

Used in chunk 429.

Numbers are shown in the user's locale, which in much of the world writes 0,5. But many users type a period anyway, especially people used to programming. A plain locale parser would read "0.5" as 5 in German, since the period is its thousands separator. So `Ui.TryParse` first tries the current culture and then the invariant one, and it accepts thousands separators only in the first attempt. The result is that "0,5" and "0.5" both mean one half under a comma locale, while under an English locale the second attempt changes nothing.

436 *⟨Win Ui: parsing a number in either notation 436⟩* ≡

```
public static bool TryParse(string text, out double value) =>
    double.TryParse(text, NumberStyles.Float | NumberStyles.AllowThousands, CultureInfo.CurrentCulture,
        ↪ out value)
    || double.TryParse(text, NumberStyles.Float, CultureInfo.InvariantCulture, out value);
```

Used in chunk 364.

Rows are laid out in a two-column grid, labels on the left, the SwiftUI `LabeledContent` of the sheet.

437 *⟨Win ParameterDialog: a labeled row 437⟩* ≡

```
private static void AddRow(Grid grid, string label, UIElement content)
{
    int row = grid.RowDefinitions.Count;
    grid.RowDefinitions.Add(new RowDefinition { Height = GridLength.Auto });
    var text = new TextBlock { Text = label, VerticalAlignment = VerticalAlignment.Center, Margin = new
        ↪ Thickness(0, 5, 14, 5) };
}
```

```

Grid.SetRow(text, row);
grid.Children.Add(text);
if (content is FrameworkElement fe) fe.Margin = new Thickness(0, 5, 0, 5);
Grid.SetRow(content, row);
Grid.SetColumn(content, 1);
grid.Children.Add(content);
}

```

Used in chunk 429.

The validation is the Swift one, with a guard that C# needs and Swift did not. Converting a double that exceeds the int range to int doesn't trap in C#; it silently produces a meaningless value, which could pass the size check. So the size is first checked in floating point, and only a plausible one is computed with `Resampler.OutputSize`.

438 *⟨Win ParameterDialog: validating a resample 438⟩* ≡

```

private void Validate()
{
    if (_request is not OperationRequest.Resample)
    {
        _apply.IsEnabled = true;
        return;
    }
    // Check in floating point first: casting a huge size to int would overflow.
    double largest = Math.Max(_source.Buffer.Width, _source.Buffer.Height) * _scale;
    bool valid = _scale > 0 && largest < 1e9;
    if (valid)
    {
        int w = Resampler.OutputSize(_source.Buffer.Width, _scale);
        int h = Resampler.OutputSize(_source.Buffer.Height, _scale);
        valid = Math.Max(w, h) <= MaxOutputSide;
        if (_result is not null) _result.Text = $"{w} × {h} px";
    }
    else if (_result is not null)
    {
        _result.Text = "-";
    }
    _apply.IsEnabled = valid;
}

```

Used in chunk 429.

The operation is built with a switch expression over the request's record type.

439 *⟨Win ParameterDialog: building the operation 439⟩* ≡

```

private ImageOperation BuildOperation() => _request switch
{
    OperationRequest.GaussianBlur => new ImageOperation.Filter(new FilterKind.GaussianBlur(_radius)),
    OperationRequest.BoxBlur => new ImageOperation.Filter(new FilterKind.BoxBlur(_radius)),
    OperationRequest.UnsharpMask => new ImageOperation.Filter(new FilterKind.UnsharpMask(_radius,
        ↪ _intensity)),
    OperationRequest.NoiseReduction => new ImageOperation.Filter(new
        ↪ FilterKind.NoiseReduction(_noiseLevel, _sharpness)),
    OperationRequest.Resample r => new ImageOperation.Resample(r.Method, _scale),
    _ => throw new InvalidOperationException(),
};

```

Used in chunk 429.

15.9 The test-pattern window

`TestPatternWindow` is the generator window of Chapter 9. The pattern analysis window has no counterpart here: a pattern is opened in an ordinary `MainWindow`, which shows its loading state while

the pattern is generated (Chapter 11). The generator keeps its spec in a field, and every control that edits it calls `SpecChanged`, which is what the SwiftUI `.task(id: spec)` reacts to implicitly.

```
440 < Windows/LineScope.App/Views/TestPatternWindow.cs 440> ≡
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Threading;
using LineScope.App.Model;
using LineScope.Core;

namespace LineScope.App.Views;

/// <summary>Window for choosing a pattern, its size and parameter before creating it.</summary>
public sealed class TestPatternWindow : Window
{
    <Win TestPatternWindow: fields 441>

    <Win TestPatternWindow: building the window 442>

    <Win TestPatternWindow: size fields 445>

    <Win TestPatternWindow: parameter field 446>

    <Win TestPatternWindow: debouncing the preview 448>

    <Win TestPatternWindow: rendering the preview 449>

    <Win TestPatternWindow: grid rows 450>
}

Root chunk, tangled to Windows/LineScope.App/Views/TestPatternWindow.cs.
```

The limits are the Swift ones: presets from 128 to 2048, at most 8192 pixels per side, and no preview above 2048×2048 pixels.

```
441 <Win TestPatternWindow: fields 441> ≡
private static readonly int[] Sizes = [128, 256, 512, 1024, 2048];
private const int MaxSide = 8192;
private const int MaxPreviewPixels = 2048 * 2048;

private TestPatternSpec _spec = new(TestPatternKind.ZonePlate);
private bool _sizeValid = true;

private readonly ComboBox _kind;
private readonly TextBlock _purpose;
private readonly TextBox _width, _height;
private readonly Grid _parameterRow = new();
private readonly Button _create;
private readonly Image _preview;
private readonly TextBlock _previewCaption;
private readonly DispatcherTimer _debounce;
private int _previewGeneration;

Used in chunk 440.
```

The window is a form on the left and the preview on the right, and sizes itself to its content. Create opens a window for the clamped spec; unlike SwiftUI's `openWindow(value:)`, it opens a new window every time.

442 *⟨Win TestPatternWindow: building the window 442⟩* ≡

```
public TestPatternWindow()
{
    Title = "New Test Pattern";
    SizeToContent = SizeToContent.WidthAndHeight;
    ResizeMode = ResizeMode.CanMinimize;
    UseLayoutRounding = true;
    WindowStartupLocation = WindowStartupLocation.CenterScreen;
```

⟨Win TestPatternWindow: pattern picker and description 443⟩

⟨Win TestPatternWindow: the size row 444⟩

```
_create = new Button { Content = "Create", IsDefault = true, MinWidth = 80, HorizontalAlignment =
    ↪ HorizontalAlignment.Right, Margin = new Thickness(0, 14, 0, 0) };
_create.Click += (_, _) => App.Current.OpenPattern(_spec.Clamped());

var form = new Grid { Width = 380 };
form.ColumnDefinitions.Add(new ColumnDefinition { Width = GridLength.Auto });
form.ColumnDefinitions.Add(new ColumnDefinition { Width = new GridLength(1, GridUnitType.Star) });
AddRow(form, "Pattern", _kind);
AddSpanning(form, _purpose);
AddRow(form, "Size", sizeRow);
AddSpanning(form, _parameterRow);
AddSpanning(form, _create);
BuildParameterRow();
```

⟨Win TestPatternWindow: the preview box 447⟩

```
Content = new StackPanel { Orientation = Orientation.Horizontal, Margin = new Thickness(16),
    ↪ Children = { form, previewPanel } };

_debounce = new DispatcherTimer { Interval = TimeSpan.FromMilliseconds(80) };
_debounce.Tick += (_, _) => { _debounce.Stop(); UpdatePreview(); };
SpecChanged();
}
```

Used in chunk 440.

Changing the kind builds a new spec that keeps the size and takes the new kind's default parameter, for the reason given in Chapter 9; the parameter row is rebuilt because its name, range and slider depend on the kind. The ! after `_purpose` tells the compiler that the field is assigned by the time the event can fire.

443 *⟨Win TestPatternWindow: pattern picker and description 443⟩* ≡

```
_kind = new ComboBox { ItemsSource = TestPatternKindInfo.All.Select(k => k.DisplayName()).ToList(),
    ↪ SelectedIndex = 0 };
_kind.SelectionChanged += (_, _) =>
{
    // Resets the parameter to the new pattern's default when the kind changes.
    _spec = new TestPatternSpec(TestPatternKindInfo.All[_kind.SelectedIndex], _spec.Width, _spec.Height);
    _purpose!.Text = _spec.Kind.Purpose();
    BuildParameterRow();
    SpecChanged();
};
_purpose = new TextBlock
{
    Text = _spec.Kind.Purpose(), TextWrapping = TextWrapping.Wrap, FontSize = 11, Foreground =
    ↪ Ui.SecondaryText,
    Margin = new Thickness(0, 6, 0, 10),
};
```

Used in chunk 442.

A size preset writes both text boxes, and their change handlers do the rest.

444 *⟨Win TestPatternWindow: the size row 444⟩* ≡

```

_width = SizeBox(_spec.Width);
_height = SizeBox(_spec.Height);
var presets = Ui.PresetsButton(Sizes.Select(s => ($"{s} × {s}"), (Action)(() =>
{
    _width.Text = s.ToString();
    _height.Text = s.ToString();
})))));
var sizeRow = new StackPanel
{
    Orientation = Orientation.Horizontal,
    Children = { _width, new TextBlock { Text = "x", Margin = new Thickness(6, 0, 6, 0),
        ↪ VerticalAlignment = VerticalAlignment.Center }, _height, presets },
};

```

Used in chunk 442.

The size boxes accept only whole numbers in range. An invalid size keeps the last valid spec but disables Create and the preview. The null check covers the first box's handler, which runs while the second box doesn't exist yet.

445 *⟨Win TestPatternWindow: size fields 445⟩* ≡

```

private TextBox SizeBox(int value)
{
    var box = new TextBox { Text = value.ToString(), Width = 64, VerticalContentAlignment =
        ↪ VerticalAlignment.Center };
    box.TextChanged += (_, _) => ReadSize();
    return box;
}

private void ReadSize()
{
    if (_width is null || _height is null) return;
    bool okW = int.TryParse(_width.Text, out int w) && w is >= 1 and <= MaxSide;
    bool okH = int.TryParse(_height.Text, out int h) && h is >= 1 and <= MaxSide;
    _width.BorderBrush = okW ? SystemColors.ControlDarkBrush : Brushes.Red;
    _height.BorderBrush = okH ? SystemColors.ControlDarkBrush : Brushes.Red;
    _sizeValid = okW && okH;
    if (_sizeValid) _spec = _spec with { Width = w, Height = h };
    SpecChanged();
}

```

Used in chunk 440.

The parameter row is the numeric field of the parameter dialog, adapted to the pattern's parameter: its name and range come from the kind, kinds without a parameter show nothing, and the noise seed gets no slider. As there, the typed value is kept even outside the slider's range, and Create clamps it.

446 *⟨Win TestPatternWindow: parameter field 446⟩* ≡

```

private void BuildParameterRow()
{
    _parameterRow.Children.Clear();
    _parameterRow.ColumnDefinitions.Clear();
    var p = _spec.Kind.Parameter();
    if (p is null) return;
    _parameterRow.Margin = new Thickness(0, 10, 0, 0);
    _parameterRow.ColumnDefinitions.Add(new ColumnDefinition { Width = GridLength.Auto });
    _parameterRow.ColumnDefinitions.Add(new ColumnDefinition { Width = new GridLength(1,
        ↪ GridUnitType.Star) });
    _parameterRow.ColumnDefinitions.Add(new ColumnDefinition { Width = GridLength.Auto });

    var label = new TextBlock { Text = p.Name, VerticalAlignment = VerticalAlignment.Center, Margin =
        ↪ new Thickness(0, 0, 10, 0) };
    _parameterRow.Children.Add(label);
}

```

```

var box = new TextBox { Text = Fmt.Number(_spec.Parameter, 4), Width = 70, Margin = new Thickness(8,
    ↪ 0, 0, 0), VerticalContentAlignment = VerticalAlignment.Center };
Grid.SetColumn(box, 2);
_parameterRow.Children.Add(box);
bool syncing = false;
Slider? slider = null;
if (_spec.Kind != TestPatternKind.WhiteNoise)
{
    slider = new Slider { Minimum = p.Min, Maximum = p.Max, Value = _spec.Parameter,
    ↪ VerticalAlignment = VerticalAlignment.Center };
    slider.ValueChanged += (_, e) =>
    {
        if (syncing) return;
        syncing = true;
        _spec = _spec with { Parameter = e.NewValue };
        box.Text = Fmt.Number(e.NewValue, 4);
        syncing = false;
        SpecChanged();
    };
    Grid.SetColumn(slider, 1);
    _parameterRow.Children.Add(slider);
}
box.TextChanged += (_, _) =>
{
    if (syncing) return;
    bool ok = Ui.TryParse(box.Text, out double v);
    box.BorderBrush = ok ? SystemColors.ControlDarkBrush : Brushes.Red;
    if (!ok) return;
    syncing = true;
    _spec = _spec with { Parameter = v };
    if (slider is not null) slider.Value = p.Clamp(v);
    syncing = false;
    SpecChanged();
};
}

```

Used in chunk 440.

The preview box is exactly 256×256 DIPs, and the image in it is not stretched, with nearest-neighbor scaling in case the system scales it anyway. Under the default mapping of Section 15.6 a DIP is an image pixel, so the preview shows pixels as the analysis window will.

447 *<Win TestPatternWindow: the preview box 447> ≡*

```

_preview = new Image { Width = 256, Height = 256, Stretch = Stretch.None };
RenderOptions.SetBitmapScalingMode(_preview, BitmapScalingMode.NearestNeighbor);
_previewCaption = new TextBlock { FontSize = 11, Foreground = Ui.SecondaryText, HorizontalAlignment =
    ↪ HorizontalAlignment.Center, Margin = new Thickness(0, 6, 0, 0) };
var previewPanel = new StackPanel
{
    Margin = new Thickness(20, 0, 0, 0),
    Children =
    {
        new Border { Width = 256, Height = 256, Background = SystemColors.AppWorkspaceBrush,
        ↪ CornerRadius = new CornerRadius(4), Child = _preview, ClipToBounds = true },
        _previewCaption,
    },
};

```

Used in chunk 442.

SwiftUI debounces by cancelling the spec's task and sleeping 80 ms in the new one. The port uses a `DispatcherTimer` of 80 ms that every change restarts, so the preview is rendered only once the spec has been left alone that long. A generation counter replaces task cancellation: each change increments it, and a preview that finishes after a newer change is discarded instead of overwriting a more recent one.

448 *⟨Win TestPatternWindow: debouncing the preview 448⟩* ≡

```
private void SpecChanged()
{
    if (_create is null) return;
    _create.IsEnabled = _sizeValid;
    _previewGeneration++;
    _debounce.Stop();
    bool previewable = _sizeValid && (long)_spec.Width * _spec.Height <= MaxPreviewPixels;
    _previewCaption.Text = previewable ? "Preview: center 256 × 256 px at 1:1" : "Too large to preview";
    if (!previewable)
    {
        _preview.Source = null;
        return;
    }
    _debounce.Start();
}
```

Used in chunk 440.

As in Swift, the preview is the center of the *full* pattern, not a small pattern, because a zone plate made small is a different image. The generation runs on the thread pool. The crop is a `CroppedBitmap` of the full bitmap, frozen so that it can be handed from the worker thread to the user-interface thread.

449 *⟨Win TestPatternWindow: rendering the preview 449⟩* ≡

```
/// <summary>
/// Generates the real pattern and shows its central pixels at 1:1, so aliasing looks as it will in the
/// analysis window.
/// </summary>
private async void UpdatePreview()
{
    int generation = _previewGeneration;
    var s = _spec.Clamped();
    var bitmap = await Task.Run(() =>
    {
        var full = TestPatternGenerator.Make(s);
        int w = Math.Min(s.Width, 256), h = Math.Min(s.Height, 256);
        var cropped = new CroppedBitmap(BitmapSource.FromBuffer(full), new Int32Rect((s.Width - w) / 2,
        ↪ (s.Height - h) / 2, w, h));
        cropped.Freeze();
        return (BitmapSource)cropped;
    });
    if (generation == _previewGeneration) _preview.Source = bitmap;
}
```

Used in chunk 440.

Two small helpers add a labeled row or a row spanning both columns to the form.

450 *⟨Win TestPatternWindow: grid rows 450⟩* ≡

```
private static void AddRow(Grid grid, string label, UIElement content)
{
    int row = grid.RowDefinitions.Count;
    grid.RowDefinitions.Add(new RowDefinition { Height = GridLength.Auto });
    var text = new TextBlock { Text = label, VerticalAlignment = VerticalAlignment.Center, Margin = new
    ↪ Thickness(0, 4, 12, 4) };
    Grid.SetRow(text, row);
    grid.Children.Add(text);
    Grid.SetRow(content, row);
    Grid.SetColumn(content, 1);
    grid.Children.Add(content);
}

private static void AddSpanning(Grid grid, UIElement content)
{
}
```

```

    int row = grid.RowDefinitions.Count;
    grid.RowDefinitions.Add(new RowDefinition { Height = GridLength.Auto });
    Grid.SetRow(content, row);
    Grid.SetColumnSpan(content, 2);
    grid.Children.Add(content);
}

```

Used in chunk 440.

15.10 The inspector

The inspector is still a separate window describing the focused variant of the most recently active document window, and it still finds that window through `ActiveSession`. It is now the only reader of `ActiveSession`, since the menus know their own window's session. Without SwiftUI's observation it has to subscribe to two things by hand: the active session changing, and the active session's own `Changed` event.

451 *< Windows/LineScope.App/Views/InspectorWindow.cs 451 > ≡*

```

using System.Windows;
using System.Windows.Controls;
using System.Windows.Threading;
using LineScope.App.Model;
using LineScope.Core;

namespace LineScope.App.Views;

/// <summary>Separate window describing the focused variant of the most recently active document
    ↪ window.</summary>
public sealed class InspectorWindow : Window
{
    private readonly ScrollViewer _scroll = new() { VerticalScrollBarVisibility =
        ↪ ScrollBarVisibility.Auto, Padding = new Thickness(14, 4, 14, 14) };
    private Session? _observed;
    private bool _refreshQueued;

    <Win InspectorWindow: placing the window 452>

    <Win InspectorWindow: following the active session 453>

    <Win InspectorWindow: rebuilding the content 454>

    <Win InspectorWindow: section and row helpers 462>
}

```

Root chunk, tangled to Windows/LineScope.App/Views/InspectorWindow.cs.

The inspector opens as a narrow window near the top-right corner of the work area, where it covers the least of a document window. Closing it unsubscribes from both events; otherwise the static `ActiveSession.Changed` would keep the closed window alive and busy.

452 *< Win InspectorWindow: placing the window 452 > ≡*

```

public InspectorWindow()
{
    Title = "Inspector";
    Width = 340;
    Height = Math.Min(760, SystemParameters.WorkArea.Height - 40);
    MinWidth = 300;
    MinHeight = 320;
    UseLayoutRounding = true;
    ShowInTaskbar = false;
    var work = SystemParameters.WorkArea;
    WindowStartupLocation = WindowStartupLocation.Manual;
}

```

```

Left = work.Right - Width - 20;
Top = work.Top + 20;
Content = _scroll;
Background = SystemColors.WindowBrush;

ActiveSession.Changed += OnActiveChanged;
Closed += (_, _) =>
{
    ActiveSession.Changed -= OnActiveChanged;
    Observe(null);
};
OnActiveChanged();
}

```

Used in chunk 451.

The inspector shows nearly everything, so it rebuilds on any change. But a line drag raises one change per mouse move, and rebuilding a whole panel of text boxes that often would make the drag stutter. So changes are coalesced: the first one queues a rebuild at **Background** priority, below input and rendering, and later ones are ignored until it has run. A drag thus updates the inspector at most once per pass of the dispatcher, after the canvas and charts have been drawn.

453 *⟨Win InspectorWindow: following the active session 453⟩* ≡

```

private void OnActiveChanged()
{
    Observe(ActiveSession.Current);
    Rebuild();
}

private void Observe(Session? session)
{
    if (_observed is not null) _observed.Changed -= OnSessionChanged;
    _observed = session;
    if (_observed is not null) _observed.Changed += OnSessionChanged;
}

/// <summary>Coalesces bursts of changes (such as a line drag) into one rebuild.</summary>
private void OnSessionChanged(SessionChange change)
{
    if (_refreshQueued) return;
    _refreshQueued = true;
    Dispatcher.BeginInvoke(DispatcherPriority.Background, () =>
    {
        _refreshQueued = false;
        Rebuild();
    });
}

```

Used in chunk 451.

The content has the five sections of the Swift inspector, in the same order and with the same rows. Like the Swift version, it asks the session for the same profile and spectrum as the charts, so the numbers agree with the zones.

454 *⟨Win InspectorWindow: rebuilding the content 454⟩* ≡

```

private void Rebuild()
{
    var session = _observed;
    if (session is null)
    {
        ⟨Win InspectorWindow: the empty state 455⟩
    }

    var variant = session.FocusedVariant;
}

```

```

var profile = session.Profile(variant);
var spectrum = session.Spectrum(profile);
var info = session.SourceInfo;
var original = session.Original.Buffer;
var panel = new StackPanel();

```

⟨Win InspectorWindow: the file section 456⟩

⟨Win InspectorWindow: the focused-image section 457⟩

⟨Win InspectorWindow: the operation-chain section 458⟩

⟨Win InspectorWindow: the line section 459⟩

⟨Win InspectorWindow: the channel-statistics section 460⟩

⟨Win InspectorWindow: keeping the scroll position 461⟩

}

Used in chunk 451.

With no document open, the inspector says so, as the Swift ContentUnavailableView does.

455 *⟨Win InspectorWindow: the empty state 455⟩* ≡

```

_scroll.Content = new StackPanel
{
    VerticalAlignment = VerticalAlignment.Center,
    Margin = new Thickness(0, 120, 0, 0),
    Children =
    {
        new TextBlock { Text = "No Image", FontSize = 18, FontWeight = FontWeights.SemiBold,
        ↪ HorizontalAlignment = HorizontalAlignment.Center },
        new TextBlock { Text = "Open an image to inspect it.", Foreground = Ui.SecondaryText,
        ↪ HorizontalAlignment = HorizontalAlignment.Center, Margin = new Thickness(0, 6, 0, 0) },
    },
};
return;

```

Used in chunk 454.

The file section reports what the WIC decoder found (Chapter 14).

456 *⟨Win InspectorWindow: the file section 456⟩* ≡

```

Section(panel, "File");
Row(panel, "Name", session.FileName);
Row(panel, "Type", info.FileType);
Row(panel, "Stored size", $"{info.PixelWidth} × {info.PixelHeight} px");
Row(panel, "Bit depth", $"{info.BitsPerComponent} bpc, {info.BitsPerPixel} bpp");
Row(panel, "Color space", info.ColorSpaceName);
Row(panel, "Alpha", info.HasAlpha ? "Yes" : "No");
if (info.Dpi is double dpi) Row(panel, "Resolution", $"{Fmt.Fixed(dpi, 0)} dpi");

```

Used in chunk 454.

The focused-image, operation-chain and line sections are row for row those of Chapter 9: the scale per axis, the full lineage, and the line in this variant's pixels with its length, sample count and spacing.

457 *⟨Win InspectorWindow: the focused-image section 457⟩* ≡

```

Section(panel, "Focused Image");
Row(panel, "Tab", variant.Name);
Row(panel, "Size", $"{variant.Buffer.Width} × {variant.Buffer.Height} px");
Row(panel, "Scale vs. original",
    $"{Fmt.Fixed((double)variant.Buffer.Width / original.Width, 4)} ·
    ↪ {Fmt.Fixed((double)variant.Buffer.Height / original.Height, 4)}");
Row(panel, "Working format", "RGBA Float32, sRGB-encoded");

```

Used in chunk 454.

The chain starts with the original and numbers every step after it.

458 *⟨Win InspectorWindow: the operation-chain section 458⟩* ≡

```
Section(panel, "Operation Chain");
panel.Children.Add(new TextBlock { Text = "Original", Foreground = Ui.SecondaryText, Margin = new
    ↳ Thickness(0, 2, 0, 2) });
for (int i = 0; i < variant.Lineage.Count; i++)
    panel.Children.Add(new TextBlock { Text = $"{i + 1}. {variant.Lineage[i].DisplayName}", Margin = new
        ↳ Thickness(0, 2, 0, 2), TextWrapping = TextWrapping.Wrap });
```

Used in chunk 454.

The line's endpoints are converted from normalized coordinates to this variant's pixels.

459 *⟨Win InspectorWindow: the line section 459⟩* ≡

```
Section(panel, "Line");
double w = variant.Buffer.Width, h = variant.Buffer.Height;
var l = session.Line;
Row(panel, "Start", $"{Fmt.Fixed(l.Start.X * w, 1)}, {Fmt.Fixed(l.Start.Y * h, 1)} px");
Row(panel, "End", $"{Fmt.Fixed(l.End.X * w, 1)}, {Fmt.Fixed(l.End.Y * h, 1)} px");
Row(panel, "Length", $"{Fmt.Fixed(profile.Samples.PixelLength, 1)} px");
Row(panel, "Samples", $"{profile.Values.Length}, spacing {Fmt.Fixed(profile.Samples.Spacing, 3)} px");
```

Used in chunk 454.

The statistics guard against an empty profile, where `Min` and `Max` of LINQ would throw instead of returning a value, and the peak is shown only if the spectrum has one.

460 *⟨Win InspectorWindow: the channel-statistics section 460⟩* ≡

```
Section(panel, $"{session.ColorModel.DisplayName()} · {session.ChannelName}");
var v = profile.Values;
Row(panel, "Min", Fmt.Fixed(v.Length > 0 ? v.Min() : 0, 4));
Row(panel, "Max", Fmt.Fixed(v.Length > 0 ? v.Max() : 0, 4));
Row(panel, "Mean", Fmt.Fixed(v.Sum() / Math.Max(v.Length, 1), 4));
if (spectrum.PeakIndex is int k)
{
    Row(panel, "Peak frequency", $"{Fmt.Fixed(spectrum.CyclesPerPixel[k], 4)} c/px ·
        ↳ {Fmt.Fixed(spectrum.CyclesPerLine[k], 2)} c/line");
    Row(panel, "Peak amplitude", Fmt.Fixed(spectrum.Amplitudes[k], 4));
}
```

Used in chunk 454.

Replacing the content would scroll back to the top on every change, which makes a long inspector unreadable during a drag. So the offset is saved and restored around the swap.

461 *⟨Win InspectorWindow: keeping the scroll position 461⟩* ≡

```
double offset = _scroll.VerticalOffset;
_scroll.Content = panel;
_scroll.ScrollToVerticalOffset(offset);
```

Used in chunk 454.

A section title is a bold line. A row puts the label on the left and the value on the right, in a read-only, borderless text box: that is how WPF makes text selectable, the equivalent of `.textSelection(.enabled)` in Swift.

462 *⟨Win InspectorWindow: section and row helpers 462⟩* ≡

```
private static void Section(Panel panel, string title) =>
    panel.Children.Add(new TextBlock { Text = title, FontWeight = FontWeights.SemiBold, Margin = new
        ↳ Thickness(0, 12, 0, 4) });

private static void Row(Panel panel, string label, string value)
{
    var grid = new Grid { Margin = new Thickness(0, 2, 0, 2) };
    grid.ColumnDefinitions.Add(new ColumnDefinition { Width = GridLength.Auto });
    grid.ColumnDefinitions.Add(new ColumnDefinition { Width = new GridLength(1, GridUnitType.Star) });
    grid.Children.Add(new TextBlock { Text = label, Margin = new Thickness(0, 0, 12, 0) });
    // A read-only, borderless TextBox so values can be selected and copied.
    var text = new TextBox
    {
        Text = value, IsReadOnly = true, BorderThickness = new Thickness(0), Background = null, Padding
        ↳ = new Thickness(0),
        TextAlignment = TextAlignment.Right, TextWrapping = TextWrapping.Wrap,
    };
    Grid.SetColumn(text, 1);
    grid.Children.Add(text);
    panel.Children.Add(grid);
}
```

Used in chunk 451.

Bibliography

- [1] Donald E. Knuth. Literate Programming. *The Computer Journal* 27(2):97–111, 1984.
- [2] Norman Ramsey. Literate Programming Simplified. *IEEE Software* 11(5):97–105, 1994.
- [3] Don P. Mitchell and Arun N. Netravali. Reconstruction Filters in Computer Graphics. *Computer Graphics (SIGGRAPH '88)* 22(4):221–228, 1988.
- [4] Robert G. Keys. Cubic Convolution Interpolation for Digital Image Processing. *IEEE Transactions on Acoustics, Speech, and Signal Processing* 29(6):1153–1160, 1981.
- [5] Fredric J. Harris. On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform. *Proceedings of the IEEE* 66(1):51–83, 1978.
- [6] XcodeGen. <https://github.com/yonaskolb/XcodeGen>.
- [7] James W. Cooley and John W. Tukey. An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of Computation* 19(90):297–301, 1965.
- [8] Microsoft. Windows Presentation Foundation and Windows Imaging Component documentation. <https://learn.microsoft.com/dotnet/desktop/wpf/>.

Index of Chunks

Every chunk name, the chunks that define it, and the chunks that use it. Root chunks are tangled to the file named by the chunk.

- ⟨*Charts.swift: the profile chart* 164⟩
Defined in 164. Used in 163.
- ⟨*Charts.swift: the spectrum chart* 166⟩
Defined in 166. Used in 163.
- ⟨*ColorSpaces: channel extraction* 56⟩
Defined in 56. Used in 52.
- ⟨*ColorSpaces: color model* 51⟩
Defined in 51. Used in 50.
- ⟨*ColorSpaces: conversions* 52⟩
Defined in 52. Used in 50.
- ⟨*ColorSpaces: HSL to RGB* 54⟩
Defined in 54. Used in 52.
- ⟨*ColorSpaces: RGB to CMYK and back* 55⟩
Defined in 55. Used in 52.
- ⟨*ColorSpaces: RGB to HSL* 53⟩
Defined in 53. Used in 52.
- ⟨*Commands: File menu, export and close* 198⟩
Defined in 198. Used in 195.
- ⟨*Commands: File menu, test patterns* 197⟩
Defined in 197. Used in 195.
- ⟨*Commands: Filter menu* 199⟩
Defined in 199. Used in 195.
- ⟨*Commands: finding the target session* 196⟩
Defined in 196. Used in 195.
- ⟨*Commands: Resample menu* 200⟩
Defined in 200. Used in 195.
- ⟨*Commands: View menu* 201⟩
Defined in 201. Used in 195.
- ⟨*Filters: 3x3 convolution* 38⟩
Defined in 38. Used in 29.
- ⟨*Filters: Core Image cases* 33⟩
Defined in 33. Used in 32.
- ⟨*Filters: CPU cases* 34⟩
Defined in 34. Used in 32.
- ⟨*Filters: dispatching a filter* 32⟩
Defined in 32. Used in 29.

- ⟨*Filters: number formatting for names* 31⟩
Defined in 31. Used in 29.
- ⟨*Filters: per-pixel color maps* 37⟩
Defined in 37. Used in 29.
- ⟨*Filters: running a Core Image filter on a buffer* 36⟩
Defined in 36. Used in 29.
- ⟨*Filters: Sobel gradient magnitude* 39⟩
Defined in 39. Used in 29.
- ⟨*Filters: the `FilterKind` enumeration* 30⟩
Defined in 30. Used in 29.
- ⟨*Filters: the Core Image context* 35⟩
Defined in 35. Used in 29.
- ⟨*Generator window: changing the pattern kind* 193⟩
Defined in 193. Used in 186.
- ⟨*Generator window: parameter field* 189⟩
Defined in 189. Used in 186.
- ⟨*Generator window: pattern picker and description* 187⟩
Defined in 187. Used in 186.
- ⟨*Generator window: rendering the preview* 191⟩
Defined in 191. Used in 186.
- ⟨*Generator window: size fields* 188⟩
Defined in 188. Used in 186.
- ⟨*Generator window: the create button* 190⟩
Defined in 190. Used in 186.
- ⟨*Generator window: the preview box* 192⟩
Defined in 192. Used in 186.
- ⟨*Generator window: validation and clamping* 194⟩
Defined in 194. Used in 186.
- ⟨*ImageCanvas.swift: the line overlay* 162⟩
Defined in 162. Used in 153.
- ⟨*ImageCanvas: coordinate conversions* 161⟩
Defined in 161. Used in 153.
- ⟨*ImageCanvas: drawing the image at true scale* 155⟩
Defined in 155. Used in 153.
- ⟨*ImageCanvas: hit testing the line* 157⟩
Defined in 157. Used in 153.
- ⟨*ImageCanvas: moving the whole segment without leaving the image* 159⟩
Defined in 159. Used in 158.
- ⟨*ImageCanvas: Shift snapping* 160⟩
Defined in 160. Used in 153.
- ⟨*ImageCanvas: the line gesture* 156⟩
Defined in 156. Used in 153.
- ⟨*ImageCanvas: the state of a drag* 154⟩
Defined in 154. Used in 153.
- ⟨*ImageCanvas: updating the line during a drag* 158⟩
Defined in 158. Used in 153.
- ⟨*ImageDocument: a generated test pattern* 105⟩
Defined in 105. Used in 101.

- ⟨*ImageDocument: decoding the first image* 103⟩
Defined in 103. Used in 102.
- ⟨*ImageDocument: reading a file* 102⟩
Defined in 102. Used in 101.
- ⟨*ImageDocument: recording facts about the source* 104⟩
Defined in 104. Used in 102.
- ⟨*ImageDocument: refusing to write* 106⟩
Defined in 106. Used in 101.
- ⟨*ImageDocument: source information* 100⟩
Defined in 100. Used in 99.
- ⟨*ImageDocument: the document type* 101⟩
Defined in 101. Used in 99.
- ⟨*ImageOperation: applying the step* 42⟩
Defined in 42. Used in 40.
- ⟨*ImageOperation: display name* 41⟩
Defined in 41. Used in 40.
- ⟨*ImageVariant: operation requests* 112⟩
Defined in 112. Used in 107.
- ⟨*ImageVariant: panes* 110⟩
Defined in 110. Used in 107.
- ⟨*ImageVariant: the frequency axis* 111⟩
Defined in 111. Used in 107.
- ⟨*ImageVariant: the variant type* 108⟩
Defined in 108. Used in 107.
- ⟨*ImageVariant: zones* 109⟩
Defined in 109. Used in 107.
- ⟨*InspectorContent: row and number helpers* 183⟩
Defined in 183. Used in 177.
- ⟨*InspectorContent: the channel-statistics section* 182⟩
Defined in 182. Used in 177.
- ⟨*InspectorContent: the file section* 178⟩
Defined in 178. Used in 177.
- ⟨*InspectorContent: the focused-image section* 179⟩
Defined in 179. Used in 177.
- ⟨*InspectorContent: the line section* 181⟩
Defined in 181. Used in 177.
- ⟨*InspectorContent: the operation-chain section* 180⟩
Defined in 180. Used in 177.
- ⟨*InspectorView.swift: the inspector's content* 177⟩
Defined in 177. Used in 176.
- ⟨*LineSampler: bilinear interpolation* 49⟩
Defined in 49. Used in 47.
- ⟨*LineSampler: line segment* 45⟩
Defined in 45. Used in 43.
- ⟨*LineSampler: normalized point* 44⟩
Defined in 44. Used in 43.
- ⟨*LineSampler: sample along the line* 48⟩
Defined in 48. Used in 47.

- ⟨*LineSampler: sample record* 46⟩
Defined in 46. Used in 43.
- ⟨*LineSampler: sampler* 47⟩
Defined in 47. Used in 43.
- ⟨*LineScope/Commands/LineScopeCommands.swift* 195⟩
Defined in 195. Used in root.
- ⟨*LineScope/Document/ImageDocument.swift* 99⟩
Defined in 99. Used in root.
- ⟨*LineScope/LineScopeApp.swift* 94⟩
Defined in 94. Used in root.
- ⟨*LineScope/Model/ImageVariant.swift* 107⟩
Defined in 107. Used in root.
- ⟨*LineScope/Model/Session.swift* 113⟩
Defined in 113. Used in root.
- ⟨*LineScope/Views/Charts.swift* 163⟩
Defined in 163. Used in root.
- ⟨*LineScope/Views/ImageCanvas.swift* 153⟩
Defined in 153. Used in root.
- ⟨*LineScope/Views/InspectorView.swift* 176⟩
Defined in 176. Used in root.
- ⟨*LineScope/Views/MainView.swift* 131⟩
Defined in 131. Used in root.
- ⟨*LineScope/Views/PaneView.swift* 147⟩
Defined in 147. Used in root.
- ⟨*LineScope/Views/ParameterSheet.swift* 169⟩
Defined in 169. Used in root.
- ⟨*LineScope/Views/TestPatternViews.swift* 184⟩
Defined in 184. Used in root.
- ⟨*LineScope/Views/ZoneView.swift* 138⟩
Defined in 138. Used in root.
- ⟨*LineScopeApp: document windows* 95⟩
Defined in 95. Used in 94.
- ⟨*LineScopeApp: inspector window* 98⟩
Defined in 98. Used in 94.
- ⟨*LineScopeApp: pattern generator window* 97⟩
Defined in 97. Used in 94.
- ⟨*LineScopeApp: test-pattern windows* 96⟩
Defined in 96. Used in 94.
- ⟨*MainView.swift: the window observer* 136⟩
Defined in 136. Used in 131.
- ⟨*MainView: publishing the session to menus and inspector* 134⟩
Defined in 134. Used in 131.
- ⟨*MainView: sheets, alerts and the progress indicator* 135⟩
Defined in 135. Used in 131.
- ⟨*MainView: the session is created once per window* 132⟩
Defined in 132. Used in 131.
- ⟨*MainView: three zones stacked vertically* 133⟩
Defined in 133. Used in 131.

- ⟨*Packages/LineScopeCore/Package.swift* 2⟩
Defined in 2. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/ColorSpaces.swift* 50⟩
Defined in 50. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/Filters.swift* 29⟩
Defined in 29. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/ImageOperation.swift* 40⟩
Defined in 40. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/LineSampler.swift* 43⟩
Defined in 43. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/PixelBuffer.swift* 3⟩
Defined in 3. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/Resampling.swift* 12⟩
Defined in 12. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/Spectrum.swift* 57⟩
Defined in 57. Used in root.
- ⟨*Packages/LineScopeCore/Sources/LineScopeCore/TestPattern.swift* 67⟩
Defined in 67. Used in root.
- ⟨*Packages/LineScopeCore/Tests/LineScopeCoreTests/LineScopeCoreTests.swift* 82⟩
Defined in 82. Used in root.
- ⟨*PaneView.swift: the tab strip* 150⟩
Defined in 150. Used in 147.
- ⟨*PaneView: tab strip above the zone's content* 149⟩
Defined in 149. Used in 147.
- ⟨*PaneView: which variant this pane shows* 148⟩
Defined in 148. Used in 147.
- ⟨*ParameterSheet: a numeric field* 173⟩
Defined in 173. Used in 169.
- ⟨*ParameterSheet: building the operation* 175⟩
Defined in 175. Used in 169.
- ⟨*ParameterSheet: fields for each request* 172⟩
Defined in 172. Used in 169.
- ⟨*ParameterSheet: layout* 171⟩
Defined in 171. Used in 169.
- ⟨*ParameterSheet: parameters and their defaults* 170⟩
Defined in 170. Used in 169.
- ⟨*ParameterSheet: validating a resample* 174⟩
Defined in 174. Used in 169.
- ⟨*PixelBuffer: decoding a CGImage* 10⟩
Defined in 10. Used in 3.
- ⟨*PixelBuffer: encoding a CGImage* 11⟩
Defined in 11. Used in 3.
- ⟨*PixelBuffer: pixel subscript* 7⟩
Defined in 7. Used in 5.
- ⟨*PixelBuffer: premultiplication* 8⟩
Defined in 8. Used in 5.
- ⟨*PixelBuffer: stored properties and initializers* 6⟩
Defined in 6. Used in 5.

- ⟨*PixelBuffer: the buffer type* 5⟩
Defined in 5. Used in 3.
- ⟨*PixelBuffer: the color sample RGBA* 4⟩
Defined in 4. Used in 3.
- ⟨*PixelBuffer: the sRGB color space* 9⟩
Defined in 9. Used in 3.
- ⟨*ProfileChart: a color for each channel* 165⟩
Defined in 165. Used in 164.
- ⟨*project.yml* 1⟩
Defined in 1. Used in root.
- ⟨*Resampler: area-average contribution* 22⟩
Defined in 22. Used in 20.
- ⟨*Resampler: contributions of input pixels* 20⟩
Defined in 20. Used in 14.
- ⟨*Resampler: horizontal pass* 18⟩
Defined in 18. Used in 17.
- ⟨*Resampler: kernel contribution* 23⟩
Defined in 23. Used in 20.
- ⟨*Resampler: kernel evaluation* 28⟩
Defined in 28. Used in 14.
- ⟨*Resampler: kernel supports* 25⟩
Defined in 25. Used in 14.
- ⟨*Resampler: nearest-neighbor contribution* 21⟩
Defined in 21. Used in 20.
- ⟨*Resampler: resampling by a scale factor* 16⟩
Defined in 16. Used in 14.
- ⟨*Resampler: resampling to a given size* 17⟩
Defined in 17. Used in 14.
- ⟨*Resampler: sinc radius and output size* 15⟩
Defined in 15. Used in 14.
- ⟨*Resampler: the BC-spline family* 27⟩
Defined in 27. Used in 14.
- ⟨*Resampler: the sinc function* 26⟩
Defined in 26. Used in 14.
- ⟨*Resampler: trimming and normalizing weights* 24⟩
Defined in 24. Used in 14.
- ⟨*Resampler: vertical pass* 19⟩
Defined in 19. Used in 17.
- ⟨*Resampling: the ResampleMethod enumeration* 13⟩
Defined in 13. Used in 12.
- ⟨*Resampling: the Resampler namespace* 14⟩
Defined in 14. Used in 12.
- ⟨*Session: analysis settings* 118⟩
Defined in 118. Used in 116.
- ⟨*Session: closing a variant* 123⟩
Defined in 123. Used in 115.
- ⟨*Session: computing a profile* 126⟩
Defined in 126. Used in 115.

- ⟨*Session: computing a spectrum* 127⟩
Defined in 127. Used in 115.
- ⟨*Session: exporting the focused variant* 128⟩
Defined in 128. Used in 115.
- ⟨*Session: identity, variants and the line* 117⟩
Defined in 117. Used in 116.
- ⟨*Session: initialization* 120⟩
Defined in 120. Used in 115.
- ⟨*Session: looking up variants* 121⟩
Defined in 121. Used in 115.
- ⟨*Session: panes, focus and pending work* 119⟩
Defined in 119. Used in 116.
- ⟨*Session: performing an operation in the background* 124⟩
Defined in 124. Used in 115.
- ⟨*Session: profile and spectrum point types* 114⟩
Defined in 114. Used in 113.
- ⟨*Session: selecting and splitting panes* 122⟩
Defined in 122. Used in 115.
- ⟨*Session: showing a new variant* 125⟩
Defined in 125. Used in 115.
- ⟨*Session: stored state* 116⟩
Defined in 116. Used in 115.
- ⟨*Session: the active session* 130⟩
Defined in 130. Used in 113.
- ⟨*Session: the focused-value entry* 129⟩
Defined in 129. Used in 113.
- ⟨*Session: the session class* 115⟩
Defined in 115. Used in 113.
- ⟨*Spectrum: analyzer* 60⟩
Defined in 60. Used in 57.
- ⟨*Spectrum: choose the FFT size* 61⟩
Defined in 61. Used in 60.
- ⟨*Spectrum: condition the signal* 62⟩
Defined in 62. Used in 60.
- ⟨*Spectrum: frequency axes* 66⟩
Defined in 66. Used in 60.
- ⟨*Spectrum: packed real FFT* 64⟩
Defined in 64. Used in 60.
- ⟨*Spectrum: result* 59⟩
Defined in 59. Used in 57.
- ⟨*Spectrum: single-sided amplitudes* 65⟩
Defined in 65. Used in 60.
- ⟨*Spectrum: window functions* 58⟩
Defined in 58. Used in 57.
- ⟨*Spectrum: zero-pad* 63⟩
Defined in 63. Used in 60.
- ⟨*SpectrumChart: the amplitude range* 168⟩
Defined in 168. Used in 166.

- ⟨*SpectrumChart: the Nyquist frequency on the current axis* 167⟩
Defined in 167. Used in 166.
- ⟨*TabStrip: a tab's context menu* 152⟩
Defined in 152. Used in 150.
- ⟨*TabStrip: a tab's label* 151⟩
Defined in 151. Used in 150.
- ⟨*TestPattern: chirps and gratings* 77⟩
Defined in 77. Used in 74.
- ⟨*TestPattern: display names* 70⟩
Defined in 70. Used in 68.
- ⟨*TestPattern: edges and impulses* 78⟩
Defined in 78. Used in 74.
- ⟨*TestPattern: generator* 74⟩
Defined in 74. Used in 67.
- ⟨*TestPattern: gray-level helper* 76⟩
Defined in 76. Used in 74.
- ⟨*TestPattern: noise and color ramp* 80⟩
Defined in 80. Used in 74.
- ⟨*TestPattern: pattern kinds* 68⟩
Defined in 68. Used in 67.
- ⟨*TestPattern: set up the canvas* 75⟩
Defined in 75. Used in 74.
- ⟨*TestPattern: Siemens star* 79⟩
Defined in 79. Used in 74.
- ⟨*TestPattern: specification* 73⟩
Defined in 73. Used in 67.
- ⟨*TestPattern: SplitMix64* 81⟩
Defined in 81. Used in 67.
- ⟨*TestPattern: the cases* 69⟩
Defined in 69. Used in 68.
- ⟨*TestPattern: the tunable parameter* 72⟩
Defined in 72. Used in 68.
- ⟨*TestPattern: what each pattern is for* 71⟩
Defined in 71. Used in 68.
- ⟨*TestPatternViews.swift: the generator window* 186⟩
Defined in 186. Used in 184.
- ⟨*TestPatternViews.swift: the pattern analysis window* 185⟩
Defined in 185. Used in 184.
- ⟨*Tests: area average halves* 88⟩
Defined in 88. Used in 85.
- ⟨*Tests: color conversions* 90⟩
Defined in 90. Used in 82.
- ⟨*Tests: constants stay constant* 89⟩
Defined in 89. Used in 85.
- ⟨*Tests: Core Graphics bridge and filters* 92⟩
Defined in 92. Used in 82.
- ⟨*Tests: gradient fixture* 83⟩
Defined in 83. Used in 82.

- ⟨*Tests: identity at scale one* 87⟩
Defined in 87. Used in 85.
- ⟨*Tests: kernel shapes* 86⟩
Defined in 86. Used in 85.
- ⟨*Tests: line sampler* 91⟩
Defined in 91. Used in 82.
- ⟨*Tests: resampling* 85⟩
Defined in 85. Used in 82.
- ⟨*Tests: spectrum* 84⟩
Defined in 84. Used in 82.
- ⟨*Tests: test patterns* 93⟩
Defined in 93. Used in 82.
- ⟨*Win App: creating a document window* 210⟩
Defined in 210. Used in 207.
- ⟨*Win App: opening a file* 212⟩
Defined in 212. Used in 207.
- ⟨*Win App: opening a test pattern* 213⟩
Defined in 213. Used in 207.
- ⟨*Win App: shared state* 208⟩
Defined in 208. Used in 207.
- ⟨*Win App: startup* 209⟩
Defined in 209. Used in 207.
- ⟨*Win App: the inspector and the pattern generator* 214⟩
Defined in 214. Used in 207.
- ⟨*Win App: the Open dialog* 211⟩
Defined in 211. Used in 207.
- ⟨*Win Charts.cs: the chart base class* 417⟩
Defined in 417. Used in 415.
- ⟨*Win Charts.cs: the profile chart* 426⟩
Defined in 426. Used in 415.
- ⟨*Win Charts.cs: the spectrum chart* 428⟩
Defined in 428. Used in 415.
- ⟨*Win Charts.cs: what a chart draws* 416⟩
Defined in 416. Used in 415.
- ⟨*Win ChartView: axis titles* 421⟩
Defined in 421. Used in 417.
- ⟨*Win ChartView: data, pens and text* 418⟩
Defined in 418. Used in 417.
- ⟨*Win ChartView: grid, ticks and axes* 420⟩
Defined in 420. Used in 417.
- ⟨*Win ChartView: laying out the plot area* 419⟩
Defined in 419. Used in 417.
- ⟨*Win ChartView: nice tick values* 424⟩
Defined in 424. Used in 417.
- ⟨*Win ChartView: the Nyquist rule* 422⟩
Defined in 422. Used in 417.
- ⟨*Win ChartView: the series as one polyline* 423⟩
Defined in 423. Used in 417.

- ⟨*Win ChartView: tick labels* 425⟩
Defined in 425. Used in 417.
- ⟨*Win ColorSpaces: channel extraction* 277⟩
Defined in 277. Used in 273.
- ⟨*Win ColorSpaces: CMYK* 276⟩
Defined in 276. Used in 273.
- ⟨*Win ColorSpaces: HSL* 275⟩
Defined in 275. Used in 273.
- ⟨*Win ColorSpaces: names, channels and plot ranges* 274⟩
Defined in 274. Used in 273.
- ⟨*Win Core project: optimizing Debug builds* 217⟩
Defined in 217. Used in 215.
- ⟨*Win Core project: target and language options* 216⟩
Defined in 216. Used in 215.
- ⟨*Win Fft: bit-reversal permutation* 288⟩
Defined in 288. Used in 287.
- ⟨*Win Fft: butterflies* 289⟩
Defined in 289. Used in 287.
- ⟨*Win Filters: 3x3 convolution* 258⟩
Defined in 258. Used in 245.
- ⟨*Win Filters: 3x3 median* 253⟩
Defined in 253. Used in 245.
- ⟨*Win Filters: dispatching a filter* 246⟩
Defined in 246. Used in 245.
- ⟨*Win Filters: display names* 244⟩
Defined in 244. Used in 242.
- ⟨*Win Filters: noise reduction* 254⟩
Defined in 254. Used in 245.
- ⟨*Win Filters: noise reduction, scanning the neighborhood* 255⟩
Defined in 255. Used in 254.
- ⟨*Win Filters: noise reduction, smoothing or sharpening* 256⟩
Defined in 256. Used in 254.
- ⟨*Win Filters: one record per filter* 243⟩
Defined in 243. Used in 242.
- ⟨*Win Filters: per-pixel color maps* 257⟩
Defined in 257. Used in 245.
- ⟨*Win Filters: running on premultiplied data* 247⟩
Defined in 247. Used in 245.
- ⟨*Win Filters: separable convolution* 248⟩
Defined in 248. Used in 245.
- ⟨*Win Filters: Sobel gradient magnitude* 259⟩
Defined in 259. Used in 245.
- ⟨*Win Filters: the FilterKind records* 242⟩
Defined in 242. Used in 241.
- ⟨*Win Filters: the Filters class* 245⟩
Defined in 245. Used in 241.
- ⟨*Win Filters: the box kernel* 250⟩
Defined in 250. Used in 245.

- ⟨ *Win Filters: the Gaussian kernel* 249⟩
Defined in 249. Used in 245.
- ⟨ *Win Filters: the two blurs* 251⟩
Defined in 251. Used in 245.
- ⟨ *Win Filters: unsharp mask* 252⟩
Defined in 252. Used in 245.
- ⟨ *Win Fmt: a fixed number of decimals* 265⟩
Defined in 265. Used in 263.
- ⟨ *Win Fmt: at most a few decimals* 264⟩
Defined in 264. Used in 263.
- ⟨ *Win ImageCanvas.cs: the line overlay* 414⟩
Defined in 414. Used in 403.
- ⟨ *Win ImageCanvas: coordinate conversions* 413⟩
Defined in 413. Used in 403.
- ⟨ *Win ImageCanvas: fields and the state of a drag* 404⟩
Defined in 404. Used in 403.
- ⟨ *Win ImageCanvas: hit testing the line* 409⟩
Defined in 409. Used in 403.
- ⟨ *Win ImageCanvas: image pixels and DIPs* 407⟩
Defined in 407. Used in 403.
- ⟨ *Win ImageCanvas: moving the whole segment without leaving the image* 411⟩
Defined in 411. Used in 410.
- ⟨ *Win ImageCanvas: Shift snapping* 412⟩
Defined in 412. Used in 403.
- ⟨ *Win ImageCanvas: showing a variant* 406⟩
Defined in 406. Used in 403.
- ⟨ *Win ImageCanvas: the mouse handlers* 408⟩
Defined in 408. Used in 403.
- ⟨ *Win ImageCanvas: the scrolling surface* 405⟩
Defined in 405. Used in 403.
- ⟨ *Win ImageCanvas: updating the line during a drag* 410⟩
Defined in 410. Used in 403.
- ⟨ *Win ImageDocument: a generated test pattern* 334⟩
Defined in 334. Used in 327.
- ⟨ *Win ImageDocument: converting to sRGB* 335⟩
Defined in 335. Used in 327.
- ⟨ *Win ImageDocument: converting to the working buffer* 332⟩
Defined in 332. Used in 330.
- ⟨ *Win ImageDocument: decoding the first frame* 331⟩
Defined in 331. Used in 330.
- ⟨ *Win ImageDocument: detecting alpha* 336⟩
Defined in 336. Used in 327.
- ⟨ *Win ImageDocument: reading a file* 330⟩
Defined in 330. Used in 327.
- ⟨ *Win ImageDocument: recording facts about the source* 333⟩
Defined in 333. Used in 330.
- ⟨ *Win ImageDocument: rendering a buffer* 338⟩
Defined in 338. Used in 327.

- ⟨ *Win ImageDocument: source information* 328 ⟩
Defined in 328. Used in 327.
- ⟨ *Win ImageDocument: stored data* 329 ⟩
Defined in 329. Used in 327.
- ⟨ *Win ImageDocument: the ICC profile's name* 337 ⟩
Defined in 337. Used in 327.
- ⟨ *Win ImageOperation: applying the step* 262 ⟩
Defined in 262. Used in 260.
- ⟨ *Win ImageOperation: display name* 261 ⟩
Defined in 261. Used in 260.
- ⟨ *Win ImageVariant: operation requests* 344 ⟩
Defined in 344. Used in 339.
- ⟨ *Win ImageVariant: panes* 342 ⟩
Defined in 342. Used in 339.
- ⟨ *Win ImageVariant: the frequency axis* 343 ⟩
Defined in 343. Used in 339.
- ⟨ *Win ImageVariant: the variant type* 340 ⟩
Defined in 340. Used in 339.
- ⟨ *Win ImageVariant: zones* 341 ⟩
Defined in 341. Used in 339.
- ⟨ *Win InspectorWindow: following the active session* 453 ⟩
Defined in 453. Used in 451.
- ⟨ *Win InspectorWindow: keeping the scroll position* 461 ⟩
Defined in 461. Used in 454.
- ⟨ *Win InspectorWindow: placing the window* 452 ⟩
Defined in 452. Used in 451.
- ⟨ *Win InspectorWindow: rebuilding the content* 454 ⟩
Defined in 454. Used in 451.
- ⟨ *Win InspectorWindow: section and row helpers* 462 ⟩
Defined in 462. Used in 451.
- ⟨ *Win InspectorWindow: the channel-statistics section* 460 ⟩
Defined in 460. Used in 454.
- ⟨ *Win InspectorWindow: the empty state* 455 ⟩
Defined in 455. Used in 454.
- ⟨ *Win InspectorWindow: the file section* 456 ⟩
Defined in 456. Used in 454.
- ⟨ *Win InspectorWindow: the focused-image section* 457 ⟩
Defined in 457. Used in 454.
- ⟨ *Win InspectorWindow: the line section* 459 ⟩
Defined in 459. Used in 454.
- ⟨ *Win InspectorWindow: the operation-chain section* 458 ⟩
Defined in 458. Used in 454.
- ⟨ *Win LineSampler: bilinear interpolation* 272 ⟩
Defined in 272. Used in 270.
- ⟨ *Win LineSampler: line segment* 268 ⟩
Defined in 268. Used in 266.
- ⟨ *Win LineSampler: normalized point* 267 ⟩
Defined in 267. Used in 266.

- ⟨*Win LineSampler: sample along the line* 271⟩
Defined in 271. Used in 270.
- ⟨*Win LineSampler: sample record* 269⟩
Defined in 269. Used in 266.
- ⟨*Win LineSampler: sampler* 270⟩
Defined in 270. Used in 266.
- ⟨*Win MainWindow: adding a menu item and its shortcut* 381⟩
Defined in 381. Used in 368.
- ⟨*Win MainWindow: building the window* 370⟩
Defined in 370. Used in 368.
- ⟨*Win MainWindow: dropping files* 375⟩
Defined in 375. Used in 368.
- ⟨*Win MainWindow: fields* 369⟩
Defined in 369. Used in 368.
- ⟨*Win MainWindow: loading a document* 372⟩
Defined in 372. Used in 368.
- ⟨*Win MainWindow: reacting to session changes* 374⟩
Defined in 374. Used in 368.
- ⟨*Win MainWindow: running and requesting operations* 382⟩
Defined in 382. Used in 368.
- ⟨*Win MainWindow: the File menu* 377⟩
Defined in 377. Used in 376.
- ⟨*Win MainWindow: the Filter menu* 378⟩
Defined in 378. Used in 376.
- ⟨*Win MainWindow: the menu bar* 376⟩
Defined in 376. Used in 368.
- ⟨*Win MainWindow: the Resample menu* 379⟩
Defined in 379. Used in 376.
- ⟨*Win MainWindow: the start screen and the loading state* 371⟩
Defined in 371. Used in 368.
- ⟨*Win MainWindow: the View menu* 380⟩
Defined in 380. Used in 376.
- ⟨*Win MainWindow: three zones stacked with splitters* 373⟩
Defined in 373. Used in 372.
- ⟨*Win PaneView: a tab's context menu* 402⟩
Defined in 402. Used in 400.
- ⟨*Win PaneView: a tab's label* 401⟩
Defined in 401. Used in 400.
- ⟨*Win PaneView: fields* 396⟩
Defined in 396. Used in 395.
- ⟨*Win PaneView: tab strip above the zone's content* 397⟩
Defined in 397. Used in 395.
- ⟨*Win PaneView: the tab strip* 400⟩
Defined in 400. Used in 395.
- ⟨*Win PaneView: updating from session changes* 399⟩
Defined in 399. Used in 395.
- ⟨*Win PaneView: which variant this pane shows* 398⟩
Defined in 398. Used in 395.

- ⟨*Win ParameterDialog: a labeled row 437*⟩
Defined in 437. Used in 429.
- ⟨*Win ParameterDialog: a numeric field 435*⟩
Defined in 435. Used in 429.
- ⟨*Win ParameterDialog: building the operation 439*⟩
Defined in 439. Used in 429.
- ⟨*Win ParameterDialog: fields for each request 432*⟩
Defined in 432. Used in 429.
- ⟨*Win ParameterDialog: layout 431*⟩
Defined in 431. Used in 429.
- ⟨*Win ParameterDialog: parameters and their defaults 430*⟩
Defined in 430. Used in 429.
- ⟨*Win ParameterDialog: the scale field and the resulting size 433*⟩
Defined in 433. Used in 432.
- ⟨*Win ParameterDialog: validating a resample 438*⟩
Defined in 438. Used in 429.
- ⟨*Win PixelBuffer: pixel indexer 222*⟩
Defined in 222. Used in 220.
- ⟨*Win PixelBuffer: premultiplication 223*⟩
Defined in 223. Used in 220.
- ⟨*Win PixelBuffer: premultiplied BGRA bridge 224*⟩
Defined in 224. Used in 220.
- ⟨*Win PixelBuffer: properties and constructors 221*⟩
Defined in 221. Used in 220.
- ⟨*Win PixelBuffer: rounding to a byte 225*⟩
Defined in 225. Used in 224.
- ⟨*Win PixelBuffer: the class 220*⟩
Defined in 220. Used in 218.
- ⟨*Win PixelBuffer: the color sample Rgba 219*⟩
Defined in 219. Used in 218.
- ⟨*Win PixelBuffer: value equality 226*⟩
Defined in 226. Used in 220.
- ⟨*Win ProfileChart: a color for each channel 427*⟩
Defined in 427. Used in 426.
- ⟨*Win Resampler: area-average contribution 237*⟩
Defined in 237. Used in 235.
- ⟨*Win Resampler: contributions of input pixels 235*⟩
Defined in 235. Used in 229.
- ⟨*Win Resampler: horizontal pass 233*⟩
Defined in 233. Used in 232.
- ⟨*Win Resampler: kernel contribution 238*⟩
Defined in 238. Used in 235.
- ⟨*Win Resampler: nearest-neighbor contribution 236*⟩
Defined in 236. Used in 235.
- ⟨*Win Resampler: resampling by a scale factor 231*⟩
Defined in 231. Used in 229.
- ⟨*Win Resampler: resampling to a given size 232*⟩
Defined in 232. Used in 229.

- ⟨*Win Resampler: sinc radius and output size* 230⟩
Defined in 230. Used in 229.
- ⟨*Win Resampler: the kernels* 240⟩
Defined in 240. Used in 229.
- ⟨*Win Resampler: trimming and normalizing weights* 239⟩
Defined in 239. Used in 229.
- ⟨*Win Resampler: vertical pass* 234⟩
Defined in 234. Used in 232.
- ⟨*Win Resampling: method names as extension methods* 228⟩
Defined in 228. Used in 227.
- ⟨*Win Resampling: the **Resampler** class* 229⟩
Defined in 229. Used in 227.
- ⟨*Win Session: change flags* 347⟩
Defined in 347. Used in 345.
- ⟨*Win Session: closing a variant* 357⟩
Defined in 357. Used in 345.
- ⟨*Win Session: color model and channel* 352⟩
Defined in 352. Used in 345.
- ⟨*Win Session: computing a profile* 360⟩
Defined in 360. Used in 345.
- ⟨*Win Session: computing a spectrum* 361⟩
Defined in 361. Used in 345.
- ⟨*Win Session: exporting the focused variant* 362⟩
Defined in 362. Used in 345.
- ⟨*Win Session: identity, variants and events* 348⟩
Defined in 348. Used in 345.
- ⟨*Win Session: initialization* 349⟩
Defined in 349. Used in 345.
- ⟨*Win Session: looking up variants* 355⟩
Defined in 355. Used in 345.
- ⟨*Win Session: panes, focus and running operations* 354⟩
Defined in 354. Used in 345.
- ⟨*Win Session: performing an operation in the background* 358⟩
Defined in 358. Used in 345.
- ⟨*Win Session: raising a change* 350⟩
Defined in 350. Used in 345.
- ⟨*Win Session: selecting and splitting panes* 356⟩
Defined in 356. Used in 345.
- ⟨*Win Session: showing a new variant* 359⟩
Defined in 359. Used in 345.
- ⟨*Win Session: spectrum and display settings* 353⟩
Defined in 353. Used in 345.
- ⟨*Win Session: the active session* 363⟩
Defined in 363. Used in 345.
- ⟨*Win Session: the line* 351⟩
Defined in 351. Used in 345.
- ⟨*Win Session: the profile record* 346⟩
Defined in 346. Used in 345.

- ⟨ *Win Spectrum: analyzer* 281⟩
Defined in 281. Used in 278.
- ⟨ *Win Spectrum: choose the FFT size* 282⟩
Defined in 282. Used in 281.
- ⟨ *Win Spectrum: complex FFT of the padded signal* 284⟩
Defined in 284. Used in 281.
- ⟨ *Win Spectrum: condition the signal* 283⟩
Defined in 283. Used in 281.
- ⟨ *Win Spectrum: frequency axes* 286⟩
Defined in 286. Used in 281.
- ⟨ *Win Spectrum: result* 280⟩
Defined in 280. Used in 278.
- ⟨ *Win Spectrum: single-sided amplitudes* 285⟩
Defined in 285. Used in 281.
- ⟨ *Win Spectrum: the radix-2 FFT* 287⟩
Defined in 287. Used in 278.
- ⟨ *Win Spectrum: window functions* 279⟩
Defined in 279. Used in 278.
- ⟨ *Win TestPattern: chirps and gratings* 301⟩
Defined in 301. Used in 298.
- ⟨ *Win TestPattern: display names* 294⟩
Defined in 294. Used in 293.
- ⟨ *Win TestPattern: edges and impulses* 302⟩
Defined in 302. Used in 298.
- ⟨ *Win TestPattern: generator* 298⟩
Defined in 298. Used in 290.
- ⟨ *Win TestPattern: gray-level helper* 300⟩
Defined in 300. Used in 298.
- ⟨ *Win TestPattern: names, purposes and parameters* 293⟩
Defined in 293. Used in 290.
- ⟨ *Win TestPattern: noise and color ramp* 304⟩
Defined in 304. Used in 298.
- ⟨ *Win TestPattern: pattern kinds* 291⟩
Defined in 291. Used in 290.
- ⟨ *Win TestPattern: set up the canvas* 299⟩
Defined in 299. Used in 298.
- ⟨ *Win TestPattern: Siemens star* 303⟩
Defined in 303. Used in 298.
- ⟨ *Win TestPattern: specification* 297⟩
Defined in 297. Used in 290.
- ⟨ *Win TestPattern: SplitMix64* 305⟩
Defined in 305. Used in 290.
- ⟨ *Win TestPattern: the parameter record* 292⟩
Defined in 292. Used in 290.
- ⟨ *Win TestPattern: the tunable parameter* 296⟩
Defined in 296. Used in 293.
- ⟨ *Win TestPattern: what each pattern is for* 295⟩
Defined in 295. Used in 293.

- ⟨*Win TestPatternWindow: building the window 442*⟩
Defined in 442. Used in 440.
- ⟨*Win TestPatternWindow: debouncing the preview 448*⟩
Defined in 448. Used in 440.
- ⟨*Win TestPatternWindow: fields 441*⟩
Defined in 441. Used in 440.
- ⟨*Win TestPatternWindow: grid rows 450*⟩
Defined in 450. Used in 440.
- ⟨*Win TestPatternWindow: parameter field 446*⟩
Defined in 446. Used in 440.
- ⟨*Win TestPatternWindow: pattern picker and description 443*⟩
Defined in 443. Used in 442.
- ⟨*Win TestPatternWindow: rendering the preview 449*⟩
Defined in 449. Used in 440.
- ⟨*Win TestPatternWindow: size fields 445*⟩
Defined in 445. Used in 440.
- ⟨*Win TestPatternWindow: the preview box 447*⟩
Defined in 447. Used in 442.
- ⟨*Win TestPatternWindow: the size row 444*⟩
Defined in 444. Used in 442.
- ⟨*Win Tests: area average halves 314*⟩
Defined in 314. Used in 311.
- ⟨*Win Tests: bitmap bridge 318*⟩
Defined in 318. Used in 307.
- ⟨*Win Tests: color conversions 316*⟩
Defined in 316. Used in 307.
- ⟨*Win Tests: constants stay constant 315*⟩
Defined in 315. Used in 311.
- ⟨*Win Tests: every filter 320*⟩
Defined in 320. Used in 319.
- ⟨*Win Tests: filters 319*⟩
Defined in 319. Used in 307.
- ⟨*Win Tests: filters keep the size 321*⟩
Defined in 321. Used in 319.
- ⟨*Win Tests: helpers 308*⟩
Defined in 308. Used in 307.
- ⟨*Win Tests: identity at scale one 313*⟩
Defined in 313. Used in 311.
- ⟨*Win Tests: kernel shapes 312*⟩
Defined in 312. Used in 311.
- ⟨*Win Tests: kernels are normalized 323*⟩
Defined in 323. Used in 319.
- ⟨*Win Tests: line sampler 317*⟩
Defined in 317. Used in 307.
- ⟨*Win Tests: resampling 311*⟩
Defined in 311. Used in 307.
- ⟨*Win Tests: smoothing filters keep constants 322*⟩
Defined in 322. Used in 319.

- ⟨*Win Tests: spectrum* 309⟩
Defined in 309. Used in 307.
- ⟨*Win Tests: SplitMix64 reference values* 326⟩
Defined in 326. Used in 325.
- ⟨*Win Tests: test patterns* 325⟩
Defined in 325. Used in 307.
- ⟨*Win Tests: the FFT against a naive DFT* 310⟩
Defined in 310. Used in 309.
- ⟨*Win Tests: the median and the noise filter* 324⟩
Defined in 324. Used in 319.
- ⟨*Win Ui.cs: a command for menu items* 365⟩
Defined in 365. Used in 364.
- ⟨*Win Ui: a presets button* 434⟩
Defined in 434. Used in 364.
- ⟨*Win Ui: a segmented control* 367⟩
Defined in 367. Used in 364.
- ⟨*Win Ui: parsing a number in either notation* 436⟩
Defined in 436. Used in 364.
- ⟨*Win Ui: secondary text* 366⟩
Defined in 366. Used in 364.
- ⟨*Win ZoneView: a combo box and a check box* 392⟩
Defined in 392. Used in 383.
- ⟨*Win ZoneView: controls specific to each zone* 388⟩
Defined in 388. Used in 383.
- ⟨*Win ZoneView: describing the line* 394⟩
Defined in 394. Used in 383.
- ⟨*Win ZoneView: fields* 384⟩
Defined in 384. Used in 383.
- ⟨*Win ZoneView: image-zone controls* 389⟩
Defined in 389. Used in 388.
- ⟨*Win ZoneView: profile-zone controls* 390⟩
Defined in 390. Used in 388.
- ⟨*Win ZoneView: side-by-side panes with splitters* 387⟩
Defined in 387. Used in 383.
- ⟨*Win ZoneView: spectrum-zone controls* 391⟩
Defined in 391. Used in 388.
- ⟨*Win ZoneView: split and unsplit buttons* 393⟩
Defined in 393. Used in 383.
- ⟨*Win ZoneView: the header row* 385⟩
Defined in 385. Used in 383.
- ⟨*Win ZoneView: updating from session changes* 386⟩
Defined in 386. Used in 383.
- ⟨*WindowKeyObserver: the observing NSView* 137⟩
Defined in 137. Used in 136.
- ⟨*Windows/LineScope.App/app.manifest* 204⟩
Defined in 204. Used in root.
- ⟨*Windows/LineScope.App/App.xaml* 206⟩
Defined in 206. Used in root.

- ⟨ *Windows/LineScope.App/App.xaml.cs* 207⟩
Defined in 207. Used in root.
- ⟨ *Windows/LineScope.App/AssemblyInfo.cs* 205⟩
Defined in 205. Used in root.
- ⟨ *Windows/LineScope.App/LineScope.App.csproj* 203⟩
Defined in 203. Used in root.
- ⟨ *Windows/LineScope.App/Model/ImageDocument.cs* 327⟩
Defined in 327. Used in root.
- ⟨ *Windows/LineScope.App/Model/ImageVariant.cs* 339⟩
Defined in 339. Used in root.
- ⟨ *Windows/LineScope.App/Model/Session.cs* 345⟩
Defined in 345. Used in root.
- ⟨ *Windows/LineScope.App/Views/Charts.cs* 415⟩
Defined in 415. Used in root.
- ⟨ *Windows/LineScope.App/Views/ImageCanvas.cs* 403⟩
Defined in 403. Used in root.
- ⟨ *Windows/LineScope.App/Views/InspectorWindow.cs* 451⟩
Defined in 451. Used in root.
- ⟨ *Windows/LineScope.App/Views/MainWindow.cs* 368⟩
Defined in 368. Used in root.
- ⟨ *Windows/LineScope.App/Views/PaneView.cs* 395⟩
Defined in 395. Used in root.
- ⟨ *Windows/LineScope.App/Views/ParameterDialog.cs* 429⟩
Defined in 429. Used in root.
- ⟨ *Windows/LineScope.App/Views/TestPatternWindow.cs* 440⟩
Defined in 440. Used in root.
- ⟨ *Windows/LineScope.App/Views/Ui.cs* 364⟩
Defined in 364. Used in root.
- ⟨ *Windows/LineScope.App/Views/ZoneView.cs* 383⟩
Defined in 383. Used in root.
- ⟨ *Windows/LineScope.Core.Tests/LineScope.Core.Tests.csproj* 306⟩
Defined in 306. Used in root.
- ⟨ *Windows/LineScope.Core.Tests/LineScopeCoreTests.cs* 307⟩
Defined in 307. Used in root.
- ⟨ *Windows/LineScope.Core/ColorSpaces.cs* 273⟩
Defined in 273. Used in root.
- ⟨ *Windows/LineScope.Core/Filters.cs* 241⟩
Defined in 241. Used in root.
- ⟨ *Windows/LineScope.Core/Formatting.cs* 263⟩
Defined in 263. Used in root.
- ⟨ *Windows/LineScope.Core/ImageOperation.cs* 260⟩
Defined in 260. Used in root.
- ⟨ *Windows/LineScope.Core/LineSampler.cs* 266⟩
Defined in 266. Used in root.
- ⟨ *Windows/LineScope.Core/LineScope.Core.csproj* 215⟩
Defined in 215. Used in root.
- ⟨ *Windows/LineScope.Core/PixelBuffer.cs* 218⟩
Defined in 218. Used in root.

- ⟨ *Windows/LineScope.Core/Resampling.cs* 227⟩
Defined in 227. Used in root.
- ⟨ *Windows/LineScope.Core/Spectrum.cs* 278⟩
Defined in 278. Used in root.
- ⟨ *Windows/LineScope.Core/TestPattern.cs* 290⟩
Defined in 290. Used in root.
- ⟨ *Windows/LineScope.sln* 202⟩
Defined in 202. Used in root.
- ⟨ *ZoneView: controls specific to each zone* 141⟩
Defined in 141. Used in 138.
- ⟨ *ZoneView: describing the line* 146⟩
Defined in 146. Used in 138.
- ⟨ *ZoneView: header above side-by-side panes* 139⟩
Defined in 139. Used in 138.
- ⟨ *ZoneView: image-zone controls* 142⟩
Defined in 142. Used in 141.
- ⟨ *ZoneView: profile-zone controls* 143⟩
Defined in 143. Used in 141.
- ⟨ *ZoneView: spectrum-zone controls* 144⟩
Defined in 144. Used in 141.
- ⟨ *ZoneView: split and unsplit buttons* 145⟩
Defined in 145. Used in 138.
- ⟨ *ZoneView: the header row* 140⟩
Defined in 140. Used in 138.